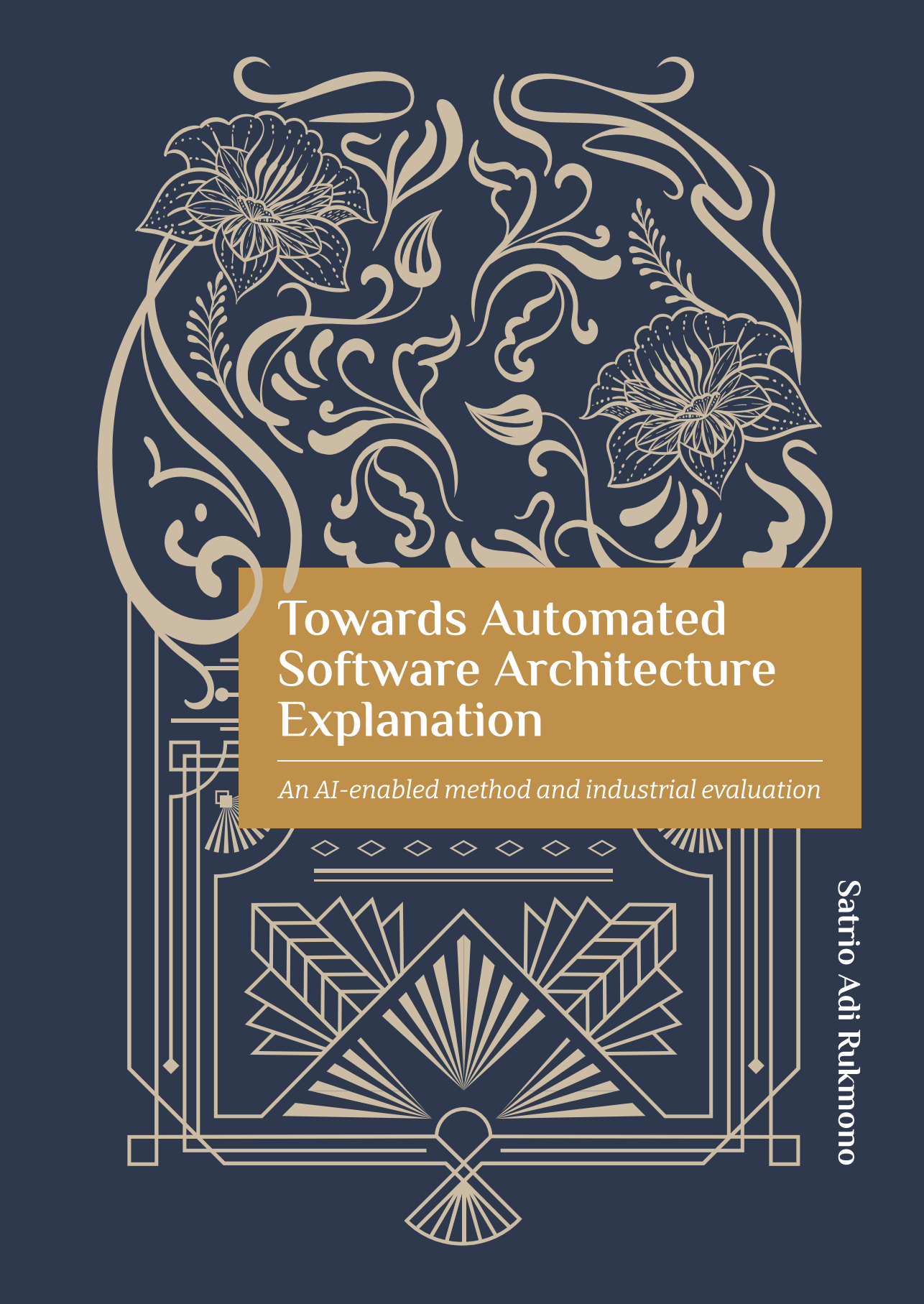


The background of the cover is a dark blue-grey color. It features a large, intricate white line-art design. The upper half is dominated by a symmetrical floral motif, possibly a stylized lily or iris, with long, flowing, Art Deco-inspired scrollwork and leaves extending from it. The lower half features a geometric, Art Deco-style design with a central fan-like shape, radiating lines, and various rectangular and triangular patterns. A horizontal orange-brown band is positioned across the middle, containing the title and subtitle.

Towards Automated Software Architecture Explanation

An AI-enabled method and industrial evaluation

Satrio Adi Rukmono

Towards Automated Software Architecture Explanation

An AI-enabled method and industrial evaluation

PROEFSCHRIFT

ter verkrijging van de graad van doctor aan de Technische Universiteit
Eindhoven op gezag van de rector magnificus prof.dr. S.K. Lenaerts,
voor een commissie aangewezen door het College voor Promoties, in
het openbaar te verdedigen op woensdag 20 mei 2026 om 11:00 uur.

door

Satrio Adi Rukmono

geboren te Jakarta, Indonesië

Dit proefschrift is goedgekeurd door de promotoren en de samenstelling van de promotiecommissie is als volgt:

Voorzitter: prof.dr. J.N. Kok

Promotor: prof.dr. M.R.V. Chaudron

Copromotor: dr. L.M. Ochoa Venegas

Leden: prof.dr. A. Vilanova
 prof.dr. A. van Deursen (Technische Universiteit Delft)
 prof.dr. J.M.W. Visser (Universiteit Leiden)
 prof.dr. G. Robles (Universidad Rey Juan Carlos)
 prof.dr. M.A.M. Soliman (Universität Paderborn)

Het onderzoek of ontwerp dat in dit proefschrift wordt beschreven is uitgevoerd in overeenstemming met de TU/e Gedragscode Wetenschapsbeoefening.



The work in the thesis has been carried out under the auspices of the research school IPA. Thesis number 2026-09.

A catalogue record is available from the Eindhoven University of Technology Library.

ISBN: 978-90-386-6695-2

Cover art © Aninda Purnamashari

All rights reserved. This book or parts thereof may not be reproduced in any form, stored in any retrieval system, or transmitted in any form by any means—electronic, mechanical, photocopy, recording, or otherwise—without prior written permission of the copyright owner.

Copyright © 2026 Satrio Adi Rukmono

Acknowledgments

“Sorry, you’re much too big. Simply impassible.”

“You mean impossible?”

“No, impassible. Nothing’s impossible!”

from Walt Disney’s *Alice in Wonderland*,
adaptation of Lewis Carroll’s *Alice’s Adventures in Wonderland*
and *Through the Looking-Glass*

Everything that has a beginning has an end. One could say that research began when humanity began and possibly will only end when humanity itself ends. All that to say, research is far from over. A doctoral research project, however, has a concrete beginning and end; I now stand at its end. Still, it is best to return to where it all began.

I was a junior lecturer at ITB when I first (e-)met Michel, who would later become my doctoral promotor. My employer had invited him to give a guest lecture in the course I was teaching. We talked afterwards; I was in need of a doctoral degree to pursue a career in academia, and he had a couple of vacancies. We then worked together on a small research project, which served as an opportunity for both of us to see whether we would be a good fit for working together. He then agreed to let me fill one of the university-funded vacancies. This gave me the wonderful opportunity not only to pursue a doctoral degree but also to experience life in a country very different from my homeland. For this, my first expression of gratitude is therefore extended to Michel, my academic father.

Speaking of a very different country, it was quite an experience to move from a tropical island of Java (30°C) to the Netherlands in the dead of winter (0°C), during a pandemic and a housing crisis, no less. I flew into the country alone, without any certainty about housing at the time. During this difficult

period, help came from fellow Indonesians in Eindhoven. I thank Majid and Binda, Pak Eko and his family, Mas Andy, and Willie, all of whom helped ensure I had a place to rest my head during my first month in the Netherlands.

A new country, and a new life, came with new friends. I am forever grateful to have called David, Nan, Mazyar, and Lina my friends, and to have been considered theirs in return. I am not the type of person who makes a lot of friends (which, I suppose, helps keep these acknowledgments short); I struggle to find the words to express how grateful I am to be blessed with friends who care so deeply about one another. I will miss the drinks, dinners, gossip, and little (mis)adventures we shared. Although I now live far away, I remain hopeful that I will see you again in the future.

Lina, I have to mention you twice because, beyond being such a good friend, you also became the co-promotor for my graduation! I still recall the moment we decided to invite you into this role. I realised early on that while Michel and I were both great at generating ideas, we were not always the best at seeing them through to completion. I suggested to him that our project would benefit from a daily supervisor who could keep me in line and prevent me from straying too far from the path—sort of like a mother. And so, my academic mother, you became! Your role in my graduation is immeasurable.

Never forgetting our homeland and, mainly, its delicious food while living in another country, I would like to thank my fellow countrymen and neighbours on Van der Meystraat: Arya and Delia and Arga and Zakiah, and other Indonesians in the Netherlands whom I cannot mention one by one—there were so many of us! We ended up helping each other find places to live, move, and take care of other everyday necessities that would be trivial back in Indonesia but were difficult to manage in a foreign land without relatives nearby. Summer barbecues in the “backyards” and noisy Monkey Town playdates are among the unforgettable moments from this chapter of our lives.

I thank the cool people in SET: Alex, Alex (that’s not a typo—we might have had too many Alexes), Mark, Loek, Ivan, Anton, Jacob, Gijs, Iman, Yanja, Katja, and Jelle. Fellow promovendi, past and present: Nathan, Mauricio,

Hossain, Lars, Leonard, Lavinia, Annie, Ema, Jan. Michel's academic family: Filip (I'm still saving for ClassViz island!), Felipe, Supun, Kevin, Merel—and our predecessors from other universities: Truong, Rodi, Dave, Arif, and our “eldest brother” Giovanni. From our neighbouring groups: Rick, Tom, Olav, Maurice, Anna, and Mark. And our superheroines Agnes and Natasja!

Sometimes friendship comes from unexpected encounters. I am grateful for the kindling of these kinships: Delina, although our collaboration did not really come to life, I thank you for always being there so enthusiastically. Santiago, coming from opposite sides of the world, does not mean we cannot love the same type of food; perhaps in the future, we can enjoy ramen together again! Matteo, you visited our group for a short time, but then you returned and became our neighbour; I am thankful for our small conversations over cups of coffee.

I thank everyone who thought I was Japanese. You gave me fuel for endless inside jokes!

A home for my thirst for movement—I thank the Eindhoven class of Capoeira Reversão: Mestre Macumba, professora Pomba, Brutus, Dragão, Antia, Trakinas, and others whom I cannot mention one by one. It is unfortunate that I had to stop training with you and move away, and speaking of the end of things . . .

Here we are, at the end of the chapter. Everything that has a beginning has an end, and at the end of this doctoral project, and of this acknowledgments section, I give the most thanks to my little family: my beloved wife, Yudea; our forever little angel, Stella[†]; and our sources of many laughs—and sometimes exhaustion, Samuel and Judika. I really could not have done this without you. My final weeks in the Netherlands without you were really, really miserable! I am glad we get to experience this life in Eindhoven, with all the joys and troubles in it. I enjoyed our little family trips, with just a dash of regret that we did not manage to visit all the places we wanted to see around Europe. However, with this end, I remain hopeful for our future new chapter. A new beginning awaits . . .

S.A.R., Bandung, April 2026

Contents

Acknowledgments	v
Abstract	xxvii
Samenvatting	xxix
Glossary	xxx
I Foundation	1
1 Introduction	3
1.1 Software Complexity and the Need for Explanations	3
1.2 Problem Statement and Research Questions	5
1.2.1 Q1 – Explanations in Practice	5
1.2.2 Q2 – Representation of Software Knowledge	6
1.2.3 Q3 – Enrichment of Structures	6
1.2.4 Q4 – Presentation through Visualisation	7
1.2.5 Q5 – Presentation through Natural Language	7
1.3 Map of the Dissertation	8
1.4 Contributions	14
2 Related Work and Concepts	17
2.1 Foundational Concepts and State-of-the-Art	17
2.1.1 Software Architecture and its Explanation	18
2.1.2 Program Comprehension	19
2.1.3 Software Architecture Recovery	20
2.1.4 Software (Architecture) Visualisation	20

2.1.5	Knowledge Representation for Software	21
2.1.6	AI in Software Engineering	22
2.2	Terminology	23
2.2.1	Foundations: Software Architecture, Recovery, and Explanation.	24
2.2.2	Representation: Graphs, Ontologies, and Architectural Abstractions	25
2.2.3	Enrichment: Deductive Reasoning, Summarisation, and LLM-based Techniques	26
2.2.4	Presentation: Visualisation and Interaction.	27
2.3	Situating This Dissertation	27
2.3.1	Software Architecture Representation and Recovery.	27
2.3.2	Explanation and Program Comprehension in Software Engineering	28
2.3.3	Software Visualisation and Interactive Exploration.	28
2.3.4	Natural Language Generation from Structured Software Artefacts.	28
3	An Explanation of Software Architecture Explanations	31
3.1	Introduction	32
3.2	Related Work.	34
3.3	Methodology.	36
3.3.1	Study Planning	38
3.3.2	Data Collection & Processing.	42
3.3.3	Data Analysis	43
3.4	Results	46
3.4.1	Explanation Topics	46
3.4.2	Explanation Plan	51
3.4.3	Supporting Artefacts	59
3.4.4	Typical Questions.	63
3.4.5	Expectations and Challenges.	66

3.5	Discussion	70
3.5.1	Explanation Through the Twin Peaks Lens.	70
3.5.2	The Explanation Window	72
3.5.3	Implications for Practice	75
3.6	Trustworthiness	76
3.6.1	Credibility	76
3.6.2	Analysability	78
3.6.3	Transparency	78
3.6.4	Usefulness	78
3.7	Conclusions	79

II Software System Representation 81

4 Unified Knowledge Graph Representation for Code and Architecture 83

4.1	Introduction	84
4.2	Requirements for A Software Knowledge Representation.	85
4.3	Related Work.	86
4.3.1	Software Metamodels and Code Representations	87
4.3.2	Architecture Recovery and Related Tools	88
4.3.3	Comparison Summary	89
4.4	Software Architecture/Analysis Basic Ontology (SABO)	89
4.4.1	SABO 1.0: Original Ontology	90
4.4.2	SABO 2.0.	92
4.5	Illustrative Graph Instance and Visualisation	97
4.5.1	JHotDraw example	97
4.5.2	Publish–subscribe example	100
4.6	Ontology Instantiation	104
4.6.1	Implementation Architecture.	104
4.6.2	Language Support Overview	105
4.6.3	Limitations and Challenges	106
4.7	Summary.	106

III	Enrichment	109
5	Evaluating DSAR Explanatory Artefacts	111
5.1	Introduction	112
5.2	DSAR and Related Work	114
5.2.1	Deductive Software Architecture Recovery	114
5.2.2	Related Work	116
5.3	Methodology.	118
5.3.1	ASML: An Evaluative Case Study	119
5.3.2	Study Planning	119
5.3.3	Data Collection and Processing	123
5.3.4	Data Analysis	125
5.4	Results	126
5.4.1	Themes	126
5.4.2	Properties	134
5.5	Discussions	141
5.5.1	Implications for Researchers.	141
5.6	Trustworthiness	142
5.7	Conclusion	143
IV	Explanation Presentation	147
6	Visualisation Prototypes for Architecture Explanation	149
6.1	Introduction	150
6.2	Related Work.	151
6.3	ClassViz: From Inspection Tool to Research Vessel	152
6.3.1	Design and Origin.	153
6.3.2	Adoption, Extension, and Appropriation.	156
6.3.3	Frontend for Architectural Analysis and Explanation	160
6.3.4	Demonstration: Visualising ClassViz in ClassViz.	161
6.3.5	Reflection	164
6.4	BubbleTea: A Layered Architecture Visualisation	165
6.4.1	Divergence from Related Work.	166

6.4.2	BubbleTea’s Approach	168
6.4.3	Illustration of Use.	172
6.4.4	Reflections	174
6.5	Threats to Validity	176
6.6	Conclusions	176
7	FLASE: Explanations Across Abstraction Levels	181
7.1	Introduction	182
7.2	Background and Related Work	184
7.2.1	Code Explanations Using LLMs.	184
7.2.2	LLM-based Agents	185
7.3	Software Explanations with FLASE	186
7.3.1	Knowledge Representation and Storage.	186
7.3.2	Context Retrieval and Explanation Generation	187
7.4	Experiment Methodology	191
7.4.1	Evaluation Codebase	191
7.4.2	Participant Recruitment	191
7.4.3	Procedure	193
7.4.4	Qualitative Analysis.	194
7.4.5	Technical Details	195
7.5	Results	195
7.5.1	Evaluation by Dimension (Deductive Coding)	195
7.5.2	Emergent Themes (Inductive Coding)	197
7.5.3	Performance by Agent	199
7.6	Discussion	202
7.6.1	Comparative Strengths and Weaknesses	202
7.6.2	Addressing the Research Questions	204
7.6.3	Design Implications for Automated Explanation Systems	205
7.7	Trustworthiness	206
7.8	Conclusion and future work.	208

V	Closing	211
8	Conclusions	213
8.1	Consolidated Answers to the Research Questions	214
8.1.1	Q1 – Explanations in Practice	214
8.1.2	Q2 – Representation of Software Knowledge	215
8.1.3	Q3 – Enrichment of Structures	215
8.1.4	Q4 – Presentation through Visualisation	216
8.1.5	Q5 – Presentation through Natural Language	217
8.2	Synthesis: A Framework for Automated Software Architecture Explanations	217
8.2.1	Pillar 1: Representation (Factual Software Structure)	218
8.2.2	Pillar 2: Enrichment (Architectural Interpretation)	219
8.2.3	Pillar 3: Presentation (User-Aligned Insights)	219
8.2.4	Overarching Principle: Context Adaptation	220
8.3	Implications	221
8.3.1	Implications for Research	221
8.3.2	Implications for Practice	222
8.3.3	Implications for Education	223
8.4	Limitations	224
8.4.1	Transferability of Empirical Validation	224
8.4.2	Technical and Methodological Constraints	224
8.4.3	Inherent Volatility of AI Models	225
8.5	Reflections	225
8.6	Outlook and Future Work	226
8.6.1	Integration of Dynamic and Behavioural Analysis	226
8.6.2	Multi-Modal Explanation Generation	227
8.6.3	Closing the Loop: From Explanation to Refactoring	227
8.6.4	Holistic Knowledge Graph Integration	227
8.6.5	Longitudinal and Comparative Field Studies	227

A	Mapping of SABO Node and Edge Labels to Language Constructs	229
A.1	Java	229
A.1.1	Nodes	229
A.1.2	Edges	230
A.2	C#.	231
A.2.1	Nodes	232
A.2.2	Edges	232
A.3	C/C++ via M3 Model	233
A.3.1	Nodes	234
A.3.2	Edges	234
A.4	JavaScript	235
A.4.1	Nodes	235
A.4.2	Edges	235
A.5	Go	238
A.5.1	Nodes	238
A.5.2	Edges	238
A.6	Simulink	239
A.6.1	Nodes	240
A.6.2	Edges	240
B	Interview Questions for DSAR Evaluation at ASML	243
B.1	Background Questions	243
B.2	Correctness Questions	244
B.3	Completeness Questions	244
B.4	Discriminative Consistency Questions	245
B.5	Compactness Questions.	246
B.6	Composition Questions	246
B.7	Task Relevance Questions.	247
B.8	Coherence Questions	247
B.9	Closing	248
C	FLASE User Study Documents	249
C.1	Background Form	249

C.2	Main Survey	250
C.3	Post-study Questionnaire	250
Bibliography		252

List of Figures

1.1	Structure and flow of the dissertation across four parts: Foundation, Representation, Enrichment, and Presentation. Arrows indicate how studies inform, guide, and feed back into one another. The icon and text beneath each box denote the technical output type: conceptual framework, ontology, method, or tool.	9
2.1	Activity diagrams of prompt response generation with (a) an LLM and (b) an LLM with RAG setup.	23
3.1	Interview study methodology.	38
3.2	The Twin Peaks model: Software architecture explanation requires explicit alignment of business and technical context, including tacit background (the context “halos”) on both the problem and solution domains.	70
3.3	The Explanation Window can be situated on and across different planes (project, component, module, function) representing natural abstraction levels, constraining how much of the system is visible at once.	74
3.4	The trade-off between depth and scope: The Explanation Window occupies a position on a curve defined by detail depth (x -axis) and functionality coverage (y -axis). The effective region corresponds to rectangles whose area remains within cognitive limits.	74

4.1	Detailed (a) and abstracted (b) versions of SABO 1.0 ontology. The detailed view encodes fine-grained code elements; the abstracted view aggregates them into a coarser, architecture-level graph used for higher-level reasoning.	91
4.2	Version 2.0 of SABO, including development elements (a), functional elements (b), and analysis aspects (c).	93
4.3	Example of <i>lifting</i> an edge from detailed-level “invokes” (between Operations) to abstracted-level “calls” (between Structures) in SABO 1.0.	96
4.4	A subset of the abstract knowledge graph instance extracted from JHotDraw, depicting classes involved in a strategy design pattern, presented in UML-like styling.	98
4.5	Some class-level elements (and their enclosing packages) from the JHotDraw project that are involved in a “strategy” design pattern instance. We can use this information to reason about the codebase’s maintainability, for instance.	101
4.6	Slices of a SABO graph instance, depicting (a) development structure and (b) functional structure of the pubsub demo. . .	102
4.7	Analysis (a) annotation and (b) visualisation of the same graph instance.	103
4.8	The SABO extraction pipeline parses source code, resolves names, constructs the graph using the syntax tree and name table, and outputs a serialised graph.	105
5.1	DSAR summarises and classifies code elements using structural context and architectural definitions. It integrates bottom-up code extraction with top-down architectural descriptions. . .	115
5.2	Cascading summarisation and classification in DSAR. Actions are applied bottom-up and left-to-right to respect dependency order.	115
5.3	Methodology of our evaluation study.	118

5.4	An example of the explanatory artefact accessible by participants, showing open source data, with box colours denoting architectural classification.	121
5.5	Implicit dependency: Class B may depend on Y as a specific implementation of X.	128
5.6	Participants' Likert responses for each evaluation property, showing the extent to which the explanatory artefacts were judged to exhibit the corresponding qualities.	135
6.1	Example visualisations using different techniques.	152
6.2	An early ClassViz version showing classes involved in an instance of the strategy design pattern in JHotDraw 5.1. All other classes are filtered out.	154
6.3	An early ClassViz version showing all packages and classes of JHotDraw 5.1 (that fit the viewport) and the "calls" relations among the classes.	155
6.4	He [86] overlays structure and behaviour information in one visualisation.	157
6.5	SABER-VR [60] puts structure and behaviour information in a VR environment.	157
6.6	Kloet [107] explores varying layout algorithms for different levels of hierarchy.	158
6.7	Atisomya [11] puts code elements (in this case, classes) in a grid of architectural layer $\times$ role stereotype, and numerical metric "number of methods" for the colour scale.	159
6.8	Arvisan [101] shows a narrow neighbourhood of components at a time.	160
6.9	A recent version of ClassViz showing classes from JHotDraw 5.1 classified into role stereotypes [222] and call relations lifted into package level.	162
6.10	ClassViz source code organisation as depicted by ClassViz itself. In this visualisation, line direction is represented from light to dark colour, e.g., from the graphProcessing folder to the utilities.	163

6.11	The K-9 Mail application v5.304 visualised in BubbleTea, with annotations on the visual elements.	173
7.1	Acquisition and storage of <i>a) Textual knowledge</i> and <i>b) Graph-based knowledge</i> about the software system in question.	186
7.2	The high-level process of generating an explanation.	188
7.3	The package-and-class structure of JPacman codebase.	192
8.1	A data flow illustration of the proposed framework for automated software architecture explanation.	219

List of Tables

3.1	Interview questions specific to explainers and explainees. . . .	41
3.2	Interview study participants. <i>Comp.</i> stands for <i>company</i> and <i>Proj.</i> for <i>project</i>	44
3.3	Number of study participants by employer’s annual revenue and organisation size.	45
3.4	Dimensions and themes revolving around RQ3.1.	47
3.5	Dimensions and themes revolving around RQ3.2.	52
3.6	Dimensions and themes revolving around RQ3.3.	59
3.7	Dimensions and themes revolving around RQ3.4.	63
3.8	Dimensions and themes revolving around RQ3.5.	66
4.1	Comparison of related approaches against our six design re- quirements (R1–R6).	89
4.2	Node labels in SABO 2.0 with descriptions.	94
4.3	Edge labels in SABO 2.0 with source, target, and definitions. .	95
4.4	Overview of language support and extraction tooling for SABO instantiation.	105
5.1	Overview of our study participants.	124
5.2	Mapping between inductive <i>themes</i> and evaluation <i>properties</i> . A bullet indicates that participants’ comments for a theme relate to the given property.	140
6.1	Illustration: the aggregation of layer indications in each class, calculated as the percentage of methods assigned to each layer.	169
6.2	Illustration: the aggregation of layer indications in each pack- age, calculated as the average percentage of methods assigned to each layer.	169

7.1	Types of questions supported by the system, with examples. .	189
7.2	Summary of retriever chains.	190
7.3	Participants in the user study.	193
7.4	Net likes by agent (Most-Liked – Least-Liked).	200
7.5	Overview of agent performance across question scope, question type and Co-3 dimension. Underlines mark the highest score for their corresponding question scope × question type × dimension.	201
A.1	Mapping of SABO 2.0 node labels in the V2 extractor to Java concepts.	230
A.2	Mapping of SABO 2.0 edge labels in the V2 extractor to Java concepts.	231
A.3	Mapping of SABO 2.0 node labels in the C# extractor to C# concepts.	232
A.4	Mapping of SABO 2.0 edge labels in the C# extractor to C# concepts.	233
A.5	Mapping of SABO 1.0 node labels in the M3GraphBuilder to M3 model concepts.	234
A.6	Mapping of SABO 1.0 edge labels in the M3GraphBuilder to M3 model concepts.	235
A.7	Mapping of SABO 2.0 node labels in the JavaScript extractor to JavaScript concepts.	236
A.8	Mapping of SABO 2.0 edge labels in the JavaScript extractor to JavaScript concepts.	237
A.9	Mapping of SABO 2.0 node labels in the Go extractor to Go concepts.	238
A.10	Mapping of SABO 2.0 edge labels in the Go extractor to Go concepts.	239
A.11	Mapping of SABO 1.0 node labels to Simulink concepts. . . .	240
A.12	Mapping of SABO 1.0 edge labels to Simulink concepts. . . .	241

Listings

- 2.1 Example of a Cypher query: find the names of all methods
invoked by a method named “saveUser”. 21
- 4.1 A Cypher query for detecting strategy pattern instances in a
SABO 1.0 graph. 99
- 6.1 Responsibility-oriented layer assignment for BubbleTea. . . . 170

*“Untuk Tuhan, Bangsa, dan Alma Mater,”
but also for my family: Yudea, Stella[†], Samuel, and Judika*

Summary

Many modern software systems are large, long-lived, and continuously evolving. As systems grow in size and complexity, understanding their architecture—how components are structured, how they interact, and why design decisions were made—becomes increasingly difficult. In practice, architectural knowledge is often incomplete, outdated, or scattered across informal artefacts and personal expertise, and is therefore frequently conveyed through ad-hoc explanations rather than systematic documentation. This dissertation examines how software architecture explanations can be supported, structured, and partially automated in a way that aligns with real-world practice.

The central contribution of this dissertation is a *framework for automated, context-aware software architecture explanation*. The work is structured around five research questions spanning explanation practice, knowledge representation, enrichment, and presentation. Rather than treating explanation as a purely linguistic or visual task, the dissertation conceptualises explanation as a pipeline that incrementally transforms raw code into user-aligned architectural insight.

The dissertation begins with an empirical study of practising software engineers, investigating how architecture explanations are produced and consumed in practice. The study shows that explanations are layered, goal-driven, and highly sensitive to audience and task. Explanations typically combine multiple artefacts and operate within bounded regions of scope and detail, motivating the need for explanation mechanisms that can adapt content and abstraction levels to situational needs.

Building on these findings, the dissertation introduces the *Software Architecture/Analysis Basic Ontology (SABO)*, a language-agnostic representation of software structure. SABO models software systems as labelled property

graphs that capture code elements and their relations while preserving traceability across abstraction levels. Designed explicitly to support explanation, SABO provides a unified substrate for subsequent architectural analysis and interpretation.

To derive architectural meaning from structure, the dissertation proposes *Deductive Software Architecture Recovery (DSAR)*, an enrichment approach that combines bottom-up code extraction with top-down architectural descriptions expressed in natural language. DSAR maps structural elements to architectural concepts such as layers and responsibilities, producing representations that better align with how practitioners reason about systems.

The dissertation further investigates how enriched architectural knowledge can be presented to human explainees. Interactive visualisation tools support exploration and abstraction over architectural structures, while natural-language explanations generated using *Retrieval-Augmented Generation (RAG)* ground fluent explanations in SABO-based knowledge graphs. Together, these approaches demonstrate how architectural knowledge can be translated into usable explanations across visual and textual modalities.

In synthesis, the dissertation integrates these contributions into a framework structured around three pillars: *representation* of factual software structure, *enrichment* through architectural interpretation, and *presentation* of user-aligned explanations, coordinated by the principle of *context adaptation*. Rather than replacing human architectural reasoning, the framework aims to reduce friction in recurring explanation tasks such as onboarding and maintenance, and to support sustained understanding of complex software systems.

Samenvatting

Veel moderne softwaresystemen zijn groot, langlevend en continu in ontwikkeling. Naarmate systemen groeien in omvang en complexiteit, wordt het steeds moeilijker om hun architectuur te begrijpen—hoe componenten zijn gestructureerd, hoe zij met elkaar interageren, en waarom bepaalde ontwerpbeslissingen zijn genomen. In de praktijk is architecturale kennis vaak onvolledig, verouderd of verspreid over informele artefacten en persoonlijke expertise, en wordt zij daarom veelal overgebracht via ad-hoc uitleg in plaats van via systematische documentatie. Dit proefschrift onderzoekt hoe uitleg over softwarearchitectuur kan worden ondersteund, gestructureerd en gedeeltelijk geautomatiseerd op een wijze die aansluit bij de praktijk.

De centrale bijdrage van dit proefschrift is een *framework voor geautomatiseerde, contextbewuste uitleg van softwarearchitectuur*. Het werk is gestructureerd rond vijf onderzoeksvragen die gezamenlijk uitlegpraktijk, kennisrepresentatie, verrijking en presentatie bestrijken. In plaats van uitleg te benaderen als een louter linguïstische of visuele taak, conceptualiseert dit proefschrift uitleg als een pijplijn die ruwe broncode stapsgewijs transformeert tot architecturaal inzicht dat is afgestemd op de gebruiker.

Het proefschrift start met een empirische studie onder professionele softwareontwikkelaars, waarin wordt onderzocht hoe architectuuruitleg in de praktijk wordt geproduceerd en geconsumeerd. Deze studie laat zien dat uitleg gelaagd is, doelgericht plaatsvindt en sterk afhankelijk is van publiek en taak. Uitleg combineert doorgaans meerdere artefacten en opereert binnen begrensde regio's van omvang en detail, wat de noodzaak onderstreept van uitlegmechanismen die inhoud en abstractieniveau kunnen aanpassen aan de situatie.

Op basis van deze bevindingen introduceert het proefschrift de *Software Architecture/Analysis Basic Ontology (SABO)*, een taalagnostische representa-

tie van softwarestructuren. SABO modelleert software als gelabelde property graphs die code-elementen en hun relaties vastleggen, terwijl traceerbaarheid over abstractieniveaus behouden blijft. SABO is expliciet ontworpen ter ondersteuning van uitleg en vormt een uniforme basis voor verdere architecturale analyse en interpretatie.

Om architecturale betekenis uit structurele gegevens af te leiden, introduceert het proefschrift *Deductive Software Architecture Recovery (DSAR)*, een verrijkingsaanpak die bottom-up code-extractie combineert met top-down architecturale beschrijvingen in natuurlijke taal. DSAR koppelt structurele elementen aan architecturale concepten zoals lagen en verantwoordelijkheden, en levert zo representaties op die beter aansluiten bij de manier waarop ontwikkelaars over systemen redeneren.

Daarnaast onderzoekt het proefschrift hoe verrijkte architecturale kennis kan worden gepresenteerd aan menselijke gebruikers. Interactieve visualisatietools ondersteunen exploratie en abstractie van architecturale structuren, terwijl uitleg in natuurlijke taal, gegenereerd met behulp van *Retrieval-Augmented Generation (RAG)*, vloeiende uitleg verankert in SABO-gebaseerde kennisgrafieën. Samen laten deze benaderingen zien hoe architecturale kennis kan worden vertaald naar bruikbare uitleg in een combinatie van visuele en tekstuele vorm.

In synthese integreert dit proefschrift deze bijdragen in een framework dat is opgebouwd rond drie pijlers: *representatie* van feitelijke softwarestructuur, *verrijking* door architecturale interpretatie, en *presentatie* van gebruikersgerichte uitleg, gecoördineerd door het principe van *contextadaptatie*. In plaats van menselijke architecturale redenering te vervangen, beoogt het framework frictie te verminderen bij terugkerende uitlegactiviteiten zoals onboarding en onderhoud, en duurzame ondersteuning te bieden bij het begrijpen van complexe softwaresystemen.

Glossary

ADD	Architectural Design Decision
ADR	Architecture Decision Record
AI	Artificial Intelligence
ASCS	Automated Source Code Summarisation
DSAR	Deductive Software Architecture Recovery
LLM	Large Language Model
LPG	Labelled Property Graph
RAG	Retrieval-Augmented Generation
RDF	Resource Description Framework
SABO	Software Architecture/Analysis Basic Ontology
SAR	Software Architecture Recovery
XAI	Explainable Artificial Intelligence

Part I

Foundation

1

Introduction

1.1 Software Complexity and the Need for Explanations

Many modern software systems are vast, distributed, and constantly evolving. Their increasing complexity makes it difficult to understand how systems are structured, how they behave, and why they were designed in a particular way [22, 216]. *Software architecture* is meant to provide these perspectives. It captures the system's principal components and their relations, allows reasoning about qualities such as scalability or reliability, and serves as a shared frame of reference among diverse stakeholders [16, 119]. Architecture plays this role not only in design but throughout software maintenance, which consumes the majority of the lifecycle cost and effort [45, 152].

However, architectural knowledge is rarely captured in a way that remains consistent, accessible, and useful over time. Architecture documentation and implementation often drift apart [123]. Wan et al. [216] show that models and tools do not ensure complete documentation; artefacts quickly become obsolete as systems evolve; and tool support for sharing, version control, and tracing across scattered design documents is still inadequate. As a result, architectural knowledge is frequently fragmented, outdated, or missing [2, 59, 214]. This erosion undermines maintenance, forcing engineers to rely on tacit knowledge and ad-hoc explanations when evolving or fixing systems [123, 193]. Existing empirical work highlights “walking architectures,” where a few

1

experts informally maintain and transmit knowledge through explanations rather than systematic artefacts [32, 44, 215]. This reliance on informal channels often stems from fundamental issues in architectural knowledge management, with information scattered and developers reluctant to engage in structured documentation [162].

Research in software engineering includes several strands that address architectural explanation from different angles. The software visualisation community studies how interactive diagrams support comprehension [35], though many approaches struggle with scalability and semantic clarity [47, 133]. Software architecture recovery develops techniques to reconstruct architectural views from code [75, 157], while program summarisation and comprehension investigate how to explain or navigate code [230]. These efforts are valuable but fragmented: explanations focus on design decisions, structures, or code in isolation, with little integration into a systematic approach to architectural explanation.

Robillard et al. [165] argue that static documentation is insufficient, proposing an *on-demand developer documentation* paradigm that dynamically surfaces information tailored to developer needs. Building on this, Razavian et al. [162] extend the vision specifically for architects, proposing an on-demand architectural knowledge system as a decision-making companion, and confirming the need for systematic support beyond static artefacts.

At the same time, advances in Artificial Intelligence (AI) opened new possibilities. When this doctoral project began, automated explanation was largely limited to program comprehension and documentation generation. The public release of ChatGPT in late 2022 [28, 153], early in our project, shifted the landscape. Suddenly, Large Language Models (LLMs) demonstrated the ability to produce fluent explanations from code at unprecedented scale. This led to investigations of how such models could be harnessed to generate architectural-level explanations atop structured knowledge. These developments directly influenced the trajectory of this dissertation and motivated integrating LLM-based techniques with existing structured graph-based representations and visualisation methods.

This combination sets the stage for the remainder of the dissertation. In addition to the empirical study on architecture explanations, we examine: how software knowledge can be represented and instantiated, how it can be enriched with higher-level abstractions, and how it can be presented through both visual and natural-language interfaces. Together, these efforts contribute towards a comprehensive framework for automated, context-aware software architecture explanations.

1.2 Problem Statement and Research Questions

In light of the challenges presented in Section 1.1, this dissertation tackles a central problem: the need for systematic, automated support for software architecture explanations that align with evolving implementations and are applicable to various maintenance tasks. This problem is explored through five main questions, supported by specific Research Questions (RQs) and methodological approaches.

1.2.1 Q1 – Explanations in Practice

How do practitioners explain software architecture in real-world settings, and what challenges and expectations shape these explanations?

Although software architecture explanations are central to maintenance, little is known about how practitioners actually construct and use them. Thus, this question is broken down into the following research questions:

- RQ3.1 What key topics are addressed in software architecture explanations?
- RQ3.2 What does a structured plan for explaining software architecture entail?
- RQ3.3 What supporting artefacts are used during software architecture explanations?
- RQ3.4 What are the typical questions asked during software architecture explanations?
- RQ3.5 What are expectations and challenges during and after software

architecture explanations?

These questions are empirically addressed in Chapter 3. We draw on a qualitative study of practising software engineers, using semi-structured interviews and thematic analysis to uncover patterns in how architecture explanations are produced and consumed.

1.2.2 Q2 – Representation of Software Knowledge

What kind of software knowledge representation can serve as a unified, language-agnostic foundation for automated architecture explanations?

Automated explanations require a structured substrate that unifies code-level detail with architectural abstractions. While prior work on architecture description and recovery has produced notations and clustering techniques, these do not provide a sufficiently extensible and semantically clear foundation for explanation.

We approach this question in Chapter 4 through an iterative process of specifying an ontology for software architectural knowledge, instantiating it across multiple languages, and refining it against real systems to ensure generality and expressiveness.

1.2.3 Q3 – Enrichment of Structures

How can recovered architectural structures be systematically enriched with semantic and explanatory information to support human understanding?

Recovered architectural structures are difficult to interpret in isolation. To make such representations practically useful, we explore enrichment mechanisms that add explanatory semantics, such as architectural classifications, layered abstractions, and textual summaries.

This question investigates enrichment as a transformation stage: from recovered graphs to *explanatory artefacts* that support understanding. Concretely, this includes our work on the DSAR approach, where recovered

structures are annotated, aggregated, and summarised. The impact of such enriched artefacts is evaluated through the following research questions:

- RQ5.1 What is the perceived content quality of the DSAR explanatory artefacts?
- RQ5.2 What is the perceived presentation quality of the DSAR explanatory artefacts?
- RQ5.3 What is the perceived practical usefulness of the DSAR explanatory artefacts?

To address this, we evaluate the explanatory artefacts with industry participants as described in Chapter 5. We combine structured criteria with open-ended feedback to examine how enrichment affects perceived usefulness and comprehensibility.

1.2.4 Q4 – Presentation through Visualisation

How can architectural explanations be effectively presented through interactive visualisations that are explorable, comprehensible, and actionable for practitioners?

Architectural knowledge needs representation media that allow users to explore systems interactively rather than through static diagrams. Building on the unified knowledge graph and enriched structures, this question investigates how visualisation can serve as a medium for architectural explanation.

This question is explored through the design and prototyping of interactive visualisation tools and their exploratory evaluation with students and practitioners, described in Chapter 6. We gained insights into effective layouts, filtering mechanisms, and abstraction strategies.

1.2.5 Q5 – Presentation through Natural Language

How can large language models be leveraged, in combination with structured knowledge, to generate architectural explanations that are correct, complete, and comprehensible?

1

Recent advances in LLMs enable fluent explanations of source code. However, their application to architectural-level reasoning is uncertain, particularly regarding the grounding of responses in structured system knowledge. To ensure that generated explanations are not only fluent but also correct, complete, and comprehensible, models must be guided by relevant architectural context.

Retrieval-Augmented Generation (RAG) offers a promising approach by injecting structured knowledge into the LLM's prompt. This allows explanations to remain anchored to real system structures and terminology. We therefore ask:

- RQ7.1 How do different Retrieval-Augmented Generation strategies compare to a commercial baseline tool in terms of developer-perceived *correctness*, *completeness*, and *comprehensibility* dimensions?
- RQ7.2 How do comparative performances vary across *question scope* (component vs. module) and *question type* (functionality, pairwise interaction, other interaction)?
- RQ7.3 Beyond the three dimensions, what emergent themes do developers identify as critical to explanation quality, and how might these insights guide future RAG-based explanation systems?

In Chapter 7, we investigate these questions through a controlled evaluation of RAG pipelines. We compare different strategies with a commercial baseline and analyse both quantitative performance and qualitative perceptions of explanation quality.

1.3 Map of the Dissertation

This dissertation is organised into eight chapters, each focusing on a distinct aspect of the overarching research problem. Chapter 2 presents the foundational work that underpins the contributions of this dissertation. Chapters 3 to 7 are based on a mixture of published and extended research articles as well as student projects, which are combined into a unified narrative. Figure 1.1 illustrates the overall structure and progression of the dissertation.

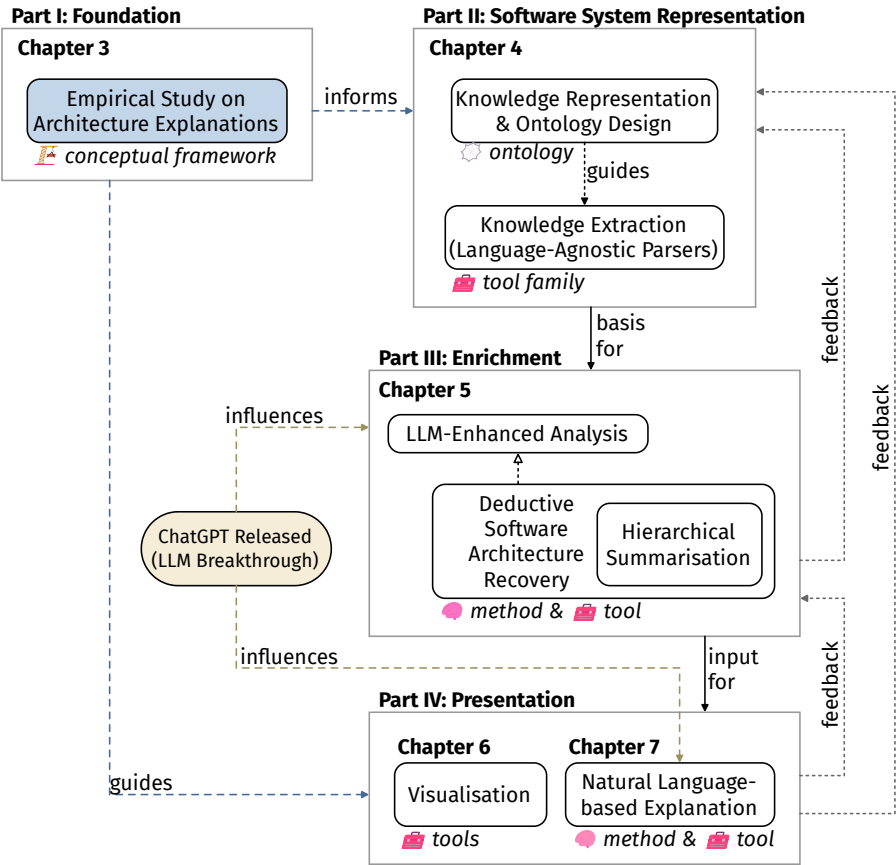


Figure 1.1: Structure and flow of the dissertation across four parts: Foundation, Representation, Enrichment, and Presentation. Arrows indicate how studies inform, guide, and feed back into one another. The icon and text beneath each box denote the technical output type: conceptual framework, ontology, method, or tool.

Chapter 2: concepts in software architecture recovery, explanation, program comprehension, and visualisation, thereby situating this dissertation within the existing body of work. The chapter also defines the terminology that is used throughout the dissertation.

Chapter 3: Provides the first systematic study of how practitioners construct and consume architecture explanations, uncovering their multi-layered nature and their reliance on rich artefacts while being tailored to different stakeholders.

This chapter builds on the following publication:

- [175] Satrio Adi Rukmono, Filip Zamfirov, Lina Ochoa, Floris Pex, and Michel R.V. Chaudron. An Explanation of Software Architecture Explanations. *Empirical Software Engineering*, 31(3):59, 2026. doi:10.1007/s10664-025-10782-3

The author is the primary contributor to this work.

Chapter 4: Presents Software Architecture/Analysis Basic Ontology (SABO), a labelled property graph model that integrates both code-level elements and architectural abstractions. The chapter further outlines tool implementations capable of extracting graph instances from a variety of programming languages.

It extends and refines the following prior work:

- [169] Satrio Adi Rukmono and Michel R.V. Chaudron. Enabling Analysis and Reasoning on Software Systems through Knowledge Graph Representation. In *20th International Conference on Mining Software Repositories*, pages 120–124. IEEE, 2023. doi:10.1109/MSR59073.2023.00029

The author is the primary contributor to this work.

Chapter 5: Introduces the DSAR approach, which transforms code-level graphs into higher-level architectural views. The chapter presents an industrial case study conducted at ASML that assesses DSAR as a support for software comprehension.

This chapter is based on the following paper (to appear in the Industry Track of The ACM International Conference on the Foundations of Software Engineering (FSE) 2026):

- [174] Satrio Adi Rukmono, Lina Ochoa, Thijs M.J. Bressers, Jacob Krüger, and Michel R.V. Chaudron. Evaluating Explanatory Artefacts of DSAR-Recovered Architectures from Industrial Codebases, 2026. Accepted at FSE-Industry 2026, to appear

The work examines the implementation of the following techniques:

- [170] Satrio Adi Rukmono, Lina Ochoa, and Michel R.V. Chaudron. Achieving High-Level Software Component Summarization via Hierarchical Chain-of-Thought Prompting and Static Code Analysis. In *International Conference on Data and Software Engineering*, pages 7–12. IEEE, 2023. doi:10.1109/ICoDSE59534.2023.10292037
- [172] Satrio Adi Rukmono, Lina Ochoa, and Michel R.V. Chaudron. Deductive Software Architecture Recovery via Chain-of-Thought Prompting. In *44th International Conference on Software Engineering: New Ideas and Emerging Results*, pages 92–96, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400705007. doi:10.1145/3639476.3639776

The author is the primary contributor to these publications.

Chapter 6: Presents interactive visualisation prototypes designed to support explanation, including the *ClassViz* prototype for class-level exploration and *BubbleTea* for multi-layer architectural views. Together, these prototypes illustrate how structured representations can be turned into practical explanatory interfaces.

The chapter is organised into two major sections. The first, on *ClassViz*, is derived from:

- [168] Satrio Adi Rukmono. *ClassViz: From Inspection Tool to Research Vessel*. In *The 24th Belgium-Netherlands Software Evolution Workshop (BENEVOL 2025)*, 2025

The second, on BubbleTea, is derived from part of the B.Sc. final project of ITB students Christopher Jeffrey and Aditya Nugroho, for which the author served as co-supervisor:

- [171] Satrio Adi Rukmono, Michel R.V. Chaudron, and Christopher Jeffrey. Layered BubbleTea Software Architecture Visualisation. In *Working Conference on Software Visualization*, pages 122–126. IEEE, 2024. doi:10.1109/VISSOFT64034.2024.00024

In addition, this chapter cites other student visualisation projects for which the author contributed as a research collaborator and co-supervisor:

- [101] Roy Kakkenberg, Satrio Adi Rukmono, Michel R.V. Chaudron, Wim Gerholt, Miguel Pinto, and Claudio R. de Oliveira. Arvisan: An Interactive Tool for Visualisation and Analysis of Low-Code Architecture Landscapes. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, pages 848–855, 2024. doi:10.1145/3652620.3688331
- [98] Christopher Jeffrey, Aditya P. Nugroho, Satrio Adi Rukmono, and Yani Widayani. CodeView: A Tool for Software Visualization in Development View. In *2024 IEEE International Conference on Data and Software Engineering (ICoDSE)*, pages 67–72. IEEE, 2024. doi:10.1109/icodse63307.2024.10829915

Chapter 7: examines how an LLM can be integrated with graph-structured knowledge to produce explanations, showcasing the promise of hybrid methods that merge structured retrieval with the flexibility of natural language generation.

This chapter is based on:

- [154] Floris M.J. Pex. Exploring Software Explanation using Retrieval Augmented Generation. Master’s thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, August 2024

The author served as co-supervisor and tutor for Floris Pex's M.Sc. thesis project and conducted an independent analysis of the raw data for incorporation into this dissertation.

Other Publications During the PhD

In addition to the work forming the core of this dissertation, several other publications were produced that are not directly related to the content of this dissertation:

- [90] Truong Ho-Quang, Arif Nurwidyantoro, Satrio Adi Rukmono, Michel R.V. Chaudron, Fabian Fröding, and Duy Nguyen Ngoc. Role Stereotypes in Software Designs and Their Evolution. *Journal of Systems and Software*, 189:111296, 2022. doi:10.1016/j.jss.2022.111296
- [18] Gustav Bergström, Fadhl Hujainah, Truong Ho-Quang, Rodi Jolak, Satrio Adi Rukmono, Arif Nurwidyantoro, and Michel R.V. Chaudron. Evaluating the Layout Quality of UML Class Diagrams Using Machine Learning. *Journal of Systems and Software*, 192:111413, 2022. doi:10.1016/j.jss.2022.111413
- [41] Bavo Coremans, Arjen L. Klomp, Satrio Adi Rukmono, Jacob Krüger, Dirk Fahland, and Michel R.V. Chaudron. Process Mining from Jira Issues at a Large Company. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 425–435. IEEE, 2023. doi:10.1109/ICSME58846.2023.00055
- [124] Grisca Liebel, Jil Klünder, Regina Hebig, Christopher Lazik, Inês Nunes, Isabella Graßl, Jan-Philipp Steghöfer, Joeri Exelmans, Julian Oertel, Kai Marquardt, Katharina Juhnke, Kurt Schneider, Lucas Gren, Lucia Happe, Marc Herrmann, Marvin Wyrich, Matthias Tichy, Miguel Goulão, Rebekka Wohlrab, Reyhaneh Kalantari, Robert Heinrich, Sandra Greiner, Satrio Adi Rukmono, Shalini Chakraborty, Silvia Abrahão, and Vasco Amaral. Human Factors in Model-driven Engineering: Future Research Goals and Initiatives for MDE. *Software*

and *Systems Modeling*, 23(4):801–819, 2024. doi:10.1007/s10270-024-01188-8

- [102] Reyhaneh Kalantari, Julian Oertel, Joeri Exelmans, Satrio Adi Rukmono, Vasco Amaral, Matthias Tichy, Katharina Juhnke, Jan-Philipp Steghöfer, and Silvia Abrahão. Systematizing Modeler Experience (MX) in Model-Driven Engineering Success Stories. *Software and Systems Modeling*, 23(4):821–832, 2024. doi:10.1007/s10270-024-01194-w

1.4 Contributions

This dissertation contributes to the field of software engineering by establishing the foundations of a framework for automated, context-aware software architecture explanations. Beyond the individual studies and artefacts presented in the later chapters, the dissertation advances the state of the art through four integrative contributions:

1. **Empirical grounding of architecture explanations.** The dissertation provides the first systematic investigation of how practitioners construct and consume architecture explanations, revealing their layered, artefact-rich, and stakeholder-sensitive nature.
2. **A unifying representation of software knowledge.** SABO is introduced as a language-agnostic model that connects code-level elements with architectural abstractions, offering a substrate for automation and cross-language applicability.
3. **Methods for enriching and presenting explanations.** Building on SABO as a foundation, the dissertation explores ways of enriching and presenting architectural knowledge. DSAR is evaluated as one example of enrichment, but the framework also accommodates measurements, clustering, and classifications that can be captured in SABO and presented through visualisation or natural language. Interactive prototypes further demonstrate how such enriched artefacts can support comprehension and explanation.

4. **A comprehensive and extensible framework for automated explanations.** The dissertation synthesises these strands into a framework that integrates knowledge representation, static code analysis, and AI-driven natural language generation to deliver context-adaptive architecture explanations for maintenance tasks. While current implementations and evaluations focus on source code, the framework is designed to be extensible: it can incorporate additional sources such as documentation, issue-tracking systems, and dynamic analyses, enabling richer and more complete explanations in the future.

Taken together, these contributions address long-standing shortcomings of static documentation and fragmented architecture tools, elevating explanation to a first-class concern in software maintenance.

They draw on ideas from several research areas, including software architecture, program comprehension, knowledge representation, visualisation, and AI-assisted code analysis. Each has largely developed on its own, focusing on a particular slice of the broader explanation challenge. To understand how they might be combined into a unified, automated explanation framework, we must first examine their underlying principles and terminology.

2

Related Work and Concepts

The contributions outlined in Chapter 1 draw on ideas from several research areas that have largely developed independently, using different terminologies, abstractions, and assumptions. This fragmentation makes it difficult to reason about software architecture explanations as a unified problem and obscures how existing approaches relate to one another.

This chapter thus consolidates key concepts, terminologies, and strands of prior work that form the intellectual backdrop of this dissertation. While individual chapters (Chapters 3 to 7) introduce related work specific to their respective contributions, this chapter provides a structured overview of the broader research context. It serves three complementary purposes: to define essential terms used throughout the dissertation, to survey relevant state-of-the-art developments across adjacent domains, and to situate this work within the wider landscape of software engineering research.

2.1 Foundational Concepts and State-of-the-Art

This dissertation builds upon and contributes to several interconnected research areas. This section reviews the foundational concepts and state-of-the-art literature in software architecture and its explanation, program comprehension, software architecture recovery, visualisation, knowledge representation, and the application of artificial intelligence in software engin-

eering.

2

2.1.1 Software Architecture and its Explanation

Software architecture provides the high-level structure of a system, encompassing its principal components, their relationships, and the rationale behind design decisions [16, 119]. It serves as a crucial frame of reference for stakeholders throughout the software lifecycle, especially during maintenance, which consumes the majority of cost and effort [45, 152].

A central challenge is the effective transfer of this architectural knowledge, for example, in an onboarding or “passing the torch” situation. In practice, architectural documentation is often incomplete, ambiguous, obsolete, or fragmented [2, 59, 214], compromising maintenance and forcing engineers to rely on informal mechanisms. This has led to the concept of the “*walking architecture*,” where knowledge is maintained and shared informally by a few experts rather than through systematic artifacts [215]. These informal interactions are often the primary source of architectural knowledge for newcomers [32, 44, 193]. To address the limitations of static documentation, the vision of *on-demand developer documentation* has been proposed, advocating for systems that dynamically discover and summarise information tailored to a developer’s needs [165]. Razavian et al. [162] expanded on this by envisioning an On-Demand Architectural Knowledge System (ODAKS) that not only provides information but also supports the human factors of architectural decision-making through assistive conversation. This aligns with the aforementioned informal practice of architectural knowledge transfer.

Several models and principles guide the description and understanding of architecture. The *4+1 View Model of Architecture* provides a framework for organising an architectural description into a set of concurrent views (e.g., logical, process, development) to address the concerns of different stakeholders [112]. The *Twin Peaks Model* describes development as the co-evolution of requirements (the problem domain) and architecture (the solution domain), suggesting that effective explanations must bridge these two areas [150, 218]. Common architectural patterns, such as the *layered architecture* pattern (decomposing a system into roughly Presentation, Service,

Domain, and Data Source layers) [69, 222], provide established structures for design. Designs are further guided by object-oriented design principles such as *SOLID*, a widely adopted acronym encompassing five key guidelines for maintainable and extensible software systems [85, 177]. These include the *Single-Responsibility Principle*—each class should have one reason to change [92]—and other principles that support modularity, encapsulation, and reuse. Complementary to these principles, the characterisation of classes using *role stereotypes* (e.g., ‘Controller’, ‘Information Holder’) provides a lens for understanding class responsibilities [222, 223]. These models and principles are meant to guide design; however, they also aid explanation by offering criteria for evaluating architectural quality and by providing conceptual anchors that make the system’s organisation and responsibilities more intelligible.

2.1.2 Program Comprehension

Program comprehension is a prerequisite for any software maintenance or evolution task. Research in this area distinguishes between the immediate, task-specific needs of programmers and the deeper, conceptual understanding required for architecture comprehension. While developers often ask targeted questions about syntax or specific functions to solve an immediate problem [115, 161, 189], architectural comprehension involves building a holistic mental model of the system’s design, rationale, and component interrelationships.

Cognitive theories describe several models for how this comprehension occurs [27]: (1) *Top-down*: Understanding begins with high-level abstractions and drills down into the code as needed. (2) *Bottom-up*: Understanding is built by inspecting low-level code details first to construct higher-level conceptual models. (3) *Opportunistic*: A hybrid model where a developer switches between top-down and bottom-up strategies based on the immediate task or available information.

2.1.3 Software Architecture Recovery

When architectural documentation is missing or unreliable, *Software Architecture Recovery (SAR)* provides a means to reconstruct the architecture of a system from its source code [48, 127, 207]. Most state-of-the-art SAR techniques lack generalisability, often being tied to a specific technology or architectural style [130, 207].

The predominant approach is what we call *Inductive SAR*, which reconstructs architectural views by clustering code-level facts, such as dependencies, structural elements, or textual artefacts, into higher-level components [17, 19, 56, 75, 176, 225]. Recent SAR methods have explored learning-based techniques that map code to architectural roles using embeddings [99], as well as specialised heuristics for specific contexts such as microservices [49].

2.1.4 Software (Architecture) Visualisation

Software visualisation supports the comprehension and explanation of a system's structure, behaviour, and evolution, particularly in complex systems [35, 47]. A systematic survey by Shahin et al. [186] identifies four main software architecture visualisation techniques.

First, graph-based visualisations dominate in software architecture recovery and system evolution tasks, due to their ability to represent relationships and dependencies at various levels of granularity. Second, notation-based techniques rely on established visual languages such as UML to convey structural and behavioural aspects in a standardised manner. Third, matrix-based approaches offer a compact and precise means of displaying dependencies, making them suitable for analysing system modularity and violations. Lastly, metaphor-based visualisations such as the “software city” metaphor [142] aim to provide intuitive and engaging overviews of large systems, although they can suffer from scalability limitations in practice [186].

Despite the breadth of techniques proposed in the literature, industrial adoption remains limited [35]. To be truly effective, architectural visualisations must serve a dual role: they must not only support structural viewpoints (i.e.,

components and their interconnections) but also decisional viewpoints, that is, the rationale underlying architectural choices [186].

2.1.5 Knowledge Representation for Software

A foundational challenge in automated analysis is bridging the semantic gap between low-level code (e.g., abstract syntax trees) and high-level architectural abstractions. This requires a unified and expressive *software knowledge representation*. The *knowledge graph* approach has emerged as a promising solution, enabling the integration of heterogeneous data and supporting complex queries [34, 55, 91].

This dissertation specifically adopts a *Labelled Property Graph (LPG)* model [7], which extends traditional graph structures by allowing both nodes and edges to carry multiple labels and arbitrary key-value properties. LPGs are commonly queried using declarative pattern-matching languages such as Cypher [70].

As an illustrative example, consider an LPG in which nodes labelled `:Method` have a string property `name`, and edges labelled `:invokes` represent method-to-method calls. A Cypher query that retrieves all methods invoked by a method named “saveUser” is shown in Listing 2.1.

Listing 2.1: Example of a Cypher query: find the names of all methods invoked by a method named “saveUser”.

```
1 MATCH (m:Method {name: "saveUser"})-[:invokes]->(callee:Method)
2 RETURN callee.name
```

Several metamodels and ontologies have been proposed to structure software knowledge, for example:

SEON (Software Engineering Ontology Network): A domain-agnostic high-level ontology for modelling software artefacts and processes, but lacks source-level granularity [24].

ISO/IEC/IEEE 42010:2022: An abstract standard for formalising the description of architectures but does not provide concrete representation mechanisms for the code [94].

FAMIX: A language-independent metamodel for object-oriented source code, strong in representing code structure but lacking explicit architectural concepts [50].

Rascal M3: A metamodel that separates language-specific abstract syntax trees from a generic relational layer to support cross-language analysis [106].

CodeOntology: Transforms Java code into RDF triples, enabling semantic querying, but is limited to object-oriented Java features [12].

While each of these representations serves specific analytical needs, they fall short for architecture-oriented explanation: some lack architectural constructs, others do not integrate well across abstraction levels, and most are not readily extensible for hybrid analyses combining structure, semantics, and stakeholder concerns. Further analysis and examples of these metamodels, along with a detailed discussion of their limitations in the context of this dissertation's goals, are provided in Chapter 4.

2.1.6 AI in Software Engineering

Recent advances in Artificial Intelligence, particularly LLMs, are reshaping software engineering and architecture [61, 97]. LLMs are increasingly used for code understanding and explanation. However, research shows that they often struggle with step-by-step reasoning and can omit important details [109, 178, 191]. Coding assistants such as GitHub Copilot generally operate at the code snippet level and have limited ability to provide coherent system-level explanations [33, 167].

To overcome these limitations, *Retrieval-Augmented Generation (RAG)* has been proposed [122]. This technique enhances LLM outputs by first retrieving relevant information from an external knowledge base (such as a vector database or knowledge graph) and providing it as context (see Figure 2.1). The concept has been extended to *LLM-based agents*, autonomous systems that can reason, plan, and act to carry out complex tasks, such as answering user queries about software [217].

However, the use of LLMs for programming-related tasks is not without risk. Recent studies have shown that LLMs can generate incorrect or even

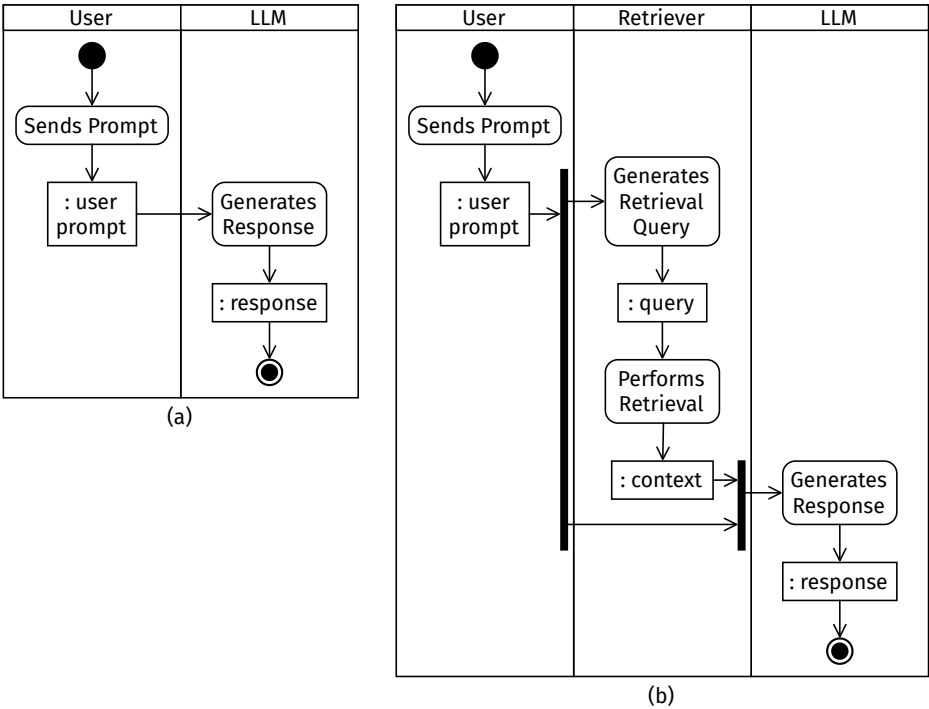


Figure 2.1: Activity diagrams of prompt response generation with (a) an LLM and (b) an LLM with RAG setup.

harmful outputs, such as insecure code or misleading explanations, particularly when used without oversight or domain-specific grounding [3]. These concerns underscore the importance of incorporating retrieval mechanisms and structured representations into the generation process.

2.2 Terminology

This section consolidates the key terminology used throughout the dissertation. To improve clarity, the terms are grouped according to the conceptual structure underpinning this work: *Foundations*, *Representation*, *Enrichment*, and *Presentation*.

Some terms originate from prior literature, while others are introduced or operationalised in this dissertation. To avoid blurring conceptual contributions with related work, each item explicitly notes whether it is (i) a term

introduced or adapted in this dissertation or (ii) a term *drawn from prior work*.

2

2.2.1 Foundations: Software Architecture, Recovery, and Explanation

Software Architecture The high-level structure of a system, its components, their relationships, and the principles guiding its evolution [16, 119]. (Prior work)

Architectural Knowledge The design decisions, structures, behaviours, and rationales that define a system's architecture [23]. (Prior work)

Walking Architecture The informal, socially distributed architectural knowledge maintained by key experts [215]. (Prior work)

4+1 View Model of Architecture A multi-view architectural description comprising Logical, Process, Physical, Development, and Scenario views [112]. (Prior work)

Twin Peaks Model A model describing the co-evolution of requirements and architecture [150, 218]. (Prior work)

Layered Architecture Pattern A design pattern structuring systems into layers with distinct responsibilities [69, 222]. (Prior work)

Software Architecture Recovery (SAR) The reconstruction of a system's architecture from code or artefacts when documentation is absent or outdated [75, 225]. (Prior work)

Inductive SAR A bottom-up family of SAR techniques that infer architecture from code-level similarities (structural, textual, or dependency-based). This dissertation uses the term to contrast with deductive SAR [172]. (Introduced in this dissertation; cf. Chapter 5)

Software Architecture Explanation The act of conveying a system's structures, behaviours, and rationale from a knowledgeable party to a less-informed audience [175]. (Introduced in this dissertation; cf. Chapter 3)

Explanation Window A conceptual framework for managing trade-offs between the breadth and depth of an explanation to maintain cognitive manageability [175]. (Introduced in this dissertation; cf. Chapter 3)

Cognitive Comprehension Models Theories describing how developers understand software using top-down, bottom-up, and opportunistic strategies [27]. (Prior work)

Judgment Study An empirical method where participants evaluate the utility of artefacts or techniques [201, 202]. (Prior work)

2.2.2 Representation: Graphs, Ontologies, and Architectural Abstractions

Knowledge Graph A structured representation of entities and their relationships [55]. (Prior work)

Labelled Property Graph (LPG) A knowledge-graph model where nodes and edges may contain key-value properties [7]. (Prior work)

Resource Description Framework (RDF) A graph data model that represents information as triples of subject-predicate-object, where properties are expressed as additional triples [91]. (Prior work)

Software Architecture/Analysis Basic Ontology (SABO) A graph-based ontology introduced in this dissertation that unifies code-level constructs with architectural abstractions. (Introduced in this dissertation; cf. Chapter 4)

Role Stereotypes A design-based classification scheme describing the responsibilities of classes (e.g. Controller, Information Holder) [222, 223]. (Prior work)

Single-Responsibility Principle A SOLID design principle stating that a class should have only one reason to change [92]. (Prior work)

SOLID Principles A set of foundational object-oriented design guidelines promoting modularity and maintainability [177]. (Prior work)

2.2.3 Enrichment: Deductive Reasoning, Summarisation, and LLM-based Techniques

2

Deductive SAR (DSAR) An approach introduced in this dissertation that interprets source code through a predefined reference architecture to infer architectural views [172]. (Introduced in this dissertation; cf. Chapter 5)

Explanatory Artefacts Textual or visual materials (or a combination of both) meant for conveying architectural structure, behaviour, or rationale [174]. (Introduced in this dissertation; cf. Chapter 5)

On-Demand Developer Documentation A paradigm advocating dynamically generated, task-specific documentation [165]. (Prior work)

Hierarchical Summarisation A technique (operationalised in this dissertation) that structures explanations across multiple abstraction levels. (Adapted/introduced in this dissertation; cf. Chapter 5)

Generative AI Techniques for generating novel content (e.g., text, images, code) using data-driven models [14, 65]. (Prior work)

LLM-based Agents Autonomous systems using large language models to reason or act [217]. (Prior work)

Retrieval-Augmented Generation (RAG) Enhancing LLM outputs by retrieving relevant context from external sources [122]. (Prior work)

Retriever Chain In the FLASE agent, a modular retrieval strategy selected based on question classification, specifying retrieval source and abstraction level. (Introduced in this dissertation; cf. Chapter 7)

Hybrid Retrieval A FLASE strategy combining vector-based and graph-based retrieval results to construct a richer context for explanation generation. (Introduced in this dissertation; cf. Chapter 7)

FLASE An LLM-based explanation agent that performs RAG over graph and vector stores to produce multi-level architectural explanations [154]. (From related student work; cf. Chapter 7)

2.2.4 Presentation: Visualisation and Interaction

ClassViz An interactive visualisation prototype for class-level structure exploration, originally built for verifying code-to-graph extraction [169]. (Introduced in this dissertation; cf. Chapter 6)

BubbleTea A prototype for visualising layered architectures using DSAR classifications [171]. (Introduced in this dissertation; cf. Chapter 6)

Co-12 Framework for XAI A multidimensional model for assessing explanation quality across content, presentation, and user-related aspects [145, 146]. (Prior work; adapted in this dissertation)

Relation Lifting A mechanism (operationalised in this dissertation) for elevating relations across abstraction levels—e.g., method-level dependency into class- or package-level. (Prior work)

2.3 Situating This Dissertation

This dissertation explores how software architectures can be represented, recovered, and explained in ways that support human understanding. While its contributions are laid out in Chapter 1, the broader intellectual space it inhabits spans several adjacent research domains. This section outlines four key “neighbourhoods” around the core of this work: research communities and traditions that inform, motivate, and complement the approaches taken in this dissertation.

2.3.1 Software Architecture Representation and Recovery

One foundational area is the study of how software architecture can be observed, described, or recovered from implementation artefacts. This includes models for expressing structural relationships and responsibilities, such as ISO/IEC/IEEE 42010-compliant viewpoints and model-based representations. The SABO graph (Chapter 4) draws from this tradition, aiming to offer a language-agnostic, extensible, and analysable graph representation of architectural constructs grounded in code. Similarly, the DSAR approach

(Chapter 5) addresses how architectural properties can be recovered and annotated through classification and summarisation processes.

2

2.3.2 Explanation and Program Comprehension in Software Engineering

A second neighbourhood concerns the problem of software explanation and how it relates to program comprehension. While explanation is sometimes implicit in tools and documentation, this dissertation treats it as a deliberate design goal. This framing aligns with recent calls to investigate how architectural knowledge can be surfaced to support understanding, decision-making, and maintenance. The Explanation Window concept introduced in Chapter 5 serves as a bridge between abstract comprehension models and practical artefact design, drawing on both empirical findings and stakeholder perspectives on what makes an explanation useful.

2.3.3 Software Visualisation and Interactive Exploration

Another surrounding area is the visualisation of software structure and behaviour. Numerous approaches aim to make software artefacts easier to inspect, from class diagrams to layered views and interactive graph-based tools. The visualisation efforts in this dissertation, ClassViz and BubbleTea (Chapter 6), are situated within this context. They focus on usability, readability, and semantic clarity, prioritising lightweight mechanisms for practitioners to inspect architectural responsibilities and layering without requiring deep interaction with raw source code or low-level metrics.

2.3.4 Natural Language Generation from Structured Software Artefacts

Finally, this dissertation draws from methods in natural language generation, especially when grounded in structured inputs such as source code or graphs. The FLASE agent (Chapter 7) exemplifies this connection, combining retrieval-augmented generation techniques with a knowledge graph-based context. This reflects a broader trend toward hybrid systems where LLMs are scaffolded by formal representations to ensure coherence and factual grounding in the generated explanations.

These four domains form the disciplinary context of this dissertation. Each offers tools, concepts, and questions that shape the research direction. However, they also reveal a key limitation: while substantial work exists on representation, recovery, visualisation, and generation, there is comparatively little systematic understanding of how software architecture explanations are actually constructed and used in practice.

As a result, current approaches often optimise individual facets of the problem, lacking a unified pathway that connects explanations to the comprehension requirements of practical contexts.

To address this gap, the dissertation begins not with a new representation or tool, but with an empirical investigation of architectural explanation as it occurs in practice.

3

3

An Explanation of Software Architecture Explanations

Software architecture knowledge transfer is essential for software development, yet documentation is often incomplete or ambiguous, making oral explanations a common means. Despite extensive work on architecture representation, recovery, visualisation, and generation, there is limited systematic understanding of how such explanations are constructed and used in practice. To inform better support and eventual automation of software architecture explanations, we first investigate how explanations are actually conducted in practice across five aspects: topics, plans, supporting artefacts, typical questions, and expectations and challenges. We report on semi-structured interviews with 17 software professionals from diverse organisations and countries. We find that explanations must balance problem and technical domains while accounting for the explainee's role, experience, and goals. Explainees frequently want not only structure and behaviour but also decision rationale. We propose the Explanation Window, a framework for focusing information by adjusting functionality scope and detail, and emphasise the need to include system context. These findings provide an empirical basis for improving architecture explanations and guiding future tool support and automation.

3.1 Introduction

Effective transfer of software architectural knowledge is critical and challenging in large-scale software development and maintenance [111, 147]. *Software architecture* refers to the high-level structuring of a system, including its components, their relationships, and the rationale behind design decisions [16, 119]. In practice, architectural knowledge is mainly transferred through documentation or informal explanations [165]. However, architectural documentation is often incomplete, ambiguous, obsolete, or fragmented [2, 59, 214], which harms onboarding and knowledge continuity [32, 59, 180, 199, 212]. At the same time, empirical evidence shows that informal, interactive explanations by architects or experienced developers are an effective alternative to transferring knowledge to colleagues and newcomers [32, 44, 215]. We refer to the process by which an architect or experienced developer conveys a system's structures, behaviours, and rationale to a less-informed audience as *software architecture explanation*.

Despite the prevalence of these software architecture explanations in practice, empirical research on the topic is scarce [13]. To our knowledge, the closest work is the vision of Robillard et al. [165], which advocates an on-demand developer documentation paradigm. Although it uses the term *documentation*, this vision shifts from traditional documentation tied to a snapshot of a system to discovering and summarising information tailored to developers' needs, much like experts explaining concepts to less-informed audiences. As a vision, no underlying research is provided; thus, we still lack a systematic understanding of how software architecture explanations are conducted, which topics and artefacts are involved, what questions are typically asked, and what recurring challenges and expectations IT professionals have. More recently, this vision has been refined into on-demand architectural knowledge systems that act as a "decision-making companion" for architects, explicitly addressing scattered information and cognitive biases [162].

Gaining further understanding on software architecture explanations will clarify what makes them effective and lay the foundation for automation, which is valuable because one-to-one expert explanations are costly yet crucial

for maintaining software quality amid frequent turnover and onboarding [68]. Our findings can then inform future work to derive criteria for designing and evaluating automated software architecture explanations. This study serves as an empirical first step in characterising how software architecture explanations occur in real industrial contexts. We therefore pose the following research questions:

- 3
- RQ3.1 What key *topics* are addressed in software architecture explanations?** This question examines the specific content discussed, from technical elements (e.g., architectural patterns, views) to contextual issues practitioners prioritise during software architecture explanations.
 - RQ3.2 What does a structured *plan* for explaining software architecture entail?** This question explores the diverse activities that typically take place during explanations and their relations.
 - RQ3.3 What *supporting artefacts* are used during software architecture explanation?** This question identifies existing or on-the-fly assets such as diagrams and tools. It helps understanding which and how visual and textual aids support comprehension, thereby guiding their refinement to better support architectural explanations.
 - RQ3.4 What are the *typical questions* asked during software architecture explanation?** This question examines recurring inquiries from explainees and reveals which aspects of the system need to be elaborated, helping explainers anticipate and address these issues.
 - RQ3.5 What are *expectations and challenges* during and after software architecture explanations?** This question investigates both what participants expect and the challenges they face to uncover gaps between communication and learning, guiding improvements in knowledge transfer.

We systematically investigate our research questions through a judgment study [202] in the form of semi-structured interviews with practising software

professionals. We interviewed 17 participants occupying either *explainer* or *explainee* roles. To ensure a robust and ethically sound investigation, we follow Strandberg's guidelines [2019]. The interview data was analysed using the Gioia methodology [77], combining open and in-vivo coding [40, 134] to identify themes relevant to the practice of software architecture explanation.

3

Our study shows that: (1) *Topics*: Software architecture explanations focus on system structures, rationale, and evolution, but under-communicate quality attributes; (2) *Plans*: Effective explanations combine documentation review, artefact exploration, questioning, and feedback, favouring top-down, incremental, interactive, in-situ styles; (3) *Supporting artefacts*: Explanations are anchored in both architecture- and implementation-focused documentation, with structural and behavioural diagrams dominant and increasing use of high-level and data views, while development tools are largely avoided; (4) *Typical questions*: Explanations address exploratory questions (rationale, impacts, alternatives) and confirmatory questions (validating partial knowledge) across abstraction levels and viewpoints; and (5) *Expectations and challenges*: Explainers and explainees expect clarity, structure, and actionable closure but struggle with abstraction management, content curation, and documentation gaps. We overlay the Twin Peaks model on our findings to stress separating the “how” from the “what” and deliberately bridge the (often implicit or presumed) business context to technical rationale. We also introduce the Explanation Window framework to tailor the scope and depth of architectural explanations to audience needs.

The remainder of this chapter is organised as follows. Section 3.2 reviews related work. Section 3.3 details our study methodology, including participant selection, data collection, and analysis procedures. Section 3.4 presents our findings, which are then discussed in Section 3.5. Section 3.6 examines the trustworthiness of our qualitative study and, finally, Section 3.7 concludes with future research directions.

3.2 Related Work

Despite the significance of software architecture explanation for effective development and onboarding, there is a remarkable absence of empirical

research examining how such explanations are actually conducted in practice. Prior studies predominantly focus on the effectiveness of documentation formats [59, 187], the role of specific artefacts or tools [89, 100], or how to effectively perform knowledge transfer [30, 32], but stop short of analysing the explanation process itself.

For instance, Ernst and Robillard [59] found that the document format (narrative vs. structured) does not significantly affect newcomers' understanding; prior code exposure and architectural information needs matter more. Similarly, Shahin et al. [187] showed Architectural Design Decisions (ADDs) enhance comprehension but not necessarily task completion time.

The persistent gap between documentation and implementation has led to studies on the informal mechanisms by which architectural knowledge is shared. Unphon and Dittrich [215] highlight the role of a “walking architecture” (through chief architects or lead developers) in managing design and communication beyond documentation. Related work on tool support, such as RoleViz by Ho-Quang et al. [89] and graphical design description tools as explored by Jolak et al. [100], show that visual artefacts improve usability, communication, and productivity, but textual descriptions can lower perceived explanation quality.

However, knowledge transfer is not simply about accessing artefacts. Camacho et al. [30] stated that knowledge transfer in software engineering requires effective application, not just dissemination. Canfora et al. [32] noted that informal, face-to-face interactions are the primary source of architectural knowledge for newcomers. This dynamic supports the explainer/explainee distinction and aligns with cognitive apprenticeship models.

Notably, several studies have explored how programmers ask questions when seeking information [115, 161, 189], but these focus on immediate, task-specific information needs. In contrast, architectural explanations are conceptual, aiming to build a deep understanding of system design, including decision rationales and component relationships. While these studies examine developers' question-asking, architectural information transfer differs fundamentally. Below, we contrast programming questions with architecture

comprehension as our initial framing and assumption:

Technical task orientation vs. conceptual understanding: Programming aims for immediate and specific solutions (e.g., code snippets), often found in documentation or forums. Architectural explanation is conceptual, seeking deep understanding of system design, decision rationales, and component relationships.

3

Detailed vs. broad information: Programming tasks need immediate and detailed data focused on syntax or functions. Architectural understanding covers broader topics like design patterns, scalability, and trade-offs, building a mental model of the whole system.

Exploration vs. transfer: Programming often involves trial and error. Architecture knowledge transfer applies prior expertise to develop lasting insight into complex system interactions.

As Arab et al. [8] suggest, there is a fundamental difference between code sharing and strategic knowledge sharing. Existing work leaves open the question of how architectural explanations are structured, what topics and artefacts are prioritised, how explainer and explainee roles interact, and where the main challenges and breakdowns occur in practice. This study directly addresses that gap. We offer the first open-ended empirical analysis of software architecture explanation as a situated practice, examining not only what information is conveyed, but *how*, *why*, and with what artefacts, strategies, and outcomes. In doing so, we move beyond artefact-centric or tool-focused perspectives, and shed light on the actual communicative work that sustains architectural knowledge in software organisations.

3.3 Methodology

We carried out a *judgement study* [202] with the primary aim to collect accounts of software architecture explanation practices as they occur in professional environments. To do so, we conducted semi-structured interviews with practitioners who reflected on recent authentic experiences in their own work contexts.

We initially sought to conduct a *field study* via in-situ observations of

architecture explanation sessions; however, all but one organisation we approached declined participation due to confidentiality concerns regarding their software. This challenge is not uncommon in software engineering research, where architectural discussions often involve proprietary or security-sensitive information. As a result, we relied on interviews, following best practices for qualitative research when direct access to field settings is restricted [182, 202].

The ABC framework [202] recognises that, when observational access is impractical, interviews grounded in lived experience are an accepted alternative to capture the practitioner's context, although they necessarily involve trade-offs in realism and depth. In our case, this strategy offered advantages in terms of actor and context diversity: interviews allowed us to sample practices across varied organisational settings, capturing a broader range of explanation scenarios than a limited number of observable sessions would have permitted. Additionally, architectural explanations often involve implicit reasoning and adaptation to the explainee's role and needs, which are not readily inferable through observation alone. Interviews, when anchored in concrete activities, support access to such internal perspectives. Accordingly, participants were prompted to recount specific, recent explanation experiences to reduce retrospection bias and ground responses in real-world events.

As judgement study, i.e., in the absence of direct observation or artefact collection, our data rely on participants' self-reported experiences. In particular, participants were explicitly prompted to anchor their accounts in actual projects and situations, and demographic and background questions were used to contextualise each account. As such, our approach provides context-rich practitioner perspectives, but does not offer the maximal contextual realism or behavioural evidence of immersive fieldwork. We therefore do not claim ethnographic depth, but rather present our results as derived from *contextually rich and practitioner-anchored interviews*, acknowledging both their authenticity and their inherent limitations. This is in line with interpretive qualitative traditions in empirical software engineering, where

practical restrictions prevent direct access to field sites [51]. Researchers and readers should interpret our findings with these constraints in mind. We discuss the limitations of this approach in detail in Section 3.6.

Figure 3.1 outlines our methodology in three phases: (1) study planning—stakeholder identification, questionnaire development, ethical considerations; (2) data collection—conducting and transcribing interviews; and (3) data analysis—coding, member checking, and reporting.

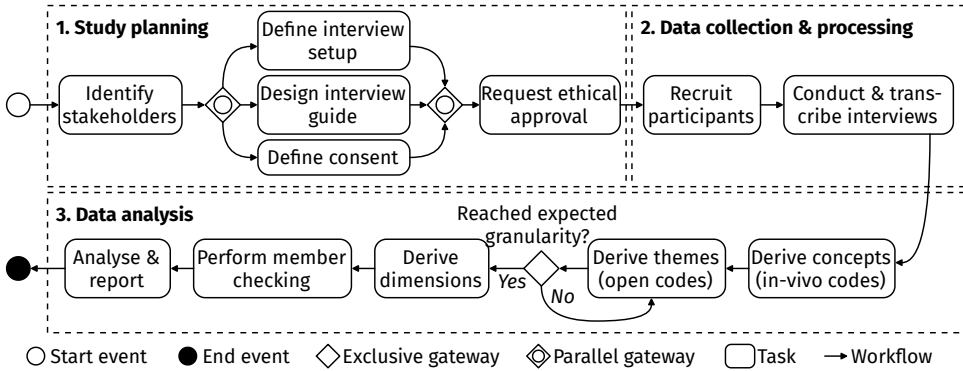


Figure 3.1: Interview study methodology.

3.3.1 Study Planning

The planning phase aims to develop a rigorous and ethical study design.

Identify stakeholders: *Stakeholders* in our study context include all people involved in giving or receiving software architecture explanations, as well as their project supervisors or managers [205]. However, only two groups served as study *participants*: *explainers*, typically architects, senior developers, technical leads, or solution architects with deep system knowledge; and *explainees*, who lack familiarity with (parts of) the system. We classified participants by their predominant role in the context of the system under discussion—rather than general professional seniority—to ensure focused, context-specific feedback.

We used a combination of convenience, purposive, and snowball sampling to recruit participants who recently engaged in a software architecture explanation in real-world contexts as either explainers or explainees. Convenience

sampling allowed efficient recruitment from accessible professional networks. Purposive sampling targeted professionals with relevant first-hand experience, while snowball sampling extended reach to under-represented or specialised roles. These approaches are widely accepted in empirical software engineering when depth and context matter [51, 53, 182, 183, 203, 210].

We did not use general online platforms such as Prolific, as these are designed for studies seeking a broad demographic reach and cannot guarantee access to professionals embedded in specific, confidential organisational settings [163, 203]. This does not align with the objectives of a judgement study, which requires careful selection of experts [202]. Previous studies [52, 163] also indicated that such platforms risk data noise and low trustworthiness when used for context-dependent topics. With the increasing availability of generative AI, ensuring the authenticity and quality of responses on such platforms has become even more challenging.

Given the small number of participants, we do not claim statistical generalisability. This study is positioned as a qualitative study, aimed at an insight into professional practices rather than broad representativeness.

Define interview setup: Interviews were conducted in English and held online using Microsoft Teams. Only the interviewer and interviewee were present (except for the first interview, which included a silent observer for interviewer feedback). Microsoft Teams was chosen for secure recording and GDPR-compliant transcript generation. All personal and confidential data were anonymised by the first author before being shared with co-researchers to avoid leaving traces that could identify participants. Conducting interviews in a private setting helped reduce distractions [202], bias, or retaliation risk [205].

Design interview guide: The interview guide design is connected to the initial plan for in-situ observation. As mentioned earlier, we succeeded in observing a real-world knowledge transfer session at one company, where a departing architect (*A*) explained system architecture to another architect from a different team (*B*), who was going to assume temporary responsibility for the project. This observation informed the formulation of our research

questions and thus the drafting of the interview guide.

Subsequently, we conducted a pilot interview with a newly hired architect (C), who later participated in an onboarding session with *B*. Following this, *B*, having both received an explanation and acted as explainer, provided detailed feedback on the interview process and questions. This dual-role perspective was instrumental in refining the guide. The pilot interview served as a realistic chance to enhance our interview design prior to conducting the full study [125, 205]. Data from the pilot were excluded from the main analysis.

3

The resulting interview guide is structured as a flexible script [20, 190] with separate versions for explainers and explainees. It begins with instructions to ground responses in a real-work context, i.e., a specific architecture explanation the participant had experienced, not limited to face-to-face situations. Next were demographic questions on experience, company, project, role, tenure, and context, used to assess suitability for the assigned role. Core questions address five dimensions aligned with our research questions: explanation topics (RQ3.1), explanation plans (RQ3.2), supporting artefacts (RQ3.3), typical questions (RQ3.4), and expectations and challenges (RQ3.5). We use both specific and open questions to obtain in-depth, nuanced views [93, 182]. Table 3.1 lists these questions and their mapping to the research questions. To control for quality, two authors drafted the guide, which was then refined with input from a software architecture expert and the earlier pilot interview.

The selection of the five dimensions is grounded in empirical observation, conceptual framing, and operational considerations. First, the initial observation session revealed that an architecture explanation unfolds across several interrelated layers: what is explained, how the explanation is structured, how it is mediated (for example, through diagrams or other artefacts), how the explainee engages with the explanation, and what outcomes or difficulties arise. These layers provided a natural basis for analytically decomposing the phenomenon.

Second, as prior work characterises architectural knowledge transfer as a

Table 3.1: Interview questions specific to explainers and explainees.

ID	Question	RQ
For Explainers		
Q_{1_E}	What needs to be explained about a system's architecture?	RQ3.1
Q_{2_E}	What are important considerations you keep in mind when explaining a software system?	RQ3.1, RQ3.2
Q_{3_E}	How do you start an architecture explanation?	RQ3.2
Q_{4_E}	Can you elaborate on the whole explanation process?	RQ3.2
Q_{5_E}	How do you wrap up the explanation?	RQ3.2
Q_{6_E}	Do you use any tools, artifacts, or frameworks at any point in your explanation?	RQ3.3
Q_{7_E}	Could you describe the typical questions you receive from team members during architecture explanations?	RQ3.4
Q_{8_E}	Have you encountered any challenging or unexpected questions during architecture explanations? How did you handle them?	RQ3.4
Q_{9_E}	What are your expectations after explaining an architecture to the other software professional within your project?	RQ3.5
For Explainees		
Q_{1_e}	What do you expect from a software architecture explanation?	RQ3.5
Q_{2_e}	What makes a good software architecture explanation?	RQ3.1, RQ3.2
Q_{3_e}	What level of detail would you prefer an architecture explanation to have?	RQ3.1
Q_{4_e}	What considerations are needed to comprehend the architecture of a software system?	RQ3.1, RQ3.2
Q_{5_e}	How do you start approaching (to understand) the architecture of a software system?	RQ3.2
Q_{6_e}	What other aspects do you consider important in that process?	RQ3.1, RQ3.2
Q_{7_e}	What do you expect to get at the end of a software architecture comprehension task?	RQ3.5
Q_{8_e}	Do you use any tools or artifacts at any point when trying to understand the architecture of a system?	RQ3.3
Q_{9_e}	Could you describe the type of relevant questions you ask software architects during architecture explanations?	RQ3.4
Q_{10_e}	Have you encountered any difficulties in understanding architecture explanations? If so, what were they, and how did you address them?	RQ3.1, RQ3.2, RQ3.3
Q_{11_e}	Are there any specific types of information or details that you find particularly useful in software architecture explanations?	RQ3.1

form of workplace-based apprenticeship [32], we draw on theories of situated learning and mediated instruction. In apprenticeship models [37], experts articulate domain knowledge (topics), organise and sequence it through instructional strategies (plans), and use tools that serve as mediational scaffolds [64]. The apprentice's questions reflect their zone of proximal development and inform the pacing and depth of the explanation [209], while expectations and challenges capture the motivational and affective conditions under which learning occurs.

Third, from a research design perspective, we required dimensions that are meaningful to practitioners, relevant to both explainers and explainees, and sufficiently focused to structure a semi-structured interview while remaining open to unanticipated insights. All five aspects were tested in the pilot and yielded substantive responses, reinforcing their suitability for this study.

Define consent: We follow Kitchenham et al. [105] for ethical considerations. Ethical approval was granted by the Ethics Review Board (ERB) of Eindhoven University of Technology on 10 July 2023, ensuring compliance with all relevant standards, laws, and regulations. All participants signed informed consent before participation; they were told participation was voluntary and could be withdrawn at any time.

3.3.2 Data Collection & Processing

Recruit participants: We invited 20 professionals: 10 explainers and 10 explainees. Nine explainers and eight explainees accepted, yielding acceptance rates of 90 % and 80 %. Prior work suggests thematic saturation after 12–15 interviews [82, 83]. Participant characteristics are summarised in Table 5.1. They represent diverse cultural and professional backgrounds and are based in the Netherlands, Sweden, Germany, the United States, Argentina, and Colombia. While individual employers cannot be disclosed, Table 3.3 reports annual revenue ranges of participants' organisations, based on publicly available financial information, showing that most work in large multinationals and that our study covers explanations in real, large-scale industrial projects. Explainers typically have 8+ years of relevant experience. In one microen-

terprise, the explainer had three years' tenure, which is the norm in that organisation. In another case, an explainer had only one year at the current company but many years as an architect in a similar domain and system type. Conversely, some explainees have extensive professional experience, but all were consistently less familiar with the system being explained in their specific context.

Onboarding was expected as the main scenario for explanation, but only four participants reported it; other scenarios included research, where a data scientist examines the team's software to identify data schemas and flows; solution-provider-to-client explanation, where a solutions architect tries to match their solutions with the client's requirements; cross-team collaboration, where different companies are involved in engineering an (eco)system; and architecture evaluation and audit, where a team member must explain the architecture to an auditor. These were identified from demographic data and thus not included as findings.

Conduct & transcribe interviews: The first author conducted all interviews between August 2023 and January 2024. Interviews were transcribed using automatic speech recognition built into Microsoft Teams, followed by manual proofreading and anonymisation (removal of names, company references, etc.).

3.3.3 Data Analysis

We adopted the Gioia methodology [77] for its strengths in providing transparency and rigour in inductive qualitative analysis. In this context, **concepts** refer to meaning-laden units drawn directly from participant statements (in-vivo codes), **themes** represent recurring patterns that capture shared ideas or concerns, and **dimensions** are broader, higher-level categories that organise and relate themes.

Our analysis relied on a common foundational assumption in interview-based research that participants can reflect on and articulate their experiences. ATLAS.ti¹ software was used to manage and facilitate the coding process.

¹<https://atlasti.com/>

Table 3.2: Interview study participants. *Comp.* stands for *company* and *Proj.* for *project*.

ID	Experience	Comp. type	Proj. type	Role	Comp. time	Proj. time
Explainers						
E_1	15 years	Equipment manufacturer	Metrology	Software architect	11 years	N/A
E_2	15 years	University	Information system	Software architect	N/A	1.5 years
E_3	14 years	Location technology	Microservices	Senior software engineer	4 years	2–3 years
E_4	15 years	Technology	Various	Solutions architect	2 years	2 years
E_5	19 years	Technology	Consultancy	Solutions architect	12 years	N/A
E_6	25 years	Software engineering	Embedded system for medical device	Software architect	>1 year	1 year
E_7	3 years	Software engineering	POS system	Technical lead	3 years	3 years
E_8	8 years	Transportation products and services	Search engine and automotive	Software / System architect	4 years	1.5 years
E_9	21–22 years	Research organisation	Metaprogramming toolkit	Consultant	24 years	20 years
Explainees						
e_1	8 years	Consultancy	Data analytics	Data analytics specialist	5 months	3 months
e_2	1.5 months	Research organisation	Maintenance of legacy system	Researcher	1.5 months	1.5 months
e_3	2 months	Applied research	Formal methods in industry	Researcher	2 months	2 months
e_4	2 months	Health technology	Embedded system for medical device	Intern	2 months	2 months
e_5	35 years	Consultancy	Various	Architecture reviewer	30 years	1–3 years
e_6	2 months	Vehicle manufacturer	Real-time embedded system	Functional application manager, user acceptance tester	2 months	3–4 months
e_7	9 years	Research agency	Embedded systems	Technologist, system architect, system engineer	5 months	5 months
e_8	4.5 years	Equipment manufacturer	Refactoring legacy code	Intern	6 months	6 months

Table 3.3: Number of study participants by employer’s annual revenue and organisation size.

Number of participants	Employer’s recent annual revenue	Organisation category
1	(N/A)	Freelance (client not disclosed)
3	(N/A)	Academic organisation
1	(N/A)	Micro enterprise*
3	€ 10–50 million	Medium enterprise*
2	€ 500–700 million	Large enterprise*
5	€ 10–50 billion	Large enterprise*
2	€ 200–700 billion	Large enterprise*

*European Commission classification [63].

Derive concepts: First-order concepts were identified using in-vivo coding [134] by the first four authors. The first three authors split the interview transcripts evenly among themselves and analysed the complete transcripts. Separately, the fourth author analysed all transcripts, but focusing solely on parts related to questions being asked during explanations (i.e., for RQ3.4).

Derive themes: Second-order themes were developed through open coding [40], with the second and third authors independently coding the same transcripts in iterative rounds. Each round covers four transcripts. After each round, the coders met to discuss and align their codes, documenting the rationale for additions or changes in individual codebooks. Each author maintains an individual codebook to record any additions or changes made during coding. These records include notes on the rationale for each change. During alignment meetings, overlapping or redundant codes are identified, merged, and updated in both the shared ATLAS.ti project and a consolidated codebook. All agreed-upon code additions, removals, or modifications are systematically documented in the shared codebook to track the development of the coding scheme across interviews.

Although the Gioia approach discourages imposing theoretical constructs prematurely, we pragmatically used Kruchten’s 4+1 view model [112] and comprehension models [4, 204] as pre-existing thematic categories. While these frameworks provided analytic structure and terminology, they did not constrain inductive openness; any codes and themes that fell outside these frameworks were retained as novel contributions.

Derive dimensions: Once coding was complete and all themes had been identified, the first three authors sat together to cluster related themes into higher-level dimensions, capturing broader facets of the data.

Perform member checking: To enhance validity and transparency, we performed member checking [21] by sharing the evolving set of codes, descriptions, and linked participant quotes via email. Participants were invited to review and suggest corrections. Three participants expressed agreement with the coding, and one requested a change to a code description, which we incorporated.

Analyse and report: The final set of concepts, themes, and dimensions was analysed and reported in alignment with participant perspectives.

3.4 Results

We present the main results answering our research questions. We follow the Gioia methodology, with dimensions highlighted in dark violet boxes and themes in light blue. Each theme indicates whether it applies to Explainers, Explainees, or Both perspectives. Themes exclusive to one perspective are marked accordingly.

3.4.1 Explanation Topics

To answer RQ3.1, we identify two key dimensions capturing topics during software architecture explanations: system context and structure, and factors influencing architectural decisions. These range from system contexts and architectural views to design rationales and system evolution implications. Table 3.4 shows the dimensions and themes around this RQ. In this table and the subsequent tables about the dimensions and themes, column **E** shows whether any *explainer* is concerned with the corresponding theme, and likewise **e** for *explainees*.

System Context and Structure

This dimension covers the system’s context and architectural composition. While architectural views were commonly discussed, quality attributes and specific architectural styles or patterns received notably less attention.

Table 3.4: Dimensions and themes revolving around RQ3.1.

RQ3.1 Explanation Topics			
Dimension	Theme	E	e
System Context and Structure	The system's context	✓	✓
	Architectural views	✓	✓
	Quality attributes	✓	
	Architectural styles/patterns	✓	
	Implementation	✓	✓
	Semantics	✓	✓
Architectural Decision Factors	Compliance and conformance	✓	
	Design implications	✓	✓
	Changes and evolution	✓	✓

E – explainer’s concern; e – explainee’s concern.

The system’s context. **Both:** Explainers (E_2, E_5) and explainees (e_1, e_5) agree on the importance of understanding the organisational context, enterprise architecture, and business goals for architectural decisions. As e_5 states, “a good explanation puts the architecture in its business context.”

Explainees: Multiple explainees (e_5 – e_8) highlight that, lacking familiarity, they rely on explanations that tie architectural decisions to the business context. Explainee e_5 observes that explainers often omit this linkage, causing consistency issues, a concern echoed by E_9 . This reflects an asymmetry: explainers tend to focus on technical knowledge, assuming domain context is known or less relevant, while explainees require the “why” before the “how” becomes meaningful [15].

Architectural views. **Both:** Discussions about behaviour are common among participants ($E_1, E_2, E_5, E_7, E_8, e_3, e_6$). Similarly, discussions on structures were widely raised (E_1, E_3 – E_5, E_7, e_1 – e_3, e_5 – e_8), specifically on the responsibilities and interactions between components. Both explainers and explainees point out the importance of considering complementary views to comprehend the system: “You want to decouple [the architecture] in different views.” (E_5)

Explainers: Deployment and development views are predominantly mentioned by explainers (E_1, E_2, E_3, E_9) and were largely absent from explainee responses. This suggests that explainers often aim for sharing a more com-

plete picture of the system's architecture, while explainees take the structural or conceptual perspective as a first step to build a mental model of the system. The discussion on architectural views also anticipates the importance of diagram usage in explanations (cf. Section 3.4.3).

3

Quality attributes. Quality attributes remain prevalent in software architecture literature [16, 131] and are seen as key drivers of architectural decisions, so we expected them to feature prominently in architecture explanations. Our findings do not support this assumption. **Explainers:** Attributes such as security, availability, scalability, and flexibility were mentioned by only a few explainers (E_2 , E_5), mainly in review or compliance contexts. For instance, E_2 notes: *“We review the quality scenarios and the architectural significant requirements imposed to the software architecture, and we study how those quality scenarios are driven there . . . or how the architecture tries to solve those quality attributes and scenarios.”* No explainees raised quality attributes as topics of interest.

This does not mean quality attributes are unimportant; rather, they seem implicit or backgrounded in *explanation practices*. This may stem from assumed shared understanding, emphasis on structural or functional aspects in early explanations, or difficulty in expressing quality-related reasoning. However, under-communicating quality attributes risks explainees missing the rationale behind decisions, overlooking system limitations, or making changes that threaten architectural integrity.

These observations suggest a gap between architecture theory and explanatory practice. Future work should explore how to integrate quality attributes more explicitly into explanations, while aligning with explainees' information needs.

Architectural styles/patterns. **Explainers:** Different architectural styles, e.g., layered architectures for embedded systems versus microservices, demand tailored explanations (E_4 , E_5). Experienced developers may specifically ask for the rationale behind the choice of style or pattern. An identified barrier is that many practitioners lack formal training in architectural terminology, limiting the use of these concepts in explanations (E_3 , E_9). This reinforces the

terminology gap between academia and industry and may hinder effective communication with explainees.

Implementation. Explainers: Implementation of a system is mainly expressed in terms of concrete executable artefacts, most commonly source code, but also related artefacts such as repository structure, configuration files, and API definitions. Discussing implementation details is essential to verify the alignment of the design and the implementation and identify technical debt, especially during code reviews or resolving blockers (E_1, E_3, E_7). As E_7 explains: “... when I then have to explain such a feature, then I’m also actively trying to help them ... both in the sense of where everything should be and what all the parts should be but also how it should look like from an implementation standpoint.” This characterises a design–implementation drift issue, i.e., “as-is” vs. “as-intended” architecture.

Explainees: Explainees tend to ask code-level questions (e.g., understanding a specific method or file) to move their tasks forward (e_2, e_4). Both groups seek actionable understanding, but for different purposes: explainers focus on maintaining architectural integrity; explainees seek to navigate unfamiliar codebases.

Semantics. Both: Shared understanding of notations used in an explanation is repeatedly mentioned as essential for effective explanation. This includes clarifying how explanation (or artefact) elements should be interpreted—for example, what boxes or arrows represent in informal sketches (E_6, E_9, e_3), and agreeing on the meaning of domain and architectural terminology. As E_9 describes, “... somebody draws an arrow or a box on the white board and it’s like, what does that arrow mean? Is it the data flow?” Informal or nonstandard diagrams are common, often requiring clarification of meaning (E_4, E_5, E_8, e_5, e_6). Explainees frequently ask for definitions of domain, business, and architectural terms (E_3, E_9, e_3), and organisational jargon can be a source of misunderstanding (E_1, E_3, e_4, e_6). Misaligned semantics can lead to incorrect mental models and, subsequently, design or implementation errors, underscoring the importance of establishing shared interpretive ground during architecture explanations.

Architectural Decision Factors

Architectural decision factors include requirements, business goals, design implications, and compliance or conformance drivers. These elements help shape architectural choices and are crucial for explanations that align technical decisions with broader organisational needs.

3

Compliance and conformance. **Explainers:** Explainers consistently stress that architecture must align with both business goals and compliance requirements, which are often poorly documented or implicit (E_5 – E_7). Making these constraints explicit and traceable is a recurring challenge that frequently causes inconsistencies or rework. Technical decisions must also respect business constraints such as budget and time (E_2 , E_5 , E_9); as E_5 notes, *“it’s not about only making a nice architecture, but if it [is going to] cost something that is out of budget, . . . it won’t work.”* Explainers highlight the need to communicate with both technical and business audiences, reuse lessons from past projects to match client needs (E_9), and clarify how features fit the overall architecture (E_7). They also emphasise internal guidelines, such as process requirements and documentation protocols (E_6), reflecting attention to organisational compliance even when not formally labelled as such. Several participants (E_4 – E_6) point to the importance of documenting rationale, linking requirements to architectural decisions, and managing technical debt. External compliance with laws, regulations, or contracts further constrains design, and explainers note the difficulty of reconciling business decisions with these policies (E_2). Overall, explainers act as translators of often hidden or shifting constraints, ensuring traceability from context to design, while compliance needs constrain both the design space and how architecture can be discussed.

Impacts of decisions. **Explainers:** Understanding past architectural decisions, especially when documentation is lacking, is considered essential to avoid inconsistencies or inefficiencies (E_1 , E_3 , E_5 – E_7). Architecture Decision Records (ADRs) are valued for capturing decision rationale (E_6 , E_7). Explainers also emphasise the need to discuss alternatives, trade-offs, and impacts on the system with stakeholders (E_1 , E_2 , E_5), echoed by explainee e_3 , something that

documentation practice recently started embracing via ADRs. E_8 highlights the utility of tools for visualising the impact of design choices. However, not every decision warrants detailed documentation; E_6 notes that explainers often exercise judgment about what is relevant to share.

Explainees: Transparency about the reasoning behind key design decisions is critical for understanding system intent (e_5, e_6, e_7). Explainee e_5 specifically expects to see “*key decisions . . . and how they relate to the business goals.*” This theme underscores the importance of visible rationale in enabling comprehension, not only for future maintainers but also for new team members.

Changes and evolution. **Explainers:** A core part of the explainer role is to guide explainees in identifying which components can or should be modified, ensuring that evolution maintains conformance and does not lead to architectural drift or erosion (E_8, E_9). Explainers thus act as gatekeepers, helping others navigate system change responsibly.

Explainees: Tracing the history of changes is essential for understanding current implementations, particularly when documentation lags behind the system (e_3). Explanation becomes especially critical when changes are not properly recorded, as this can cause confusion during ongoing evolution.

RQ3.1 Answer: Software architecture explanations typically orbit around the system’s context and views, with supporting detail on implementation, semantics, and styles or patterns. They also extend to decision factors such as compliance, rationale, and system evolution. In practice, explainers act as translators: linking design to business goals, constraints, and past choices, while guiding responsible change. Notably, quality attributes, though central to architecture, remain under-communicated.

3.4.2 Explanation Plan

A deliberate plan enhances the effectiveness of software architecture explanations. To answer RQ3.2, we structure the plan into three dimensions: activities (what to do), comprehension model (how understanding devel-

ops), and explanation style (how it is delivered). These guide a successful explanation. Table 3.5 shows the dimensions and themes around this RQ.

Table 3.5: Dimensions and themes revolving around RQ3.2.

RQ3.2 Explanation Plan			
Dimension	Theme	E	e
Activities	Gain prior knowledge	✓	✓
	Prepare explanation meeting	✓	✓
	Review documentation	✓	✓
	Explore source code	✓	✓
	Interact with the system	✓	✓
	Visualise system architecture	✓	✓
	Provide/receive feedback	✓	✓
	Support Q&A	✓	✓
	Derive working plan	✓	✓
	Document the outcomes	✓	✓
Comprehension Model	Top-down model	✓	✓
	Bottom-up model		✓
	Opportunistic model	✓	✓
	Hybrid model		✓
Explanation Style	Incremental explanation	✓	✓
	Interactive explanation	✓	✓
	In-situ explanation	✓	✓
	Non in-situ explanation		✓

Activities

Participants from both perspectives identified a set of core explanation activities, but reported no uniform sequence. We enumerate these activities without suggesting a fixed order, as execution varies by situation, role, and personal preference.

Gain prior knowledge. **Explainees:** Explainees emphasise the need for some level of prior knowledge, whether general software engineering (e_2) or system- or domain-specific (e_7). However, opinions are split: some (e_1 , e_8) argue domain knowledge is essential to grasp architecture, while others (e_2 , e_7) think that good architectural explanations can bridge knowledge gaps for those with technical backgrounds. Explainee e_2 states that, with an effective explanation, “most people with a software background should be able

to understand,” without needing domain knowledge.

Explainers: E_5 agrees domain knowledge supports understanding, but also signals the importance of tailoring explanations to the explainee’s background. This highlights the need for adaptive strategies and aligning frames of reference, as not all explainees approach explanations with the same knowledge or needs.

Prepare explanation meeting. While we did not restrict the format of architecture explanations, nearly all participants recalled dedicated *explanation meetings*, a face-to-face meeting between explainer and explainee with architecture explanation as the agenda. Both explainers and explainees described that these meetings often require preparation on both sides.

Explainers: Many explainers expect explainees to review background materials and prepare relevant questions in advance (E_1, E_5, E_9). E_1 describes sometimes sending explainees away to research and return with questions or proposals: “Go back and do your research and come back again with the current design and where you want to go.”

Explainees: Explainees e_4 and e_6 echo the need for preparation as a shared responsibility: explainees should formulate questions, and explainers should provide clear and structured explanations.

Review documentation. Both: Documentation review is a recurring activity, used at various points before or after live explanations (E_4, e_5, e_7). Documentation spans architecture diagrams, configuration details, use cases, results from analysis tools, and rationale for key design choices. Explainees often rely on documentation as a first point of contact (E_8, e_2), or when system experts are unavailable (e_1). It also serves as a reference during later development (e_4). However, most participants recognise documentation is rarely complete or fully aligned with the system, reinforcing the need for supplementary explanation and up-to-date rationale.

Explore source code. Explainers: Several (E_3, E_4, E_7) use source code exploration, often in pair programming activity, to reinforce architectural concepts and verify alignment with design.

Explainees: Perspectives diverge: some (e_8) find code walkthroughs dis-

tracting during explanations, saying, *“I would rather have a more overview explanation of the whole system and then [look] myself into the very specific implementations . . . rather than someone telling me the very specific implementations and having to figure out the domain and the overview of the system;”* others (e_2) see code review as vital to reinforcing understanding. This split reflects different comprehension models: Some prefer a conceptual foundation first, while others want hands-on implementation details early.

Interact with the system. Both: Direct interaction through demos or user exploration helps bridge the gap between abstract architecture and concrete functionality (E_4, e_2, e_7, e_8). As e_8 puts it: *“When I can interact with the system it helps with the explanation. What does a button do? What does this field do? It’s easier to follow the explanation.”* Demos reveal system behaviour, clarify feature purpose, and sometimes expose gaps between documentation and reality. (E_4 : *“Part of that first explanation is showing them or giving them a demo: This is what we built. This is our software. This is where you’re going to be living.”*)

Visualise system architecture. Both: Diagrams and drawings are widely seen as essential for understanding, more effective than text or code alone ($E_1, E_4, E_6, E_8, e_5, e_6, e_7, e_8$). Visuals clarify structure, foster peer discussion, and make complex interactions tractable. E_1 : *“What [is] helpful for me to explain my proposal to others is to create diagrams. I think those are the best way to to interact with the fellow architects and other engineers.”* Explainees (e_8) often describe visualisations as the main explanation aid. (See Section 3.4.3 for more information on visual artefact types.)

Provide/receive feedback. Explainers: Timely feedback from explainees helps clarify ambiguities and improve future sessions (E_3). When presenting to external customers, architects expect pushback and use feedback to assess alignment with requirements and compliance (E_5, E_6), as noted by E_5 , *“We will go into the architecture presentation and it’s just about feedback in our case. Basically, we’re proposing something and of course it’s not gonna be perfect. Of course, it’s gonna be pushed back. I am talking about the good one, good feedback. Hey, you missed this one or why do we do this this way or no; based*

on our compliance rules, we cannot do that.”

Explainees: e_3 stresses feedback as a tool for clarifying ambiguous points and shaping explanations, shifting from passive reception to active construction of understanding.

Support Q&A. Explainers: About half (E_1, E_3, E_8, E_9) stress that supporting Q&A sessions is essential not just to clarify details, but to examine the understanding of the explainee and to uncover new perspectives. E_3 : *“You need to give more space for [explainees] to ask questions. Actually, I would say that is one of the most important things.”* E_9 notes that recurring questions from past clients help reinforce or adjust explanations using prior insights.

Explainees: All explainees (e_1 – e_8) see Q&A as crucial, both to address knowledge gaps and to follow up incomplete documentation. Several recommend preparing for explanations by reviewing documentation and formulating questions (e_2 – e_4). When documentation is lacking, peer consultation becomes even more important (e_1 – e_3).

Derive working plan. Both: Before concluding an explanation, both sides often agree on concrete next steps, like running or debugging the system (E_1, E_5), carrying out a pilot task (E_3, e_2, e_3), or pair programming (E_3). E_3 describes a hybrid approach: new hires start with pair programming but are encouraged to work independently before seeking help. Later, the team evaluates requirements to define future features (E_2, E_6, E_9). This transition marks the shift from guided explanation to autonomous work.

Document the outcomes. Both: Documenting key takeaways from explanations (notes or minutes) ensures a shared understanding of decisions and discussions (E_6, e_7). As E_7 notes: *“I take notes [of] what things are important for what I’m interested in.”* Documentation supports explainees in subsequent tasks and helps explainers improve future sessions. In addition, explanation sessions often expose outdated or missing documentation, prompting valuable updates to architectural records.

Comprehension Model

The sequence and nature of explanation activities are shaped by the comprehension model in use. We define the comprehension model as a cognitive framework describing how an individual forms an abstract representation of what is being understood [204]. We build on cognitive models from prior work [4, 204]: top-down, bottom-up, opportunistic, and hybrid, mapping our themes to this literature.

Top-down model. In this model, architecture is understood by linking high-level abstractions down to code as needed [27].

Explainees: Explainees e_5 , e_7 , and e_8 endorse this model: e_8 explains, “*it is easier to know first what the system is doing as a whole and then focus on different parts of functionality.*”

Both: Participants emphasise avoiding information overload. E_1 : “*I want to limit the information . . . the more information I provide, the harder it will be for the scope and problem to be understood.*” Some, especially in management roles (E_1 , E_4 , E_6 , E_7), stay at a high level, focusing on design layout and interactions without deep dives. Explanations often end at the interfaces of key components or services (E_1 , E_4 , E_5 , e_2 , e_6). This points to the need to calibrate abstraction and tailor explanations to the audience.

Bottom-up model. This approach starts with inspecting code and low-level details, then builds higher-level models [27].

Explainees: Only e_1 explicitly prefers a bottom-up start, usually as part of a combination strategy, beginning with code, then constructing a conceptual model and checking it with the explainer.

Opportunistic model. **Both:** Here, professionals attend to what is most relevant for the immediate task, e.g., refactoring, bug-fixing, or feature work [4, 204] (E_7 , e_7). Explanation is not for holistic understanding, but for supporting a specific goal [120, 128]. E_7 : “*What I’m considering when explaining things is that [it is] relevant for the tasks that they’re doing.*” Focus may shift between architectural views (Section 3.4.1) (E_5 , e_6 , e_7), or across abstraction levels (e_2). Some (E_3) describe actively jumping between

perspectives to refine their mental model. This underlines the need for explanations that are responsive to evolving goals and contexts.

Hybrid model. This blends the above strategies, switching as needed for the situation [4]. Flexibility allows cross-checking and fills knowledge gaps.

Explainees: e_1 , for example, rarely follows a linear path: starting with the system's purpose, they alternate between structure (directories, classes) and detailed code, iteratively refining understanding: *"The starting point should be ... what the system tries to do, then ... the hierarchy or directories, and then ... go to each directory ... and explain what the classes here are doing or how they communicate with each other."* This iterative, integrative style appears common and pragmatic in handling software complexity.

Explanation Style

Explanation style concerns how architectural knowledge is conveyed, e.g., its pacing, structure, and supporting resources, distinct from interpersonal communication style. Nevertheless, participants consistently emphasise open communication (E_1 , E_2), minimising conflict (E_2), and prioritising clarity and simplicity (E_5 , E_7 , e_4 , e_6 , e_8), aligning with assertive communication principles [149].

Incremental explanation. **Both:** Both explainers and explainees emphasise the need to spread explanations over multiple sessions, allowing for stepwise knowledge internalisation. e_8 stresses the difficulty of absorbing everything at once: *"It's better [if there are] different sessions, so I can understand the information ... I need time to process a little information and then move on to the next part."* E_5 notes that architectural explanation is rarely "one meeting," but instead, an iterative, interactive process. Rapid, continuous explanations from experts can overwhelm newcomers.

Interactive explanation. **Explainers:** Explainers (E_1 , E_8) structure sessions as interactive, not just presenting but continuously inviting questions and updating diagrams or artefacts. This enables immediate feedback and real-time adjustment.

Both: Many (E_2 , e_2 , e_4) note that discussions, often with several architects

or teammates, are critical for understanding complex issues. e_2 highlights the value of “on-demand” clarification via chat or informal discussions, reinforcing that accessibility and availability of team expertise are essential for effective architecture comprehension.

3

In-situ explanation. **Both:** Face-to-face meetings allow explainers to monitor explainees’ reactions for signs of confusion or understanding (E_1 , E_8). E_1 : *“It helps a lot ... to be face to face with that person and to get feedback on their reaction, because most of the time when I have to explain these terms, I see the other person confused.”* Such in-situ feedback loops drive more adaptive, transparent explanations.

Ex-situ explanation. **Explainees:** When direct sessions are unavailable, explainees such as e_1 rely on documents or diagrams in the knowledge base, interpreting them without guidance. This approach is more effortful and prone to misunderstanding, and e_1 expresses a preference for live, interactive sessions: *“Having live sessions ... would be much better to have an easier understanding on how things work.”* This underscores the need for more robust asynchronous or mediated alternatives when face-to-face explanation is not feasible.

RQ3.2 Answer: An effective architecture explanation plan weaves together three elements: activities, comprehension models, and explanation styles. Activities cover the mechanics: preparation, documentation, system exploration, visualisation, Q&A, and outcome capture. Comprehension follows different models: top-down, bottom-up, opportunistic, or hybrid, with the top-down model dominating participant’s preferences. Explanation styles: incremental, interactive, in-situ or otherwise, govern delivery and pacing. What emerges is not a fixed recipe but a flexible approach: explanations succeed when adapted to audience background, task at hand, and desired depth, ensuring both shared understanding and actionable next steps.

3.4.3 Supporting Artefacts

Supporting artefacts are essential for clarifying and improving understanding in software architecture explanations. These include documentation, diagrams aligning with the 4+1 view model, and tools for their creation and management. Table 3.6 shows the dimensions and themes around RQ3.3.

Table 3.6: Dimensions and themes revolving around RQ3.3.

3

RQ3.3 Supporting Artefacts			
Dimension	Theme	E	e
Documentation	Architecture-related documentation	✓	✓
	Implementation-related documentation	✓	✓
Diagrams	Functional diagrams	✓	✓
	Process diagrams	✓	✓
	Development diagrams	✓	✓
	Deployment diagrams	✓	✓
	Data diagrams	✓	✓
	High-level diagrams	✓	✓
Tools	Visual tools	✓	✓
	Documentation tools	✓	✓
	Development tools	✓	

Documentation

Documentation supports explanations both during sessions and independently, especially when explainers are unavailable. Participants describe a wide range of documents, which we group as architecture- and implementation-related.

Architecture-related documentation. **Both:** Beyond diagrams, textual documentation is highly valued ($E_4, E_6, E_9, e_2, e_5, e_6, e_7$). Participants point out that architecture documents are often in unstructured, narrative (“storytelling”) formats: “Every time we have someone onboarding, we ... explain all these decisions ... It’s more of a storytelling than [structured documentation], but [it’s] is useful for newcomers” (E_4). This embedded context helps new team members grasp rationale and history. By contrast, ADRs provide a structured alternative, capturing decisions, context, alternatives, and stakeholders (E_5, E_6).

Implementation-related documentation. **Both:** Implementation documentations, like API specs (E_7, e_1, e_4), code documentation (E_5, E_6, e_3, e_4), integration manuals (E_6), service interaction docs (E_4), help explainees understand the system and enable independent exploration and debugging across diverse tech stacks. E_5 notes: “*The most important documentation is how to run the system . . . [to] debug the system is the best way of learning it.*” Even low-level documents serve as a bridge for understanding architecture and design, though their completeness and consistency remain a recurring issue.

Diagrams

Diagrams are central to explanation, typically aligning with the 4+1 view model [112], but also reflecting high-level and data-centric perspectives. Participants discussed both diagrams that are persisted within documentation or modelling tools, and those that are drawn ad-hoc, e.g., on a whiteboard, during explanations.

While we refer to several known diagram types (e.g., component diagrams, sequence diagrams, context diagrams), we do not equate diagram types with architectural views or formal viewpoints. Our classification follows how participants themselves described and used these artefacts in real-world explanation contexts. As such, the categories reflect the functional role of diagrams in explanation rather than a one-to-one mapping to architectural viewpoints.

Functional structure diagrams. **Both:** Functional structure diagrams, such as UML class diagrams ($E_1, E_2, E_8, E_9, e_2-e_4$), component diagrams ($E_1-E_3, E_6, E_8, e_3, e_4, e_8$), dependency diagrams (e_2), and layered architecture diagrams (E_3), are mentioned frequently. We surmise that this category of diagrams is particularly valuable at the outset of architectural explanations, as they provide a high-level overview of the system’s main components and their interactions.

Process diagrams. **Both:** Process diagrams are almost as popular as structure view diagrams, suggesting that both structural and behavioural aspects are expected to facilitate comprehension. These are mostly UML sequence

diagrams (E_1 – E_3 , E_6 , E_9 , e_2 – e_5 , e_7), but there are also references to activity diagrams (e_2) and state transition diagrams (E_6 , E_8), as well as informal process diagrams (E_8). State diagrams are mentioned especially for embedded systems, whose behaviour often depends on the system's mode. This indicates that the usefulness of a type of diagram is context-dependent, i.e., they need to align with the specific characteristics of the system under study.

Development diagrams. **Both:** Development view diagrams (e.g., package diagrams, folder structures) are rarely mentioned (E_9 , e_5 , e_7), and appear underutilised or replaced by direct code inspection. This relative absence may suggest that development diagrams are perceived as irrelevant in architectural explanations, with a preference for concrete artefacts (i.e., direct source file access) over abstractions.

Deployment diagrams. **Explainers:** Many explainers use deployment diagrams, network diagrams, infrastructure diagrams, and platform-specific visualisations such as Kubernetes diagrams (E_2 , E_3 , E_5 , E_6) to illustrate physical view and how functional components map to the runtime context.

Explainees: Explainees are less likely to note deployment diagrams as useful early on (e_5), focusing first on structure and process. Their value seems to grow as the explainee's mental model matures or as specific operational concerns arise.

Data diagrams. **Explainers:** Two explainers (E_4 , E_5) mention the use of entity-relationship diagrams for clarifying data flow.

Explainees: One data analytics specialist (e_1) highlights the need for dedicated “data views” to comprehend data flow and bridge domain concepts and technical artefacts. This may become increasingly relevant in response to the increasingly prevalent AI- and data-driven systems [25].

High-level diagrams. **Both:** Participants mention *context diagram*, *use case diagram*, *conceptual diagram*, and general high-level system sketches (E_1 , E_3 – E_5 , E_8 , e_3 , e_5 , e_7). While these artefacts vary in formality and underlying viewpoint, they share a common characteristic: a deliberately abstract representation of the system intended to support comprehension. These

diagrams are typically used to convey system-wide framing, simplify complex structures, or establish a shared mental model. Although often used at the beginning of an explanation to provide orientation, they also appear later when re-establishing context and for providing anchor points. Importantly, their high abstraction level makes them effective scaffolds for explanation, on which more detailed system models can subsequently be built.

Tools

A wide range of tools support architecture explanation: diagramming, documentation, and development platforms.

Visual tools. **Explainers:** Tools like Draw.io (E_5), PlantUML (E_1 , E_6), and GraphViz (E_9) are valued for rapid diagram creation, either via visual manipulation, written as code, or generated from other artefacts. **Both:** Some participants rely on custom tools for bespoke needs (e.g., e_7 , e_8 , E_8), reflecting dissatisfaction with off-the-shelf solutions, although policy constraints may limit adoption (e_3). E_3 mentions a dedicated microservice observability tool.

Documentation tools. **Both:** Confluence is a common choice for collaborative documentation (E_8 , E_3 , e_7), with Microsoft Word and PowerPoint used for writing and presenting textual/visual artefacts (e_4 , e_5).

Development tools. **Explainers:** A few explainers report using IDEs (E_6 , E_7) to clarify code-level details, though architecture explanations generally avoid deep dives into source. Interestingly, IDE usage is more often cited by explainers, challenging the expectation that explainers tend to stay at a high level of abstraction.

RQ3.3 Answer: Effective software architecture explanations rely on diverse artefacts and tools. Documentation spans architecture- and implementation-focused materials, including decision rationales and API specifications. Diagrams aligned with the 4+1 model play a central role, with functional and behaviour views most common. Data views may

gain importance; high-level diagram hints the need to manage system complexity. Tools facilitating dynamic diagram creation and collaboration enhance explanations. Occasional use of custom visualisation and development tools reflects varied needs and constraints.

3.4.4 Typical Questions

Q&A support is a key part of software architecture explanations (Section 3.4.2). Questions relate to topics from Section 3.4.1 but vary in purpose and abstraction. Table 3.7 shows the dimensions and themes around RQ3.4.

Table 3.7: Dimensions and themes revolving around RQ3.4.

RQ3.4 Typical Questions			
Dimension	Theme	E	e
Question Purposes	Exploratory questions	✓	✓
	Confirmatory questions	✓	✓
Question Dimensions	Level of abstraction	✓	✓
	Architectural viewpoint	✓	✓

Question Purposes

This dimension explores the purposes underlying questions posed in software architecture explanations.

Exploratory questions. **Both:** Exploratory questions seek to uncover system structure, behaviour, and rationale—asked when the explainee *does not know* and *wants to know*. They typically begin with “what”, “how”, or “why”. As e_6 notes: “[I need] to have [a] full overview of what’s happening, why it’s happening, how it’s happening, choices made. I need to understand the full concepts and . . . why, how, what, when.”

Two notable subtypes emerge: (1) *Decision-related questions* probe the rationale for design choices (E_2 , E_4 , E_5 , e_5 , e_6). E_4 emphasises: “The first question is always, ‘Why?’ ‘Why was this done this way?’ That’s the main question.” Such questions elicit detailed reasoning and often reveal inconsistencies between the implemented system and its intended or documented

architecture. They may also trigger discussions on the need for refactoring (E_1, E_6).

(2) *Impact questions* examine hypothetical changes and their consequences (E_9, e_7), e.g., “What if I change the API?” or “What if we reduce the RAM ... from 192 GB to 128 GB?” Addressing such queries requires deep understanding of how components function and interconnect, and how changes propagate through the system (e_8). As e_8 notes: “If you have a what-if question, then you are already close to the answer; if you have that question, you are on the right direction.”

Confirmatory questions. **Both:** Confirmatory questions are asked when the explainee wants to *confirm what they (think they) know*. They are used to validate or clarify existing understanding, typically yes/no or inductive, and often based on artefacts like diagrams. This helps to deepen baseline comprehension and uncover omissions or errors (E_6, e_7). For example, e_7 describes: “For instance, one block is connected to five other blocks. You inductively see it, but the person explains only three; then the question [is], ‘Is it only these three or are there more?’ Sometimes there are no more ... there might be a reason for that, or it might be an error.”

Question Dimensions

Questions during software architecture explanations vary in their level of abstraction and architectural viewpoint. This dimension illustrates how participants navigate between high-level system views and detailed technical inquiries.

Level of abstraction. **Explainers:** Explainers frequently receive detailed questions about modules or components: their responsibilities, behaviours, and roles (E_1, E_3, E_5). E_1 observes that queries about component behaviour are common; E_3 notes that detail-oriented questions often come from those previously involved with the system. This suggests explainees often seek operational detail, especially in preparation for hands-on engineering tasks. **Explainees:** Explainees, however, ask questions across the abstraction spectrum. High-level questions focus on the overall system architecture

and its main elements (e_2, e_8): as e_7 puts it, “For the very high level, what are the big pieces of the system?” Yet, explainees also delve into specifics: component responsibilities, interfaces, and data flows (e_1, e_4, e_8). These variations reflect not only knowledge gaps, but differing explanatory needs: high-level questions seek coherence and context, while low-level ones are more action-oriented.

Architectural viewpoint. **Explainers:** Two primary viewpoints dominate: functionality (what components do, their capabilities and behaviours) (E_1, E_5), and interaction (how components relate and communicate) (E_1, E_3). **Explainees:** Explainees, similarly, seek both: they ask about individual component roles and how these contribute to system goals (e_4, e_8), as well as about integration points and data exchanges across components (e_2, e_3, e_4, e_1). As e_2 notes: “Most [questions are] pretty specific. So what does this class do? Or what does this method do?”

The use of both perspectives suggests that architectural understanding is dynamic and situational, requiring iterative synthesis. Rather than treating functionality and interaction as distinct question types, explainees use both in combination to triangulate meaning and uncover rationale.

RQ3.4 Answer: Typical architecture questions fall into two purposes: exploratory questions probe the unknown, asking “what”, “how”, or “why” to uncover structure, rationale, and potential impacts of change, while confirmatory questions validate or clarify partial knowledge, often against diagrams or artefacts. These questions operate across two dimensions: level of abstraction (from high-level system overviews to detailed component behaviour) and architectural viewpoint (functionality versus interaction). Together, they show that architectural questioning is dynamic and situational: explainees navigate between broad context and fine-grained detail, using both exploration and confirmation to build, test, and refine their understanding.

3.4.5 Expectations and Challenges

Participants shared diverse expectations and challenges concerning both the process and outcomes of architectural explanations. These reveal what participants aim to achieve, how they assess explanation quality, and what difficulties arise. To answer RQ3.5, we structure findings across three dimensions: expectations during the explanation, expectations after, and challenges encountered during the process. Table 3.8 shows the dimensions and themes around this RQ.

Table 3.8: Dimensions and themes revolving around RQ3.5.

RQ3.5 Expectations and Challenges			
Dimension	Theme	E	e
Expectations During Explanation	Good system documentation		✓
	Knowledgeable explainer		✓
	Actionable closure	✓	
Expectations After Explanation	System and organisation knowledge	✓	✓
	Business–architecture alignment	✓	✓
	Working independence	✓	✓
	Improved process and results	✓	
Challenges	Manage multiple levels of abstraction	✓	✓
	Relevance to task-at-hand	✓	✓
	Unknown knowledge	✓	
	Outsourced services	✓	
	Work ownership	✓	✓

Expectations During Explanation

Expectations during explanations relate to both the quality of the process and the produced outputs. While explainees focus on explanation quality and supporting documentation, explainers' expectations centre on the closure of the session.

Good system documentation. As outlined in Section 3.4.3, documentation may be referenced during explanations or used independently. **Explainees:** Explainees consistently expect documentation that is clear, detailed, up-to-date, and directly relevant (e_2 – e_4 , e_6). The absence, obsolescence, or inconsistency of documentation—particularly for architectural artefacts like

ADRs—is frequently cited as a barrier (E_4, e_3-e_8). Despite the recognised value, documentation is often incomplete or outdated due to the manual effort required to maintain it.

Knowledgeable explainer. **Explainees:** Explainee e_6 expect explanations to be clear, concise, well-structured, and delivered by explainers with sufficient domain knowledge. This expectation underlines the role of explainer credibility in fostering trust and effective knowledge transfer (Section 3.4.2, Section 3.4.1).

Actionable closure. **Explainers:** Explainers prioritise ending sessions with explicit agreements on next steps or follow-up actions (E_1, E_2, E_5, E_6, E_9): “Take a note and some actions that you need to do and, yeah, we wrap up with some agreement” (E_1). This reflects the role of architectural explanations within the broader software development feedback loop. Further research should assess how these closures affect development cycles.

Expectations After Explanation

Participants expect that explanations will deliver tangible outcomes. Four themes recur:

System and organisation knowledge. **Both:** Explanations are expected to provide system and organisational understanding. Most ($E_3, E_7, E_8, e_2, e_4, e_7$) anticipate gaining a high-level overview or “big picture”; a few (e_6, e_7) seek complete and detailed understanding, although this depends on context and complexity. Across the board, understanding system’s operational context is a baseline expectation.

Business–architecture alignment. **Both:** Both roles expect explanations to clarify how architectural choices serve specific business needs and constraints (E_5, e_5). Generic justifications are not considered sufficient: “It has to be ... this particular architecture is optimised for modifications of that type in this business context” (e_5). This underscores the importance of contextualisation and the need to bridge problem and solution spaces (see twin peaks model [150]).

Working independence. **Explainers:** Explainers (E_3, E_4, E_7) expect explainees to become self-sufficient, beginning with simple tasks post-explanation: *“The first step after explanation is to work on something . . . really simple and [then] build up from that”* (E_4). **Explainees:** e_4 echoes this expectation, but warns that assumptions of full autonomy may not always be realistic and can result in frustration.

Mutual refinement of understanding. **Explainers:** Some explainers note that explanations also refine their own understanding and can lead to improved architectural solutions: *“It’s not only that their internal model aligns to mine, it’s also that mine aligns to theirs . . . very often the outcome is actually something that is better than what we went into the discussion with . . . I am becoming a better architect”* (E_6). Such alignment fosters more effective collaboration and a shared vision.

Challenges

Certain questions and topics routinely complicate architecture explanations, often due to required cross-functional knowledge, organisational silos, or lack of direct control. Five challenges are prominent:

Manage multiple levels of abstraction. **Explainers:** Explainers (E_2 – E_4, E_9) must tailor the level of abstraction to the audience: some need a broad overview, others require technical detail for design or quality evaluation. **Explainees:** Explainees (e_7, e_2) agree that too much detail is overwhelming, while insufficient detail leaves gaps. As e_2 puts it: *“[The explanation] should consist of different levels . . . classes, components, packages, and the whole file structure.”* The responsibility is on the explainer to strike the right balance.

Relevance to task-at-hand. **Explainers:** Explainers (E_7, E_1, E_8) emphasise focusing content on the explainee’s current tasks, to avoid overload and maintain focus: *“What I’m considering when I’m explaining things is that it’s relevant for the tasks that they’re doing”* (E_7). **Explainees:** e_5 prefers explanations connected to concrete activities (e.g., implementing features or refactoring), reinforcing the opportunistic model of comprehension. Explainers once again bear the responsibility of curating content.

Unknown knowledge. **Explainers:** Explainers (E_3) may encounter questions outside their expertise, often due to team turnover or unclear ownership: *“They are really asking something that I don’t know. For example, something specific about [an] interfaces, but I may not know why it exists”* (E_3). Such gaps require effort to bridge, sometimes involving redirection to other experts.

Outsourced services. **Explainers:** Questions about cloud or other external services are difficult when the relevant systems are beyond the team’s control: *“We get questions [about things happening in] the cloud . . . which we don’t have under control”* (E_6). The organisation often acts as an intermediary, and incomplete knowledge can block full explanation.

Work ownership. **Explainers:** Unclear or missing ownership complicates explanations and maintenance (E_6). **Explainees:** Explainees (e_1, e_6) find it difficult to proceed when responsibility for system parts is ambiguous or monopolised by a single person: *“[You do not want to] become dependent on . . . one person knowing that information, because . . . some things will forever be unanswered.”* (e_6). Clear ownership and knowledge sharing are critical for continuity and learning.

RQ3.5 Answer: Explainees expect clear, structured explanations from knowledgeable explainers, supported by accurate and accessible documentation. They value relevance to their immediate tasks and aim to gain a contextual understanding of the system, its architecture, and its business alignment, ultimately enabling independent work. Explainers, in turn, focus on delivering actionable closure and see explanations as a means to refine their own understanding and architectural decisions. They face challenges: explainers must manage appropriate abstraction levels, curate relevant content, and navigate gaps in knowledge, especially around outsourced services or unclear ownership. Documentation issues further complicate the process for both parties.

3.5 Discussion

We now interpret and synthesise our empirical findings by first, presenting software architecture explanation through the Twin Peaks lens [150, 218]; second, introducing the Explanation Window framework as an integrative device to manage the complexity of explaining software architecture; and third, describing implications for practitioners. Throughout, we indicate how empirical themes and participant accounts motivated these theoretical contributions.

3.5.1 Explanation Through the Twin Peaks Lens

The Twin Peaks model, depicted in Figure 3.2, describes software development as the co-evolution of requirements (problem domain) and architecture (solution domain), where each peak informs and constrains the other through iterative refinement [150]. This framing is particularly relevant for software architecture explanations, which, based on our findings, require articulation of both structural design decisions and the rationale linking them to stakeholder needs and system constraints. Explanations therefore continuously jump across both peaks: they clarify how requirements motivate architectural choices and how architectural possibilities reshape requirements. Hereby, we ground some of our findings on the Twin Peaks model by addressing iterative alignment alongside cognitive and contextual constraints.

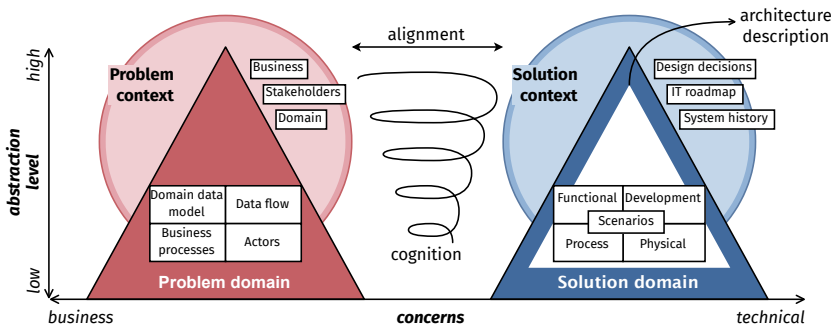


Figure 3.2: The Twin Peaks model: Software architecture explanation requires explicit alignment of business and technical context, including tacit background (the context “halos”) on both the problem and solution domains.

Iterative Alignment

A central empirical finding is that software architecture explanations operate as processes of *iterative alignment* between the mental models of explainers and explainees. This emerged from themes such as **business–architecture alignment**, **system and organisation knowledge**, and frequent mentions of the need to connect architectural choices with business constraints and stakeholder needs (see Section 3.4.1). This dual alignment is not static: several explainers reported refining their explanations in response to real-time feedback (**interactive explanation**, **provide/receive feedback**). This also reflects a two-way negotiation of understanding (**mutual refinement of understanding**).

The empirical data also reveal a recurring challenge: participants often encountered gaps in contextual knowledge, which we interpret as the “halos”, representing tacit contextual knowledge on both sides, that we added to the Twin Peaks model (Figure 3.2). These halos reflect the often unstated but influential background, such as organisational practices, compliance expectations, or domain conventions, which our participants repeatedly highlighted as shaping explanations. Reflecting our own experience and interpretation in light of our findings, business stakeholders rarely articulate the operational logic behind their needs; developers, in turn, often omit the rationale behind design choices, assuming technical knowledge. These tacit assumptions can quietly shape the explanation. Practitioners must actively surface these contextual dependencies. Otherwise, critical background may be misaligned or omitted in the explanation despite accurate factual transfer.

When explanations fail, they often expose underlying issues with the peaks-alignment, which manifests as undocumented system changes, unclear **work ownership**, or **unknown knowledge**. Thus, these failures act as diagnostic probes, revealing systemic issues. Viewed this way, explanations are not merely pedagogical events but continuous architectural alignment across roles and concerns.

Cognitive and Contextual Constraints

Empirical codes such as `manage multiple levels of abstraction`, `incremental explanation`, and `top-down model` directly support our identification of cognitive constraints. Participants consistently described the challenge of not overwhelming the audience, reporting strategies like starting broad before zooming into details. These concerns about information overload and adapting to the explainee's prior knowledge (`gain prior knowledge`) resonate with cognitive load theory [206] and theories of working memory [151], showing that explanation must be calibrated to avoid overload. The opportunistic shifts that we observed align with prior accounts of non-linear comprehension in software engineering [120, 128].

Additionally, task context was highlighted: Our initial assumption was that architecture explanation is primarily conceptual and abstract, distinct from task-driven, operational comprehension at the source code level. However, the data shows that explanations are often tightly coupled to the explainee's immediate goals (`relevance to task-at-hand`) and structured accordingly. Yet, calibration relied almost entirely on the explainer's judgment (e.g., reflecting the preferred comprehension model) instead of structured processes.

3.5.2 The Explanation Window

Building on the need for iterative alignment (Section 3.5.1) and the cognitive and contextual constraints (Section 3.5.1), we conceptualise the *Explanation Window* as a device to balance scope and depth in software architecture explanations.

We define the Explanation Window as a *rectangle* through which the system is viewed during an explanation. Its *height* corresponds to the depth of detail (ranging from high-level abstractions down to code-level specifics), while its *width* corresponds to the scope of functionality covered (ranging from a narrow slice of the system to a broad set of features). The window constrains the explanation: widening the scope necessarily reduces the feasible depth, while increasing depth requires narrowing the scope. Explanations remain cognitively manageable only when the “window area”, i.e., the product

of depth and scope, stays within practical limits yet to be defined through further investigation.

Figure 3.3 situates the Explanation Window on three hierarchical planes of abstraction: the *project level* (capturing entire projects and their components), the *component level* (showing modules within components), and the *module level* (breaking down to individual functions). Each plane provides a natural unit of reasoning, but only a bounded portion can be viewed at once. An architect may open a wide but shallow window at the project level to explain overall organisation and dependencies to business stakeholders. In contrast, when explaining to a developer trying to solve a bug, they may open a narrow but deep window at the module level to reason about the behaviour of specific functions, with only a small fraction of the broader architecture in view. This underlines that explanations are inherently selective: the same system can be framed differently depending on role, purpose, and the trade-off between scope and depth.

Figure 3.4 frames this trade-off between depth and scope more abstractly. The *x*-axis represents detail depth, the *y*-axis functionality coverage, and the curve bounds the region in which explanations remain effective. Deviating from the curve risks “lacking” information or “taxing” cognition.

The window also has a temporal dimension: its *trajectory*, i.e., how it moves across depth and scope during an explanation. These trajectories can follow different comprehension models. Top-down approaches begin with wide, shallow windows and gradually roll down into narrower, deeper ones; opportunistic approaches shift dynamically in response to tasks or questions; bottom-up and hybrid approaches, though less frequently observed, represent alternative ways the window can move.

These trajectories are further shaped by variables such as the explainee’s role (e.g., programmer vs. business stakeholder), familiarity with the project, and immediate task context (e.g., onboarding, debugging, integration). Each combination demands a differently sized and sequenced window: onboarding may require a wide but shallow rectangle; integration may require cross-cutting windows across planes; debugging concentrates on narrow, deep

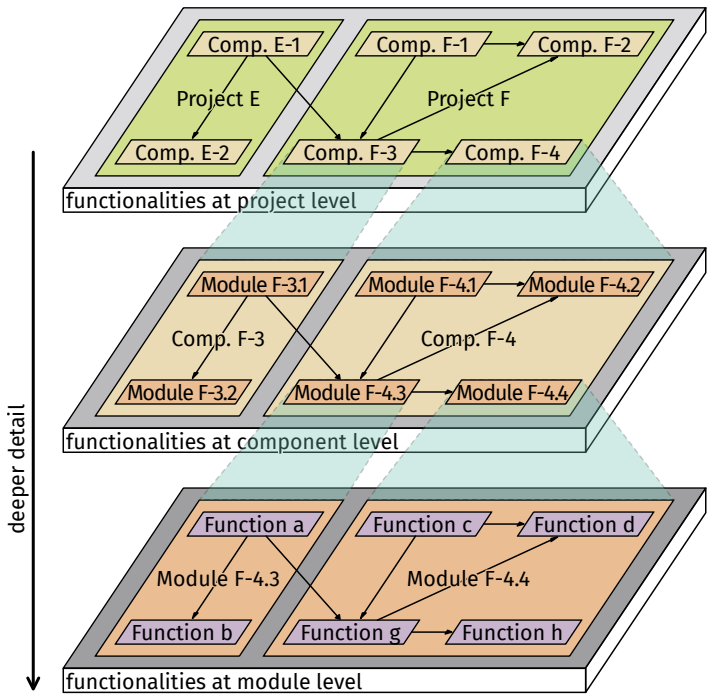


Figure 3.3: The Explanation Window can be situated on and across different planes (project, component, module, function) representing natural abstraction levels, constraining how much of the system is visible at once.

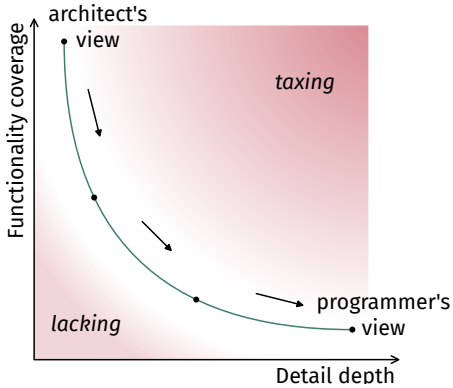


Figure 3.4: The trade-off between depth and scope: The Explanation Window occupies a position on a curve defined by detail depth (x-axis) and functionality coverage (y-axis). The effective region corresponds to rectangles whose area remains within cognitive limits.

windows. While these examples are interpretative illustrations rather than

empirical results, they clarify how the Explanation Window can be operationalised in practice as a flexible, adaptive guide for balancing scope, depth, and task relevance.

3.5.3 Implications for Practice

Our findings suggest several actionable implications for improving architectural explanations.

Plan explanations around scope and abstraction level: Participants' frequent challenges to **manage multiple levels of abstraction** imply the importance of deliberate planning. The interpretative Explanation Window concept directly addresses this empirically-supported need by providing a practical structure for adapting explanations to the explainee's role, task, and comprehension model.

Shift explanation towards joint reasoning: Participants explicitly valued interactive Q&A and feedback mechanisms (**interactive explanation, provide/receive feedback**). Explainers adjust their framing based on feedback; explainees challenge gaps or assumptions. This interactivity helps align mental models on both sides, and often improves the explainer's own grasp of the architecture.

Ensure actionable closure: Participants clearly expect explanations to lead to concrete actions (**actionable closure**). This not only orients explainees toward **working independence**, but also tests whether understanding was sufficient to proceed. Closure reinforces **relevance to task-at-hand** already implicit in most explanation requests.

Prioritise automated documentation: Participants explicitly identify documentation problems as frequent (**good system documentation**). Although participants do not explicitly label manual documentation untenable, repeated mentions of outdated and inconsistent documentation strongly support investment in automated solutions to maintain accuracy. Automated, extractive documentation, e.g., visualisation of current module dependencies, recent changes, or active interfaces, can ensure explanations are grounded in what the system actually is, not what it was.

Additionally, we lay the groundwork for knowledge criteria to be used in such automation. This study does not aim to define a complete set of criteria for automated explanations. However, it surfaces knowledge dimensions that are necessary (without claiming sufficiency) for this vision: coverage of multiple views, cross-abstraction navigation, explicit rationale, and separation of problem- and solution-domain concepts. These provide a foundation for future work to consolidate them into a more comprehensive set of knowledge criteria. Such criteria would then guide the design of suitable knowledge representations and support the evaluation of automated architectural explanations.

Ultimately, an effective architectural explanation involves balancing comprehensiveness with cognitive manageability. The Explanation Window provides both a conceptual and a practical lens for doing so. We believe that it can inform tools, practices, and training materials that are sensitive to the dual challenges of technical complexity and human cognition.

3.6 Trustworthiness

We assess the trustworthiness using the Total Quality Framework, which is a more suitable approach for qualitative studies than the threats-to-validity lens common in quantitative research, because it addresses quality holistically across the design, conduct, and interpretation of qualitative studies rather than reducing it to a binary validity judgment [166]. We consider the credibility, analysability, transparency, and usefulness facets.

3.6.1 Credibility

Credibility refers to the completeness and accuracy of the collected data. To strengthen it, novice participants first articulated their understanding of software architecture, ensuring suitability for the study and helping us contextualise later responses. We piloted and revised the interview guide to reduce ambiguity. To ground answers in real practice, we asked participants to reflect on their actual work contexts.

Because the study relied on retrospective self-report, it is subject to recall

bias and cannot fully capture naturalistic behaviour; participants may omit, reinterpret, or selectively recall experiences. We sought to mitigate this by asking them to anchor responses in recent, concrete work situations and by triangulating accounts across participants, but the lack of in situ data collection remains a limitation. Anonymity and data privacy were guaranteed to promote honest participation. Member checking was conducted by returning codes derived from each participant's transcript; only one participant requested a minor correction.

Our recruitment combined convenience, purposive, and snowball sampling. These approaches may introduce sample bias and limit diversity across organisations, industries, or cultures. Relying on professional and academic networks likely excluded practitioners from less connected environments. We did not employ paid online platforms such as Prolific for participant recruitment; our rationale is that such platforms rarely provide targeted access to practitioners in specialised software architecture roles, and quality control for professional background is difficult to guarantee. Although these services may suit studies targeting broad end-user populations, they were not suitable for our aim of in-depth, context-sensitive inquiry among experienced professionals. We acknowledge that this decision limits generalisability, and future research should combine sampling methods to address this gap.

Because key constructs can be interpreted differently in technical interview studies, we explicitly asked all novice or early-career participants to define "software architecture" prior to the main interview. This served two purposes: first, it confirmed that participants possessed a relevant baseline understanding; second, it allowed us to interpret their subsequent responses in light of their own conceptualisation. In doing so, we reduce the risk of credibility threats arising from inconsistent or assumed meanings of central terms. This practice makes explicit how participants' perspectives were anchored and how our interpretations accounted for their definitions.

3.6.2 Analysability

Analysability concerns the completeness and rigour of data analysis and interpretation. We followed the Gioia methodology, using systematic thematic coding with iterative development and refinement of codes. Four researchers participated in coding and interpretation, mitigating individual bias and increasing consistency. Interpretive disagreements were resolved through discussion and consensual decision rules. The analysis was anchored in established frameworks and relevant literature.

3.6.3 Transparency

Transparency is supported by detailed reporting of all methodological decisions, including the rationale for semi-structured interview design, pilot testing, stepwise coding, member checking, and participant selection. Anonymised transcripts and the codebook are publicly accessible for audit and secondary analysis.

3.6.4 Usefulness

Usefulness refers to the practical value and transferability of the findings. Our results provide grounded, practice-oriented insights into the realities of software architecture explanation, potentially relevant to professionals in similar roles or organisational contexts. However, our sampling was not random or exhaustive, so transferability is limited to contexts with similar characteristics. Future research should seek broader and more heterogeneous samples, potentially leveraging online professional communities or mixed-method sampling, to improve representativeness and robustness.

Because interviews were conducted remotely—often outside the immediate physical workspace of participants—our method does not maximise contextual realism as immersive ethnography or in situ participant observation would. We could not independently verify accounts against artefacts, live practices, or organisational context. As such, our findings should be interpreted as authentic perspectives of practitioners, not direct observations of behaviour. This necessarily restricts generalisability; transferability depends on the reader's judgment regarding similarity of their own context to those

reported here.

3.7 Conclusions

This study examined how professionals explain software architecture in practice, using semi-structured interviews with explainers and explainees from various organisations. We found that effective explanation goes beyond technical accuracy: it depends on the explainee's goals and role, the explainer's judgment, and the need to connect business context with system structure. Rather than being a purely abstract process, architecture explanation is grounded in practical needs, especially changing requirements, onboarding, and routine operational work.

Our findings show that explanations must cover both problem and solution domains, integrating stakeholder concerns, business rules, technical constraints, and implementation details. Rather than repeating documentation, effective explanations draw on diverse sources around the system such as business context (stakeholder interests, compliance), development history (past decisions, evolution), and system rationale. They often make explicit the relationships and traceability between these elements, providing context and justification that are rarely captured in a single artefact.

Managing the complexity of explanation is a central challenge. Practitioners address this by adapting abstraction levels and scoping content to the explainee's role and goals, often refining their approach interactively in response to feedback. Explanation thus emerges less as a one-way transfer than as collaborative, iterative reasoning.

To make sense of these practices, we introduced the Explanation Window as a conceptual tool to balance the scope and depth of explanations, and we related our findings to the Twin Peaks model. The Explanation Window frames how content is selected and presented according to task, role, and context, and its movement over time reflects varying comprehension strategies. These frameworks provide a foundation for structuring, diagnosing, and adapting explanations to manage both cognitive and contextual constraints.

Our study provides a first systematic characterisation of architectural explanation in industry, but we emphasise its exploratory nature. As a judge-

ment study based on self-reported accounts, our findings capture context-rich practitioner perspectives rather than naturalistic observations.

More importantly, these findings expose a set of requirements for any approach that aims to support or automate architecture explanation. Explanations must integrate heterogeneous sources of information, span multiple levels of abstraction, and maintain traceability between structural elements, design decisions, and contextual factors. They must also support flexible scoping, allowing relevant information to be selected and adapted to the needs of different stakeholders and tasks.

Addressing these requirements calls for a representation that can capture software systems in a unified, extensible, and analysable form, bridging low-level implementation details and higher-level architectural concepts.

Part II

Software System Representation

4

4

Unified Knowledge Graph Representation for Code and Architecture

The empirical findings in Chapter 3 show the need for a representation that integrates heterogeneous information sources, spans multiple abstraction levels, and supports traceability between code and architectural concepts. This chapter presents Software Architecture/Analysis Basic Ontology (SABO), a graph-based representation that unifies low-level code structure with high-level architectural abstraction. SABO enables scalable, architecture-aware analysis by modelling source-level entities, development artefacts, and analytical classifications in a semantically rich, architecture-aware knowledge graph. We define the design requirements for this representation, review existing ontologies and metamodels, and introduce SABO 2.0, a generalised, extensible model for multiple languages and paradigms. A worked example demonstrates its use in visualisation and analysis. SABO provides a foundation for reasoning, tooling, and intelligent assistants in modern software engineering.

4.1 Introduction

The increasing complexity of software systems presents serious challenges for developers seeking to understand, analyse, and maintain them. Manual and strictly rule-based analysis techniques no longer scale to large, evolving codebases. In response, the field has shifted toward heuristic and data-driven techniques that rely on extracted knowledge, often sourced from large corpora of code, to enable reasoning, tooling, and automation.

This shift raises a central representational challenge: how to bridge the gap between low-level implementation detail and higher-level architectural abstraction. Most existing tools and models are biased toward one of these extremes. At the low end, abstract syntax trees and control flow graphs capture fine-grained structural detail but lack architectural context. At the high end, architectural models omit critical implementation linkages, making them hard to validate or enforce. This disconnect hampers both human comprehension and automated reasoning, particularly in large systems where architectural intent is frequently lost over time.

To address this, we argue for a unified representation of software knowledge that serves as a foundation for scalable, architecture-aware analysis. The representation should be expressive enough to accommodate implementation and architecture together, while remaining flexible and extensible to evolving development practices. We aim for this representation and its corresponding fact-extraction tools to serve as building blocks in software analysis infrastructures [76, 88]. We adopt a knowledge graph approach to encode this representation, enabling integration of heterogeneous data sources and supporting both queries about the code elements and for higher-level reasoning, or even a combination thereof.

Our long-term vision is to support intelligent assistants for software development that can operate on this representation: answering questions, detecting inconsistencies, surfacing design patterns, or proposing improvements. As a step toward this vision, we introduce Software Architecture/Analysis Basic Ontology (SABO), a structured, multi-perspective framework designed to integrally model software systems from both code-centric and

architecture-aware viewpoints. This chapter presents the requirements that guide our design, reviews related work, and details the evolution and capabilities of SABO.

4.2 Requirements for A Software Knowledge Representation

To support architecture-aware analysis at scale, a software knowledge representation must meet a set of requirements. These requirements derive from the central challenge outlined in the introduction: bridging the semantic gap between low-level implementation and high-level architectural understanding. They also reflect practical demands of modern software analysis workflows, including coping with heterogeneous technology stacks, tolerating incomplete or uneven architecture-to-implementation mappings, and remaining extensible enough to accommodate analyses that cannot be anticipated in advance.

The following six criteria form the basis for evaluating existing approaches and inform the design of our proposed representation.

R1 – Multiple levels of abstraction: The representation must support both abstract (i.e., architectural) and concrete (i.e., implementation) perspectives. It should enable architectural reasoning without discarding implementation details and allow architectural elements to be traceably linked to their corresponding implementation artefacts. This duality is essential to connect design intent with its realisation in software.

R2 – Rich relational semantics: It must capture a wide variety of relations among artefacts, beyond basic invocations or inheritance. Relations such as composition, variable usage, parameter and return types, and containment provide additional nuances that support understanding software functional structure and behaviour at scale. Moreover, a semantically rich model supports more expressive queries and meaningful architectural interpretation.

R3 – Language-agnosticism: The model should generalise across programming languages and paradigms. Although early implementations may focus on specific ecosystems, the underlying schema must remain extensible and avoid language-specific assumptions or encodings. This also caters to

representing (and reasoning about) heterogeneous implementations (“programming cocktails” [42]) and technology evolution.

R4 – *Partial and evolving knowledge*: The representation must support reasoning about incomplete or evolving information. Many software systems are under-documented, inconsistently structured, or subject to constant change. Unlike formal modelling approaches that require closed and complete systems, our setting demands robustness in the face of uncertainty and the ability to incrementally enrich the knowledge base.

4

R5 – *Multi-source integration*: Software knowledge is scattered across code, documentation, models, configuration, and runtime observations. The representation must accommodate these sources and allow them to be integrated within a coherent semantic framework. This enables richer analysis pipelines that leverage diverse evidence.

R6 – *Extensibility and enrichment*: The representation must remain open to incorporating future analysis results, emerging design concepts, or domain-specific abstractions. Examples include roles, responsibilities, architectural layers, or metrics derived from code analysis. The model should serve as a living artefact that evolves alongside the software system and the tooling that supports it. While R4 concerns gaps or uncertainties within known categories of information, R6 addresses the ability to incorporate entirely new categories that may emerge in the future. In other words, R4 handles uncertainty within known concepts; R6 enables expansion to new concepts.

These requirements frame our evaluation of existing software ontologies and metamodels and provide the rationale for the design of SABO presented in the following sections.

4.3 Related Work

Numerous models have been proposed to represent software systems, ranging from source-level metamodels and industrial toolchains to high-level ontologies and architecture standards. This section reviews representative approaches across these categories, focusing on how well they meet the six requirements introduced in Section 4.2, especially the ability to bridge

implementation details and architectural abstraction.

4.3.1 Software Metamodels and Code Representations

SEON: The Software Engineering Ontology Network (SEON) [24] offers a high-level, domain-agnostic ontology covering artefacts, stakeholders, activities, and resources in software engineering. However, its abstraction level limits its usefulness for reasoning about source code. It lacks constructs for representing code-level elements and their interrelations. SEON is suitable for conceptual modelling and meta-level analysis, but does not support traceability across architecture and implementation.

ISO/IEC/IEEE 42010:2022: The ISO 42010 standard [94] defines how to describe software and system architectures. It introduces concepts such as stakeholders, concerns, viewpoints, and correspondences, and distinguishes between the system architecture and its description. The 2022 update strengthens traceability and formalises view correspondences. However, the standard remains intentionally abstract and does not specify mechanisms for linking architecture with source-level artefacts, limiting its applicability for automated analysis.

FAMIX: FAMIX [50], the foundation of the Moose reengineering platform, is a language-independent metamodel focused on object-oriented code. It defines entities like classes, methods, and attributes, along with relations such as accesses and invocations. FAMIX 3.0 improves clarity and extensibility, but remains limited to structural code representation. It lacks architectural constructs, role annotations, and stakeholder viewpoints. Our work builds on the core concepts of FAMIX, extending them beyond the object-oriented paradigm and introducing higher-level abstractions for architectural analysis.

Rascal M3: The M3 metamodel [106] in the Rascal framework separates language-specific syntax trees from a common relational layer that encodes semantic facts. It supports cross-language analysis and uses URIs to reference code elements. While powerful for capturing static structure and dependencies, M3 lacks a unified abstraction hierarchy and provides no consistent semantic model across languages. The set of captured relations is tied to the

input language, limiting generalisation.

CodeOntology: CodeOntology [12] translates Java code into RDF triples using an OWL-based ontology, enabling semantic queries via SPARQL [38]. It offers detailed representation of Java-specific constructs but does not generalise to other languages or support architectural reasoning. Its scope is restricted to object-oriented features and documentation within the Java ecosystem.

srcML and src2neo: srcML [36] converts source code into XML format while preserving original structure and formatting. It captures syntactic detail for languages like C, C++, Java, and C#, but lacks semantic resolution, e.g., it does not resolve identifier references or types. src2neo [87] builds on srcML to generate Neo4j graphs from C# code, but the resulting graph remains bound to syntactic constructs and does not support architectural modelling or abstraction.

Renaissance: Renaissance [116] is an industrial tool for analysis and transformation of Ada, C, C++, and C# codebases. It uses custom extractors per language and defines a schema per deployment. While effective for static analysis and transformation, this approach limits schema reuse and interoperability. It is not designed for unified architectural modelling or long-term knowledge integration.

Graph model by Mens and Lanza: Mens and Lanza [139] proposed a graph-based code model that includes method calls, line numbers, and other structural facts. However, it does not capture higher-level groupings such as modules or namespaces, and lacks extensibility mechanisms for role or architectural classification.

4.3.2 Architecture Recovery and Related Tools

SAR intermediate representations: Early work in software architecture recovery (SAR) [74, 213] used dependency graphs to extract views from source code, often limited to call graphs or inheritance. DP-CORE [46] advances this by detecting design patterns through combined structural and behavioural analysis. However, these intermediate models are task-specific

Table 4.1: Comparison of related approaches against our six design requirements (R1–R6).

Approach	R1	R2	R3	R4	R5	R6
SEON [24]	×	△	✓	△	✓	△
ISO 42010 [94]	×	△	✓	△	✓	△
FAMIX [50]	✓	✓	✓	△	×	△
Rascal M3 [106]	✓	✓	△	✓	△	✓
CodeOntology [12]	×	△	×	×	×	✓
SABO 2.0 (this work)	✓	✓	✓	✓	✓	✓

R1 – Dual abstraction; **R2** – Rich relational semantics; **R3** – Language-agnosticism; **R4** – Partial/evolving knowledge; **R5** – Multi-source integration; **R6** – Extensibility/enrichment.

and not designed for reuse, extension, or general architectural reasoning.

inspect4py: inspect4py [66] extracts facts from Python code to support comprehension. Its schema targets Python-specific structures and lacks support for architectural modelling or cross-language analysis.

SAIN: SAIN [76] is a platform for constructing modular software analysis pipelines. It is not a representation model or ontology, but rather an integration framework. SABO is designed to serve as a foundational, reusable component within such environments, providing structured, extensible knowledge that downstream tools can consume.

4.3.3 Comparison Summary

Table 4.1 summarises how the reviewed approaches align with the six design requirements introduced in Section 4.2. A check mark (✓) indicates explicit and comprehensive support for a requirement; a triangle (△) indicates partial or incidental support; and a cross (×) indicates little or no coverage. These assessments reflect our interpretation based on publications and documentation; applicability may vary depending on deployment context.

4.4 Software Architecture/Analysis Basic Ontology (SABO)

SABO is a structured, graph-based representation of software systems, designed to support automated analysis at both the implementation and architectural levels. It encodes core software entities and relations in a form suitable for reasoning, visualisation, and integration with analysis pipelines.

SABO uses a Labelled Property Graph (LPG) model, where nodes and edges can have labels and key–value properties. In contrast, the Resource Description Framework (RDF) represents data as triples (subject, predicate, object), with edges as labelled relations and most properties encoded as additional nodes or triples [91].

We prefer LPGs to RDF for practical and semantic reasons. Many software facts (e.g., a method’s name or source location) are more naturally properties of an entity than standalone nodes, whereas RDF’s triple model adds indirection and inflates the graph. RDF also lacks native edge properties, requiring reification or extra triples. LPGs instead allow each edge to carry key–value metadata, supporting richer relational annotations. Finally, the Cypher query language [70], widely used for property graphs, is often seen as more accessible and expressive than SPARQL [110].¹ Cypher’s pattern-centric syntax matches common software queries (e.g., call-graph or dependency traversals), making it more intuitive for developers than SPARQL’s triple-based logic.

The remainder of this section introduces SABO in two stages. We first describe the original ontology (SABO 1.0), which focused on object-oriented systems and fine-grained structural representation. We then present SABO 2.0, an expanded and generalised model that introduces support for non-object-oriented paradigms, explicit development-level structure, and integration of architectural analysis results.

4.4.1 SABO 1.0: Original Ontology

Our first ontology, introduced in [169] and retroactively named SABO 1.0, models source-level software structure across two levels of abstraction. The detailed graph schema (Figure 4.1a) encodes individual entities such as types, methods, fields, and their relations. The abstracted graph schema (Figure 4.1b) collapses these into higher-level concepts, i.e., containers and types, deriving simplified edges based on structural patterns.

The ontology defines node types such as **Type**, **Operation**, **Variable**,

¹<https://news.ycombinator.com/item?id=21004610>, accessed 2023-01-25

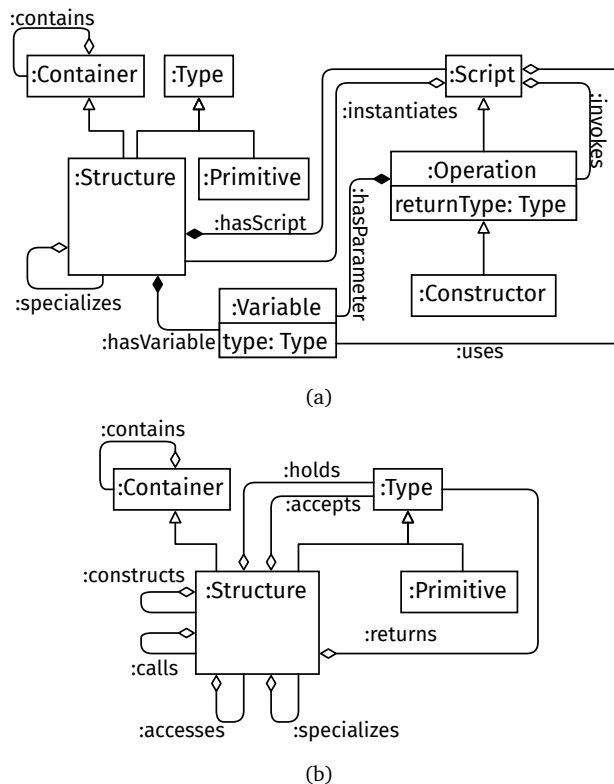


Figure 4.1: Detailed (a) and abstracted (b) versions of SABO 1.0 ontology. The detailed view encodes fine-grained code elements; the abstracted view aggregates them into a coarser, architecture-level graph used for higher-level reasoning.

Container, and **Primitive**, and edges like *invokes*, *constructs*, *accesses*, and *specializes*. (We refer to the original publication [169] for detailed definitions of the node and edge labels.) This ontology supports use cases such as design pattern recognition, role classification, and high-level dependency queries, while having precise linking (in the detailed schema) to code artefacts.

SABO 1.0 is focused toward the object-oriented programming paradigm, with partial accommodation for structured and functional programming constructs. Although it served effectively in earlier analysis pipelines, several limitations emerged, particularly around paradigm generality, architectural extensibility, and traceability from development artefacts. These motivate the improvements introduced in SABO 2.0.

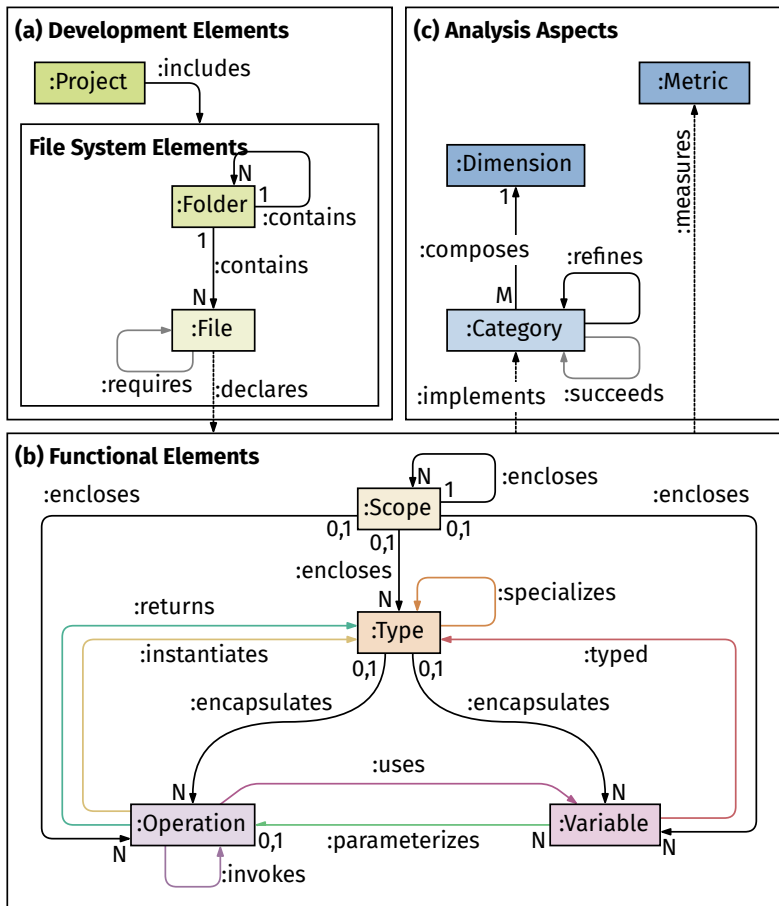
4

4.4.2 SABO 2.0

SABO 2.0 introduces three primary extensions to address the shortcomings of its predecessor:

- (1) *Support for non-object-oriented paradigms*: The ontology no longer assumes that variables and operations are scoped exclusively within class-like structures, making it applicable to procedural and functional languages.
- (2) *Explicit representation of development structure*: SABO now distinguishes between the physical hierarchy of development artefacts (e.g., folders, files) and the logical structure defined by the programming language (e.g., packages, types); this clarifies provenance and supports development view-aware reasoning.
- (3) *Analysis and architectural integration*: A new ontology layer we call *analysis aspects* supports classification and grouping based on external analyses, enabling dynamic architectural interpretation grounded in implementation data.

SABO 2.0 is thus structured around three conceptual groups: development elements, functional elements, and analysis aspects. See Tables 4.2 and 4.3 for the enumeration of the node and edge labels in each group.



Unwritten multiplicity means $M \rightarrow N$.

Figure 4.2: Version 2.0 of SABO, including development elements (a), functional elements (b), and analysis aspects (c).

Table 4.2: Node labels in SABO 2.0 with descriptions.

Label	Description
Development Elements	
Project	Root container of a software system.
Folder	Directory; hierarchical grouping of files.
File	Individual file.
Functional Elements	
Scope	Namespace-like scope (e.g., Java package, C# namespace).
Type	Data type (class, struct, enum, etc.).
Operation	Function, method, or procedure.
Variable	Field, parameter, or global or local variable.
Analysis Aspects	
Dimension	Analytical axis (e.g., role stereotype, architectural layer).
Category	Classification within a dimension.
Metric	Quantitative property (e.g., number of methods).

Development elements: This group models the physical organisation of software within the development environment, reflecting the development view in Kruchten [112]’s 4+1 model and grounding all higher-level constructs in concrete artefacts.

Functional elements: This group captures the logical structure of the program as expressed in its source language. All functional elements are linked back to their source location, enabling traceability between development and logical structure. This corresponds with the logical view in the 4+1 model [112].

Analysis aspects: This group introduces support for interpretation and classification of code elements based on analytical results. This layer enables the mapping of implementation-level entities into architectural abstractions and supports dynamic aggregation across abstraction levels, making it a key mechanism for architecture-aware tooling.

Lifted or composed relations: In SABO 1.0, the abstracted graph was systematically derived from the detailed graph by lifting or composing lower-level relations into higher-level hyper-edges. These lifted edges were given

Table 4.3: Edge labels in SABO 2.0 with source, target, and definitions.

Label	Source	Target	Definition
Development Elements			
<i>includes</i>	Project	Folder	Project includes Folder.
<i>contains</i>	Folder	Folder/File	Folder contains Folder or File (hierarchical containment).
<i>requires</i>	File	File	File statically depends on another File (include/import).
Functional Elements			
<i>declares</i>	File	(Any func. elmnt. node)	File declares a functional element.
<i>encloses</i>	Scope	(Any func. elmnt. node)	Logical scoping of identifiers.
<i>encapsulates</i>	Type	Variable/Operation	Data/object-structure encapsulation.
<i>specializes</i>	Type	Type	Type specialisation, e.g., inheritance or mix-ins.
<i>invokes</i>	Operation	Operation	Operation invocation by another Operation.
<i>returns</i>	Operation	Type	Return Type of an Operation.
<i>instantiates</i>	Operation	Type	Instantiation or object creation within an Operation.
<i>uses</i>	Operation	Variable	Operation uses Variable.
<i>parameterizes</i>	Operation	Variable	Operation is parameterised by Variable.
<i>typed</i>	Variable	Type	Variable is of a certain Type.
Analysis Aspects			
<i>implements</i>	(Any func. elmnt. node)	Category	Functional element implements a Category.
<i>measures</i>	(Any func. elmnt. node)	Metric	Functional element measured by a Metric.

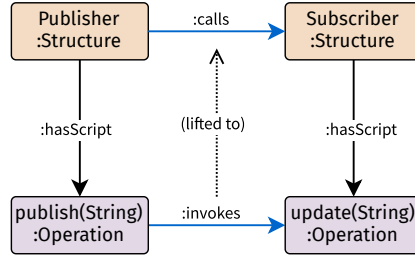


Figure 4.3: Example of *lifting* an edge from detailed-level “invokes” (between Operations) to abstracted-level “calls” (between Structures) in SABO 1.0.

4

dedicated labels to support specific analyses. In particular, those abstracted edges are defined as follows.

S holds T: When *Structure S* has a *Variable V* of *Type T*.

S accepts T: When *Structure S* has a *Script M* that has a parameter *P* of *Type T*.

S returns T: When *Structure S* has a *Script M* that returns an instance of *Type T*.

S constructs T: When *Structure S* has a *Script M* that invokes a *Constructor C* of *Structure T*.

S calls T: When *Structure S* has a *Script M* that invokes an *Operation N* of *Structure T*. Figure 4.3 illustrates this lifting, where *S* corresponds to *Publisher*, *M* to *publish(String)*, *N* to *update(String)*, and *T* to *Subscriber*.

S accesses T: When *Structure S* has a *Script M* that uses a *Variable V* of *Structure T*.

In SABO 2.0, we retain the concept of lifted relations, but leave their derivation and interpretation to downstream analysis processes or visualisation tools. The ontology itself does not prescribe a fixed set of lifted edge labels; instead, it supports their computation via graph traversal over the detailed representation. This delegation reduces ontology complexity and allows more flexible and context-specific lifting strategies.

4.5 Illustrative Graph Instance and Visualisation

To ground the concepts introduced in this chapter, we present two worked examples. The first one comes from our original publication and uses a Java software system JHotDraw. The second example uses a small and simple publish–subscribe design pattern demo program.

4.5.1 JHotDraw example

We demonstrate SABO 1.0 graph² instance extraction and two kinds of analysis on the graph: software architecture recovery and design pattern instance detection. We apply our graph extraction pipeline on JHotDraw version 5.1, a vector drawing application and framework developed to showcase design pattern usages. The JHotDraw project consists of 154 Java classes or class-like structures (i.e., interfaces and enumerations).

Software architecture recovery: Graph community detection algorithms can be used for software architecture recovery, where communities of classes correspond to cohesive components. We experiment with the Leiden technique [211] on a SABO 1.0 abstract graph instance describing JHotDraw and identify 14 components that by manual inspection seem to contain semantically related classes. For example, one component contains application controller- and corresponding GUI-classes, while another component contains classes related to data structure and related persistence classes. We find that patterns in the relations correspond to architectural layers and functional decomposition. For instance, the *constructs* relations prominently come from a higher architectural layer while *calls* and *accepts* usually occur within a layer. However, this is only an initial experiment and requires further refinement and proper evaluation.

Design pattern detection: Existing techniques can be used to detect design patterns by analysing patterns of relation types between classes [46, 160, 185]. We apply such analysis to our language-agnostic representation using

²We use SABO 1.0 version for this example solely because it provides explicit labels for abstracted relationships, which makes raw (sub)graphs more legible. SABO 2.0 is more expressive and allows for more complex queries, but it is less user-presentable in its raw format.

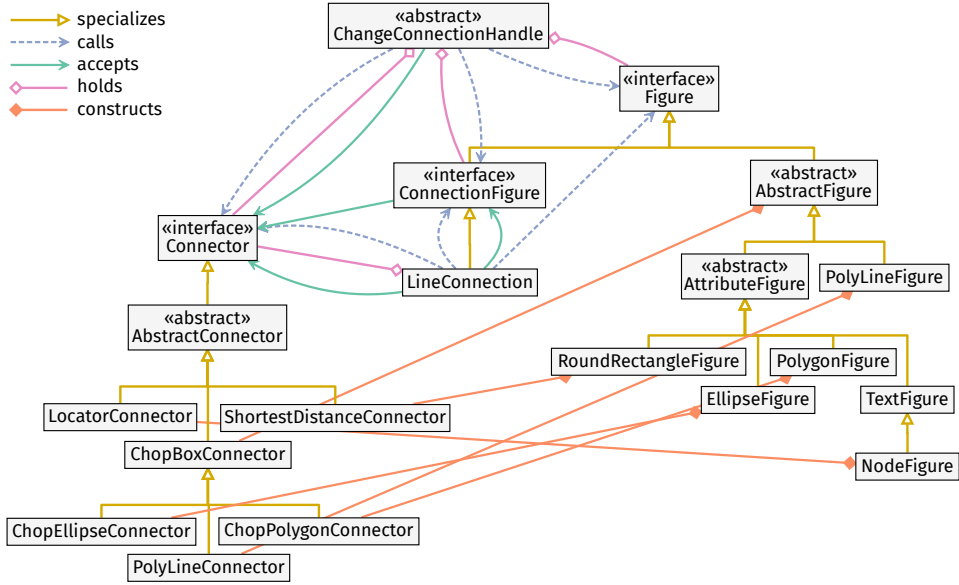


Figure 4.4: A subset of the abstract knowledge graph instance extracted from JHotDraw, depicting classes involved in a strategy design pattern, presented in UML-like styling.

Cypher queries. To illustrate an analysis on our knowledge graph, we start with ground truth information on a design pattern instance in JHotDraw from the P-MARt repository [81]. We select an interface `Connector` that acts as an abstract *strategy interface* in a Strategy pattern instance. We then use our interactive visualisation tool (cf. Section 6.3) to inspect the neighbouring nodes connected with edge types relevant to a Strategy pattern, resulting in a sub-graph depicting this particular pattern instance (Fig. 4.4). The figure is stylised using familiar UML notations.

The classes `ChangeConnectionHandle`, `ConnectionFigure`, and `LineConnection` act as the context that accepts a reference to a `Connector` and calls its methods. The class `Figure` provides an interface that allows the strategy to access its data. Concrete strategies (a group of `Connector` subclasses) are constructed based on the concrete `Figure` involved (a group of `AbstractFigure` subclasses). We cross-check this finding with the ground truth and find that we discover 16 out of 19 classes that experts manually identified to belong to this pattern instance.

Based on the relation structure in this example, we construct a Cypher query (shown in Listing 4.1) as a starting point for detecting instances of the Strategy design pattern in SABO 1.0 graphs.

This query identifies potential *context* classes *c* that are structurally connected to an abstract *strategy* class *s* via three types of relations: *c* may accept *s* as a parameter, call one of its methods, or hold it as a field. The strategy class *s* is specialised by one or more concrete strategy classes *cs*, which are in turn instantiated by an implementation of an interface *i*—a reverse construction chain is used to capture this pattern.

Listing 4.1: A Cypher query for detecting strategy pattern instances in a SABO 1.0 graph.

```

1 MATCH p = (c)-[:accepts]->(s)<-[:specializes*1..]-(cs)
2   <-[:constructs]-()-[:specializes*]->(i)<-[:holds*]-(c),
3   q=(c)-[:calls]->(s),
4   r=(c)-[:holds]->(s)
5
6 WHERE (i)<-[:calls|holds]-()-[:specializes*]-(cs)
7
8 RETURN p,q,r

```

The query does not aim to capture all Strategy instances precisely. Rather, it provides an initial structural hypothesis that can be iteratively refined: querying the graph, inspecting candidate matches, and adjusting constraints to improve precision and reduce false positives.

Quality attribute reasoning: Once design patterns are annotated in the code, a logical next step is to reason about quality attributes by tracing their structural and semantic underpinnings. For example, Kazman et al. [104] describes *maintainability* as being supported by architectural tactics such as *defer binding* and *encapsulate*, both of which are commonly employed by the *strategy* design pattern. These tactics, in turn, influence measurable properties such as *test efficiency* and *coupling*: defer binding may hinder test efficiency, but it can also reduce coupling.

These conceptual relationships can be explicitly encoded in the knowledge graph as typed nodes and edges—design pattern roles and instances, the tactics they employ, the metrics they affect, and their relation to higher-level

quality attributes. Figure 4.5 shows a worked example of this: a Strategy instance in JHotDraw (as represented in a SABO 2.0 (sub)graph) is linked to maintainability through tactics and metrics. This allows us to trace how local design decisions may support or hinder desired quality attributes, providing a concrete foundation for quality reasoning grounded in structural evidence.

4.5.2 Publish–subscribe example

This example uses a simplistic publish–subscribe (pubsub) demo program in Java consisting of 4 classes and 8 operations (6 methods and 2 constructors). Figures 4.6 and 4.7 illustrate how SABO 2.0 represents the structural and analytical aspects of this demo program in an integrated form.

Figure 4.6a shows the development structure: a project that includes a few folders and files, linked to their declared functional elements. Figure 4.6b shows the functional structure: classes, methods, variables, and their logical enclosures. This illustrates how SABO enables traceability between functional and physical perspectives.

In Figure 4.7a, each **Type** node is enriched with analysis results: class role-stereotype [222] classification (e.g., PubSubDemo is a *Controller* class), dependency profile [26] (e.g., Subscriber is an *Inbound* class, i.e., it is being used but not using classes from another package), and metrics (e.g., method count). These are modelled as **Categories** within **Dimensions** and **Metrics**, showcasing SABO’s extensibility for architecture-aware analysis.

Figure 4.7b illustrates a visualisation mockup using the same graph data. Type enclosure hierarchy defines grouping; architectural roles define horizontal placement; dependency profiles define vertical placement; and the number of methods defines colour. This demonstrates how a unified, semantically rich representation enables expressive, multidimensional views.

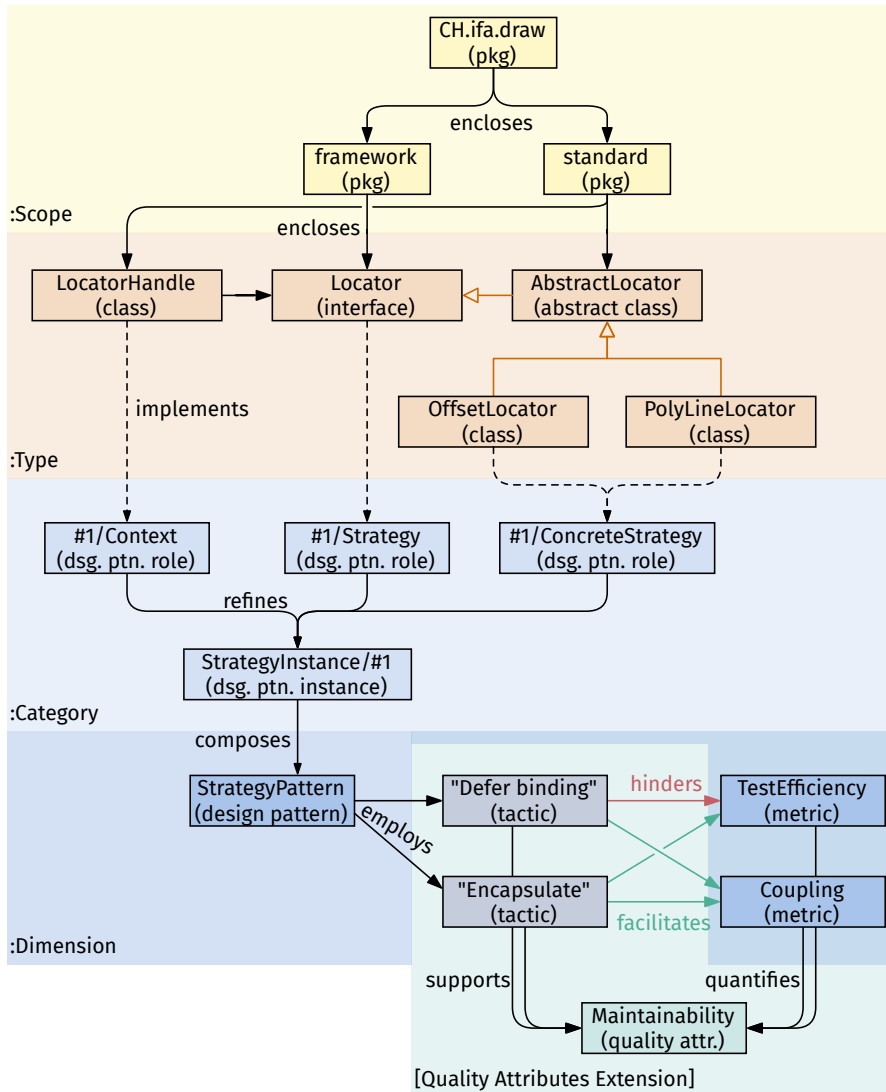
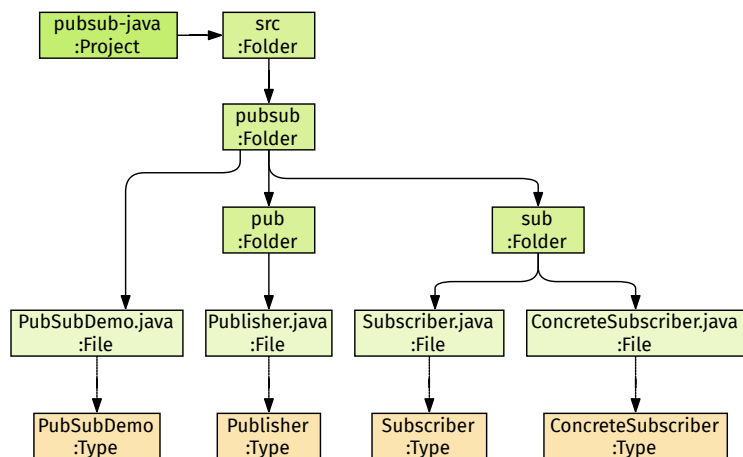
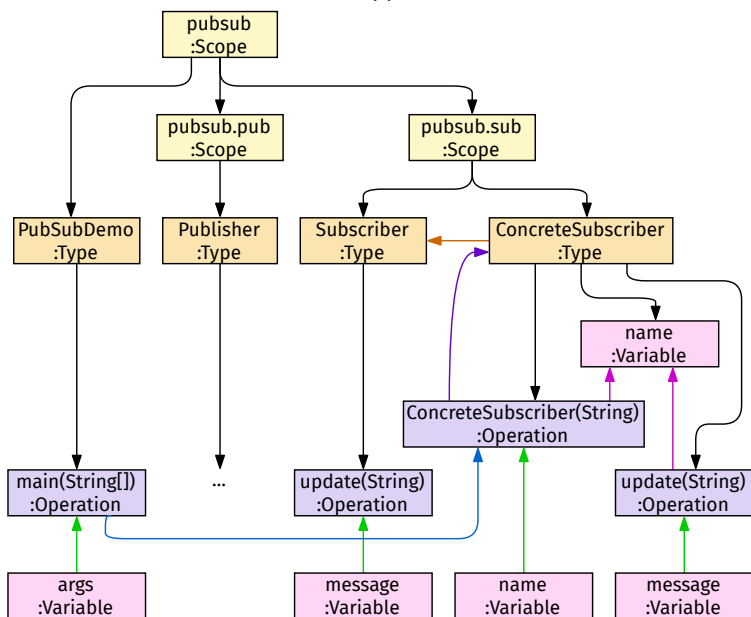


Figure 4.5: Some class-level elements (and their enclosing packages) from the JHotDraw project that are involved in a “strategy” design pattern instance. We can use this information to reason about the codebase’s maintainability, for instance.

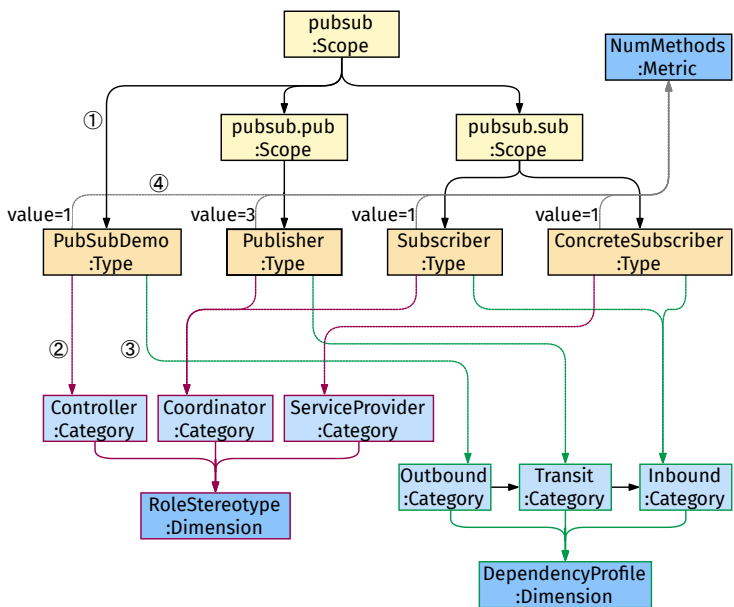


(a)

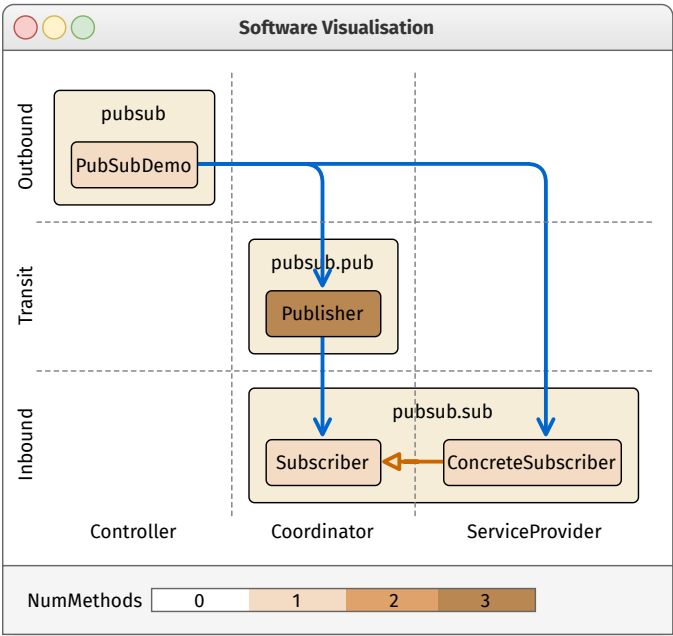


(b)

Figure 4.6: Slices of a SABO graph instance, depicting (a) development structure and (b) functional structure of the pubsub demo.



(a) Annotated graph with ① functional hierarchy, ② class role stereotype, ③ dependency profile category, and ④ “number of methods” metric.



(b) Visualisation mockup using dimensions as visual variables.

Figure 4.7: Analysis (a) annotation and (b) visualisation of the same graph instance.

4.6 Ontology Instantiation

This section details the instantiation process of the SABO ontology from source code artefacts, highlighting the practical considerations and technical workflow involved. The extraction pipeline is designed for language-agnostic operation, yet requires language-specific parsing and mapping logic to populate the graph with accurate and semantically meaningful nodes and edges.

We begin with an overview of the end-to-end extraction process and the input/output formats used. This is followed by a summary of current language support and extraction coverage. The actual mapping of SABO labels to language constructs can be seen in Appendix A.

4

4.6.1 Implementation Architecture

The typical instantiation pipeline comprises four main stages:

1. **Parsing:** The source code is parsed into a syntax tree (e.g., an abstract syntax tree or similar intermediate form) using a language-specific parser. This structure captures the syntactic hierarchy of the program.
2. **Name resolution:** The syntax tree is analysed to resolve names and types where applicable. This phase disambiguates identifiers, resolves references across scopes, and constructs a name table mapping symbols to their declarations.
3. **Graph construction:** The extractor traverses the syntax tree while consulting the name table to construct the SABO graph. During this phase, program entities and their relationships are mapped to typed nodes and edges, accurately reflecting both structural and semantic connections.
4. **Output encoding:** The resulting SABO graph is serialised, typically in a JSON-based format. This format extends Cytoscape JSON to include explicit labels for node and edge types, and arbitrary key-value properties, supporting downstream tasks such as visualisation, analysis, and integration.

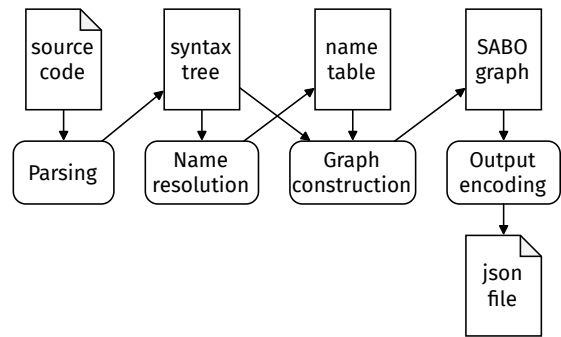


Figure 4.8: The SABO extraction pipeline parses source code, resolves names, constructs the graph using the syntax tree and name table, and outputs a serialised graph.

Table 4.4: Overview of language support and extraction tooling for SABO instantiation.

Language	Analysis Framework	Coverage	SABO Versions	Notes
Java	Spoon	High	1, 2	GitHub: rsatrioadi/javapers
C#	Roslyn/Microsoft.CodeAnalysis	High	2	GitHub: rsatrioadi/csharpers
C/C++	Rascal/M3 model	Partial	1	GitHub: Software-Analytics-Visualisation-Team/M3GraphBuilder
JavaScript	acorn	Partial	2	GitHub: rsatrioadi/jspers
Go	go/ast, go/types, go/packages, gopls	High	2	GitHub: rayhanp1402/gophers
Simulink	srcML	Partial	1	GitHub: Software-Analytics-Visualisation-Team/SimulinkParser

Figure 4.8 shows this pipeline. The entry point for extraction is typically a directory path or project root; the output is a single graph file per codebase, ready for ingestion by property graph databases or analysis tooling.

4.6.2 Language Support Overview

Extraction tooling currently targets the following programming languages: Java, C#, C/C++ , JavaScript, Go, and Simulink. For each, we have implemented dedicated extraction modules built atop existing analysis frameworks. Table 4.4 summarises the state of extraction support, including core frameworks and coverage notes.

For languages not explicitly supported, the architecture is designed to enable rapid addition of new language modules, provided that an AST and minimal semantic analysis can be performed.

4.6.3 Limitations and Challenges

Several technical limitations and open challenges remain:

Incomplete resolution: Static extraction cannot always resolve dynamic behaviour (e.g., reflection, dynamic imports, late binding), which may result in incomplete call graphs or type hierarchies.

Cross-language constructs: Systems employing polyglot architectures or foreign function interfaces present challenges in tracing relations across language boundaries.

Generated/preprocessed code: Handling of generated sources, preprocessor macros (notably in C/C++), or post-compilation artefacts is inherently brittle.

Dynamic code loading: Languages such as JavaScript permit runtime code generation or injection, which cannot be fully captured by static analysis.

Parser/tool limitations: Extraction quality is gated by the underlying parser's correctness, ability to recover from malformed input, and support for evolving language features.

These limitations are mitigated where possible by fallback heuristics, explicit markers for unresolved entities, and careful documentation of extraction coverage.

4.7 Summary

This chapter has introduced SABO, a graph-based ontology intended to connect low-level code structure with higher-level architectural abstraction. We formulated six key requirements for representing software knowledge, surveyed representative related work, and explained the design and evolution of SABO using illustrative examples and schema tables.

The extraction and instantiation pipeline supports multiple programming languages through a modular, extensible design. In SABO 2.0, node and

edge labels cover both development-oriented and functional concepts, and allow analysis results to be attached in a flexible manner, thereby enabling architecture-aware reasoning and tooling.

Despite the progress in integrating code and architecture, there are still limitations related to dynamic behaviour, polyglot systems, and generated artefacts. These open issues point to promising avenues for future research.

The following chapters build on SABO as a foundational representation for downstream analysis and explanation. However, representation alone does not yield architectural insight: the structural information captured in the graph must be interpreted, classified, and related to higher-level architectural concepts.

The next step, therefore, is to enrich this representation with semantic information that reflects how practitioners reason about systems, such as responsibilities, layers, and architectural roles.

Acknowledgments

We would like to thank Filip Zamfirov and Matteo Asuni for their contributions in implementing the C/C++ extractor, Rayhan H.M. Pradana for the Go extractor, and Felipe de Azeredo Coutinho Xavier for the Simulink extractor.



Part III

Enrichment

5

Evaluating Explanatory Artefacts of DSAR-Recovered Software Architectures from Industrial Codebases

5

Building on the graph-based representation introduced in Chapter 4, the next step towards automated architecture explanation is to enrich structural information with architectural meaning. Understanding large, complex software architectures is difficult and time-consuming. Software architecture recovery aims to extract explanatory artefacts from code, but existing methods often lack generalisability. To address this, we previously proposed Deductive Software Architecture Recovery (DSAR) to extend current techniques. This paper presents a qualitative evaluation of DSAR in an industrial setting. Using a large language model-assisted prototype, we examined the recovered architecture's content quality, presentation quality, and usefulness for understanding and maintaining real-world systems. To ensure practical relevance, we conducted an in-depth judgement study with software engineering teams at ASML applying DSAR to 17 Java repositories across diverse teams. Our contributions are: (1) an empirical assessment indicating that DSAR supports comprehension, architectural

reasoning, and communication; (2) qualitative evidence on the strengths and limitations of the generated explanatory artefacts; and (3) recommendations for practitioners and researchers on adopting and further developing DSAR for architectural sense-making. These contributions can aid developers in practice and suggest directions for broader research validation.

5.1 Introduction

Understanding the architecture of a software system from its source code is a difficult and time-consuming task [121, 156]. Although source code represents a system's definitive state and program comprehension often focuses on it [180], architectural documentation and diagrams can aid developers in understanding a system [179, 229]. Unfortunately, even when such artefacts exist, they tend to be outdated, incomplete, or inconsistent with the source code [67, 114, 148, 198, 227]. This leaves developers without reliable means to understand, maintain, or evolve complex systems [2, 59, 214].

Software Architecture Recovery (SAR) has been proposed as a means to reconstruct a system's architecture when documentation is missing or unreliable [127]. SAR aims to extract architectural insights from source code to enhance documentation [48, 207], but state-of-the-art techniques often lack generalisability: They are tied to a particular architecture or technology, limiting their reuse across systems [130, 207]. Most SAR techniques employ what we call *inductive recovery*, reconstructing architectural structures by clustering code-level facts like entities, dependencies, or textual artefacts [17, 19, 56, 75, 176, 225].

To complement such SAR techniques, we introduced *Deductive SAR* (DSAR), which interprets source code using a natural-language description of a reference architecture [172]. The main output of DSAR is a set of what we call *explanatory artefacts*: visualisations and summaries of the recovered architecture and its entities, mapped to the reference architecture. These artefacts help programmers and architects navigate unfamiliar codebases using recovered architectural knowledge, without needing prior documentation.

Although promising, DSAR lacked systematic evaluation. This paper fills that gap with an empirical evaluation through a judgement study [202]

on DSAR use at ASML, a European semiconductor company. We evaluate the explanatory artefacts from our DSAR prototype, which integrates Large Language Model (LLM)-augmented code summarisation, architectural classification, and interactive visualisation. We adopted the Co-12 framework for explainable AI (XAI) [146], which treats explanation quality as a multifaceted concept spanning *content* (e.g., completeness, correctness, contrastivity), *presentation* (e.g., compactness, composition), and *user-related* dimensions (e.g., context, coherence). We thus define the following research questions (RQs):

RQ5.1 Content quality: *How do practitioners perceive the content quality of the DSAR explanatory artefacts?* If the artefacts omit critical details, contain errors, or are incongruent, they fail to effectively support developers' understanding.

RQ5.2 Presentation quality: *How do practitioners perceive the presentation quality of the DSAR explanatory artefacts?* Even accurate content can be unhelpful if poorly presented, so clear organisation and succinctness are key to making explanatory artefacts useful for different stakeholders.

RQ5.3 Usefulness: *How do practitioners judge the practical usefulness of the DSAR explanatory artefacts?* This question connects to the user-related dimension defined by Nauta et al. [146]. Explanatory artefacts must ultimately serve practical purposes, and examining how they fit into practitioners' tasks provides evidence of their utility.

We applied DSAR to several ASML code repositories (Section 5.3) and conducted semi-structured interviews with engineers to assess the explanatory artefacts' perceived content quality (RQ-1), presentation quality (RQ-2), and usefulness (RQ-3). We provide a replication package with the interview guide and resulting codebook [173]. Our results (Section 5.4) show that DSAR's effectiveness depends on codebase size and quality, as well as available domain context and dependency information. Overall, DSAR supports comprehension, architectural reasoning, and communication.

5.2 DSAR and Related Work

In this section, we revisit our DSAR method and related work.

5.2.1 Deductive Software Architecture Recovery

DSAR¹ implements a deductive recovery process by combining bottom-up extraction of code artefacts with a top-down application of architectural descriptions to infer architectural views [172]. Unlike inductive SAR approaches based solely on clustering or summarisation, DSAR interprets code using architectural knowledge provided as natural-language rules (e.g., stereotypes [222], layering rules) that define how higher-level responsibilities manifest at the code level. Henceforth, we use the language-agnostic terminology of Rukmono and Chaudron [169], which distinguishes *operations*, *structures*, and *containers* in software projects (e.g., methods, types, and packages in Java).

Figure 5.1 shows the components that implement DSAR. A *Static Analyser* extracts structural and dependency information from source code [169], stored in a graph representation. Predefined architectural indicators (e.g., for stereotypes, layers) are provided as to support the following actions. The *DSAR Core* then applies an *action* (e.g., summarise, classify) to an *object* (e.g., operation, structure) within a *context* derived from local code evidence and structural dependencies, producing natural-language summaries and architectural classifications. These resulting artefacts are reused for higher-level actions rather than raw code.

Figure 5.2 illustrates the cascading workflow induced by this setup. Actions are applied bottom-up (operations → structures → containers) and left-to-right to respect dependency order (e.g., *b* invokes *a*), so that intermediate summaries and classifications become context for later actions. Structure- and container-level classifications are obtained by aggregating lower-level classifications rather than re-running classification from scratch [170].

We now revisit DSAR as a conceptual framework with three interdependent pillars that underlie and operationalise this workflow: graph-based

¹DSAR was introduced in prior work [172]. We refer readers there for a full overview.

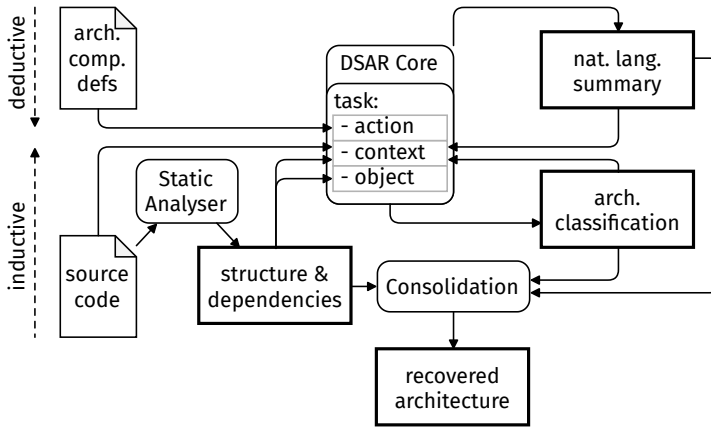


Figure 5.1: DSAR summarises and classifies code elements using structural context and architectural definitions. It integrates bottom-up code extraction with top-down architectural descriptions.

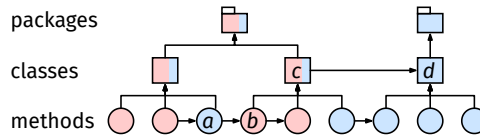


Figure 5.2: Cascading summarisation and classification in DSAR. Actions are applied bottom-up and left-to-right to respect dependency order.

representation, hierarchical summarisation, and deductive classification.

Graph-based representation: We use a programming language-agnostic Labelled Property Graph (LPG) to encode program entities (e.g., operations, structures) and their structural (e.g., containment, subtyping) and behavioural (e.g., invocation, data access) relations [169]. This graph supports *vertical aggregation*, which composes higher-level constructs from their contained elements, and *horizontal propagation*, which transfers context across peers (e.g., operation invocation).

Hierarchical summarisation: Using LPG traversals, DSAR performs bottom-up summarisation from operations to structures to containers. At the operation level, summaries use the operation’s source code plus its callees and accessed attributes. At the structure level, they use operation summaries, subtyping relations, and declared attributes. At the container level, summaries are aggregated over structures and subcontainers in topological order to avoid unresolved dependencies [170].

Deductive classification: DSAR integrates deductive classification into the workflow to infer architectural roles and responsibilities. Prior architectural descriptions, such as stereotypes or layering rules, guide the interpretation of code-level manifestations [172]. Combining code evidence with explicit architectural knowledge distinguishes DSAR from approaches based solely on implementation artefacts.

5.2.2 Related Work

Recent work on software architecture recovery (SAR) spans a broad spectrum of learning-based and heuristic techniques. Many approaches treat recovery as a classification problem, in which source-code artefacts are mapped to architectural modules based on lexical or structural features. For example, Johansson et al. [99] use large-language-model embeddings to associate code elements with architectural components defined in documentation. They report acceptable accuracy when there is strong lexical overlap between the architectural description and the implementation, but performance drops considerably when the vocabulary diverges. A complementary line of work by Jabeen et al. [96] employs graph convolutional networks to label code elements using dependency-graph information. These learning-based classifiers show promise but require representative training data and may struggle to generalise across domains.

Other researchers favour rule-based heuristics to improve scalability and reduce dependence on training data. Amalfitano et al. [5] describe a semi-automated design-recovery pipeline for embedded C systems that leverages domain-specific patterns and folder structures to infer component diagrams. Their approach produces complete architectural views for large codebases but remains tightly coupled to the conventions of the target domain. Duarte [49] proposes MicroPAD, a tool that detects microservice pattern instances from Infrastructure-as-Code artefacts by querying large language models. In early experiments on 22 GitHub projects, MicroPAD correctly identifies around 83 % of pattern instances with minimal cost; however, the prototype currently supports only a handful of microservice patterns and its accuracy varies across projects.

Specialised studies explore how generative AI can support architects directly. Quevedo et al. [158] evaluate ChatGPT's ability to answer questions about a system's microservice architecture using static analysis models as context. They find that the model can respond sensibly to many design queries but that providing additional architectural context does not reliably improve accuracy, highlighting limitations in the model's ability to leverage structured knowledge. Fuchß et al. [72] investigate the use of LLMs to recover trace links between software architecture documentation and code by extracting component names. Their approach achieves a weighted F1 score of 0.86, comparable to state-of-the-art techniques that require manually created intermediate models while avoiding manual labour. These domain-specific experiments demonstrate the potential of generative models but also underline the need for careful validation.

Broader studies emphasise the promise and pitfalls of generative AI for architectural tasks. Esposito et al. [61] perform a multivocal literature review on generative AI in software architecture. They observe growing adoption of LLMs for decision support, requirement-to-architecture translation and architecture reconstruction, but also note common challenges including hallucinations, ethical concerns, privacy issues and the absence of rigorous evaluation frameworks. Mastropaolo et al. [136] discuss the transformative potential and risks of generative AI for architectural practice. Finally, Stapleton et al. [197] conduct a controlled study comparing human- and machine-generated code summaries. They show that automatic metrics such as BLEU and ROUGE do not predict how well summaries aid comprehension and that human-written summaries remain superior. These findings reinforce the need for human-centred evaluation when integrating generative AI into architectural tooling.

In summary, existing SAR techniques either rely on learning-based classifiers that require significant aligned data, or heuristics that exploit domain knowledge but are hard to generalise. Recent efforts to apply large language models to microservice-specific tasks and traceability show promising accuracy but still lack interpretability and stability across contexts. Surveys

consistently emphasise the need for rigorous evaluation and human oversight when deploying generative AI for architecture recovery and design support.

5.3 Methodology

In reference to the software engineering research framework of Stol and Fitzgerald [201, 202], we conducted a *judgement study*, gathering empirical data from a group of participants who are asked to judge the utility of a method. In our setup, the judgment stimuli are DSAR-generated explanatory artefacts: participants were asked to rely on these artefacts for comprehension and related tasks instead of their usual practice of navigating code and documentation. Therefore, we implemented an evaluation study at ASML (cf. Section 5.3.1), where software engineers and architects interacted with the explanatory artefacts produced from codebases they routinely work on. Subsequently, we evaluated these artefacts through semi-structured interviews guided by the ethical procedures of Strandberg [205] and the Co-12 interpretability framework for XAI proposed by Nauta et al. [145, 146]. Figure 5.3 presents an overview of our three-phase methodology. In this section, we elaborate on each phase and its associated activities.

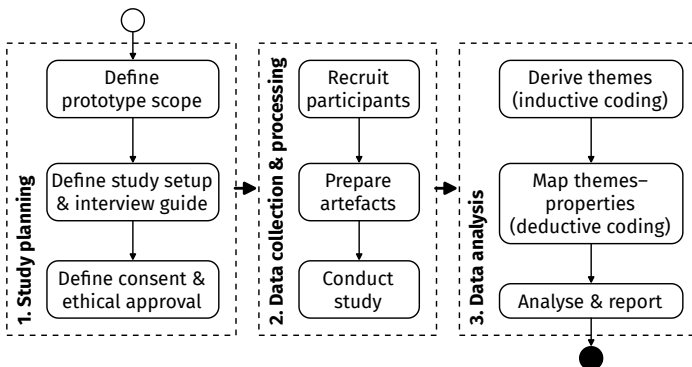


Figure 5.3: Methodology of our evaluation study.

5.3.1 ASML: An Evaluative Case Study

ASML² is a leading semiconductor company specialised in producing lithography machines. Since its founding in 1984, software has been integral to ASML's products, with roughly 45 million lines of code powering its machines [219]. Based in the Netherlands, ASML employs over 44,000 people globally. Software development is distributed across multiple teams, each responsible for subsets of the overall system. Newcomers often struggle with navigating existing systems, and obtaining architectural insights from experts is rarely feasible. This issue is significant at ASML, where 54% of employees have joined in the past five years [9], requiring intensive onboarding and architectural comprehension. These factors make ASML an ideal setting to test the DSAR method for enhancing the understanding and communication of large software architectures.

5.3.2 Study Planning

Define prototype scope: To evaluate the DSAR method in practice, we iteratively developed a Java-focused prototype implementing the DSAR method shown in Figure 5.1, consisting of three components: extraction, analysis, and visualisation.

The first component, refined from our prior work *Javapers*³ [169], corresponds to the *Static Analyser* in Figure 5.1. It converts Java source code into its corresponding LPG instance.

The second component, *DSAR Core* in Figure 5.1, extends our original DSAR prototype *Arcana*⁴ [172]. In the general DSAR framework, this component may employ any mechanism capable of (i) constructing hierarchical summaries and (ii) determining whether code elements satisfy some architectural description. In this implementation, both tasks are carried out using LLM-based reasoning. We use *Llama-3.3-70B-Instruct* (8-bit quantised), which receives source-level representations and architectural descriptions and returns method-level layer assignments and class-level role

²<https://www.asml.com/>

³GitHub:rsatrioadi/javapers

⁴GitHub:rsatrioadi/arcana

stereotypes [222, 223]. The architectural descriptions themselves are supplied by the study participants and represent the reference architecture against which DSAR performs its deductive classification.

The third component, *ClassViz*⁵, visualises DSAR outputs: structural dependencies, natural-language summaries, and classification results. It shows packages and classes as nested boxes, with relations as connecting lines. Method-level relationships are abstracted to classes or packages. Line colours encode relation types, and node colours encode architectural roles. This is the component participants used in our study (see Figure 5.4), evolved from our earlier exploratory visualisation prototypes [169, 171].

5

Define study setup and interview guide: We conducted our study in English and in a private setting to minimise distractions, bias [202], and risks of retaliation [205]. The study comprised three sequential phases: a *preparation phase*, an *exposure phase*, and a *semi-structured interview*.

Preparation phase: Before each session, we prepared the DSAR explanatory artefacts corresponding to the repositories relevant to that participant. For each selected repository, we used its previously defined architectural descriptions and executed the DSAR pipeline to generate summaries, classifications, and visualisations. This ensured that, during the session, participants interacted with artefacts grounded in their own systems and architectural conventions.

Exposure to explanatory artefacts: In this first in-session phase, participants explored the DSAR explanatory artefacts corresponding to their chosen repositories. We asked them to begin with components they were most familiar with and to verbalise their thoughts while interacting with the tool. We observed their actions and remarks, providing technical support as needed. This phase helped establish how participants naturally approached and interpreted the artefacts in the context of their system knowledge.

Semi-structured interview In this phase, we conducted a semi-structured interview to assess the DSAR method within the context of each participant's

⁵GitHub:rsatrioadi/classviz, demo: <https://satrio.rukmono.id/classviz/?p=jhotteddraw-5.1>

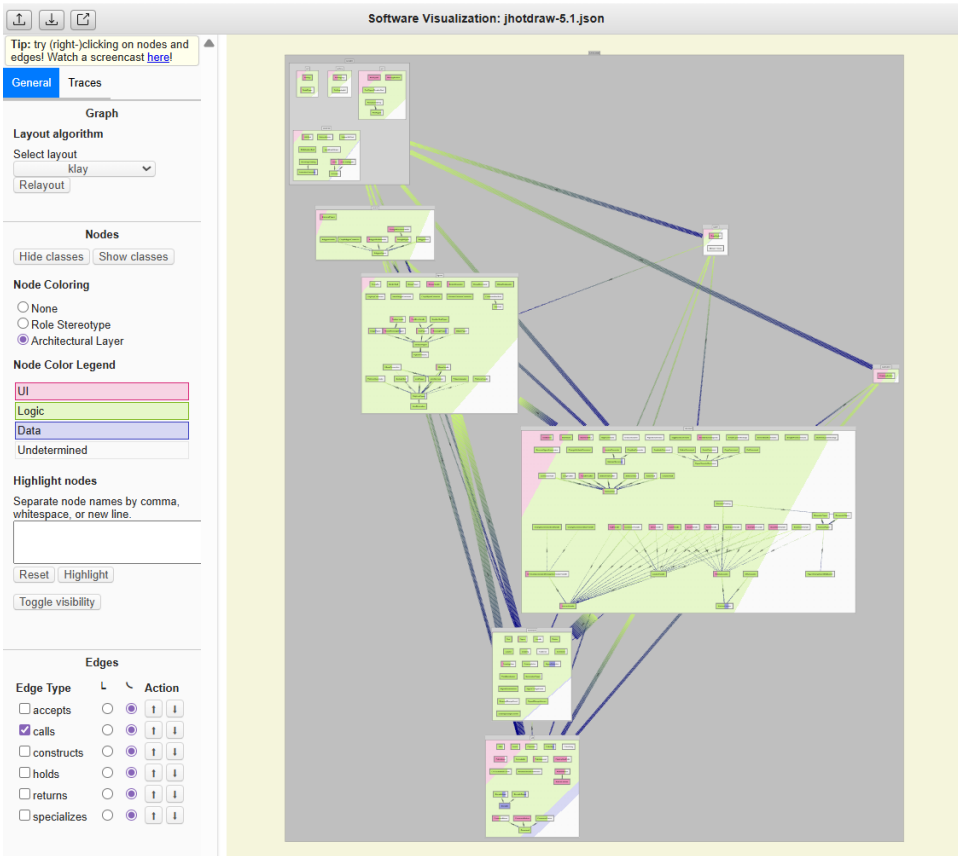


Figure 5.4: An example of the explanatory artefact accessible by participants, showing open source data, with box colours denoting architectural classification.

system. The interview guide followed established best practices for empirical software engineering research [20, 190]. It began with background questions covering years of professional experience, company tenure, current role, familiarity with the selected system, and typical development and maintenance tasks.

Subsequent questions focused on evaluating the DSAR artefacts. We used a customised version of the Co-12 framework for explainable AI [145, 146], organising ten of the twelve properties into seven, aligned with our research questions:

(1) **Correctness** “describes how faithful the explanation is w.r.t. the black box. Key idea: Nothing but the truth” [146].

(2) **Completeness** “describes how much of the black box behaviour is described in the explanation. Key idea: The whole truth” [146].

(3) **Discriminative consistency:** We interpret the original properties *contrastivity*, *consistency*, and *continuity* together as a spectrum we call *discriminative consistency*:

Contrastivity “describes how discriminative the explanation is w.r.t. other events or targets. Key idea: Answers ‘why not?’ or ‘what if?’ questions” [146].

Consistency “describes how deterministic and implementation-invariant the explanation method is. Key idea: Identical inputs should have identical explanations” [146].

Continuity “describes how continuous and generalisable the explanation function is. Key idea: Similar inputs should have similar explanations” [146].

(4) **Compactness** “describes the size of the explanation. Key idea: Less is more” [146].

(5) **Composition:** We interpret *composition* and *covariate complexity* together as they both refer to how easy it is to understand the explanation:

Composition “describes the presentation format and organisation of the explanation. Key idea: How something is explained” [146].

Covariate complexity “describes how complex the (interactions of) features in the explanation are complexity. Key idea: Human-understandable concepts in the explanation” [146].

(6) **Task relevance:** To avoid confusion with the LLM concept of a “context window,” we rename the *context* property to *task relevance*:

Context “describes how relevant the explanation is to the user and their needs. Key idea: How much does the explanation matter in practice?” [146].

(7) **Coherence** “describes how accordant the explanation is with prior knowledge and beliefs. Key idea: Plausibility or reasonableness to users” [146].

Linking the research questions to the above properties, *content quality* (RQ5.1) is represented by properties (1) to (3), *presentation quality* (RQ5.2)

by properties (4) and (5), and *usefulness* (RQ5.3) by properties (6) and (7). We excluded the remaining two Co-12 properties, *confidence* and *controllability*. Confidence (i.e., probabilistic accuracy) was inapplicable, as the DSAR prototype does not generate confidence scores. Although participants selected the codebase and defined the architectural layers, controllability (i.e., user influence over outputs) was not applicable as the tool generated explanatory artefacts in a fixed, non-interactive manner.

We combined 7-point Likert-scale statements (strongly disagree to strongly agree) with open-ended questions [93, 182]. Likert-scale statements served as a way to interpret the open-ended question responses. The full interview guide was developed and reviewed to ensure quality, and can be seen in Appendix B. It was piloted in January 2025 with a company representative, simulating a real session [125, 205]. The data from this pilot was excluded from our actual analysis.

Define consent and ethical approval: To ensure ethical integrity [105, 205], we submitted our study design to the Ethics Review Board (ERB) of Eindhoven University of Technology (TU/e) for review. This reinforces compliance with the General Data Protection Regulation (GDPR) and other relevant standards like the ACM Publications Policy on Research Involving Human Participants and Subjects. Approval was granted with reference ERB2024MCS39. Our legal basis for processing personal data was consent, which we obtained from all participants via a signed form prior to the study. Participation was voluntary and could be withdrawn at any time.

5.3.3 Data Collection and Processing

Recruit participants: We used purposive and convenience sampling to recruit participants [51, 53, 182, 183, 203, 210]. Purposive sampling targeted individuals with relevant roles, while convenience sampling targeted professionals through existing networks. We focused on ASML employees working with Java-based software in architectural or engineering roles.

The company representative identified 15 candidates from 10 teams; 13 accepted the invitation. Additionally, one participant was recruited at a soft-

Table 5.1: Overview of our study participants.

ID		PEY	CTY	PTY	AA	PA	CBF	Role	Team ID	Repository type
P01	8	5	2	Yearly	Never	4	Software Developer	A	Math/algorithms library	
P02	16	7	7	Daily	Never	5	Software Architect	B	Desktop app backend for data analysis and transformation	
P03	18	6	5	Monthly	Weekly	6	Software Architect	C	Job automation, triggered by user request or document change	
P04	25	7	4.5	Weekly	Daily	5	Software Developer	C	Job automation, triggered by user request or document change	
P05	14	6.5	4	Weekly	Daily	5	Software Developer	A	Math/algorithms library	
P06	17	5.5	2.5	Daily	Daily	6	Functional Engineer	D	GUI for measurements and calculations	
P07	9	2.5	1	Never	Monthly	2	Software Engineer	E	Backend for converting between data formats	
P08	12	12	4	Monthly	Never	5	Software Architect	B	Desktop app backend for data analysis and transformation	
P09	18	14	3	Monthly	Incidental	5	Software Architect	F	API wrappers for services	
P10	16	12	5	Daily	Never	5	Functional Engineer	G	UI frontend-user application	
P11	20	6	1	Monthly	Weekly	5	Software Architect	H	Desktop app for loading documents, computing, and exporting	
P12	10	8	5	Weekly	Weekly	6	Software Architect	I	User-facing simulations application	
P13	10	3.5	3.5	Weekly	Daily	6	Software Architect	I	User-facing simulations application	
P14	6	0.5	0.5	Incidental	Weekly	1	Software Developer	J	Plugin to an external information-system platform	

PEY: Professional experience (years).

CTY: Company time (years).

PTY: Project time (years).

AA: Architecting activity, i.e., how often they work in architecting for the codebase.

PA: Programming activity, i.e., how often they work in programming for the codebase.

CBF: Self-reported codebase familiarity, 0: not familiar, 6: highly familiar.

ware engineering conference, totalling 14 participants comprising *functional architects*, *system architects*, and *software engineers*. As Table 5.1 shows, all had over six years of experience, with varying tenure at ASML and familiarity with the codebase. Only two participants (P07, P14) reported low familiarity.

Prepare artefacts: Across the study, we prepared DSAR artefacts for 17 Java repositories maintained by the participating teams, spanning libraries (e.g., maths, algorithms), services, standalone applications, data-processing pipelines, and plugin modules (see column *Repository type* in Table 5.1).

Each repository required a definition of its architectural layers. For this, we asked a knowledgeable expert for each repository to draft layer indicators specific to their system. These indicators were iteratively refined when necessary to ensure clarity and alignment with the DSAR method.

The finalised indicators formed the reference architectures used throughout the study. Using these, we executed the DSAR pipeline on all selected repositories to produce the explanatory artefacts later used in the participant sessions.

Conduct study: We conducted the study between March 2025 and May 2025 at ASML's facilities, in dedicated meeting rooms where only the researcher and participant were present. During each session, we took extensive notes, which were later anonymised to protect participant identity and confidentiality before being shared with the research team for analysis.

5.3.4 Data Analysis

Derive themes (inductive coding): Two researchers processed the study notes and interviews using in-vivo coding [134]. Afterwards, they independently refined and derived themes from the in-vivo codes via open coding [40], keeping two independent codebooks. These two steps were executed in three rounds, each including 4–5 interviews and the respective notes. After each round, we analysed the independent inductive codes and aligned them into one shared codebook, reaching consensus on the interpretation. As a final step, we used ChatGPT with model `gpt-4o` to suggest merges of highly similar codes. We manually reviewed and decided whether to incorporate a

merge suggestion. We did not calculate an inter-rater reliability metric, as we opted for consensus by resolving disagreements through discussion.

Map themes–properties (deductive coding): After the inductive coding was finalised, we clustered related themes into the seven properties we described in Section 5.3.2.

Analyse and report: The finalised set of themes and properties was analysed and reported, reflecting the perspectives of the participants on our research questions.

5.4 Results

This section presents the study findings revolving around our inductive *themes* and how they relate to the evaluated *properties*.

5

5.4.1 Themes

From the interviews, we identified themes depicting the assessment of the DSAR implementation by the participants, and linked them to the studied properties. We documented which participants provided each observation, with their IDs shown in parentheses.

Impact of codebase complexity: In our study, the impact of “size” manifested primarily at the code elements level (e.g., classes with many methods, packages with many classes) rather than at the level of the whole repository. Having large or dense elements (e.g., high fan-in/out or deep nesting) tends to reduce the *correctness*, *compactness*, and *composition* of generated artefacts (P02, P05). Summaries for complex classes become noisy or incomplete (P05, P13), whereas small classes produced clearer overviews (P02). Participants noted that classification accuracy drops when the code contains many nested or private elements (P08, P13), and that large diagrams hinder effective interpretation (P02, P04, P05, P10).

Rendering and layouting struggle under structural density (P01, P04, P09, P11), especially in deep or entangled regions (P02). While these effects often co-occur with poor abstraction, they originate from the volume and connectivity of the code elements rather than their design quality. The effect

of poor system design is instead reflected in the theme “garbage in, garbage out.”

Confidently wrong—mediation necessary: Participants noted that artefacts sometimes included incorrect but confidently presented summaries, impacting *correctness* (P04, P06, P07, P08). These subtle errors involved misuse or invented domain terms (P06, P07) and lacked hedging language. This reduced *coherence* and misled participants who were less familiar with the domain. A common sub-theme was *terminology leakage*, where internal tool names or irrelevant jargon were included (P04). Expert users, familiar with domain terms, became more sceptical about the entire artefact’s correctness when encountering confidently wrong output, even if other parts were accurate. These problems diminished trust and pointed to the need for mechanisms to restrict vocabulary to domain-relevant terms or validate statements against code context (P08).

This theme and the one that follows can result in a similar “confidently wrong” tone, but they differ in source. This theme centres on the LLM misinterpreting or misrepresenting terminology, whereas the next concerns the LLM assuming the user’s input as correct.

Garbage in—garbage out: Closely related to the previous theme, participants observed that the tool assumes user input as canonical, meaning that flaws in the codebase, such as misleading names, typos, or poor structural organisation, are directly reflected in the generated artefacts (P05, P06, P11). This affects *correctness*: summaries inferred false meanings from incorrect identifiers (P02, P11), misclassified elements due to unclear structure (P06), or failed to clarify what the code actually did (P09). Well-structured codebases led to more accurate results, while messier ones amplified inaccuracies (P04, P06). This undermines *composition* and *task relevance*, as the artefacts fail to support comprehension. Unlike the previous theme, which focuses on complexity as a neutral design characteristic, this one highlights how poor-quality input actively reduces the usefulness of the generated artefacts.

Domain-specific knowledge: Participants suggested that feeding relevant domain-specific information into the model could substantially improve the

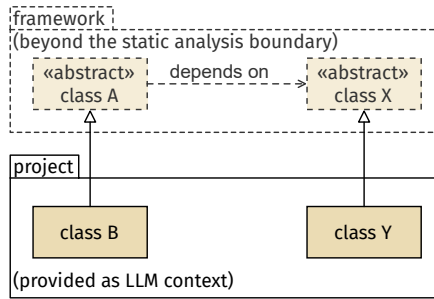


Figure 5.5: Implicit dependency: Class B may depend on Y as a specific implementation of X.

5

quality of the explanatory artefacts (P02, P05, P07, P12). Doing so would not only increase *completeness* but also enhance *coherence* by ensuring that the terminology aligned with project- or organisation-specific vocabulary. Without this coherence, the artefacts require user’s prior domain knowledge to comprehend (P02, P11, P13). Feeding glossaries or curating inputs could help prevent vague or generic phrasing, grounding the artefact in the team’s real-world language.

Rich dependency information: Participants appreciated the artefact’s rich representation of relations beyond typical invocation-based dependency graphs (affecting *completeness*) (P12). For example, we include relations like inheritance, field access, and method return types. However, this richness demands careful visual handling to maintain effective *composition* and *compactness* (P02, P04, P05, P10). One participant noted the absence of a “throws” relation, relevant for error handling (P12). Others highlighted missing relation extents that affect *completeness*:

- (i) *implicit dependencies* [80] at the framework level not explicitly encoded (P05, see Figure 5.5);
- (ii) *inbound dependencies*, which were excluded when generating summaries (P01, P08); and
- (iii) *external dependencies*, such as libraries or another team’s repository within the company (P03, P09, P11, P14).

Dynamic information: The lack of dynamic information, such as logs or call trees, was seen as a limitation in *completeness* (P06, P08, P10, P11, P14).

Participants noted that dynamic aspects like execution traces and data flow are important for identifying key system behaviour. Their absence limits not only what an artefact can convey but also how it can support architectural and comprehension tasks. For instance, diagrams could better capture the weight of method invocations if informed by execution traces, thereby improving *composition*. Integrating such data could also sharpen textual summaries by emphasising behaviourally significant elements. However, collecting and maintaining dynamic information involves substantially more effort, which must be carefully weighed when considering such extensions in practice.

Shallow context for stubs: Abstract methods and stub functions lack executable bodies, leaving the tool with insufficient information to construct meaningful summaries. As a result, the generated summaries are often shallow or generic, lowering perceived *completeness* (P05, P06). Participants recognised that this is an inherent limitation of static analysis but suggested ways to manage expectations, such as explicitly flagging these elements as placeholders or linking to relevant implementing subclasses. One participant also proposed leveraging existing “TODO” comments, which developers often leave in their code, to indicate incompleteness (P14) and further support more transparent and trustworthy summaries. These mechanisms help prevent a false impression of completeness and aid in accurate codebase comprehension.

Architectural assessment (nice to have): Participants expressed an interest in going beyond factual description towards evaluative and diagnostic capabilities. They understood that such features were beyond the scope of the prototype under evaluation, yet nevertheless envisioned that the artefacts could, in future iterations, support *architectural assessment* by identifying violations of design principles (P06 specifically mentioned SOLID), measuring metrics like cohesion or complexity (P08), surfacing known design smells (P14), or highlighting misalignments with architectural rules (P01, P02, P09). The absence of these features does not imply lack of *completeness*, however, their inclusion would enrich *task relevance* by offering actionable insights and helping users evaluate structural quality.

Unwanted or excess information: Summaries occasionally include redundant or trivial content, reducing *compactness* by forcing users to sift through irrelevant details (P08, P10–14). Participants pointed out that restating method names as sentences or enumerating all fields of a class adds little value (P04, P07, P08, P11, P14). This verbosity dilutes the usefulness of summaries, even when correct.

Compactness via composition and abstraction: Participants emphasised that *compactness* is achieved through deliberate *composition* and appropriate abstraction, not by omitting details indiscriminately. This involved being able to filter relationships, zoom between package-level and class-level summaries (P05, P09, P10), or “lift” relations across granularity levels (P11, P12, P14), enabling the same system to be presented with varying levels of detail. Compactness and composition were seen as mutually reinforcing: abstraction, layout clarity, and filtering together allowed more information to be shown without overwhelming the viewer (P08, P10, P12, P14). These mechanisms were particularly appreciated in diagrams, where complexity otherwise led to visual clutter (P04, P05).

Audience role and needs: Participants noted that perceived usefulness varied with their roles. Developers preferred concrete behavioural details, while architects sought higher-level structural overviews (P04, P06, P10, P14). This role-dependent need affects both *composition* and *task relevance*: no single summary style satisfies all users equally. Flexible artefacts that adapt to role-specific views, for example by filtering edges (P08, P10) or tailoring detail levels (P04, P10), were seen as more effective. Role stereotypes helped some participants quickly grasp responsibilities (P14), but unfamiliarity with the concept of role stereotypes limited their value to others (P12).

Document–summary synergy: Participants evaluated summaries in relation to existing documentation comments. When summaries merely echoed identifiers or had insufficient details, they were seen as redundant (P01, P05). However, participants found the artefacts especially valuable when these filled documentation gaps, such as missing, outdated, or inaccurate comments, by offering more up-to-date or contextually enriched explanations

(P01, P08, P11, P14). This complementary relationship enhanced both *task relevance* and *composition*, as participants could rely on the artefact to serve as a “living document” that can fill gaps and correct misrepresentations in the documentation.

Visualisation value: Participants saw visualisations as essential complements to textual summaries, especially for revealing structural and behavioural relationships (P06, P07, P11, P12). Diagrams showing layers, object composition, and control or data flow supported onboarding, and architectural tasks (P04, P05, P07, P08–12, P14), directly aiding *composition* and *task relevance*.

Three factors shaped the visualisation’s utility. First, colour helped differentiate elements (P01, P03, P06, P07, P10, P13), although the palette had not been extensively validated for accessibility (P08, P10, P13). Second, stable hierarchical layouts, such as Klay [181, 196], supported mental models and traceability (P01, P03, P05, P09, P11–P14), while unstable layouts hurt clarity. Third, participants often inferred importance from “hotspots” (nodes connected to many edges), but noted that this could mislead; for example, abstract classes often appeared central even when having minor role (P01, P08). These factors show that visual *composition* shapes interpretation.

General and project-specific analyses: Participants valued both general role stereotypes and project-specific architectural layers. General role stereotypes (e.g., controller, information holder) helped with cross-system comparisons and quick understanding during onboarding (P03, P10, P14), while project-specific classifications reflected local architectural realities (P12, P13). This duality enhanced both *coherence* and *task relevance*, showing that consistent local context can be valuable.

However, mismatches between general terms and local vocabulary may reduce *coherence*, requiring translation or reinterpretation (P12). When aligned, the combination of roles and layers helped surface architectural issues such as violations of the single-responsibility principle [85, 92] or layer pollution (P01, P08, P13, P14). These findings highlight the importance of balancing abstraction with system-specific insight.

Systematised consistency: Participants emphasised the need for consistent treatment of similar elements in both summaries and classifications. When structurally or behaviourally similar elements were described differently, or classifications were inconsistently applied, it undermined trust and clarity (P01, P08, P12–14). Participants expected summaries to align with their roles and to reflect distinctions between elements without redundancy or contradiction.

To achieve this, participants called for a systematic mechanism to identify similar or related elements and ensure they are treated in a unified way (P05, P09, P13). Rather than generating explanations independently for each class or method, they wanted DSAR to detect recurring patterns (such as shared responsibilities, common architectural roles, or inherited behaviour) and to let those similarities guide both the phrasing of summaries and the application of classifications. They suggested leveraging inheritance relationships, common contexts, or templated summary structures for this mechanism.

This points to two intertwined needs: discriminative consistency, where summaries and classifications correspond meaningfully to architectural roles (P08, P14); and system-level coherence, where related elements are explained in a harmonised manner that reflects their shared characteristics (P12, P13). Participants viewed both as essential for accurate and trustworthy comprehension.

Taken together, the next three themes describe how participants might use the explanatory artefacts to support their day-to-day activities. These task-oriented themes reflect the practical value participants derived from the artefacts and how they integrated them into real workflows:

Comprehension task: Participants viewed the explanatory artefacts as an up-to-date reference that aided system comprehension (P01, P02, P08–10, P12). It helped build a mental model of the codebase, especially in legacy or undocumented systems. This support was foundational, as comprehension was often cited as a prerequisite for subsequent architectural and communication tasks.

Architectural task: The explanatory artefacts were regarded as useful in architectural-related tasks, especially when documentation was missing. Participants mentioned using it for impact analysis (P02, P05–10, 12, 14) and guiding testing effort based on perceived element importance (P06). The artefacts were deemed unnecessary for local programming tasks, where IDEs provide sufficient access. This matches our expectations as SAR is aimed at higher-level discourse.

Communication task: The explanatory artefacts also supported communication tasks, from onboarding newcomers (P04, P07) to conveying architectural overviews to non-technical stakeholders (P08, P10, P14). Participants used the artefacts to explain architectural decisions and justify refactoring needs. They also valued them for composing executive-level documentation, which reduced manual effort and improved clarity (P04, P14).

Across these tasks, the artefacts supported different roles in different ways, but their usefulness depended on clear distinctions, accurate abstraction, and the availability of contextual cues such as domain vocabulary and system conventions.

Terminology clarity: Participants stressed that clear, precise, and context-appropriate terminology is essential for ensuring *coherence*. Summaries that used unfamiliar theoretical terms (e.g., role stereotypes) were harder to interpret for teams with own naming conventions (P2, P04–06, P12, P14). Differences in assumed notations, such as Java-, UML-, or project-specific terms, further complicated understanding (P14). Beyond familiarity, vague expressions like “file” instead of “configuration file” led to misinterpretation of an element’s role or importance (P02, P07, P08). Sparse or overly generic wording made summaries feel disconnected from actual usage (P02, P04, P05, P07, P12). Thus, terminology must balance familiarity with precision to support effective comprehension.

Handling tests: Participants highlighted the necessity of distinguishing test from production code in both summaries and diagrams (P09, P11). Artefacts frequently misrepresented test classes as regular system elements,

overlooking their role in development. This blurring reduced *discriminative consistency*, complicating the differentiation between conceptually distinct artefacts. Moreover, participants anticipated varying *composition* forms for tests—emphasising intent, coverage, or verification over functionality. Thus, integrating them with production code introduced unnecessary noise. Separating test artefacts was deemed crucial for clarity and maintaining significant distinctions in the explanation.

Transparency and explainability: Participants wanted to understand the reasoning behind how elements were classified or organised, particularly when results were surprising. The need for clarity was noted in areas like classification (P09, P12), layout decisions (P12), and emphasis methods like visual hotspots (P08). Without transparency, even accurate artefacts may be misjudged or doubted (P08, P09, P12). This highlights the necessity for explainable AI tools (e.g., rule surfaces, architectural rationales) to support generated artefacts.

5

5.4.2 Properties

We synthesised the qualitative insights from the preceding themes along our seven explanation properties (Section 5.3.2), with the mapping presented in Table 5.2. For clarity, we organise the discussion in three clusters that mirror our research questions: *content quality* (Correctness, Completeness, Discriminative consistency; RQ5.1), *presentation quality* (Compactness, Composition; RQ5.2), and *usefulness* (Task relevance, Coherence; RQ5.3). Within each cluster, we first describe the relevant properties and then provide an explicit answer to the corresponding research question.

Figure 5.6 supports this analysis by showing participants' ratings of each property on the 7-point Likert scale, ranging from dark magenta -3 (strongly disagree) to dark green $+3$ (strongly agree). Overall, the majority selected $+2$ or $+3$, indicating a positive perception that the explanatory artefacts met the properties. These numerical results are complementary to the qualitative findings and should not be seen as a statistical conclusion.

We begin with *content quality* (RQ5.1), reflected in the properties of

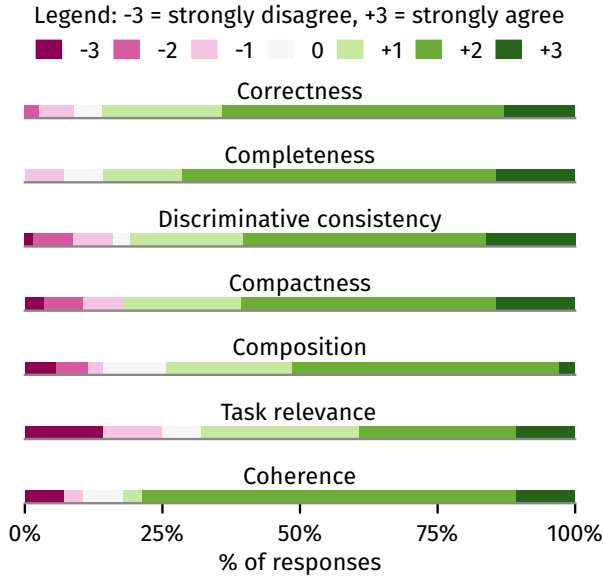


Figure 5.6: Participants’ Likert responses for each evaluation property, showing the extent to which the explanatory artefacts were judged to exhibit the corresponding qualities.

Correctness, Completeness, and Discriminative consistency.

Correctness: Participants generally agreed that the artefacts reflected the structure and behaviour of the system well. Layer and role classifications were often accurate, and most summaries faithfully paraphrased the code. However, correctness was weakened when the model echoed flawed code inputs, such as misleading identifiers or typos, and when hallucinated domain terms appeared without qualification. These issues were especially problematic when phrased confidently, leading participants to question the explanatory artefacts’ reliability as a whole. Suggestions included restricting summaries to validated vocabulary, surfacing architectural rationale for classifications, and applying a two-pass review process to flag risky inferences. Correctness was thus not only about factual accuracy but also about safeguarding trust.

Completeness: Participants considered the artefacts broadly comprehensive in capturing structure and architectural roles. They consistently pointed to enhancements that could deepen system understanding: including dy-

dynamic data, incorporating broader dependencies, and integrating domain- or project-specific context. Especially for the first two, participants framed them as opportunities to extend the artefacts' explanatory power rather than as missing elements. Incorporating such signals would thus strengthen the artefacts' usefulness.

Discriminative consistency: Consistency across elements, both in classification and summarisation, was a key trust factor. Participants expected similar components to be treated similarly, and for assigned architectural roles to be reflected in the summaries. When similar classes were classified differently or described with mismatched terminology, it undermined perceived reliability. Participants suggested using structural cues (e.g., inheritance) and shared templates to enforce consistency, while also highlighting subtle differences where relevant. This balance between sameness and contrast was useful for identifying architectural deviations and supporting reasoning about system structure.

5

RQ5.1 Answer: *How do practitioners perceive the content quality of the DSAR explanatory artefacts?*

The perceived *content quality* of the explanatory artefacts generated by our DSAR prototype is subject to the quality and complexity of the codebase itself: errors or smells in the codebase result in issues and complexity in the explanatory artefacts. This phenomenon supports our approach to combine inductive (bottom-up) with deductive (top-down) analysis. Furthermore, LLMs sometimes confidently framed wrong descriptions, providing misleading artefacts. The absence of relevant information in the knowledge graph (e.g., dynamic behaviour, broader dependencies, or domain-specific context) also impacted the perceived quality of the explanatory artefacts; participants' expectations about the type of information to be represented were not entirely met. Lastly, the explanatory artefacts were occasionally inadequate at highlighting the differences between elements of similar nature (e.g., methods implementing the same interface), or at describing non-implemented stubs. These issues

pointed to a need for validation safeguards and more systematic treatment of similarity and differences across elements.

We next turn to *presentation quality* (RQ5.2), which concerns how information is structured and displayed through the properties of Compactness and Composition.

Compactness: Participants valued artefacts that conveyed rich information without overwhelming detail. Compactness was not achieved by mere brevity but through abstraction, filtering, and selective emphasis. Summaries are most effective when balanced: concise for simple elements, but more detailed for complex ones. Redundant or trivial information (e.g., restated identifiers or exhaustive listings) diminishes readability, whereas adjustable detail helps align summaries with task needs. An interactive visualisation maintains compactness by displaying high-level elements, but allows unpacking complexity via edge filtering, zooming, or relation lifting when necessary. Thus, compactness emerged as an adjustable property, meaning the capability to tailor granularity and information density to a user's cognitive load and intent.

Composition: Composition encompassed both the visual and textual structuring of the artefacts. Participants appreciated artefacts with a logical layout, stable phrasing, and consistent formatting. Visually, layered diagrams with hierarchical layouts, colour-coded roles, and meaningful edges facilitated navigation and represented system architecture well. Abstract methods, tests, and framework code were seen as needing tailored treatment to reflect their unique roles within the system.

RQ5.2 Answer: *How do practitioners perceive the presentation quality of the DSAR explanatory artefacts?*

Our participants highlighted that a balance between composition and abstraction should be achieved to provide an optimal *presentation quality* of the generated explanatory artefacts. In this regard, visual representations were highly appreciated by participants. For complex and smelly

systems, the challenge lies in simplifying the system's representation and making the summaries facilitate comprehension. Additionally, the lack of completeness mentioned in the results for RQ5.1 opens a new challenge: finding an appropriate way of depicting the missing information in the explanatory artefacts. Additionally, participants encouraged treating artefacts as complementary to, rather than replacements for, existing documentation, with attention paid to how artefacts frame or scaffold incomplete or abstract elements.

Finally, we consider *usefulness* (RQ5.3), captured by the properties of Task relevance and Coherence.

5

Task relevance: The artefacts were most valued for comprehension, impact analysis, architectural reasoning, and communication, especially in onboarding and legacy system contexts. Participants appreciated them for filling gaps in code comments and for enabling efficient stakeholder discussions. Role stereotypes and layer assignments helped surface responsibilities, while visual cues guided architectural evaluations. However, artefacts were less useful for routine coding or debugging, where IDEs sufficed. Suggestions included integrating quality metrics or principle violations to automate architectural assessments. Thus, task relevance also demonstrated the need for adaptability to align with a user's role and objectives.

Coherence: Participants trusted the artefacts more when these used domain-aligned and contextually appropriate terminology. Coherence dropped when summaries included overly theoretical labels (e.g., information holder), generic phrasing ("file" instead of the more precise "configuration file"), or terms inconsistent with the codebase or team conventions. Terminology that matched project-specific language, or clarified distinctions via precise wording, enhanced comprehension. Thus, we deduce that coherence was not just about linguistic accuracy but about interpretability and alignment with the mental models of the user. Familiar vocabulary was seen as a powerful enabler of trust and utility.

RQ5.3 Answer: *How do practitioners judge the practical usefulness of the DSAR explanatory artefacts?*

Participants generally judged the explanatory artefacts to have satisfactory *usefulness* for comprehension (e.g., in onboarding situations), architectural reasoning, and communication—especially in unfamiliar or legacy systems. However, the artefacts’ usefulness was strongly modulated by how well the terminology aligned with domain vocabulary and role-specific needs. The artefacts were noted as less suitable for low-level coding tasks, where IDEs already provide sufficient support, and their perceived value lessened when high-quality documentation was already in place. Further analyses on the codebase (e.g., design principles, quality metrics) can facilitate deeper insights into the system and help with developing a robust maintenance process.

Table 5.2: Mapping between inductive *themes* and evaluation *properties*. A bullet indicates that participants' comments for a theme relate to the given property.

Theme (Section 5.4.1)	C	P	Property & RQ (Section 5.4.2)						
			RQ5.1			RQ5.2		RQ5.3	
			Correctness	Completeness	Discr. Consistency	Compactness	Composition	Task Relevance	Coherence
Impact of codebase complexity	11	9	•			•	•		
Confidently wrong	11	9	•						•
Garbage in–garbage out	11	9	•				•	•	
Domain-specific knowledge	11	12		•					•
Rich dependency information	24	12		•		•	•		
Dynamic information	3	5		•			•		
Shallow context for stubs	7	6		•					
Architectural assessment	34	12						•	
Unwanted or excess information	4	8				•			
Compactness via compos. and abstr.	34	13				•	•		
Audience role and needs	39	14						•	
Document–summary synergy	11	7					•	•	
Visualisation value	54	14					•	•	
General & project-specific analyses	12	10	•				•	•	•
Systematic differentiation	20	10	•		•			•	
Comprehension task	39	14						•	
Architectural task	20	11						•	
Communication task	7	6						•	
Terminology clarity	18	12							•
Handling tests	3	3			•		•		
Transparency & explainability	34	13	•		•		•	•	

C: Number of codes related to this theme.

P: Number of unique participants concerned about this theme.

5.5 Discussions

This section discusses the implications of our study for industrial adoption and research advancement.

Implications for practitioners: For DSAR adoption, organisations must provide clear guidelines to help experts define architectural layer indicators and clearly distinguish roles and layers; otherwise, explanatory artefacts may be lower quality. These artefacts should embed domain knowledge, organisational terminology, and the team's representation standards (e.g., UML) to aid comprehension.

To meet user and organisational needs, the DSAR knowledge graph should integrate additional analyses for completeness and explainability, including evaluative capabilities (e.g., architectural assessment, smell detection) that participants often requested. Explanations must be tailorable to developers, architects, and managers, with appropriate levels of detail, so DSAR can act as an adaptive reference for onboarding and system understanding.

Practitioners should also complement static source code information with dynamic data or other sources to enrich recovery and improve completeness, for example, to analyse interfaces or stub functions without concrete implementations. External sources that experts may not know well (e.g., libraries, frameworks) should be carefully considered in the analysis.

These observations support the view that generative AI is most effective for architecture reconstruction when combined with architectural knowledge and static analysis, rather than used as an end-to-end solution [95]. Consistent with this, DSAR was used interactively and exploratively in our study, with generative elements aiding comprehension while static artefacts ensured contextual fidelity.

5.5.1 Implications for Researchers

To further mature DSAR, research should address how software complexity and smells affect explanatory artefact quality and how to mitigate these effects. Integrating execution traces, data flows, and domain- or organisation-specific knowledge into architectural recovery and summarisation (e.g., via

fine-tuning or retrieval-augmented generation) is key to improving artefact quality and usefulness. Advanced mechanisms such as uncertainty estimation, validation pipelines, or hybrid approaches are needed to reduce misleading DSAR-generated explanations. Studies across more organisations, languages, and domains are needed to improve transferability. Finally, developing templates and validation criteria for architectural indicator authoring is essential, as indicator quality directly bounds the quality of all generated artefacts.

5.6 Trustworthiness

In this section, we reflect on the trustworthiness of our study. We adopt the trustworthiness framework [126] for qualitative and naturalistic research, instead of the “threats to validity” framework [184] that is optimised for quantitative empirical research.

Credibility: Evaluates the trustworthiness of research data and results. We sought ERB approval at TU/e for our study design to reinforce ethical and design confidence. To mitigate bias in our questionnaire, we followed a three-step process: initial design, review, and pilots for refinement are carried out involving different members of the research team. For DSAR implementation, we used an iterative approach with feedback from distinct experts for domain alignment and operational fit, risking researcher-tailored features. During our data analysis, two researchers coded independently and discussed with another researcher to develop a shared codebook. The DSAR method was assessed through a judgment study, but additional validation is needed to guarantee thematic saturation [82, 83].

Transferability: Indicates the potential application of results to similar situations. While not asserting universal relevance, we provide study specifics in Section 5.3 to allow judgements of applicability [126]. Due to intellectual property and privacy concerns, software repository details and the explanatory artefacts cannot be publicly shared. Although designed to be transferable, our judgment study was conducted only within one organisation. We increased perspective diversity by involving participants from

diverse teams and projects. Nevertheless, convenience sampling may cause bias by over-representing readily available individuals and omitting other perspectives. Lastly, although our method is developed to be language-agnostic, our evaluation outcomes may not directly transfer to codebases with different technology stacks.

Dependability: Evaluates how well methodological choices are made. Participants' confidence may have impacted the study results. To reduce this bias, sessions remained private, involving only one researcher and participant, in separate private rooms at ASML. Data was pseudo-anonymised to prevent retaliation. Raw data access was restricted to the researchers who performed the coding task. Additionally, the interview guide ensured unbiased interactions, while coding bias was minimised with multiple coders during the qualitative analysis.

Confirmability: For confirmability, we clearly described our methodology and key design decisions in Section 5.3. We also discuss their impact and potential biases. All pseudo-anonymised data and analyses were carefully documented, with main outputs included in the study's replication package.

5.7 Conclusion

We presented the first empirical evaluation of Deductive Software Architecture Recovery (DSAR) in an industrial setting at ASML. Our qualitative study shows that DSAR-generated explanatory artefacts meaningfully support software comprehension, architectural reasoning, and communication, especially for onboarding, legacy systems, and cross-team work. Practitioners also saw DSAR as a promising form of living documentation and a useful aid for architectural assessment.

At the same time, the study highlights that the effectiveness of DSAR artefacts is contingent on three factors: (i) the quality and complexity of the input codebase, (ii) the clarity and systematic formulation of architectural indicators, and (iii) the compact, coherent, role-appropriate presentation of information. When these are lacking, issues such as confidently wrong outputs, weak contextualisation of abstractions, and inconsistent treatment

of similar components underscore the need for stronger validation and consistency mechanisms.

Participants gaps in the current implementation: dynamic behaviour, external dependencies, and organisation-specific terminology are under-represented, reducing completeness and contextual fidelity. Simultaneously, requests for evaluative features (e.g., principle-violation detection) show that explanation alone is insufficient; practitioners want tools that both describe and assess systems. These findings reinforce the need to combine structured analysis with richer contextual signals and domain knowledge.

Taken together, our results suggest that DSAR's strength lies in the combination of structured analysis and "fuzzy" natural-language reasoning, which enables insights that neither approach could provide in isolation. However, realising this potential in practice requires systematic safeguards and extensions. Four priorities emerge for future work: (1) guardrails against unreliable outputs, such as binding claims to traceable code evidence and constraining vocabulary through validated glossaries; (2) systematic support for architectural indicator authoring, including templates and validation criteria; (3) integration of dynamic information and broader dependency coverage to address completeness gaps; and (4) replication across organisations, technology stacks, and architectural styles to establish transferability boundaries.

Crucially, they relate not only to what information is produced, but to how it is presented and explored. In our evaluation, participants interacted with DSAR-generated artefacts through a visual interface ClassViz, which enabled them to inspect structures, trace relationships, and identify irregularities. This highlights that enriched architectural representations must be made explorable and interpretable in practice, rather than treated as static outputs.

Acknowledgments

We thank our study participants for their time and efforts.

Declaration of AI Use

The authors used Writefull and ChatGPT (OpenAI) for language refinement, and ChatGPT for suggesting merges of semantically similar inductive codes during analysis (Section 5.3.4). No scientific claims or conclusions were generated by these tools. All outputs were manually reviewed, edited, and verified by the authors.

Part IV

Explanation Presentation

6

Visualisation Prototypes for Architecture Explanation

6

The evaluation of DSAR in Chapter 5 shows that enriched architectural artefacts are useful not only for their content, but also for how they are presented and explored. This chapter explores how visualisation can bridge precise yet complex knowledge representations and human needs for comprehension and explanation. Building on the Software Architecture/Analysis Basic Ontology (SABO), we present two visualisation prototypes: ClassViz and BubbleTea. ClassViz began as a lightweight tool for visually inspecting SABO instantiation results, but evolved through student projects, industrial collaborations, and research integration into a versatile platform for exploring class-level structures. Its trajectory shows how a modest tool can grow into a shared environment for experimentation, evaluation, and communication. BubbleTea, in contrast, targets layered architectural viewpoints, classifying code elements into semantic layers and highlighting irregularities in functionality placement. It demonstrates an alternative path, where responsibility-oriented layering drives both the classification and visual encoding of code elements. Together, these prototypes exemplify how visualisation supports architecture explanation: by simplifying inspection, grounding analyses in recognisable views, and revealing anomalies or inconsistencies.

6.1 Introduction

Visualisation plays a central role in comprehending and explaining software systems, complementing textual and structural representations. Because software lacks a tangible or perceivable form, its complexity can easily exceed human cognitive limits when presented solely as code or abstract data [47]. Software Architecture/Analysis Basic Ontology (SABO) graphs (cf. Chapter 4) are precise and expressive, but their size and complexity make direct inspection difficult. Visualisation provides an intermediate form to inspect SABO graph extractions at a glance and communicate insights to human audiences.

This chapter presents two prototypes developed to address these challenges. *ClassViz* began as a lightweight environment for visually inspecting graph instantiation results, but evolved into a versatile platform used across research, education, and industry. *BubbleTea* was designed to visualise Deductive Software Architecture Recovery (DSAR) results, mapping source code methods into architectural layers by responsibility. Rather than emphasising dependency-based violations, it highlights elements whose functionality diverges from their assigned layer.

ClassViz also featured prominently in the industrial evaluation of DSAR in Chapter 5, where it formed the “exoskeleton” of the explanatory artefacts shown to participants. However, that chapter focused on empirically evaluating DSAR, not on the design and evolution of the visualisation itself. There, the use of Research Questions (RQs) structured systematic data collection and analysis within a controlled setting.

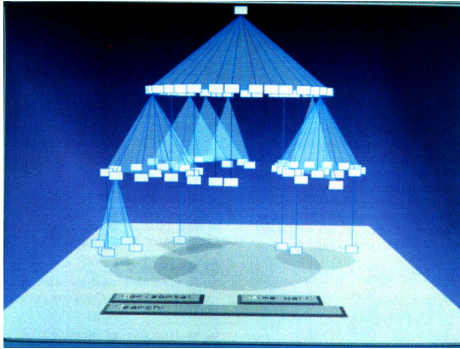
This chapter differs in both purpose and approach, following a more exploratory trajectory. The sections that follow are organised around the two visualisation prototypes and the emerging design requirements and insights from their iterative development. Readers are encouraged to view this chapter as both a report of engineering practice and a reflection on how visualisation design responds to evolving representational needs. Where relevant, examples from classroom and industry contexts are included to illustrate practical relevance, although these are not presented as formal evaluations.

6.2 Related Work

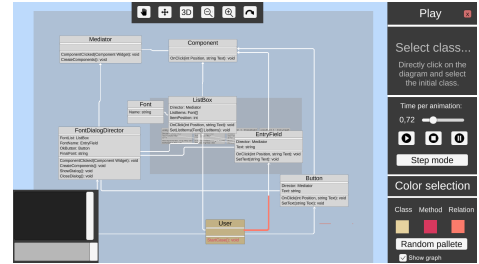
Software visualisation has addressed aspects of structure, behaviour, and evolution. A systematic review by Chotisarn et al. [35] analysed 105 articles from 2013 to 2019, finding balanced coverage of these aspects. Visualisations most often targeted design, implementation, and maintenance, with developers as main users. Techniques included multivariate visualisation, graph-based encodings, and metaphorical mappings (e.g., cities). Despite many proposals, industrial uptake remains limited, especially for maintenance and debugging, highlighting the need for tools aligned with daily practice.

In software architecture visualisation, Shahin et al. [186] surveyed 53 studies from 1999–2011. They identified four main types: graph-based (e.g., Robertson et al. [164]), notation-based (e.g., Radosky and Polasek [159]), matrix-based (e.g., Abuthawabeh and Zeckzer [1]), and metaphor-based (e.g., Misiak et al. [140], Wettel and Lanza [220]). Graph-based approaches dominate recovery and evolution tasks; metaphor-based views offer intuitive overviews but raise scalability and cognitive concerns. Most techniques were tested only on small or academic systems, with little industrial use. The review emphasised the dual role of architecture visualisation: supporting both structural viewpoints (components, connectors, layers) and decisional viewpoints (design decisions and rationale). Figure 6.1 shows an example for each visualisation type.

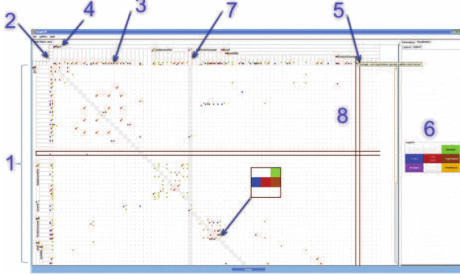
These reviews frame the approaches in this chapter. ClassViz supports structural analysis of extracted architecture graphs with filtering, layout, and layering. BubbleTea addresses layered viewpoints with a visual encoding aimed at anomalies and inconsistencies. ClassViz's adoption in academic and industrial settings illustrates one response to calls for usable, practice-oriented tools.



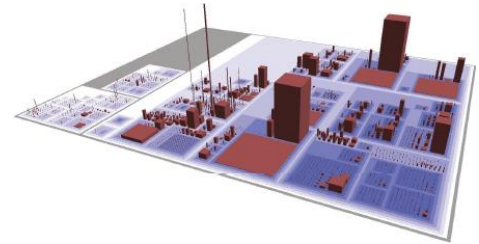
(a) Graph-based visualisation by Robertson et al. [164]



(b) Notation-based visualisation by Radosky and Polasek [159]



(c) Matrix-based visualisation by Abuthawabeh and Zeckzer [1]



(d) Metaphor-based visualisation by Wettel and Lanza [220]

Figure 6.1: Example visualisations using different techniques.

6.3 ClassViz: From Inspection Tool to Research Vessel

ClassViz began as a pragmatic tool for visually inspecting SABO graph instantiations (cf. Chapter 4) and evolved through student projects, industrial collaborations, and research integration into a versatile platform for exploring class-level structures. As a research contribution, it is best seen not as a finished product but as a research vehicle whose design, iterative evolution, and deployment in evaluation contexts show how visualisation can bridge precise but complex knowledge representations and human needs for comprehension and explanation.

The trajectory of *ClassViz* was not driven by a predetermined specification, but by successive research needs that redefined its role. Each new capability was prompted by limitations encountered once previous requirements

were (partially) addressed. The resulting design goals, which structure the remainder of this section, emerged through this cumulative process:

- (1) The tool must provide visual affordances that support effective lightweight inspection of SABO graphs for correctness and usability assessment.
- (2) The tool must be designed to support adoption, extension, and appropriation across diverse contexts, including educational and industrial settings.
- (3) The tool must serve as an effective frontend for automated analyses, enabling users to explore, interpret, and communicate results from processes such as architectural recovery and summarisation.

6.3.1 Design and Origin

ClassViz's initial design prioritised clarity and filtering over expressiveness, reflecting the primary need for lightweight correctness inspection of SABO graphs. It was developed as a personal tool to inspect SABO instantiation from Java systems. Raw SABO graphs were precise but overwhelming and too large for immediate feedback.

Nested boxes represented packages and classes, with class-level relations drawn as coloured UML-style arrows (e.g., hollow triangles for inheritance). Filtering allowed toggling of primitive types, packages, and relation types, and highlighting of classes by name. Click-through navigation revealed related elements; relations could be rendered as orthogonal or Bézier lines, with layouts from general-purpose algorithms. Figures 6.2 and 6.3 show this initial version of ClassViz applied to JHotDraw 5.1.

Implemented as a lightweight browser application, ClassViz was easy to run in short development cycles. Addressing design goal (1), ClassViz's simple encodings, filters, and minimal infrastructure provide minimal but expressive visual affordances that reduce cognitive load while preserving structural fidelity for quick sanity checks and practical debugging of graph instantiation.

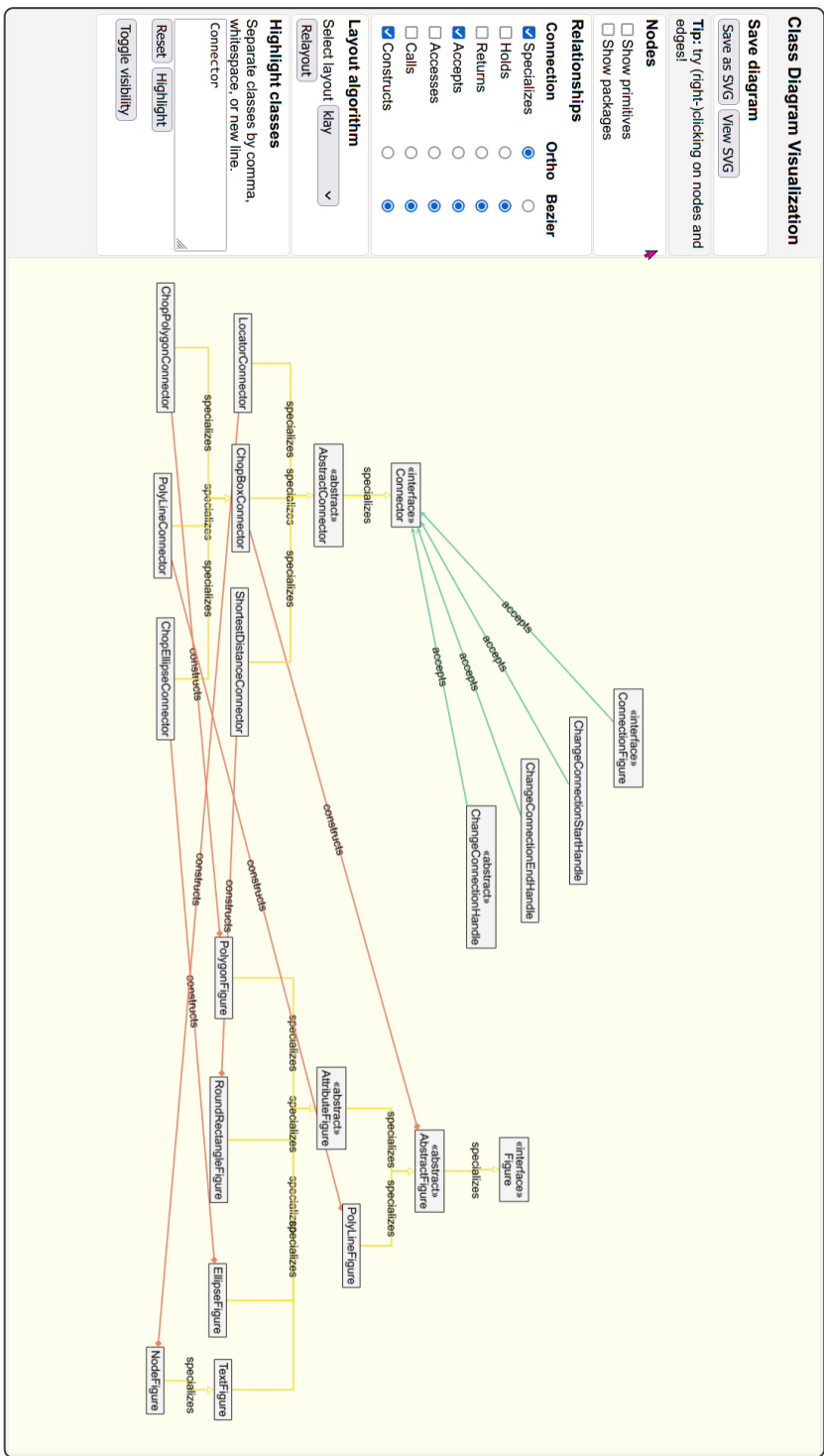


Figure 6.2: An early ClassViz version showing classes involved in an instance of the strategy design pattern in JHotDraw 5.1. All other classes are filtered out.

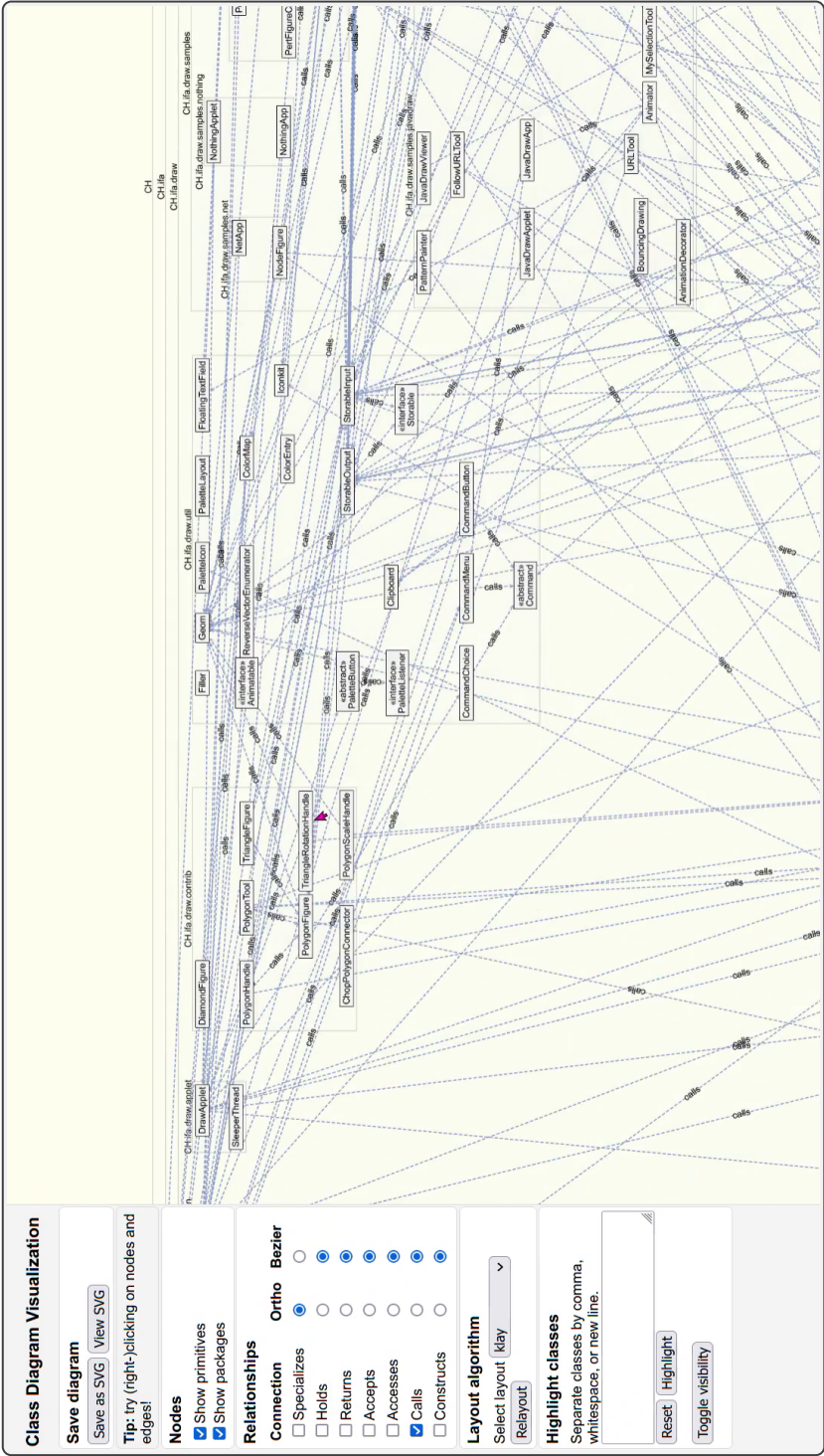


Figure 6.3: An early ClassViz version showing all packages and classes of JHotDraw 5.1 (that fit the viewport) and the “calls” relations among the classes.

6.3.2 Adoption, Extension, and Appropriation

ClassViz's adoption across student and industry contexts was driven not by feature completeness but by a deliberately minimal and open design. Three properties of this design enabled its uptake: a low barrier to entry for developers, openness to diverse input/output semantics in the same syntax, and modifiability by design. Together these qualities enabled ClassViz to function not just as a tool but as a shared platform across multiple independent projects and industry-facing efforts.

Dynamic behaviour extension: Several student projects explored how runtime information could be integrated into ClassViz's structural views. Fung [73] integrated dynamic execution traces into ClassViz, mapping information onto class nodes as attributes and colour overlays. He [86] built on ClassViz directly, adding behaviour visualisation modes and summary views to complement static structural diagrams, including interactive filtering and simplified overlays for activity traces, as shown in Figure 6.4. Van Esch [60] adopted Fung [73]'s dynamic data strand into immersive virtual-reality (VR) visualisation named SABER-VR, combining static structure (class/package layouts) with dynamic sequence visualisations rendered in 3D (see Figure 6.5). In an industry setting with Philips, Tanis [208] developed a new implementation (rather than extending ClassViz) to explore behaviour visualisation and summarisation for very large-scale codebases, validating the demand for scalable behaviour-oriented visualisation in industrial practice. Together, these works show how runtime-oriented adaptations were feasible across fidelity levels and modalities.

Structural and encoding extensions: Other projects focused on improving structural clarity and expanding the visual design space. Kloet [107] investigated per-level layout algorithms to improve the readability of clustered graphs, experimenting with stable placements across abstraction levels. An important finding is that the most effective visualisations are obtained by using different layout strategies at different hierarchy levels, as shown in Figure 6.6. This finding was possible because the design of ClassViz afforded this kind of experimentation. Atisomya [11] explored the direct mapping

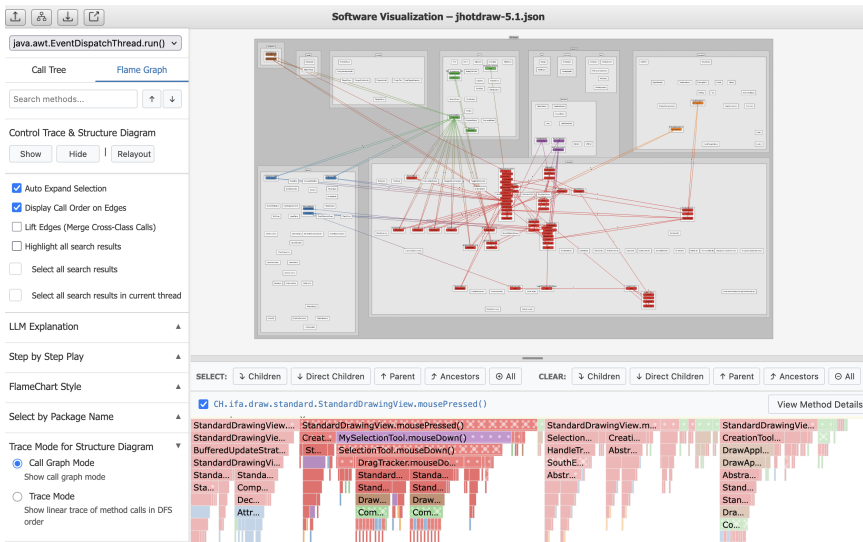


Figure 6.4: He [86] overlays structure and behaviour information in one visualisation.

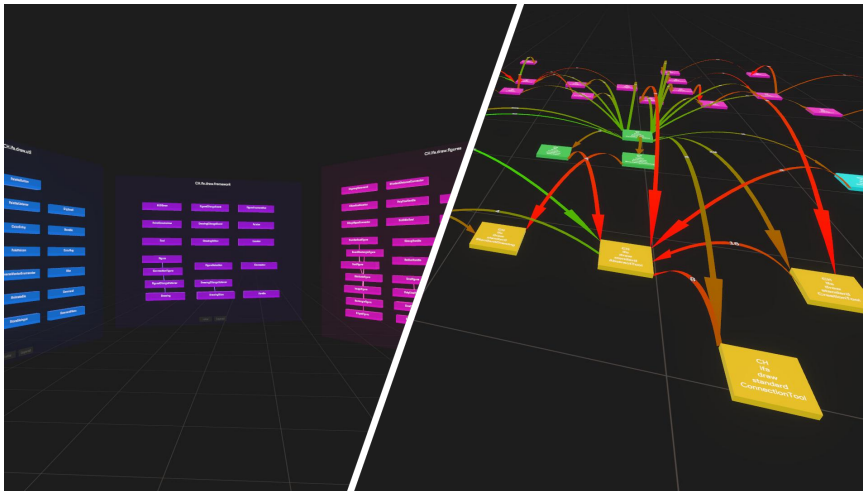
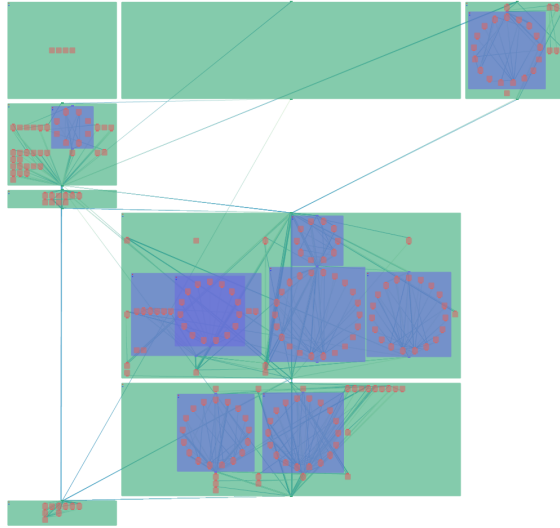


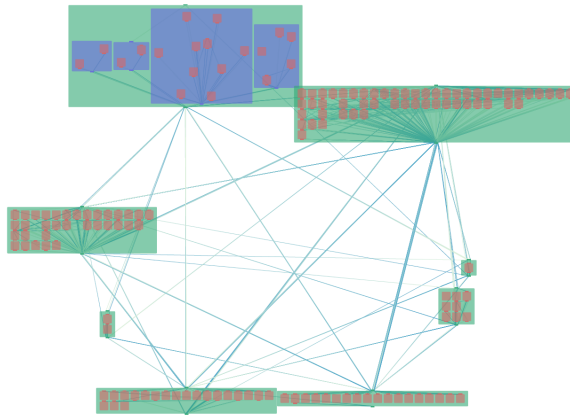
Figure 6.5: SABER-VR [60] puts structure and behaviour information in a VR environment.

of SABO dimensions to visual variables, assigning x - and y -position as well as colour to represent properties such as package, type, or metric values. This provides a systematic exploration of SABO-to-visual mapping as shown in Figure 6.7. Jeffrey and Nugroho worked on two strands: *BubbleTea* (cf. Section 6.4), which visualises layered architectures using circular packing

of classes as coloured “bubbles” across horizontal bands; and *CodeView*, which employs nested boxes and edge lifting to present the development view of software. Both strands resulted in publications (VISSOFT [171] and ICoDSE [98] respectively, the latter receiving a best paper award).



(a) Grid-like layout for outer boxes and circular layout for inner boxes.



(b) Circular layout for outer boxes and grid-like layout for inner boxes.

Figure 6.6: Kloet [107] explores varying layout algorithms for different levels of hierarchy.

Specialised analysis frontends: ClassViz also demonstrated its portability as a frontend for domain-specific analysis pipelines. Asuni [10] connected

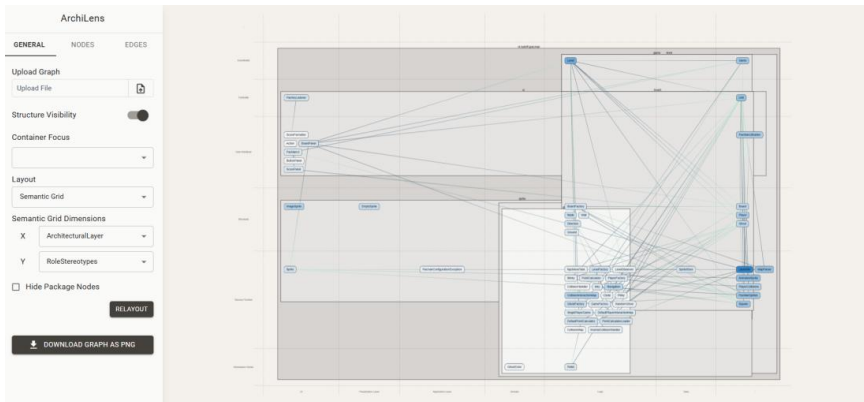


Figure 6.7: Atisomya [11] puts code elements (in this case, classes) in a grid of architectural layer $\times$ role stereotype, and numerical metric “number of methods” for the colour scale.

ClassViz to a security vulnerability detection pipeline, overlaying markers on affected classes and dependencies, an exemplar of how ClassViz can serve as a front-end for specialised analyses, i.e., in the domain of security. Kakkenberg et al. [101] developed Arvisan, a new tool inspired by ClassViz principles but targeting OutSystems low-code platforms, directly operationalising the Explanation Window framework (cf. Section 3.5.2) by only showing a narrow neighbourhood of components at a time, as shown in Figure 6.8. Arvisan continues to be useful in a different research project with different company, system domain, and technology platform [227]. Morier¹ developed an infrastructure foundation rather than exploring visual representations, resulting in a platform and API to support authentication, graph versioning, and data orchestration, laying the groundwork for a more cohesive tooling ecosystem and encouraging convergence across forks and variants.

This proliferation of forks, extensions, and parallel visualisation efforts thus underscores the value of design minimalism and architectural openness: because ClassViz made few assumptions about input semantics or output layout, it enabled projects to explore dynamic behaviour, security overlays, role stereotypes, and VR environments with minimal friction. Thus, addressing design goal (2), the low entry barrier and minimal coupling empowered

¹<https://github.com/SimonMorier/ArchManager-back>

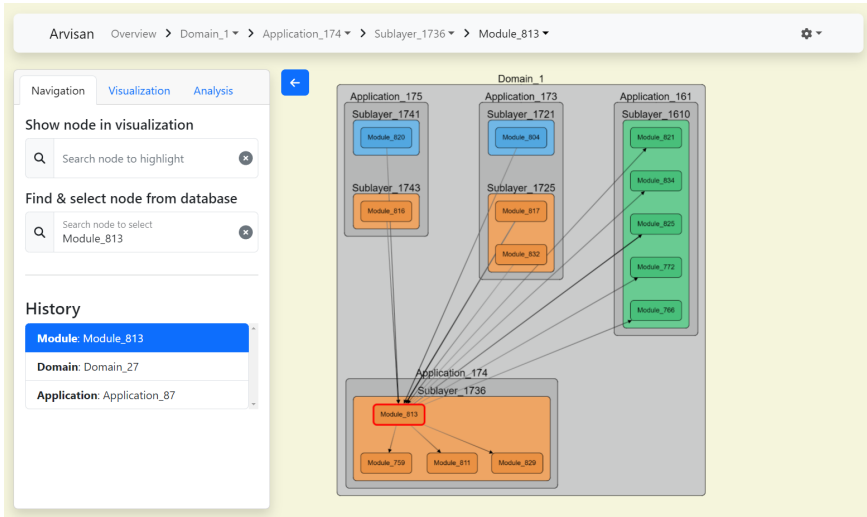


Figure 6.8: Arvisan [101] shows a narrow neighbourhood of components at a time.

6

non-core developers to repurpose ClassViz across adjacent (sub)domains without deep reengineering.

6.3.3 Frontend for Architectural Analysis and Explanation

ClassViz was extended to serve as the presentation layer for automated analysis pipelines, specifically hierarchical summarisation and DSAR outputs. A central enabler of this evolution is its use of LPGs, which allows analysis results to be integrated flexibly, either as additional nodes and edges or as properties on existing ones. These properties can be directly mapped to visual variables; for example, metrics to colour gradients, classifications to discrete colours, and ranked values to spatial layout. This makes ClassViz well-suited as a frontend for structurally anchored explanations.

To support these use cases, ClassViz was enriched with visual encodings and navigational aids, each motivated by the need to make a specific class of analysis output readable within the visual surface. Node colouring by classification (e.g., role stereotypes [222] or architectural layers) allowed users to see analysis results directly on the structural diagram without leaving the inspection context. A detail pane was added to display automatically generated summaries, grounding abstract textual outputs in the specific

class or package the user was inspecting. Additional enhancements drawn from extension projects were integrated: lifting and lowering relations across abstraction levels, gradient and thickness styling for relation edges to indicate direction and weight, and more intuitive filtering and navigation. Figure 6.9 shows this recent ClassViz version, illustrating the classification of stereotype roles as the colours of the class nodes.

ClassViz then entered the industrial evaluation at ASML, acting as the explanatory surface for DSAR-derived architectural views shown to practitioners. In this role, it was the component through which participants encountered and interpreted recovered architecture, i.e., the “exoskeleton” of the explanatory artefacts. The empirical evaluation of that role is the subject of Chapter 5; here, what matters is the design implication: the adaptations described above were not cosmetic additions but necessary conditions for ClassViz to serve as a legible, interpretable frontend for automated architectural analysis in an industrial context. Addressing design goal (3), this evaluation shows how LPGs bridge abstract analysis results and architectural explanations by enabling node and edge properties to be directly mapped to visual variables.

6.3.4 Demonstration: Visualising ClassViz in ClassViz

To demonstrate our design goals on a concrete, non-trivial codebase, ClassViz was applied to visualise the internal design of its own source code. This self-visualisation serves to exercise the inspection affordances of the tool on a real system and render DSAR-inferred role stereotypes and architectural layer assignments as visual variables, instantiating its role as a frontend for automated analysis.

Figure 6.10 presents the internal organisation of the ClassViz source code, rendered in ClassViz itself using a manually arranged layout with minor post-processing for clarity. The diagram combines two abstraction levels: filesystem folders, shown as UML-style packages, and JavaScript modules, shown as labelled rectangles. Modules are coloured by inferred role stereotype (e.g., *Controller*, *Coordinator*, *Interface*); folders by architectural layer (e.g., *Presentation*, *Domain*, *Data*). Although ClassViz typically toggles

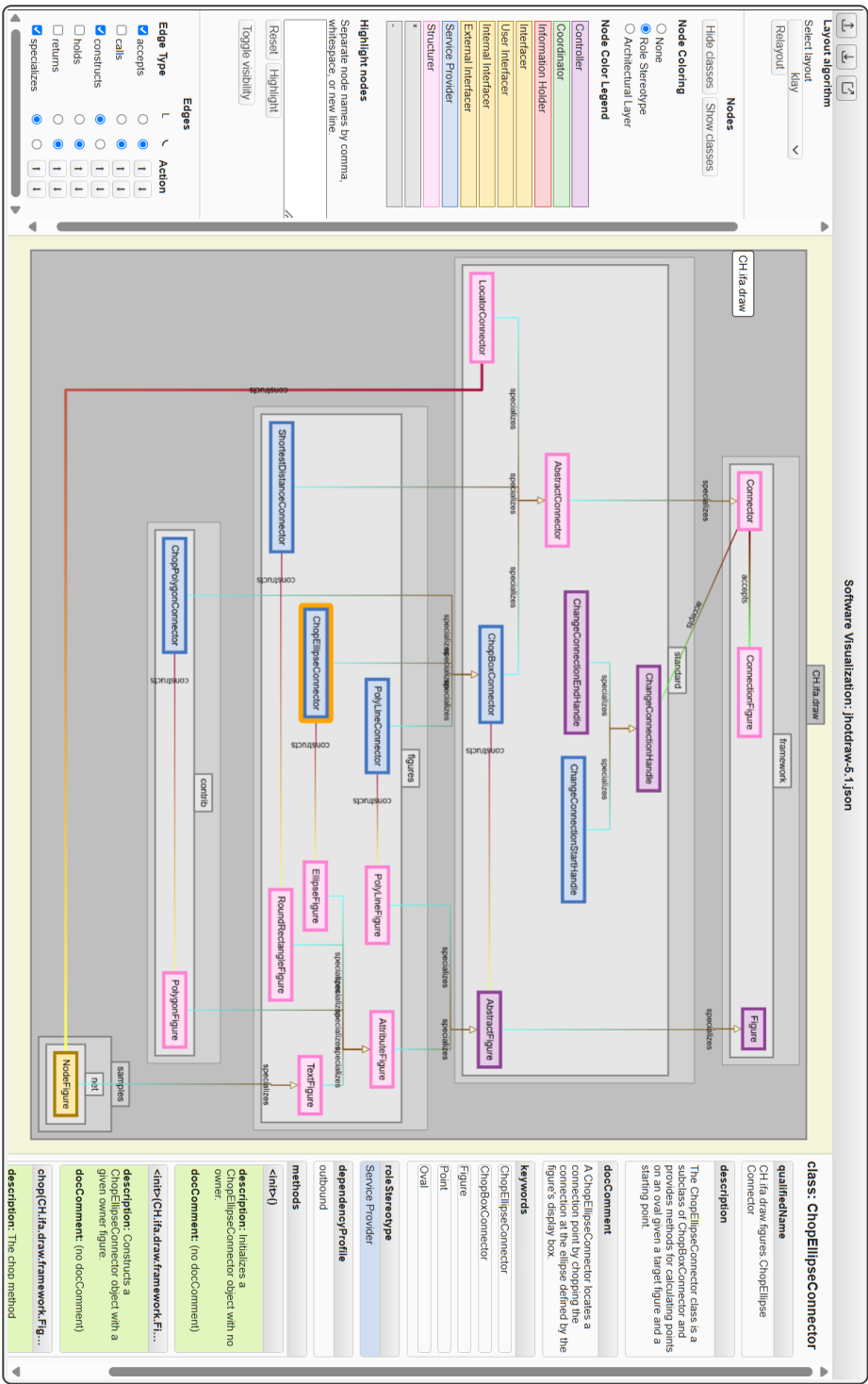


Figure 6.9: A recent version of ClassViz showing classes from JHotDraw 5.1 classified into role stereotypes [222] and call relations lifted into package level.

between these modes, the figure overlays both to provide an integrated view and avoid redundancy.

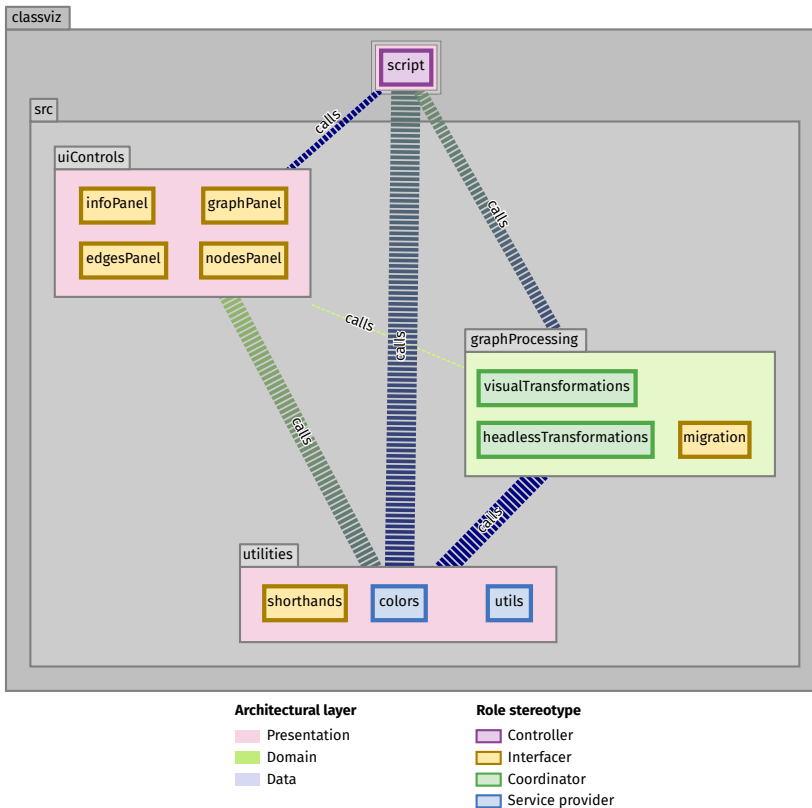


Figure 6.10: ClassViz source code organisation as depicted by ClassViz itself. In this visualisation, line direction is represented from light to dark colour, e.g., from the graphProcessing folder to the utilities.

The graphProcessing folder houses the core transformation pipeline, including logic for collapsing classes, assigning visual cues, and filtering edges. However, some fundamental concerns such as file loading and certain filter logics (e.g., class name and edge type filters) remain awkwardly placed in the main script and the graphPanel module, an artefact of early prototyping that was not fully refactored. Layout computation is delegated to external, pre-existing third party plugins, keeping the visualisation backend modular but introducing an opaque dependency boundary. Meanwhile, uiControls manages user-facing interactions, and utilities provides generic functions such

as colour mapping and graph traversal. This latter module, while convenient, is heavily depended upon across the system. This dominance of calls to utility functions suggests overcentralisation that might benefit from decomposition. Conversely, the interface code exhibits clear segmentation, reflecting a more modular and layered design.

In summary, the self-visualisation validates the tool on a real codebase and reveals its own architectural inconsistencies, serving both as a showcase of all three design goals and as a candid diagnostic of its evolution.

6.3.5 Reflection

The trajectory of ClassViz illustrates how a research tool can evolve from a pragmatic artefact into a central research instrument. Its growth was not driven by a fixed roadmap, but by successive research needs that redefined its role. This process was not incidental: ClassViz matured in tandem with the research programme itself, adapting to new inquiries while simultaneously enabling them.

Regarding lightweight inspection, this co-evolution surfaced the insight that visualisation tools in software engineering are rarely static products. Their utility often lies in their adaptability: how they can absorb new data sources, act as experimental platforms, or reveal structures and patterns that suggest new research directions. ClassViz supported all three, effectively becoming a testbed that shaped the very questions asked throughout this dissertation.

The many forks, extensions, and parallel visualisation efforts highlight the value of design minimalism and architectural openness. Because ClassViz assumed little about input semantics or output layout, student projects could easily explore dynamic behaviour, security overlays, role stereotypes, and VR environments. This indicates that generality and modifiability can matter more than completeness in early-stage research tools.

Regarding explanation as a generative act, the development of ClassViz and its derivatives reinforces the idea that explanation and inspection are not secondary activities in research, but rather generative. The need to see structure clearly often precedes the ability to articulate it analytically. Thus,

building tools that serve such explanatory needs is not just a support task; it can be considered a form of scientific inquiry by itself.

6.4 BubbleTea: A Layered Architecture Visualisation

Layering is a common design tactic for decomposing a system across levels of abstraction. A typical reference pattern consists of four types of layers [69, 222], each dedicated to a specific type of responsibility:

- The ***presentation layer*** manages the user interface (UI), defines UI elements and their behaviour, displays information, responds to user input, and updates views.
- The ***service layer*** controls the application flow, orchestrates domain operations, connects UI events with domain logic, and synchronises domain changes with the UI.
- The ***domain layer*** handles business logic, represents domain data and behaviour, and performs necessary computations for domain operations.
- The ***data source layer*** interacts with databases, filesystems, hardware, messaging systems, or other data sources, performs CRUD (Create/Read/Update/Delete) operations, handles data conversion, and ensures data integrity.

6

This reference architecture underpins BubbleTea, which maps software components onto layers. However, these four layers serve only as one illustrative example. Different types of systems across domains may adopt distinct layering schemes and definitions of what constitutes a layer [69]. BubbleTea does not assume any particular scheme; instead, it accommodates responsibility-based classification independent of the chosen vocabulary or number of layers. The visualisation therefore remains applicable across varied architectural interpretations of layering.

BubbleTea was created to visualise DSAR results, in which source code methods are classified by architectural responsibility. This differs from dependency-based analyses, where layers and violations are derived from

call graphs. BubbleTea instead focuses on the interpretation of responsibility rather than the structure of calls or other types of collaboration or dependency.

The following requirements emerged during the development of BubbleTea, reflecting the goals and challenges in using semantic layering to aid comprehension:

- (4) *Support responsibility-oriented layering.* The visualisation must reflect architectural responsibilities rather than inferred call dependencies, allowing elements to be grouped by their intended role in the system.
- (5) *Expose irregular or ambiguous elements.* Not all components fit neatly into a single layer. The tool should make such irregularities visible, prompting interpretation, discussion, or refinement of the classification.
- (6) *Remain scalable and intuitive.* The chosen visual encoding should make large systems navigable while conveying layering semantics in a form that practitioners find natural and readable.

6

6.4.1 Divergence from Related Work

Rethinking layer identification beyond dependency: Most existing layer-identification methods use topological sorting techniques based on dependency relationships between source code elements. SArF Map [108] introduces the *greedy level decomposition* algorithm combined with a *greedy cycle removal* heuristic to address cyclic dependencies, which complicate layer detection. oZone [117] extends the analysis by considering additional relationships beyond method invocation, such as class access, reference, inheritance, and extension, and removes cycles by selectively cutting edges with the least weight.

An alternative to dependency-based layering was introduced by Budi et al. [29], who applied a machine learning classifier to assign classes to architectural layers based on extracted source-level features. This shifts the focus from structural links to semantic characteristics of the code, a perspective our approach [172] also adopts through deductive means.

Addressing requirement (4), layer identification in BubbleTea is based on the semantic responsibilities of code elements rather than their structural dependencies. Methods and classes are positioned according to their intended architectural roles and behaviours, not merely their call relationships, reflecting a responsibility-oriented interpretation of layering.

Reframing violations as semantic misalignments: Architectural layers are typically represented as horizontal bands, with lower layers closer to hardware and higher layers representing user-level abstractions. Software components are placed within these bands, and dependency arrows connect the components. Layer violations are depicted by upward arrows.

Erben's SAB [58] follows this model, relying on user-supplied layers—a mapping of classes to layers provided as input to the visualisation. Van Ham [84] similarly proposed incorporating user-defined layers into a matrix-based visualisation. In this approach, each layer appears as both a row and a column; ideally, the lower triangle of the matrix should be empty, and any non-empty cells indicate a layer violation.

Addressing requirement (5), BubbleTea departs from traditional notions of layer violation. Rather than identifying forbidden dependencies, it highlights semantic outliers—elements whose behaviour deviates from the expected responsibility of their assigned layer.

Visualising layers through responsibility-driven layouts: BubbleTea employs circular packing to represent software components. A related visualisation, EVA [143], also uses circular packing but defines “layer” as a point in time in the evolution of a software system rather than an architectural layer.

Several studies have sought to increase the information density of circular packing [79, 228]. While such improvements could theoretically be incorporated into BubbleTea, our focus is not on optimising the packing algorithm itself but on how semantic layering is conveyed visually.

Addressing requirement (6), BubbleTea integrates circular packing with semantic grouping and colour coding to produce an intuitive, scalable visualisation of responsibility-oriented layers, supporting both structural overview and semantic interpretation.

These conceptual and visual departures from related work set the stage for the design of BubbleTea, which builds upon a responsibility-oriented interpretation of layering and presents its findings through an expressive and accessible visual grammar. The following section details the classification process and visual encoding strategies underpinning the tool.

6.4.2 BubbleTea's Approach

The approach underlying BubbleTea comprises two interwoven aspects: a semantic classification pipeline that assigns architectural layers based on method-level responsibilities, and a corresponding visualisation design that communicates these results in a manner aligned with architectural reasoning. In the following, we describe the classification logic, visual mapping, and design decisions that structure the user experience of BubbleTea.

Layer classification and visualisation: The process of classifying code elements into architectural layers begins with analysing the methods within each class. An AI-enhanced static analysis determines each method's functionality and corresponding layer. Concretely, we follow the Deductive Software Architecture Recovery approach [172] by providing a method's source code, along with additional context information, to a large language model (gpt-3.5-turbo). We ask the model to determine whether the provided method fits one of the four architectural layers. The classification is based on the stereotypical responsibilities of the layers: *presentation*, *service*, *domain*, and *data source*.

To illustrate, consider a Java package P_1 with a class C_1 containing methods M_1 , M_2 , and M_3 . Suppose M_1 and M_3 are classified under the *domain* layer, while M_2 belongs to the *data source* layer. The method-to-layer assignments are aggregated at the class level to calculate the percentage of

methods assigned to each layer. For class C_1 , this results in the distribution 0% *presentation*, 0% *service*, 66.67% *domain*, and 33.33% *data source* (Table 6.1).

Table 6.1: Illustration: the aggregation of layer indications in each class, calculated as the percentage of methods assigned to each layer.

Package	Class	Presentation	Service	Domain	Data Source
P_1	C_1	0.00%	0.00%	66.67%	33.33%
	...				
P_2	...				

This process is repeated for all classes within a package, and the results are aggregated at the package level. For instance, if package P_1 shows the following averages—0% *presentation*, 25% *service*, 25% *domain*, and 50% *data source* (Table 6.2)—the *data source* layer is deemed “dominant”. Consequently, P_1 is assigned to the *data source* layer.

Table 6.2: Illustration: the aggregation of layer indications in each package, calculated as the average percentage of methods assigned to each layer.

Package	Presentation	Service	Domain	Data Source
P_1	0%	25%	25%	50%
P_2	...			
...				

A layer indication is considered to “dominate” another if its percentage of methods exceeds 1.5 times that of the other indication. For a given package, we use the following criteria:

- *If one layer indication dominates all others*, the package is assigned to that dominating layer.
- *If two layer indications each dominate the other two but not each other, and are adjacent in the architecture*, the package is considered to lie between the two layers.
- *Otherwise*, no layer is assigned.

The complete algorithm for assigning layers into packages can be seen in Listing 6.1.

Listing 6.1: Responsibility-oriented layer assignment for BubbleTea.

```

1 Require:
2   Set of packages  $\mathcal{P}$ ;
3   each package  $P \in \mathcal{P}$  contains classes  $C$ , and each class contains methods  $M$ 
4   Classifier  $\text{ClassifyLayer}(m)$  that assigns method  $m$  to a layer  $L$ ,
5   where  $L \in \{\text{Presentation, Service, Domain, DataSource}\}$ 
6 Ensure:
7   Mapping  $\mathcal{A} : \mathcal{P} \rightarrow \mathcal{L} \cup \{\text{None}\}$ ,
8   where  $\mathcal{L}$  includes layers or between-layer assignments
9 for all class  $C$  in package  $P \in \mathcal{P}$  do
10    $M_C \leftarrow \text{methods in } C$ 
11   for all  $m \in M_C$  do
12      $l_m \leftarrow \text{ClassifyLayer}(m)$ 
13   end for
14   for all layer  $L$  do      (Compute percentage of methods in  $C$  per layer  $L$ .)
15      $p_C[L] \leftarrow |\{m \in M_C \mid \text{ClassifyLayer}(m) = L\}| / |M_C|$ 
16   end for
17 end for
18 for all package  $P \in \mathcal{P}$  do
19    $\mathcal{C}_P \leftarrow \text{classes in } P$ 
20   for all layer  $L$  do      (Compute average percentage.)
21      $p_P[L] \leftarrow (\sum_{C \in \mathcal{C}_P} p_C[L]) / |\mathcal{C}_P|$ 
22   end for
23    $L_{\max} \leftarrow \arg \max_L p_P[L]$ 
24   if  $p_P[L_{\max}] > 1.5 \times p_P[L]$  for all  $L \neq L_{\max}$  then
25      $\mathcal{A}(P) \leftarrow L_{\max}$ 
26   else if two adjacent layers  $L_1, L_2$  both dominate the others
27      $\mathcal{A}(P) \leftarrow (L_1, L_2)$ 
28   else
29      $\mathcal{A}(P) \leftarrow \text{None}$ 
30   end if
31 end for

```

6

No layer assignment for the third case handles both packages with no clearly dominating layer and also those dominated by non-adjacent layers (e.g., presentation and data source simultaneously). Placing such packages “between” those layers would imply a spatial proximity that misrepresents the architectural distance between them; outside placement instead signals that the package lies beyond the reference architecture guideline.

BubbleTea visualises the result of this layer assignment using a canvas divided into four horizontal colour bands. Each band corresponds to an architectural layer, from top to bottom: *presentation*, *service*, *domain*, and

data source. Each software component, such as a Java package, is depicted as a large circle. Inside these circles, smaller circles arranged in a packed layout represent individual classes, and within each class, still smaller circles correspond to methods whose colours indicate their layer assignment.

The size of these innermost circles reflects the proportion of methods in a class assigned to a particular layer. This distribution of coloured “tea-bubble” circles provides an immediate impression of how classes within a package are spread across architectural layers.

The package circles are positioned within, between, or outside of the layer bands according to their classification, as described above. An annotated example visualisation of the K-9 Mail application is shown in Figure 6.11.

BubbleTea supports zooming and panning to facilitate detailed inspection of individual packages and classes. Packages assigned to a single layer, or situated between two layers, can be repositioned along the x -axis via drag-and-drop, preserving their y -axis placement to maintain architectural meaning. In contrast, packages not assigned to any layer may be moved vertically to support easier comparison and exploratory analysis.

Key visualisation design decisions: The visualisation in BubbleTea is underpinned by several key design decisions that reflect our focus on providing an architect’s holistic view of the system.

- *The use of predefined architectural layers.* This contrasts with many visualisations that rely on code dependencies for layout. Instead, we adopt well-established definitions of layer responsibilities and assess whether components semantically conform to them.
- *The colours of classes correspond to the colours of layers.* This design choice allows users to assess the homogeneity of the design and to see why a component is mapped to a particular layer—its dominant colour indicates the prevailing responsibility among its methods.
- *Classes within a package are not grouped by layer assignment.* This avoids implying structural separation of classes within a package (i.e., internal dependencies) that may not exist in the source code.

- *Semantics are considered at method level.* We start from the responsibility of each method and its relation to the stereotypical functionality of architectural layers. This fine-grained analysis enables the identification of design issues and improvement opportunities, while maintaining a high-level overview.
- *Packages and classes are represented as circles.* The layer structure surrounding the classes and packages uses rectangular bands. Using rectangular encoding for packages and classes, such as in a treemap visualisation, risks implying a false nested substructure (sublayers). Circles avoid this false implication.

6.4.3 Illustration of Use

Figure 6.11 presents a visualisation of the source code for the Android email client K-9 Mail (version 5.304)². In this section, we discuss observations about the organisation of the source code in K-9 Mail using BubbleTea, illustrating how the visualisation can support architectural reasoning at different levels of abstraction.

Overall or high-level view: The visualisation shows 37 packages in K-9 Mail, with 22 confined to a single layer, 10 spanning two layers, and 5 outside any layer. Notably, over 40% of packages mix layer responsibilities.

As seen in Figure 6.11, packages within a layer typically show classes coloured according to that layer. However, some classes differ in colour (“outliers”), indicating mixed concerns even within well-placed packages.

Domain-related outliers are prevalent even in packages not categorised as such. This reflects the fact that a software application ultimately exists to realise domain logic; therefore, domain-related functionality often appears in presentation and data-source components.

Packages outside the coloured bands may relate to cross-cutting concerns or indicate poor design evolution. These, along with border packages, are prime candidates for refactoring.

²<https://github.com/thunderbird/thunderbird-android/tree/5.304>. This relatively old version was chosen because prior manual analysis on architectural layer assignments had been conducted on it.

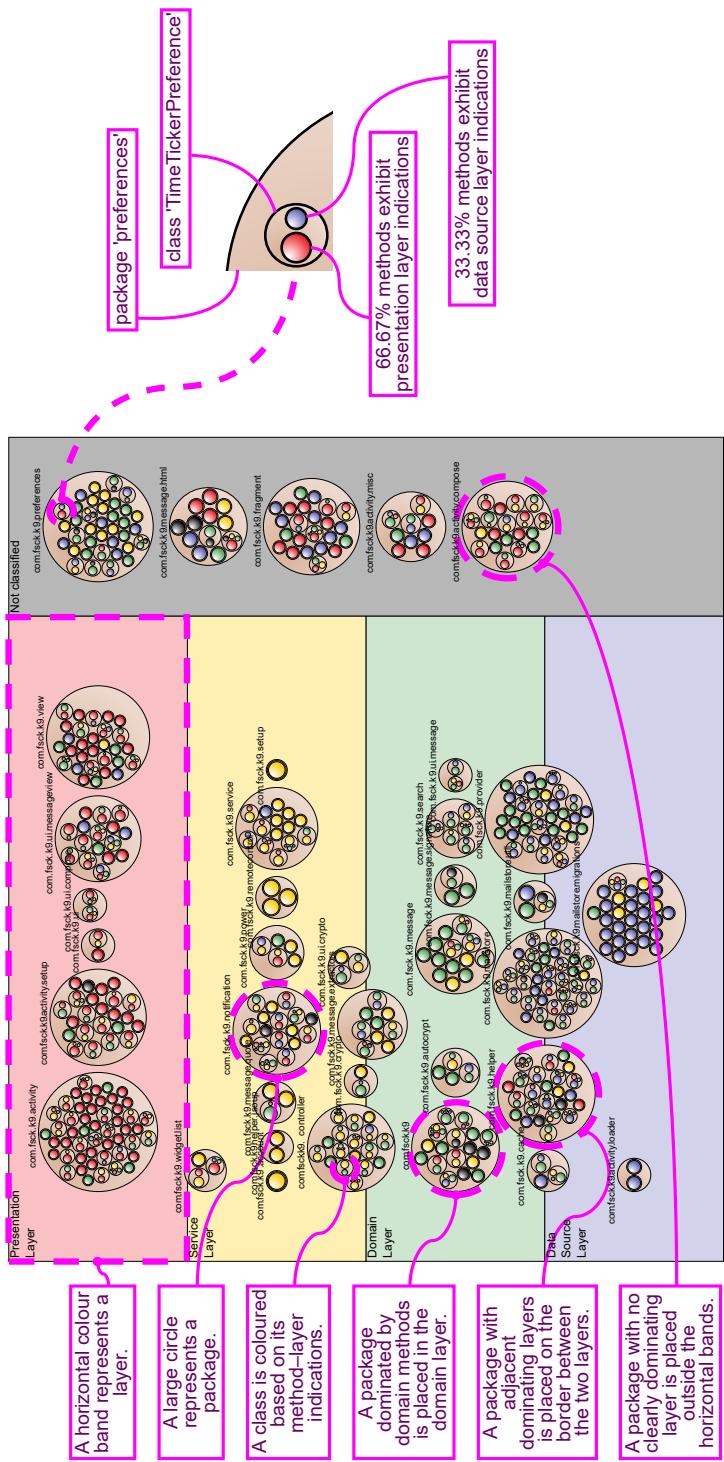


Figure 6.11: The K-9 Mail application v5.304 visualised in BubbleTea, with annotations on the visual elements.

Examining selected packages in detail: An expert analysis of K-9 Mail's package organisation reveals that the *activity* package (`com.fsck.k9.activity`) and its sub-packages belong to the *presentation layer*. Most classes within these packages have dual *presentation* and *domain* indications, consistent with earlier observations on the prevalence of domain responsibilities. However, many *activity* sub-packages appear outside layer bands, suggesting entanglement with other concerns and a need for restructuring.

The *mailstore* package (`com.fsck.k9.mailstore`) and its *migrations* sub-package belong to the *data source* layer, as they interact with databases. However, the *migrations* sub-package shows significant *domain* responsibilities, again aligning with our earlier observations about mixed layer indications.

The *preferences* package (`com.fsck.k9.preferences`) manages various settings, including both domain-level data (e.g., email accounts) and UI configurations (e.g., colour themes, display fonts). It therefore lies outside the main layers in the visualisation, reflecting its cross-cutting role.

6

The *helper* package (`com.fsck.k9.helper`), which one might expect to fall outside the layers, instead shows a focus on *domain* and *data source* concerns. Closer inspection reveals classes with three or more layer indications. For example, the *Contacts* class accesses device-stored contacts (a *domain* and *data source* concern) but also includes a method tightly coupled with the UI for contact creation.

To conclude, BubbleTea reveals the intricate interplay of responsibilities across different layers and packages, highlighting areas where the intended layering structure is maintained and where it is not. It illustrates how design decisions and system evolution often result in mixed responsibilities within a single package. The visualisation thus helps identify potential architectural improvements and guides refactoring efforts.

6.4.4 Reflections

We reflect on the visualisation and the architectural insights it enables regarding the system under inspection, and potential improvements in future iterations.

Cross-cutting vs. functional divergence: Some software systems are explicitly designed with cross-cutting components—modules that span multiple architectural concerns such as logging, configuration, or security. In such systems, the presence of components outside the standard layers may reflect intentional design. However, in systems that do not anticipate this, components that appear in the cross-cutting column often do so due to excessive mixing of unrelated responsibilities, in other words, a divergence from coherent, responsibility-aligned design.

This divergence manifests not only in entirely cross-cutting components but also within packages that span multiple layers. In an ideal scenario, each package would map cleanly to a single architectural layer. In practice, many packages exhibit blended responsibilities or ambiguous roles, and some defy classification altogether.

BubbleTea reveals such structural and semantic tensions by visualising the internal distribution of class-level responsibilities within each package. This internal view supports architectural reflection: it can highlight inconsistencies, prompt discussion about design rationale, and help identify candidates for refactoring or modularisation.

Omitted dependency information: Our current focus has been on semantic responsibility, i.e., mapping what components are intended to do, rather than how they interact. To maintain clarity, BubbleTea does not currently display inter-component dependencies. However, overlaying such dependency information could enhance the tool's explanatory power by revealing not just *where* responsibility lies, but also *how* those responsibilities are interconnected. This presents an opportunity for future integration.

Adding a column dimension: As software architectures often reflect multiple orthogonal concerns, a promising direction for future work is to introduce a column dimension to BubbleTea. For example, if packages can be grouped by functional feature or subsystem, this information could inform their horizontal (x -axis) positioning. Such an arrangement may offer a clearer visual articulation of separation of concerns, complementing the existing vertical layering.

6.5 Threats to Validity

Internal validity: The adoption evidence presented for ClassViz comes from projects that originated within or adjacent to our research group. This introduces selection bias: projects that found ClassViz's assumptions unsuitable and abandoned it early are not represented since they left no published record. For BubbleTea, the layer classification relies on LLM-based method analysis, which introduces misclassification risk that propagates through the aggregation chain from method to class to package; the dominance threshold is additionally a design choice without empirical calibration.

External validity: The ClassViz industrial evaluation involved a single company in a specific domain (ASML). Generalisability to organisations in different domains, or to codebases structured differently from those evaluated, is uncertain. That said, derivative tools built on ClassViz's design principles have been deployed in other industrial settings [101, 208, 227], suggesting that the underlying design approach transfers across domains, even if the specific DSAR frontend has not been independently replicated. BubbleTea's illustrative analysis is similarly limited to a single system, K-9 Mail v5.304, and the technique is defined for hierarchically organised OO languages; applicability to other paradigms is untested.

Construct validity: Both tools' effectiveness is assessed through usage patterns and observed behaviour rather than controlled usability studies, so claims about cognitive load reduction and legibility are grounded in design rationale and observed use, not measured outcomes. Additionally, BubbleTea's central construct of architectural layer adherence is operationalised against a four-layer reference architecture that is not universal; systems designed around different patterns may not map meaningfully onto it. Systematic usability evaluation of both tools should be addressed in future work.

6.6 Conclusions

This chapter has explored how visualisation can support architectural explanation by addressing six requirements, three centred on structural graph exploration (*ClassViz*) and three on semantic layering (*BubbleTea*). These

tools demonstrate how visual encodings bridge the gap between formal representations and human reasoning.

ClassViz emerged from the need to inspect SABO extractions more intuitively. Basic but targeted visual cues such as nested boxes, colour-coded edges, and selective filtering serves to make large graphs legible. Its lightweight design encouraged student and industry uptake, serving as a springboard for experimentation. Over time, this collaborative use reshaped *ClassViz* into a shared platform that supported a broad range of visual overlays across domains. Ultimately, it became a flexible visual frontend for higher-level techniques like DSAR, adapting to serve layered architectural abstractions.

In contrast, *BubbleTea* foregrounds architectural responsibility rather than dependency. It arranges code elements by their semantic intent, not their call relationships. This shift reframes architectural irregularities as semantic misalignments, inviting interpretation rather than rule-based violation detection. Its visual design—fixed horizontal layers, circular packing of nested components, and semantic colour coding—offers a compact yet expressive map of where architecture aligns or diverges from design intentions.

Together, these tools show that visualisation can do more than reveal structure: it can expose architectural purpose, highlight design tensions, and scaffold shared understanding. *ClassViz* anchors exploration in extracted structure; *BubbleTea* surfaces intent and interpretation. Both reflect an ongoing effort to make software architecture visible, meaningful, and improvable.

At the same time, the development and use of these tools highlights a broader challenge in software visualisation practice. The limited industrial adoption of visualisation tools may stem not only from issues of scalability or expressiveness, but also from how such tools position themselves relative to developers' everyday workflows. Many visualisation systems implicitly encourage immersion in a dedicated exploratory environment, requiring users to step outside their normal development context. In contrast, professional software work is highly flow-oriented and distributed across tightly integrated tools such as IDEs, version control systems, and issue trackers.

While immersion is appropriate in domains such as games, where users intentionally disengage from daily workflows, productivity tools risk disrupting developer flow when they demand similar context switching. This suggests that architectural visualisations may be more effective when embedded as lightweight, situational aids within existing workflows rather than positioned as stand-alone environments.

Another insight emerging from this work is that software hierarchies are inherently multidimensional. Developers organise code along multiple orthogonal dimensions: files and folders, namespaces, responsibilities, functional decomposition, and so on. These hierarchies often coexist but are rarely visualised together. For instance, in C++ systems, some components may rely on file structure for logical grouping, while others rely on nested namespaces. When a visualisation chooses one hierarchy (e.g., folder layout), it necessarily occludes the other (e.g., namespace intent), complicating comprehension and explanation.

6

This challenge is not fully addressed by the current prototypes and calls for further investigation. Future tools should explore mechanisms to overlay, switch between, or reconcile competing hierarchies, enabling users to view the system through the organisational lens most aligned with their current task or mental model.

Future work includes empirical validation of these tools in real-world settings, scaling semantic layering to larger and more complex systems, and exploring how visualisation can be combined with complementary explanation mechanisms: while visualisation is suited for interactive exploration and pattern recognition, additional modalities can complement visual interfaces by providing flexible, context-sensitive explanations in natural language. Recent advances in large language models (LLMs) offer a promising direction for this, enabling explanations that can adapt to user queries while remaining grounded in structured representations of software systems.

Acknowledgement

We extend our gratitude to Truong Ho Quang, Ph.D., for his detailed manual analysis of the K-9 Mail package organisation concerning the layered refer-

ence architecture. We also thank all the students and colleagues involved in the subsequent spin-off studies.

7

Can You Take Me Higher? Generating Explanations of Software Systems Across Abstraction Levels

7

While visualisation supports interactive exploration of software architecture, many explanation tasks instead require concise, question-driven responses that synthesise information across abstraction levels in highly complex software systems. Such systems demand explanations that bridge raw code and higher-level architectural insights. Advanced Large Language Models (LLMs) now excel in answering questions about source code. However, their ability to explain software across abstraction levels remains untested. We introduce FLASE, an LLM-based agent that uses retrieval-augmented generation over both a vector database and a knowledge graph of software components and their relationships. In a user study with 16 participants, we compared three retrieval strategies: vector-only, graph-only, and hybrid, against a commercial baseline, assessing correctness, completeness, and comprehensibility. Graph-based retrieval consistently outperformed others for high-level component queries, while vector-based methods were precise for some module-level questions but often incomplete. The hybrid

approach worked well unless overwhelmed by excessive context. Participants preferred graph-based and hybrid explanations over the commercial baseline.

7.1 Introduction

Comprehension and maintenance of complex software systems are challenging [135, 221]. Structured explanations [43, 103, 138] at different levels of abstraction help bridge the gap between code and human understanding, facilitating processes such as debugging, maintenance, and onboarding [44]. Explanations often rely on the knowledge of system experts or internal documentation [175], which presents several challenges. First, experts are frequently unavailable to provide explanations; Kruchten [113] describe this as the architecting antipatterns of “goldplating” and “ivory tower”. Second, expert turnover leads to a loss of organisational knowledge. Directly related to that, third, obtaining relevant explanations is often hindered by unstructured, outdated, or missing documentation [59, 200]. To address these issues, researchers have proposed automated, on-demand documentation generation [162, 165].

7

Recent advances in natural language processing have produced large language models (LLMs) that are used, among others, for understanding and explaining software [141, 144]. AI-powered tools such as GitHub Copilot [78] and GPTutor [33] provide valuable code-level assistance. Yet their ability to coherently explain software at multiple abstraction levels remains unexplored. While newer models allow large contexts that can technically fit entire codebases, Liu et al. [129] show that they struggle to process long contexts, especially when the relevant information is positioned in the middle of the text. There is a clear need for dynamic, context-aware explanations tailored to user needs, linking user questions and grounding responses to both code symbols and higher-level structures to support understanding of functionality and interactions.

To address this, we introduce FLASE, an LLM-based agent that leverages Retrieval-Augmented Generation (RAG) [122, 137] to identify relevant code sections for explanation. We use two knowledge sources: (1) a vector database storing embeddings of source code and code summaries, enabling fuzzy

semantic retrieval; and (2) a graph database representing code symbols (e.g., methods, classes, packages) and their relationships, preserving source code structure. We hypothesise that this dual-source strategy enables adaptable abstraction granularity during explanation generation, combining semantic and structural fidelity.

We conducted a user study with 16 developers of varying experience levels to evaluate this approach. Participants answered predefined questions about a Java codebase, where both FLASE and a commercial baseline provided system explanations. To systematically assess how different components of our approach contribute to explanation quality, we adopted an ablation-based design that examines the impact of different retrieval strategies by selectively disabling or removing them. Such ablation studies are commonly used in empirical machine learning evaluation to justify the role of specific design choices [188]. Inspired by the Co-12 framework for Explainable Artificial Intelligence (XAI) [145, 146], we surveyed participants on three dimensions: *correctness*, *completeness*, and *comprehensibility* (hereafter referred to collectively as Co-3).

Our contributions are:

1. An LLM-based agent combining RAG with vector-based and graph-based retrieval for software explanation at multiple levels of abstraction.
2. A controlled user study comparing three retrieval strategies and a commercial baseline, with quantitative and qualitative assessment.
3. Actionable design principles, i.e., structural clarity, technical precision, and user personalisation, to guide future RAG-based software explanation systems.

Authorship and roles: The experimental design was developed by Floris Pex under the close supervision of the author and other faculty members. Floris Pex implemented the tool, executed the experiment, and collected the data. The analysis and interpretation of the results were conducted by the author.

The rest of this chapter is structured as follows: Section 7.2 reviews related work; Section 7.3 details system design and implementation; Section 7.4

describes experimental setup and evaluation; Section 7.5 presents results, followed by discussion in Section 7.6; Section 7.7 addresses trustworthiness; and Section 7.8 concludes and outlines future directions.

7.2 Background and Related Work

This section situates our work within the research landscape on automated software explanation, focusing on recent developments in LLM-generated code explanations and the capabilities of LLM-based agents. We clarify how our approach differs from or extends existing methods.

7.2.1 Code Explanations Using LLMs

Research on LLM-generated code explanations reveals a recurring trade-off between high-level summaries and detailed walk-throughs. For instance, while MacNeil et al. [132] report that students favour high-level overviews, Leinonen et al. [118] show that the utility of LLM outputs varies with the required granularity. Yet, studies such as Sarsa et al. [178] and Kojima et al. [109] highlight that LLMs often struggle with step-by-step reasoning, especially in handling complex control flow, and important details may be omitted even in patch explanations [191]. Soliman and Keim [194] noted that asking architectural knowledge directly from LLM (in particular, the GPT model) is unreliable: while the model provides initial insights, it exhibits lower precision for complex architectural concepts.

Retrieval-Augmented Generation (RAG) has been proposed as a means to ground LLM outputs in external knowledge, thereby improving factual accuracy and contextual alignment. In RAG-based systems, explanations are generated not solely from the model's parameters but are conditioned on retrieved artefacts such as documents, traces, or structured knowledge. Prior work applies this idea in various domains: Sobrín-Hidalgo et al. [192] align LLM explanations with observed system behaviour in robotics; Pinto et al. [155] demonstrate improved developer documentation through retrieval of relevant code artefacts; and Edge et al. [54] show that combining local and global retrieval over knowledge graphs improves coverage in question answering tasks. However, most prior work applies RAG either with simple

vector retrieval or uses knowledge graphs in limited ways, rarely integrating both to adapt the level of abstraction. This offers little support for dynamically adapting explanations to different scopes or user intents.

LLM-based coding assistants (e.g., GitHub Copilot Chat [6], Cody [195], Chen et al. [33], Ross et al. [167]) typically operate at code or snippet level, and seldom provide consistent explanations that bridge code-level details with system architecture. Recent graph-based methods (CodeOntology [12], Javapers [169], Src2Neo [87]) enable structural modelling and hierarchical summarisation, but are not integrated with adaptive, query-driven explanation mechanisms.

Our work addresses these gaps by combining vector-based and graph-based retrieval within a single RAG-driven agent. The system dynamically selects retrieval strategies and abstraction levels based on the user's question, thus supporting flexible, context-aware explanations that span from code-level details to architectural structure.

7.2.2 LLM-based Agents

LLM-based agents refer to systems that combine a language model with explicit control logic to interpret inputs, select actions, and interact with external resources. Such agents can perform reasoning, planning, and tool use in a coordinated manner, enabling more sophisticated user interaction than single-pass prompting [71, 217]. State-of-the-art LLM agents often adopt modular designs (e.g., separate components for planning, retrieving information from memory, executing actions, observing outcomes, and reflection) to acquire external knowledge and dynamically adjust their behaviour [226]. In practice, however, most evaluations focus on general task automation or decision support, with limited attention to explainability challenges in complex domains such as software engineering. Assessment remains a mix of subjective (e.g., human annotation) and objective metrics, but explicit benchmarks for explanation quality across abstraction levels are lacking.

Our approach differs by explicitly targeting explainability for software structures and by empirically studying how retrieval strategies affect the quality and trustworthiness of agent-generated explanations. We bridge

advances in agent design with open problems in software comprehension, providing both new empirical evidence and actionable principles for future LLM-based explanatory agents.

7.3 Software Explanations with FLASE

This section presents FLASE, our LLM-based agent for generating multi-level software explanations using RAG. FLASE grounds its explanations in two complementary knowledge sources: a vector store for semantic retrieval of code and summaries, and a graph-based representation that preserves explicit structural relations between software elements. By integrating these sources, FLASE operationalises the Explanation Window (cf. Section 3.5.2) as an adaptive lens on system knowledge, enabling explanations to adjust in scope and granularity to user intent.

7.3.1 Knowledge Representation and Storage

FLASE maintains two forms of knowledge used during explanation generation: textual knowledge and graph-based knowledge. Figure 7.1 illustrates how these representations are acquired and stored.

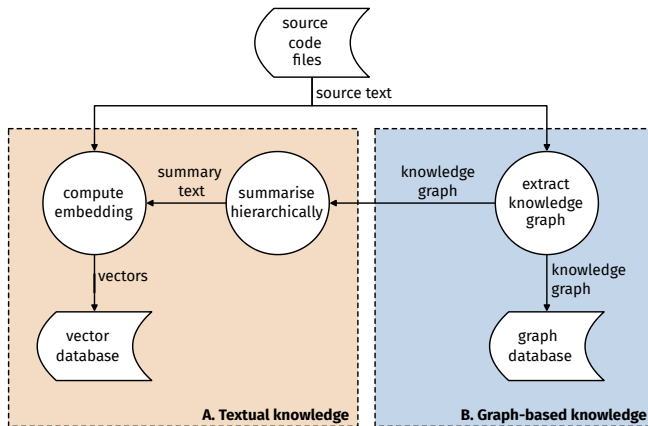


Figure 7.1: Acquisition and storage of a) *Textual knowledge* and b) *Graph-based knowledge* about the software system in question.

Textual knowledge: The vector database stores both raw source code and hierarchical code summaries (method-, class-, and package-level), embedded

to support semantic similarity search [170]. Queries are matched against these embeddings to retrieve text fragments whose semantics are closest to the user's question rather than relying on exact keyword matches. Encoding both code and its abstractions enables retrieval across a range of granularity levels, from fine-grained implementation details to coarse-grained overviews, addressing the tension between detail and summary.

Graph-based knowledge: The knowledge graph encodes software structure: nodes represent packages, classes, and methods, while edges capture containment, dependency, and inheritance relations [169]. We use the SABO graph model described in Chapter 4. Each node links to the source code it represents, allowing retrieval of both relationships and underlying implementations. The graph is queried using Cypher, a declarative graph query language, which enables precise traversal of structural relations and controlled extraction of relevant subgraphs. The graph is extensible: additional architectural views (e.g., deployment or functional decomposition) can be integrated when available, supporting evolving explanation needs across the software life cycle and views.

7.3.2 Context Retrieval and Explanation Generation

FLASE adopts a modular architecture comprising an *Orchestrator*, a *Large Language Model (LLM)*, and a set of specialised *Retriever Chains* (Figure 7.2). Together, these components interpret user questions, select an appropriate retrieval strategy, and generate context-aware explanations.

The process begins when the Orchestrator invokes the LLM to classify a user's natural-language question. Based on this classification, it selects a corresponding Retriever Chain: a modular retrieval policy that specifies both the abstraction level (e.g., component vs. module) and the retrieval mechanism (graph- or vector-based).

This design allows FLASE to adapt retrieval scope and granularity to the user's intent rather than operating at a fixed abstraction level. Broad questions trigger chains that retrieve higher-level architectural context, while more specific questions activate chains that return fine-grained structural or

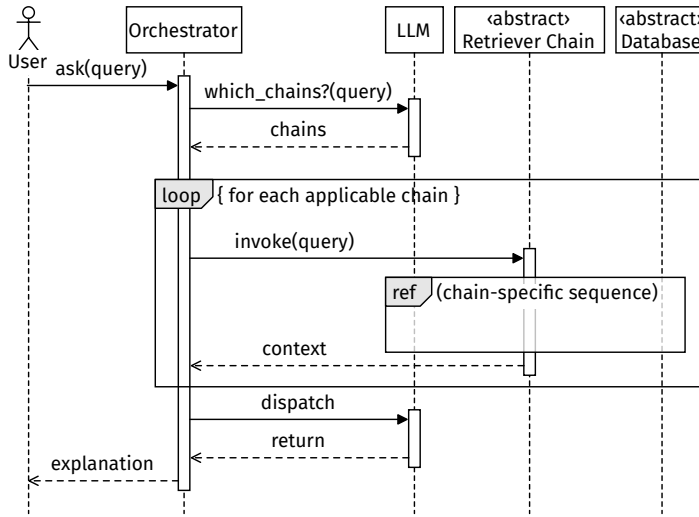


Figure 7.2: The high-level process of generating an explanation.

behavioural information.

7

The classification scheme is grounded in the empirical findings of Chapter 3, which identified recurring patterns in how practitioners seek architectural understanding. We distinguish three *question types*: *Functionality* (what an element does), *Pair-wise interaction* (how two elements interact), and *Other interaction* (what else an element interacts with). Each type is considered across two *question scopes*: component-level (e.g., packages) and module-level (e.g., classes, methods). This 3×2 space defines the supported explanation tasks and determines which Retriever Chain is selected (Table 7.1).

These question types are operationalised by nine Retriever Chains (Table 7.2). Vector-based chains share a unified semantic search strategy, while graph-based chains rely on predefined Cypher query templates tailored to the SABO schema. Each template specifies a fixed query structure with parameterised elements (e.g., node identifiers or scopes), allowing the LLM to instantiate queries without altering their structural form. Using templates rather than ad hoc query generation ensures structural validity, schema conformity, and predictable retrieval behaviour.

Table 7.1: Types of questions supported by the system, with examples.

Type	Component-level	Module-level
Functionality	“What is the functionality of package <i>X</i> ?”	“What is the functionality of class <i>A</i> ?” “What does method <i>M</i> of class <i>A</i> do?”
Pair-wise Interaction	“How do packages <i>X</i> and <i>Y</i> interact?”	“How do classes <i>A</i> and <i>B</i> interact?”
Other Interaction	“What other packages does package <i>X</i> interact with?”	“What other classes does class <i>A</i> interact with?”

Each Retriever Chain encapsulates a context-retrieval policy specifying what knowledge to extract, at which abstraction level, and from which data source. This enables FLASE to autonomously select appropriate retrieval pathways while maintaining reliability and interpretability.

This mechanism realises the Explanation Window framework (Section 3.5.2). The user’s question implicitly positions the Explanation Window: broader questions activate higher-level abstractions, while narrower questions focus retrieval on fine-grained elements. The selected Retriever Chain encodes this positioning.

Finally, the retrieved context, whether as subgraphs or text fragments, is passed to the LLM for explanation synthesis. The resulting answer reflects both the system’s architectural knowledge and the user’s immediate informational needs, filtered through an adaptive lens of relevance and abstraction.

Table 7.2: Summary of retriever chains.

#	Abstraction Level	Query Type	Data Source	Description
1	Module	Functionality	Textual	Performs semantic search on the vector database to retrieve relevant source code snippets.
2	Module	Pair-wise & Other Interaction	Textual	Extracts module names and invokes Chain #1 for each, combining results.
3	Component	All	Textual	Retrieves component summaries via semantic similarity search.
4	Module	Functionality	Graph-based	Retrieves module properties and source code from the knowledge graph.
5	Module	Pair-wise Interaction	Graph-based	Retrieves interacting modules and their properties.
6	Module	Other Interaction	Graph-based	Discovers interacting modules and invokes Chain #5 on each.
7	Component	Functionality	Graph-based	Retrieves modules within a component and applies Chain #4.
8	Component	Pair-wise Interaction	Graph-based	Retrieves interacting components and invokes Chain #7.
9	Component	Other Interaction	Graph-based	Discovers interacting components and invokes Chain #8.

7.4 Experiment Methodology

This section outlines the methodology used to evaluate FLASE in the context of software architecture explanation. The study was designed as a field experiment that combines realistic usage conditions with controlled variables [201]. We selected a comprehensible and well-documented Java codebase, formulated clear and representative question types, and employed a structured evaluation protocol to answer the following research questions:

- RQ7.1** How do different RAG strategies (vector-based, graph-based, and hybrid) compare to a commercial baseline tool in terms of developer-perceived *correctness*, *completeness*, and *comprehensibility* (Co-3) of software explanations?
- RQ7.2** How do comparative performances vary across *question scope* (component vs. module) and *question type* (functionality, pairwise interaction, other interaction)?
- RQ7.3** Beyond the Co-3 dimensions, what emergent themes do developers identify as critical to explanation quality, and how might these insights guide future RAG-based explanation systems?

7.4.1 Evaluation Codebase

We used JPacman for our evaluation: a Java PacMan-style game designed for teaching software testing at TU Delft. We selected JPacman for its clean architecture and wide familiarity of the PacMan game among software practitioners. The codebase contains 53 class-level structures (including interfaces and enums), supporting a range of question types across abstraction levels. Figure 7.3 shows the JPacman codebase structure.

7.4.2 Participant Recruitment

We recruited 16 participants (from 42 invited) through convenience sampling [62]. To mitigate selection bias, we employed purposeful sampling [126] to also select participants aiming for diversity in terms of Java proficiency levels. The pool included both novice and experienced developers, mostly B.Sc./M.Sc. Computer Science students at Eindhoven University of Technology(TU/e). Table 7.3 summarises anonymised participant

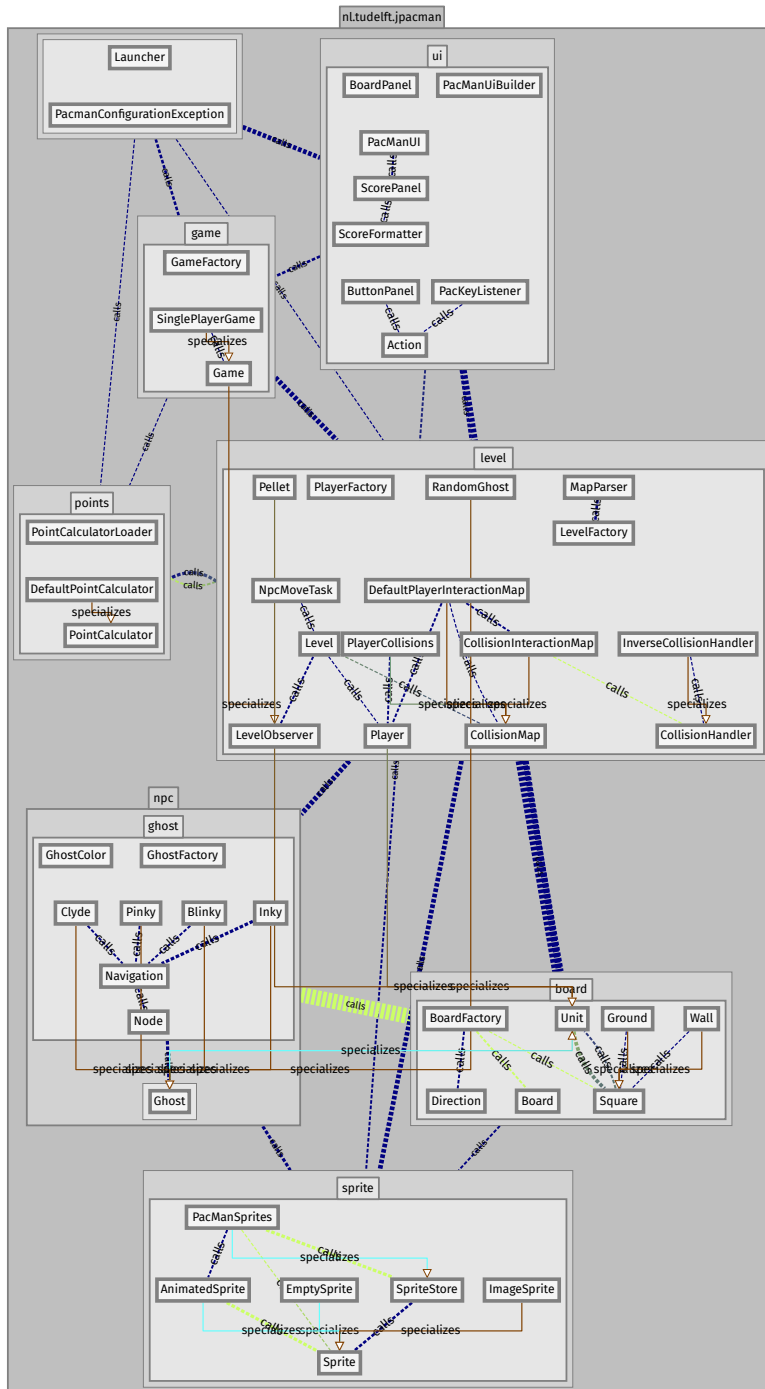


Figure 7.3: The package-and-class structure of JPacman codebase.

demographics, with self-reported Java proficiency level.

Table 7.3: Participants in the user study.

Scope	ID	Years	PPL	Java Proficiency
Component	P1	5	Python	Advanced
	P2	5	Python	Beginner
	P3	5	Python	Intermediate
	P4	5	Python	Intermediate
	P5	5	Python	Intermediate
	P6	7	C#	Intermediate
	P7	5	C#	Intermediate
	P8	6	Typescript	Beginner
	P9	5	Typescript	Beginner
Module	P10	3	Python	Advanced
	P11	3	Python	Intermediate
	P12	4	Flutter	Advanced
	P13	5	Java	Advanced
	P14	3	Python	Beginner
	P15	2	Java + Python	Intermediate
	P16	7	Python	Intermediate

Years: years of development experience,
PPL: primary programming language.

7.4.3 Procedure

The study was carried out in three phases:

Preparation: The study protocol and materials were approved by the TU/e Ethical Review Board, following best practices for ethical software engineering studies [205]. The instruments were tested in a trial run with a volunteer from the same population as the participants. The objective of the trial was to ensure clarity of the instructions for participants and to measure session duration feasibility. Data from this trial run is not included in the analysis. Participants received a study briefing, gave informed consent, and provided background information.

Explanation Generation: Knowledge was extracted from the JPacman code-base. We prepared a standard set of user questions, covering all supported types (Section 7.3.2). Explanations were generated by four agent setups, using the Retriever Chains as described in Table 7.2:

- *Arthur*: vector-only (Chains #1–#3);
- *Bibi*: graph-only (Chains #4–#9);
- *Charlotte*: hybrid (all Chains);
- *Dirk*: commercial baseline (Cody [195]).

Each agent produced explanations for the full question set.

Survey: Participants received the JPacman codebase (exploreable in an IDE), question set, and all generated textual explanations in natural language. Each participant was assigned to a survey variant, either component-level (high-level) or module-level (detailed), to ensure coverage. For each explanation, participants rated the Co-3 dimensions:

- *correctness*: accuracy with respect to codebase,
- *completeness*: coverage of relevant aspects,
- *comprehensibility*: clarity and understandability.

7

Ratings used 5-point Likert scales. Open-ended feedback was also solicited. Participants had full access to the JPacman source code for direct consultation.

The full survey presented to participants is provided in Appendix C.

7.4.4 Qualitative Analysis

The qualitative analysis followed a two-phase process, combining deductive and inductive coding [57]. All coding was performed by a single researcher, i.e., the author, which necessarily introduces interpretive subjectivity—a limitation addressed in Section 7.7.

In the first phase, open-ended responses were deductively coded using the Co-3 dimensions previously described. Codes were assigned wherever responses referenced correctness, completeness, or comprehensibility, either explicitly or implicitly. Representative quotations were recorded to substantiate coding decisions and facilitate verification.

The second phase used inductive analysis to uncover additional insights beyond the Co-3 scheme. This involved open coding to identify new topics,

thematic grouping to cluster related codes, and synthesis of broader themes capturing perceived strengths, weaknesses, and improvement suggestions. Coding continued until thematic saturation (no new codes in two consecutive passes). To enhance transparency and credibility, findings are anchored in verbatim participant extracts.

7.4.5 Technical Details

All LLM-based agents used gpt-3.5-turbo for parity with the commercial baseline (Cody). Embeddings (text-embedding-ada-002) were generated for both code and summary texts and stored in Milvus database. Graph retrieval used Cypher queries on a Neo4J database. The explanation generation length was capped at 250 words to ensure uniform results from all agents and avoid an agent's (dis)advantage over the others due to the amount of text. All explanations were post-processed for consistent formatting: emphases such as bold, italic, and monospaced styling were removed.

7.5 Results

This section presents both the qualitative and quantitative results of the user study. Note that considering the small number of participants (16), quantitative results are reported only to support the qualitative results and are not meant to provide generalisable or robust statistical descriptions of the agents' performance. We first analyse participant feedback using deductive and inductive coding, then report agent performance metrics and participant preferences.

7.5.1 Evaluation by Dimension (Deductive Coding)

All free-text comments were systematically coded according to the three selected criteria: *correctness*, *completeness* and *comprehensibility*. Illustrative findings are presented below, organised by the Co-3 dimensions and compared across the four agents.

Correctness: Participants valued accuracy, favouring explanations that faithfully reflected the codebase while penalising speculation or hallucinated details. At the **component level**, the hybrid (Charlotte) and graph-based

(Bibi) agents generally achieved higher correctness ratings. Their use of the knowledge graph provided structural information, leading to more accurate descriptions of component interactions. For instance, P5 noted Charlotte had *“the most correct information”*. In contrast, the vector-only approach (Arthur) sometimes struggled due to information loss from summarisation, and the baseline (Dirk) occasionally hallucinated non-existent components (e.g., P3: *“mentions components which do not even exist, like ghost or pellet”*; P6: *“Contains non-existent components”*). Arthur’s correctness suffered particularly on interaction questions. At the **module level**, however, direct source code access often gave Arthur and Dirk an advantage, particularly for functionality questions. P10 praised Dirk (*“provides the best information”*), while P12 preferred Arthur (*“most correct description”*). Bibi’s lack of implementation details context hurt its correctness for functionality, though it performed well when explaining multiple interactions. Hedging language like “likely” also undermined perceived correctness for Bibi (P11, P14). Arthur often excelled in explaining pair-wise interactions between classes, potentially benefiting from having the source code of both classes. Charlotte sometimes lagged behind Arthur despite having more context, possibly due to difficulties integrating information.

7

Completeness: Participants assessed completeness based on whether all relevant aspects (classes, methods, interactions—depending on the question) were covered. At the **component level**, Bibi and Charlotte consistently scored highest, leveraging the knowledge graph to ensure comprehensive coverage of relationships. P2 felt Charlotte struck *“the best balance between completeness, readability and length”*. Arthur often lacked depth due to its summary-of-summaries approach (*“[Arthur’s] was mostly just a summation ... did not really go into detail”* – P5). Dirk frequently missed crucial details or components (*“A lot of content ... is left unmentioned”* – P2; *“misses a lot of information”* – P1), likely due to its fragmented RAG strategy retrieving incomplete code snippets. At the **module level**, Arthur and Dirk often performed well for functionality questions, both having access to full class code. However, Dirk’s completeness plummeted for interaction questions

involving multiple classes (*“missed a lot of details”* – P13; *“not complete”* – P15). Bibi excelled in completeness for “other interaction” questions due to the graph queries identifying all related classes (*“most complete and intuitive”* – P13), but struggled with functionality and pair-wise interactions where implementation details matter. Charlotte’s completeness was surprisingly variable, sometimes scoring lower than intuitively expected from an agent that “knows more”, i.e., has more context included. A possible explanation for this might be that the LLM has trouble reasoning with a larger input context [129], as well as the inherent randomness of LLM outputs.

Comprehensibility: Clarity, structure, and conciseness were key drivers of perceived comprehensibility. At the **component level**, Charlotte and Bibi were often praised for their logical structure and readability. P9 found Charlotte *“clear to read, has structured senteces [sic]”*, while P8 noted its structure *“starts from the basis . . . then slowly works down”*. Arthur sometimes scored lower due to buzzwords (*“Provide an immersive gaming experience”* – P2, P4, P5). Dirk’s structure was sometimes criticised (*“leaves everything up to the user”* – P1). At the **module level**, Arthur often achieved the highest comprehensibility, particularly for functionality and pair-wise interactions. Participants liked its narrative style (*“provides a more intuitive ‘story’”* – P13) and effective summaries (*“gives information on both classes . . . summary is given in the end”* – P14). Bibi was frequently criticised for being too method-focused (*“almost completely missing the big picture”* – P13; *“not useful for understanding”* – P10) and vague (P14, P15). Charlotte’s comprehensibility suffered significantly on “other interaction” questions, where it merely listed methods without explanation (*“completely useless”* – P11; *“does not help us understand”* – P16), likely due to poor synthesis of the large hybrid context. Dirk’s comprehensibility was generally moderate.

7.5.2 Emergent Themes (Inductive Coding)

Beyond the three predefined criteria, inductive coding of participant feedback revealed several cross-cutting themes related to explanation quality and user trust.

Structured, navigable explanations: Many participants appreciated responses with clear formatting (e.g., bullet points, consistent structure) and logical flow. P8 noted that Charlotte’s “well-structured” explanation “*starts from the basis . . . and then slowly works down the different supportive classes*”. P4 remarked that Bibi’s enumeration of interactions made understanding “*a bit easier*”. Conversely, dense paragraphs or arbitrary ordering (P2) impeded comprehension. The graph-aware agents (Bibi, Charlotte) often excelled here, likely benefiting from the structure of the graph data. Some participants recommended including visual explanations, such as diagrams (P1: “*visualisations for code structure*”; P3: “*class diagram*”), expecting them to reduce cognitive load. Graph-based approaches naturally lend themselves to generating such visualisations from query results.

Specificity to the codebase: Detailed references to class names, methods, and specific interactions were valued, particularly by participants who wanted to verify explanations against the code. Bibi’s graph-based approach frequently provided this level of specificity. P2 mentioned that Bibi “*gave the most technical and detailed answers*” about component interaction. However, merely listing technical details without context, as Charlotte occasionally did (esp. module Q3), was unhelpful (“*completely useless*” – P2; “*does not help us understand*” – P16). Arthur and Dirk, relying on summaries or code snippets, sometimes lacked sufficient technical depth. Participants desired explicit links or line numbers to the code (P1, P5).

Neutral, technical tone: Several participants criticised explanations, particularly from Arthur and Dirk, for including unnecessary summary text, redundant phrasing, and “buzzwords” or marketing language. P1–2 and P4–5 complained that some answers tried to “*sell*” the code and cited examples of generic slogans about “*immersive gaming experience*” and noted that “*flavor text*” around itemised lists did not add value. P1 noted that filler text, like generic concluding sentences (P8: “*does not have any value at all*”), detracted from clarity and technical value. Bibi and Charlotte were generally less prone to this, especially when explaining interactions.

Personalisation: Results suggest user preferences vary, underscoring the

need for adaptable explanation tools. Some valued concise responses highlighting key functions (P4 liked a *“short and simple”* answer), while others preferred detailed descriptions for verification (P3 praised an answer that was *“very complete and detailed”*, P16 wanted more detail from Dirk). This contrasting feedback indicated differing needs. P1 recommended *“personalised output ... elaborate for beginners, short ... for experts,”* echoed by P2 and P4, who wanted control over the level of detail. Vector-based approaches might suit those seeking brevity, while graph/hybrid strategies offer richer context for others.

Trust: Participants frequently encountered inaccuracies or hallucinations across all agents, undermining trust. Dirk was particularly criticised for inventing components (*“mentions components which do not even exist”* – P3) or misstating interactions (*“D is just wrong. The Game class never calls ...”* – P10). Arthur also made mistakes due to over-summarisation (*“mentions incorrect details”* – P4). Even Bibi and Charlotte were not immune (P7 noted Charlotte *“makes some mistakes”*). In response, participants strongly desired a confidence score or “fact-checking” mechanism (P7) to flag dubious statements and enhance trust. Additionally, vagueness and lack of confidence also reduce trustworthiness. While Bibi was strong in technical detail, its use of uncertain qualifiers like “likely” significantly undermined confidence (*“does not seem to be certain of itself”* – P11; *“makes it less trustworthy”* – P14). Similarly, Charlotte’s explanations were sometimes criticised for being vague, especially at the module level (*“only vaguely describes what the class is”* – P15). Arthur’s high-level summaries could also be vague (*“first sentence is already vague”* – P9). Participants preferred assertive and precise language.

7.5.3 Performance by Agent

To evaluate the relative strengths and weaknesses of each agent, both net preference and detailed rating dimensions were analysed. Net likes (most-liked minus least-liked votes) provide a high-level view of participant preference, while mean ratings for correctness, completeness, and comprehensibility are reported across all combinations of scope and question type. Owing to the

modest sample size (component: 9 participants; module: 7 participants), the quantitative properties described here are best interpreted as exploratory and indicative, and is meant to accompany the qualitative analysis rather than as a robust statistical performance evaluation.

Table 7.4 shows that Bibi is most frequently selected as the preferred answer, followed by Charlotte. Dirk is most often the least liked, while Arthur receives a neutral net preference.

Table 7.4: Net likes by agent (Most-Liked – Least-Liked).

Agent	Net Likes
Arthur	0
Bibi	+10
Charlotte	+4
Dirk	−14

A complete view of each agent’s quantitative performance is presented in Table 7.5, which details mean scores for correctness, completeness, and comprehensibility for each question scope and type, accompanied by a summary note of the corresponding qualitative findings.

Table 7.5: Overview of agent performance across question scope, question type and Co-3 dimension. Underlines mark the highest score for their corresponding question scope $\times$ question type $\times$ dimension.

Agent	Qst. Scope	Question Type	Co1	Co2	Co3	Notes
Arthur	component	functionality	4.11	2.00	3.22	Accurate but superficial; low completeness.
		other interaction	2.11	3.67	3.56	Weak correctness, moderate completeness.
		pair-wise interaction	2.22	2.33	2.89	Performs poorly across metrics.
	module	functionality	3.43	2.57	4.29	High comprehensibility but lacks depth.
		other interaction	4.00	3.86	2.71	Correct but hard to follow.
		pair-wise interaction	4.29	3.86	4.43	Strong across metrics.
Bibi	component	functionality	3.44	4.22	3.56	Detailed and balanced.
		other interaction	3.89	4.22	4.22	Strong across all metrics.
		pair-wise interaction	4.33	4.11	3.78	High correctness and completeness.
	module	functionality	3.14	3.57	2.29	Adequate correctness but weak clarity.
		other interaction	4.43	4.57	3.71	Excellent correctness and completeness.
		pair-wise interaction	3.29	2.29	3.71	Completeness suffers.
Charlotte	component	functionality	4.22	4.00	4.00	Strong across metrics.
		other interaction	4.00	4.44	3.67	Highest completeness.
		pair-wise interaction	4.22	4.22	4.56	Best comprehensibility overall.
	module	functionality	3.14	2.57	3.86	Readable but lacks depth.
		other interaction	3.57	3.43	1.86	Completeness acceptable, clarity poor.
		pair-wise interaction	3.14	3.14	3.29	Middling performance.
Dirk	component	functionality	3.67	2.56	3.89	Clear but incomplete.
		other interaction	1.33	1.89	2.89	Worst performer; incorrect and shallow.
		pair-wise interaction	3.33	2.78	3.22	Average correctness, weak completeness.
	module	functionality	3.86	3.86	3.71	Well rounded.
		other interaction	3.43	1.43	3.43	Very low completeness.
		pair-wise interaction	3.29	3.43	3.86	Average performance.

Co1: Correctness, Co2: Completeness, Co3: Comprehensibility

7.6 Discussion

Here, we interpret the study's findings, comparing agent performance, addressing the stated research questions, and discussing key design implications for automated software explanation systems.

7.6.1 Comparative Strengths and Weaknesses

Arthur (Vector-only RAG): Arthur leverages source code embeddings and summaries stored in a vector database. This approach grants it direct access to implementation details. Consequently, Arthur often achieved high correctness and comprehensibility scores for **module-level** questions, especially pair-wise interactions where having the full source code of both classes proved beneficial. Participants praised its ability to provide intuitive explanations and effective summaries when dealing with specific classes. However, Arthur's performance significantly degraded for **component-level** questions. The process of summarising summaries to build component context led to loss of detail, resulting in explanations that lack depth and completeness. Arthur was often criticised for missing key details or even making incorrect statements about interactions. Its explanations sometimes included irrelevant “buzzwords” or “fluffy sales text”. Overall, Arthur excelled at detailed, low-level explanations but struggled significantly with higher abstraction levels.

Bibi (Graph-only RAG): Bibi retrieves information exclusively from the knowledge graph, which encodes structural relationships such as dependencies and containments. This structured and factual knowledge proved to be highly advantageous for **component-level** questions. Bibi consistently delivered the most accurate and complete explanations for component functionality and interactions, often receiving the highest ratings and “most liked” votes. Participants appreciated the technical detail and clear structure derived from the graph. However, Bibi's lack of access to implementation details was a major weakness for **module-level** questions, particularly regarding class functionality. Explanations were often perceived as shallow and focused too much on method names without clarifying their purpose. Furthermore, the

lack of implementation detail context sometimes led to uncertain language (“likely”), which undermined user trust. Bibi performed well explaining multiple class interactions (“other interactions”), where the graph’s ability to identify all connections was crucial. Bibi represents the strength of structured knowledge for high-level understanding but highlights the need for implementation context at lower levels. Bibi achieves the highest scores in most categories and the strongest net likes (+10).

Charlotte (Hybrid RAG): Charlotte combines context from both the vector database (like Arthur) and the knowledge graph (like Bibi), aiming for the best of both worlds. At the **component level**, Charlotte often performed very well, mirroring Bibi’s high scores in correctness and completeness, while sometimes achieving better comprehensibility. Charlotte was frequently praised for striking a good balance and providing well-structured answers. However, its performance was inconsistent at the **module level** and occasionally even at the component level. When the combined context of both sources became too large, the LLM seemed to struggle to synthesise the information effectively, leading to vague, incomplete, or poorly structured answers. This suggests that simply combining contexts is not always optimal and highlights challenges related to LLM context window limitations and information integration. Charlotte demonstrates the potential of a hybrid approach, but underscores the need for better context management strategies.

Dirk (Commercial Baseline): Dirk uses the commercial tool Cody, which employs its own black-box RAG strategy. Dirk performed adequately for simple, self-contained **module-level** functionality questions, sometimes matching Arthur’s scores. However, Dirk consistently struggled with **component-level** questions and module-level interaction questions. Its approach frequently seemed to result in fragmented context, leading to incomplete answers that missed key interactions or functionalities. Critically, Dirk was prone to hallucinating non-existent components or incorrect relationships, especially at the component level, severely damaging its perceived correctness and trustworthiness. Dirk received the most “least liked” votes overall (net −14), suggesting that current commercial tools may still lag behind specialized

approaches, particularly for higher-level architectural explanations.

7.6.2 Addressing the Research Questions

RQ7.1 – Comparison with baseline: Qualitative evidence indicates that both the graph-based strategy (Bibi) and the hybrid strategy (Charlotte) surpass the commercial baseline (Dirk) on all Co-3 dimensions, while the vector-only strategy (Arthur) occupies a middle ground. Participants repeatedly praised Bibi for its “*most technical and detailed answers*” (P2) and Charlotte for “*the most correct information*” (P5). Dirk, by contrast, was criticised as “*incorrect and shallow*” (P4) and attracted the largest share of least-liked votes. Arthur’s outputs were generally judged concise and often accurate, yet “*mainly a summation of parts*” (P5) that “*does not really explain how they work together*” (P2), leading to low completeness scores for several tasks. Thus, retrieval augmentation grounded in a knowledge graph (Bibi) or in a combined graph-plus-vector context (Charlotte) offers a demonstrable advantage over both a text-only approach and the baseline tool.

7

RQ7.1 Answer: Both the graph-based and hybrid retrieval strategies consistently outperform the commercial baseline on all Co-3 dimensions, while the vector-only approach performs moderately.

RQ7.2 – Performance across question scope and type: Performance differences are strongly modulated by question scope and type. At *component* level, Bibi and Charlotte clearly dominate: they are most liked for functionality and interaction questions and secure the highest completeness ratings. Participants applauded their ability to “*start from the basis and work down the supportive classes*” (P8). In *module*-level tasks, however, direct source access gives Arthur an edge, yielding the top correctness for module functionality. Charlotte struggles when faced with long contexts; it “*provides no useful information and only vaguely describes*” the module (P15), suggesting that the larger hybrid context can overload the LLM. Dirk performs adequately on small, self-contained modules but fares worst whenever broader structural context is needed. These findings underscore the need to match retrieval

strategy to granularity: graph and hybrid approaches excel for architectural views, while raw code snippets remain valuable for fine-grained detail.

RQ7.2 Answer: Graph-based and hybrid methods excel for high-level (component) questions, whereas vector-based retrieval is most effective for detailed module-level tasks.

RQ7.3 – Emergent themes: Beyond correctness, completeness, and comprehensibility, participants highlighted five qualities that drive trust in automated explanations: (i) *structural clarity* (bullet points, headings, optional UML), (ii) *precise technical detail* (e.g., class, method, and file names), (iii) *concise, neutral prose* free of “fluffy sales text” (P2), (iv) *personalisable explanation depth* to suit novices and experts alike, and (v) explicit links to code for *verification*. Shortcomings like using speculative terms such as “likely” and omitting key interactions or hallucinating classes were pointed out across all agents, but most frequently in Dirk’s responses. The fact that Charlotte occasionally outperforms its constituent strategies when it preserves structure, yet falters when the combined context becomes large, points to an important design implication: future RAG systems should dynamically select or weight multiple retrieval streams, filter redundant snippets, and maintain a clear narrative scaffold. Integrating confidence scores, optional diagrams, and user-controlled verbosity would further align with the emergent themes and enhance developer trust.

RQ7.3 Answer: Trustworthy automated explanations require structural clarity, precise technical detail, concise and neutral language, personalisable explanation depth, and explicit code references.

7.6.3 Design Implications for Automated Explanation Systems

While our findings are based on a small-scale user study and should be interpreted as exploratory, several implications emerge for the design of retrieval-augmented, LLM-based software explanation systems.

First, our results suggest that retrieval strategy may need to be dynamically

matched to question scope and type. In this study, graph-based retrieval tended to be more effective for architectural or component-level queries, whereas vector-based retrieval often provided useful details for module- and code-level questions. However, combining both (hybrid) strategies sometimes led to diminished performance if the resulting context was too large or insufficiently filtered. This indicates that future systems might benefit from mechanisms to prioritise, filter, and structure retrieved information according to both the user's question and the model's context limitations.

Second, user satisfaction and trust appear to be influenced not only by correctness and completeness but also by structural clarity and technical precision. Participants valued explanations that used clear formatting (e.g., bullet points, headings), explicit naming of code elements, and a direct, neutral tone. The desire for personalisable explanation depth also emerged, suggesting that offering users some control over verbosity and detail could enhance usefulness for both novices and experts.

Third, several participants indicated that trust is strengthened when explanations include explicit references or links to code, supporting verification. Systems that can trace statements back to code artefacts, or provide confidence indicators, may therefore better support user needs for transparency.

Finally, these findings highlight some possible limitations of current commercial baselines, which tended to lack structural clarity, technical detail, and adaptability. While broader validation is needed, these preliminary results suggest that future research and tool development should further investigate adaptive retrieval strategies, clearer structuring, and user-centric personalisation features.

In general, we recommend interpreting these implications as hypotheses and recommendations for further work, rather than definitive requirements for all contexts or user groups.

7.7 Trustworthiness

In this section, we assess the trustworthiness of our study. The discussion surrounding trade-offs in research design is well-established, with various frameworks proposed to evaluate the validity and reliability of scientific

contributions [31, 39, 126, 184, 224]. We adopt Lincoln and Guba's trustworthiness framework [126].

Credibility: The extent to which trust in the findings has been established [126]. First, participants were recruited through convenience sampling, primarily from the authors' network. This introduces participant bias, as the sample may not be representative of the broader population of developers. To alleviate this issue, we combined the sampling method with purposive sampling, looking for participants who had diverse (self-reported) Java proficiency levels.

The tool and survey were developed by a master's student co-supervised by the author, which may introduce experimenter bias. To mitigate this, we aim for objectivity in survey design and analysis. Each experimental setup was assigned a distinct persona with hidden internal workings, and the study design (including the questionnaire) was reviewed by co-supervisors to ensure fairness.

Transferability: The extent to which results can be transferred to a context of similar nature [126]. The study's findings may not be transferable to all software development environments or programming languages, as the participants were primarily students with Java experience. Additionally, the well-designed and well-documented JPacman project limited the assessment setting. Future studies should include a broader range of programming languages, professional developers, and less documented, more complex codebases to improve transferability.

The Retrieval Chains in FLASE were developed with specific types of user questions in mind. The study setups might have behaved differently with user questions that deviate significantly from those we evaluated. Furthermore, our implementation and Cody both used GPT-3.5, newer and more advanced LLMs could impact the quality of generated explanations, particularly for reasoning with larger input windows.

Dependability: The extent to which methodological decisions have been properly made [126]. The study is based on properties (i.e., correctness, completeness, and comprehensibility), which could have been interpreted

differently by each participant. Explicit definitions were supplied to ensure a standardised understanding, mitigating this issue.

There was a trade-off between the granularity of the survey questions and the time and effort invested by participants. Including more detailed open-ended questions about the assessment process could have enhanced the richness of the findings. Moreover, the explanations assessment could have been influenced by the participants' comprehension and learning preferences. Having validation questions to control for these preferences could have enhanced the analysis and interpretation of the results.

Finally, all qualitative coding was conducted by a single researcher, which may introduce interpretive bias and limit reliability. To strengthen dependability in subsequent work, we recommend employing at least two independent coders, calculating inter-coder agreement (e.g., Cohen's κ) and convening consensus meetings to refine the codebook. Such multi-coder validation would both reduce subjectivity and offer a more robust audit trail for the coding decisions.

7.8 Conclusion and future work

The chapter introduces FLASE, an LLM-based approach that uses RAG to produce dynamic, context-aware software explanations by integrating a vector database for source code with a knowledge graph that captures code symbol relationships. Our user study demonstrates that combining code embeddings and static analysis in a retrieval-augmented LLM agent produces explanations that are more complete, accurate, and comprehensible than those generated by a leading commercial baseline. Graph-based retrieval dominates for architectural questions, vector-based for certain code-level tasks, and hybrid strategies can outperform both, unless the context size becomes excessive. Importantly, participants valued clear structure, technical precision, and personalised depth in explanations, which were lacking in the commercial baseline.

Future work should focus on integrating natural language-based explanations with interactive diagrams, while also prioritising mechanisms for personalisability, structural clarity, and confidence estimation to address the

key concerns identified in this study. Advances in LLMs and adaptive context management are needed to prevent information overload and ensure explanations remain both comprehensive and usable.

Taken together, the contributions of this chapter, alongside the preceding work on representation, enrichment, and visualisation, establish the essential components of an automated, context-aware architecture explanation pipeline. The remaining task is to synthesise these components into a coherent framework that explains how they relate, interact, and can be applied in practice.

Part V

Closing

8

Conclusions

This dissertation set out to establish a systematic framework for automating context-aware software architecture explanations. It addresses the costly gap between the implemented reality of software systems and the architectural knowledge required to maintain and evolve them effectively.

Building on the preceding chapters, which introduced and evaluated the empirical grounding, knowledge representation, enrichment, and complementary presentation modalities of this approach, this chapter integrates these elements into a unified framework. To this end, five interconnected studies were conducted: an initial investigation into practitioners' needs; the design of a unified knowledge model; its enrichment with architectural semantics; and the development of two complementary presentation modalities—interactive visualisation and natural language generation. Together, these provide a validated foundation for establishing the pillars of an automated explanation framework.

This concluding chapter is structured to synthesise and consolidate the dissertation's contributions. Section 8.1 revisits the original research questions and provides consolidated answers based on the results of the five studies. Section 8.2 introduces and defines the core pillars of the proposed framework for automated software architecture explanations, integrating the findings into a coherent architectural vision. Section 8.3 discusses the

theoretical, practical, and academic implications of this work. Section 8.4 and Section 8.5 offer a critical examination of the approach, outlining known boundaries and broader methodological considerations. Finally, Section 8.6 outlines directions for future work, highlighting opportunities to build on and extend the proposed framework in both research and practice.

8.1 Consolidated Answers to the Research Questions

The research presented in this dissertation was designed to answer five guiding questions posed in Section 1.2. The empirical needs identified in the first question informed the design requirements for the knowledge representation in the second question. This representation then served as the necessary foundation for the enrichment techniques developed in the third question. Finally, this enriched knowledge base became the substrate for the two distinct presentation modalities explored in the fourth and fifth questions.

8.1.1 Q1 – Explanations in Practice

How do practitioners explain software architecture in real-world settings, and what challenges and expectations shape these explanations?

8

Chapter 3 presented the first systematic study of software architecture explanation, revealing it as a dynamic, context-sensitive, and interactive practice. Effective explanations are not simple information dumps but are tailored to the explainee's role, experience, and task, centering not just on what the system does and how, but crucially on why key architectural decisions were made. Explanations tend to follow a top-down model, starting with high-level overviews and drawing on a mix of artefacts—especially visual diagrams, both formal and informal—as well as code and documentation. The study also identified key challenges, including cognitive overload, outdated artefacts, and knowledge gaps, which led to the development of the Explanation Window: a conceptual framework for managing the scope and depth of explanations to fit the recipient's cognitive capacity and task at hand.

8.1.2 Q2 – Representation of Software Knowledge

What kind of software knowledge representation can serve as a unified, language-agnostic foundation for automated architecture explanations?

Chapter 4 introduced the Software Architecture/Analysis Basic Ontology (SABO), a labelled property graph designed to bridge the semantic gap between low-level code and high-level architectural concepts, forming the foundation for automated explanation. In response to the needs identified in the explanation-in-practice study, SABO offers a unified, queryable model that supports language-agnostic representation. This is demonstrated across Java, C#, C/C++, JavaScript, and Go. It encodes rich semantic relations, such as invocation, specialisation, typing, and variable usage, enabling deeper structural and behavioural analysis than conventional dependency graphs. Critically, SABO is extensible, allowing new analytical layers (e.g., design patterns, quality metrics) to be integrated over time, making it a dynamic, evolving knowledge base rather than a static model.

8.1.3 Q3 – Enrichment of Structures

How can recovered architectural structures be systematically enriched with semantic and explanatory information to support human understanding?

Chapter 5 tackled the semantic gap in raw SABO graphs by introducing Deductive Software Architecture Recovery (DSAR), a method that enriches code graphs with architectural meaning using Large Language Models to summarise and classify code elements according to a reference architecture. Unlike traditional inductive Software Architecture Recovery (SAR) techniques, DSAR applies a top-down, concept-driven approach, interpreting code in terms of established architectural constructs such as layers and stereotypes. This yields explanations that are more aligned with design intent and context. A case study at ASML validated DSAR's utility for understanding and communicating complex systems, while also revealing that output quality depends critically

on both code quality and the availability of domain knowledge to steer LLM reasoning.

8.1.4 Q4 – Presentation through Visualisation

How can architectural explanations be effectively presented through interactive visualisations that are explorable, comprehensible, and actionable for practitioners?

Chapter 6 tackled the challenge of making enriched SABO graphs accessible to users by developing two interactive visualisation tools: ClassViz and BubbleTea. ClassViz presents a filterable view of class and package relationships overlaid on structural hierarchy, enabling users to inspect DSAR output and verify analysis results within familiar structures. It evolved from a basic inspection tool into a platform adopted in both academia and industry. BubbleTea introduces a novel view of layered architectures based on responsibilities rather than dependencies. Instead of verifying layer placement, it surfaces misalignments between component meanings and functions, offering deeper insights into design integrity.

Both tools underscore the importance of managing software system complexity through abstraction, interactive filtering, stable layouts, and semantically grounded encodings. Yet a deeper insight emerged during development: software systems are inherently multi-hierarchical. Developers organise components using file system-structures, namespaces, layers, or functionality; each represents a valid but orthogonal view. Current tools, including ours, impose a fixed perspective, potentially obscuring others. Overcoming this limitation remains an open challenge for future work in visualisation and comprehension tooling.

8.1.5 Q5 – Presentation through Natural Language

How can large language models be leveraged, in combination with structured knowledge, to generate architectural explanations that are correct, complete, and comprehensible?

Chapter 7 extended natural language into our explanation pipeline with FLASE, an LLM-based agent that uses Retrieval-Augmented Generation (RAG) over the SABO graph to produce multi-level, on-demand textual explanations. A controlled user study showed that graph-informed RAG strategies (graph-only and hybrid with embedding vectors) significantly outperformed both a commercial baseline and vector-only retrieval, especially for higher, architecture-level questions where structural context is critical. These results underscore a central claim of the dissertation: the SABO graph provides the necessary grounding to constrain LLM output, reducing hallucinations, and improving technical accuracy. Qualitative feedback further validated user needs identified earlier, highlighting the importance of structural clarity, precision, and adaptable depth in effective explanations.

8.2 Synthesis: A Framework for Automated Software Architecture Explanations

The five research questions structured this dissertation into a series of inter-linked studies, each addressing a distinct facet of the architecture explanation problem. Yet the overarching goal was not to solve these facets in isolation. Rather, each study contributed to a generalisable methodology for transforming raw code into trustworthy, multimodal, and context-aware explanations. This section synthesises these contributions into a coherent conceptual framework.

As shown in Chapter 3, practitioners are often hindered by the lack of architectural documentation and the structural opacity of code. This revealed the need for a unified substrate that captures both the physical and logical organisation of software. Chapter 4 addressed this need by introducing SABO, a graph-based ontology that captures code elements and their relations in a

precise, language-agnostic form.

However, factual structure alone is insufficient for explanation. Architectural meaning must be derived from structure by interpreting code through design-oriented concepts. Chapter 5 demonstrated how deductive enrichment can overlay structural facts with architectural interpretations such as layers, responsibilities, and stereotypes, guided by reference models and assisted by LLM reasoning.

Finally, even enriched architectural knowledge does not explain itself. Insight arises only when relevant information is selected, composed, and delivered in a form aligned with a particular explainee and task. Chapter 6 and Chapter 7 explored complementary strategies: interactive visualisation and retrieval-augmented natural language generation, for translating enriched representations into usable explanations.

Across these studies, a recurring insight emerged: effective explanations are inherently situated. They depend not only on what is known, but on *what must be shown* to a particular user at a particular moment. This observation motivated the Explanation Window framework and established *context adaptation* as a unifying principle across all stages of explanation.

Together, these contributions form a cumulative pipeline in which raw structural facts are transformed into interpreted architectural meaning and finally into user-aligned insight. Figure 8.1 illustrates this framework, structured around three pillars of Representation, Enrichment, and Presentation, coordinated by the overarching principle of Context Adaptation. The remainder of this section defines the operational role of each element.

8

8.2.1 Pillar 1: Representation (Factual Software Structure)

Representation encodes the raw software system into a precise, language-agnostic form. The SABO graph records entities such as classes, functions, and types, along with their hierarchical and dependency relations. The aim of this pillar is to transform source code into a well-defined, queryable representation. At this stage, the system is faithfully observed and encoded; no interpretation or judgement is added. Representation thus provides the *factual substrate* on which all further reasoning depends.

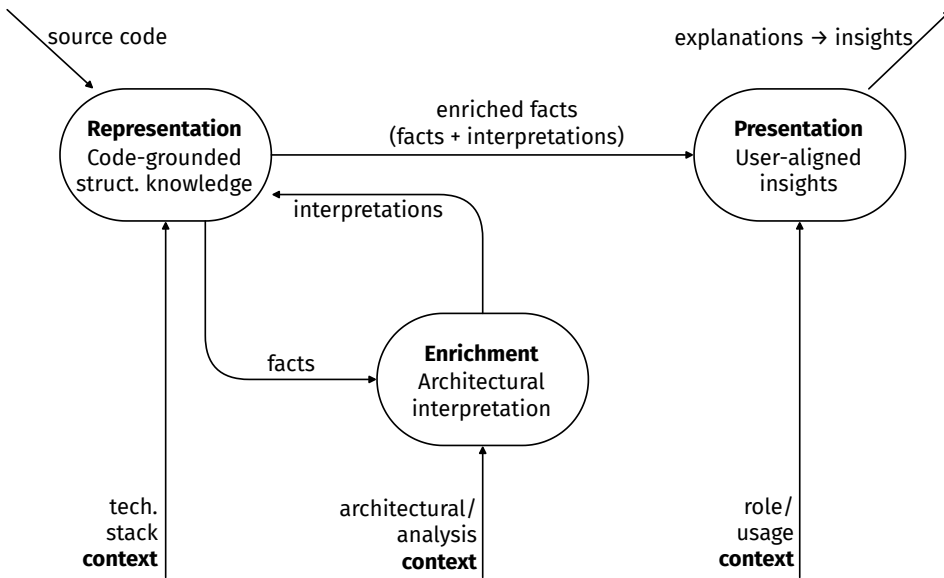


Figure 8.1: A data flow illustration of the proposed framework for automated software architecture explanation.

8.2.2 Pillar 2: Enrichment (Architectural Interpretation)

Enrichment overlays the factual structure with architectural meaning. Through DSAR and related analyses, elements drawn from Representation are mapped to higher-level concepts such as layers, components, responsibilities, or domain roles. This step integrates architectural reference models, contextual knowledge, and patterns to derive meaning that is not explicit in the raw code. Enrichment does not alter the factual layer but builds upon it, producing an integrated interpreted representation that expresses not only *what is present* but *what it signifies* architecturally.

8.2.3 Pillar 3: Presentation (User-Aligned Insights)

Presentation composes insights from the enriched representation in forms tailored to specific tasks, audiences, or purposes. It selects, filters, aggregates, and renders relevant information into diagrams, summaries, or interactive explanations. Visual tools and LLM-based agents translate complex graphs and architectural annotations into human-comprehensible, actionable outputs. If

Representation describes what the system is, and Enrichment describes what the system means, Presentation determines what the user needs to see *right now*. This pillar turns latent knowledge into usable insight.

8.2.4 Overarching Principle: Context Adaptation

Context adaptation is an overarching principle that governs how explanations can be shaped by repository-specific, domain-specific, and user-specific context.

In *Representation*, context adaptation influences how software structure is captured and organised. Different languages, ecosystems, and repositories impose different notions of hierarchy and modularity (e.g., directory structure versus namespaces, or framework-driven versus package-driven organisation). Representation aims to remain factual and non-interpretive, however, contextual assumptions determine which structural dimensions are treated as primary and how raw facts are organised into a usable substrate.

In *Enrichment*, context adaptation determines which architectural interpretations and analyses are meaningful. Problem domain, architectural style, organisational conventions, and quality concerns shape which reference models, architectural concepts, and analytical perspectives should be applied. As a result, enrichment is not a fixed set of analyses but a context-sensitive process that derives architectural meaning appropriate to the system under study.

In *Presentation*, context adaptation governs how enriched knowledge is transformed into insights for human explainees. Here, user role, task, prior knowledge, and cognitive constraints dictate the appropriate scope, abstraction level, and modality of explanation. The Explanation Window operationalises this aspect of context adaptation by regulating which subset of the enriched representation is exposed and how it is composed into visual or textual artefacts.

Taken together, context adaptation ensures coherence across the framework: it aligns what is represented, how it is interpreted, and how it is presented, so that architectural explanations remain relevant, trustworthy, and effective across systems, domains, and users.

It is worth noting that the specific techniques and technologies introduced in this dissertation, such as LLMs, RAG, SABO, DSAR, ClassViz, BubbleTea, and FLASE, are exemplars rather than prescriptions. The framework is intentionally tool-agnostic and extensible: future systems may embody the same principles using emergent representation formats, enrichment mechanisms, retrieval methods, or interface modalities. This contribution articulates not a particular toolchain, but a way of transforming code into architectural explanation.

8.3 Implications

The development and validation of this framework for automated software architecture explanation carry considerable implications that extend across software engineering research, industrial practice, and education.

8.3.1 Implications for Research

This dissertation makes three core contributions with broader implications for software engineering research.

First, it *offers a model for building reliable AI-enabled systems in software engineering*: by grounding LLM outputs in a structured knowledge graph, the framework mitigates hallucinations and ensures factual accuracy. This “hybrid” model of grounding probabilistic generation in a deterministic representation can be adapted and explored for a wide range of other software engineering tasks, including test generation, fault localisation, and traceability analysis.

Second, it *reinforces the value of empirically-driven tool design*. Beginning with a qualitative study of practitioner needs ensures that technical solutions address real-world problems. This challenges the reliance on benchmark-based evaluation and underscores that the effectiveness of AI tools, especially those using fuzzy natural language, must ultimately be assessed in human contexts. When multiple explanations may all be “correct,” static ground truths are insufficient; nuanced human judgement becomes essential.

Finally, the framework *reframes program comprehension as a collaborative process between human and automation tools*. With conversational agents and

interactive interfaces, comprehension shifts from solitary cognition to dialogic interaction. This opens up new research directions on how developers form mental models through such tools, the cognitive strategies involved, and how human–machine collaboration reshapes software expertise.

8.3.2 Implications for Practice

The framework offers practical solutions to long-standing challenges in software architecture practice, particularly in documentation, onboarding, and governance.

Most directly, it *enables automated, up-to-date (“on-demand”) architectural documentation*, addressing the widespread issue of knowledge erosion due to outdated or missing artefacts. The continuous derivation of explanations from source code means that the approach supports living documentation, reducing maintenance overhead. This thus limits architectural drift and improves system sustainability.

It also *supports scalable onboarding and knowledge transfer*. Instead of relying solely on senior developers, newcomers can use visual tools and conversational agents to explore system architecture and get targeted answers, enabling self-directed learning. This reduces bottlenecks and improves knowledge accessibility, as evidenced by positive industry feedback on generated explanatory artefacts.

Finally, the framework *strengthens architectural governance and communication* by making architectural knowledge both inspectable and actionable. Visualisations can support not only technical validation but also cross-role dialogue. They can help communicate architectural risks to non-technical stakeholders, providing evidence for issues like layering violations or architectural drift. This enables developers and architects to justify the need for refactoring or rearchitecting with system-derived insights, rather than relying solely on intuition or abstract principles. This grounding may help elevate governance from informal review to a more transparent, continuous, and evidence-driven process.

Together, these affordances point to the framework’s practical viability in addressing core architectural pain points in real-world contexts.

8.3.3 Implications for Education

Although education was not an empirical focus of this dissertation, our concurrent involvement in teaching software engineering courses and supervising bachelor and master thesis projects provided a practice-informed perspective on the educational relevance of the developed tools and concepts. The reflections below are not grounded in formal pedagogical research, but arise from recurring observations across course design, classroom experience, and supervision. They are included not as validated findings, but as grounded insights into how the framework may support learning, and as pointers to future research opportunities in software engineering education.

Visualisation tools offer *support for teaching abstract architectural concepts*, such as layering, design patterns, and role stereotypes, by grounding them in real, navigable codebases. This makes core ideas more tangible and engaging than traditional static materials. Applying these tools to student-made code may allow learners to visualise their own architectural decisions and receive feedback on design structure and alignment, effectively using the tools as reflective learning aids.

More broadly, the dissertation highlights the need to *train students in human–AI collaboration*. The developed approaches exemplify how students might learn not just to write code, but to critically query and interpret AI-generated outputs, which is an emerging competency in modern software development.

Finally, the framework counters *educational concerns around over-reliance on LLMs*. The anchoring of AI explanations in a structured, verifiable knowledge graph models responsible AI use: not as a shortcut or unquestioned oracle, but as a tool for inquiry. This reinforces critical thinking and problem-solving skills, encouraging students to treat AI as a collaborator to be interrogated, not obeyed. This mindset of critical, collaborative engagement will be essential as AI systems become embedded in software engineering education and practice.

8.4 Limitations

This dissertation proposes a solid foundation for automated software architecture explanation. However, several limitations must be acknowledged. These stem from the scope of empirical validation, methodological constraints, and the inherent volatility of generative AI models. Together, they define the boundaries of the framework's current applicability and guide directions for future work.

8.4.1 Transferability of Empirical Validation

Our empirical studies, although methodologically rigorous, were limited in breadth. The qualitative study in Chapter 3 involved 17 professionals from specific organisational settings; the industrial evaluation of DSAR (Chapter 5) was carried out in a single company (ASML) on Java systems; and the FLASE user study (Chapter 7) used student participants and an academic codebase. These findings, while robust within specific contexts, have yet to be validated across broader organisational and technological settings.

8.4.2 Technical and Methodological Constraints

The framework's current form is constrained by its reliance on static code analysis, which excludes dynamic behaviours essential for understanding runtime characteristics and emergent interactions. Extending the approach to incorporate dynamic analysis remains a key avenue for development.

SABO's language-agnostic design provides generality, but its practical coverage depends on the maturity of language-specific extractors. The approach has been most validated on object-oriented systems (notably Java) and to a lesser extent procedural (C, Go), scientific (MatLab), and low-code (OutSystems) environments, with less exploration in functional or highly dynamic languages such as Python and JavaScript.

The technical parts of this dissertation also omit a critical but distinct dimension of architectural explanation: *design rationale*. In software architecture, the reasoning behind structural decisions, e.g., why certain patterns, layers, or decompositions were chosen, is often as important as the structures themselves. However, rationale is typically undocumented, recorded in ex-

ternal artefacts (e.g., ADRs), or dispersed across informal communication channels. Because this dissertation focuses on source code as the primary input, it does not attempt to model or recover rationale. While emerging work has explored extracting rationale from code or inferring it via LLMs, such techniques remain preliminary and are not within the scope of this research.

Finally, scalability has been tested only on (sub)systems of moderate size. Neither computational performance nor cognitive usability (e.g., visual clutter in large graphs) has been systematically evaluated on massive-scale codebase ecosystems. Addressing these limitations will be essential for extending the framework's scalability and language coverage.

8.4.3 Inherent Volatility of AI Models

LLM-based implementations of framework components, such as DSAR and FLASE, inherit the volatility and non-determinism of contemporary AI systems. Model drift, configuration sensitivity, and the potential for subtle misrepresentations challenge reproducibility and long-term stability. Although the SABO knowledge graph provides grounding to mitigate these risks, human oversight remains indispensable: practitioners must act as the final arbiters of interpretive accuracy and trustworthiness.

8.5 Reflections

Situated at the intersection of software architecture, program comprehension, and artificial intelligence, this dissertation has yielded broader insights into the nature of software knowledge and the evolving role of humans in its interpretation. These reflections move beyond empirical findings to consider their conceptual and philosophical implications.

The framework does not aim to replace the software architect, but rather to elevate their role. In this emerging paradigm, the architect becomes a curator and critical validator of automatically generated explanations rather than its sole producer. Human expertise shifts from manual documentation and rote explanation toward higher-order activities: guiding automation systems in generating explanations, scrutinising their outputs, and integrating

diverse insights into coherent design understanding. The human remains central, focusing on strategic sense-making rather than mechanical detail.

This work exposes a fundamental duality in the way software can be understood. One form of knowledge is formal, verifiable, and structural—in our work, embodied in the SABO graph: a precise map of what the system is. The other is interpretive and contextual: a narrative about how (and why) the system behaves as it does. True comprehension arises from their synthesis: the structure provides rigour and evidence, while the natural-language model offers accessibility and interpretation. The future of comprehension tools lies in the fusion of these two complementary modes.

Concerns about AI-induced skill erosion are legitimate, yet the proposed framework promotes the opposite by cognitive augmentation. As it grounds explanations in a verifiable graph, it fosters active exploration rather than passive consumption. A user may ask a question, then inspect the corresponding structures to verify or extend understanding. This interplay encourages curiosity, critical thinking, and iterative learning, supporting comprehension without diminishing human agency.

8.6 Outlook and Future Work

This dissertation establishes the foundation for a new generation of intelligent software explanation tools, but it represents a beginning rather than a conclusion. Building on the findings and identified limitations, several key directions emerge for extending and maturing the approach.

8

8.6.1 Integration of Dynamic and Behavioural Analysis

Future work should extend the SABO knowledge graph beyond static structure to include dynamic and behavioural data derived from execution traces, performance profiles, and system logs. This would enable explanations that address not only the static *what* but also the dynamic *how*, capturing runtime behaviours, data flows, and emergent interactions to provide a more complete picture of the system.

8.6.2 Multi-Modal Explanation Generation

The current presentation layer treats visual and textual explanations as separate modalities. A natural evolution is to integrate them into a single, multi-modal explanation process where visuals and text are dynamically generated in tandem. For example, a future iteration of FLASE could respond to a query with both a textual summary and a customised ClassViz-like diagram that highlights the relevant components and relationships. Conversely, natural language could serve as an interface for interacting with and filtering the visualisation, creating a seamless, conversational exploration of the system.

8.6.3 Closing the Loop: From Explanation to Refactoring

Currently, our proposed framework is diagnostic. A next step is to make it prescriptive and constructive by leveraging the enriched graph to suggest or semi-automate refactorings. The system could identify architectural issues such as coupling or layer violations and propose corrective actions, which the architect would then review and approve. This would evolve the framework from explanation to active, collaborative system improvement.

8.6.4 Holistic Knowledge Graph Integration

A comprehensive explanation framework must draw from the full spectrum of software artefacts. Future versions of the system should integrate data from requirements, version control, issue tracking, and Architecture Decision Records (ADRs), linking business intent, implementation history, and design rationale within a unified knowledge graph. Such integration would support explanations that connect technical and organisational perspectives.

8.6.5 Longitudinal and Comparative Field Studies

To validate generalisability and long-term impact, future research should involve longitudinal deployments of framework-based tools across varied industrial settings. Extended field studies, spanning months or years, could assess effects on productivity, onboarding, code quality, and architectural integrity. Comparative studies across different domains and technology stacks will be essential for refining the framework and establishing its robustness

in real-world practice. These directions mark the next frontier in making architectural explanations more dynamic, inclusive, and impactful.

In closing, this dissertation has presented a systematic, empirically grounded framework for transforming raw code into architectural explanations that are trustworthy, multimodal, and context-aware. Its contribution lies not only in the specific techniques developed, but in the articulation of explanation as a structured, adaptable, and collaborative process. As software systems grow ever more complex and dynamic, the ability to generate and communicate architectural understanding faithfully and fluently will remain central to effective engineering. This work provides a foundation for that endeavour.

A

Mapping of SABO Node and Edge Labels to Language Constructs

This appendix details the mappings between SABO node and edge labels and the constructs and concepts specific to each language. These mappings are compiled according to the most recent versions of the code-to-graph extraction tool implementations applicable to each supported language.

A.1 Java

This graph extractor is implemented in compliance with the SABO 2.0 schema. It processes Java source code using the Spoon analysis framework to build typed graphs for static analysis and program understanding. The source code for the extractor is available at <https://github.com/rsatrioadi/javapers>.

A.1.1 Nodes

The Mapping of SABO 2.0 node labels to Java language constructs is shown in Table A.1. This table defines how each node type corresponds to concrete Java program elements such as types, operations, and variables.

Table A.1: Mapping of SABO 2.0 node labels in the V2 extractor to Java concepts.

Graph node label	Java concept	Mapping notes
Project	Java project	The <i>Project</i> node represents the entire Java project and serves as the root of the knowledge graph.
Folder	File system directory	Each <i>Folder</i> node corresponds to a directory in the input file system; edges labelled <i>contains</i> connect parent and child folders or files.
File	Java source file	<i>File</i> nodes model individual Java source files. They are contained in folders and declare packages and types.
Scope	Package	<i>Scope</i> nodes correspond to Java packages; packages may contain sub-packages and types (classes, interfaces, enums) via <i>encloses</i> edges.
Type	Class, interface, enum	Each <i>Type</i> node represents a Java type such as a class, interface, or enum, with the <i>kind</i> property distinguishing the exact type.
Variable	Field or method/constructor parameter	<i>Variable</i> nodes represent variables. Fields declared in a class become variables with kind <i>field</i> ; method/constructor parameters create variables with kind <i>parameter</i> .
Operation	Method or constructor	An <i>Operation</i> node models a behaviour-defining element (method or constructor). Each node has a signature and is encapsulated within a type.

A.1.2 Edges

The Mapping of SABO 2.0 edge labels to Java-specific relationships is shown in Table A.2. These edges represent structural and semantic links, such as containment, invocation, type annotations, and inheritance.

Table A.2: Mapping of SABO 2.0 edge labels in the V2 extractor to Java concepts.

Graph edge label	Java concept	Mapping notes
contains	Directory containment	The <i>contains</i> edge links a parent folder to a subfolder or file, modelling the hierarchical file system structure.
includes	Project membership	<i>includes</i> edges connect the project node to the root folders or files provided as input, indicating that these resources belong to the project.
declares	Declaration of types and packages	A <i>declares</i> edge from a file to a type or package indicates that the file declares that class, interface, enum or package.
requires	Import dependency	<i>requires</i> edges connect a file to another file whose types are imported via Java import statements.
encloses	Hierarchical containment	The <i>encloses</i> edge is used twice: packages enclose sub-packages and types, and types enclose nested types.
specializes	Inheritance and interface implementation	A <i>specializes</i> edge links a type to its superclass or implemented interface, capturing extends and implements relationships.
encapsulates	Member ownership	A type encapsulates its fields and operations; <i>encapsulates</i> edges link a type to variables (fields) and operations (methods/constructors) defined in it.
typed	Type annotation	<i>typed</i> edges connect variables (fields or parameters) to their declared types.
returns	Return type	The <i>returns</i> edge links an operation (method) to the type returned by it.
parameterizes	Parameter association	<i>parameterizes</i> edges connect parameter variables to the operation they belong to, indicating the signature of the method or constructor.
invokes	Method invocation	The <i>invokes</i> edge connects an operation to another operation that it calls, modelling method invocations.
instantiates	Object creation	When an operation creates a new instance of a class, an <i>instantiates</i> edge links the operation to the instantiated type.
uses	Field access	<i>uses</i> edges connect an operation to a variable when the operation reads or writes a field.
overrides	Method overriding	The <i>overrides</i> edge links an operation to the operation it overrides in a superclass or interface.

A.2 C#

This extractor is implemented in compliance with the SABO 2.0 schema and uses the Roslyn compiler platform (Microsoft.CodeAnalysis) to parse and analyse C# source code. The implementation is available at <https://github.com/rsatrioadi/csharpers>.

A.2.1 Nodes

The Mapping of SABO 2.0 node labels to C# constructs is provided in Table A.3. This table defines how program elements such as classes, methods, and variables are represented in the graph.

Table A.3: Mapping of SABO 2.0 node labels in the C# extractor to C# concepts.

Graph node label	C# concept	Mapping notes
Project	C# project or solution	Represents the entire C# project or solution; it serves as the root of the knowledge graph and is connected to the top-level folders and files via <i>includes</i> edges.
Folder	File system directory	Each <i>Folder</i> node corresponds to a directory in the project; <i>contains</i> edges link a parent folder to its subfolders and files.
File	C# source file (.cs)	<i>File</i> nodes model individual C# source files; a file belongs to a folder and may declare namespaces and types via <i>declares</i> edges.
Scope	Namespace	<i>Scope</i> nodes represent C# namespaces (including nested namespaces); namespace nodes enclose sub-namespaces and types via <i>encloses</i> edges.
Type	Class, interface, struct, enum or type parameter	<i>Type</i> nodes correspond to the different C# type categories such as classes, interfaces, structs, enums and type parameters; the <i>kind</i> property distinguishes the specific category.
Variable	Field or method parameter	<i>Variable</i> nodes represent member fields and parameters of methods/constructors; the <i>kind</i> property records whether the variable is a field or parameter.
Operation	Method or constructor	An <i>Operation</i> node models a behaviour-defining element such as a method or constructor; it is encapsulated by its defining type and has a signature and return type.
Metric	Software metric	<i>Metric</i> nodes denote computed metrics (e.g., number of methods or statements); these are connected to types or operations via <i>measures</i> edges that store the measured value.

A.2.2 Edges

Table A.4 shows how edge labels are mapped to C#-specific semantic and structural relationships, such as method invocations, field accesses, and inheritance.

Table A.4: Mapping of SABO 2.0 edge labels in the C# extractor to C# concepts.

Graph edge label	C# concept	Mapping notes
contains	Directory containment	The <i>contains</i> edge links a parent folder to a subfolder or file, modelling the hierarchical file system structure.
includes	Project membership	<i>includes</i> edges connect the project node to the root folders or files provided as input, indicating that these resources belong to the project.
declares	Declaration of types, variables and namespaces	A <i>declares</i> edge from a file to a type, variable or namespace indicates that the file declares that entity.
requires	Import dependency	<i>requires</i> edges connect a file to another file whose types are imported via <code>using</code> directives.
encloses	Hierarchical containment	The <i>encloses</i> edge is used when a namespace contains sub-namespaces or types, and when a type contains nested types.
specializes	Inheritance and interface implementation	A <i>specializes</i> edge links a type to its base class or implemented interface, capturing C# <code>extends</code> (inheritance) and <code>implements</code> relationships.
encapsulates	Member ownership	<i>encapsulates</i> edges link a type to the variables (fields) and operations (methods/constructors) defined in it.
typed	Type annotation	A <i>typed</i> edge connects a variable (field or parameter) to its declared type, representing the type annotation in C#.
returns	Return type	The <i>returns</i> edge links an operation (method) to the type returned by it.
parameterizes	Parameter association	<i>parameterizes</i> edges connect parameter variables to the operation they belong to, indicating the signature of the method or constructor.
invokes	Method invocation	The <i>invokes</i> edge connects an operation to another operation that it calls, modelling method calls.
instantiates	Object creation	When an operation creates a new instance of a class, an <i>instantiates</i> edge links the operation to the instantiated type.
uses	Field access	<i>uses</i> edges connect an operation to a variable when the operation reads or writes a field.
overrides	Method overriding	The <i>overrides</i> edge links an operation to the operation it overrides in a superclass or interface.
measures	Metric measurement	A <i>measures</i> edge connects a type or operation to a metric node, with the edge property storing the measured value (e.g., number of methods or number of statements).

A.3 C/C++ via M3 Model

This extractor builds on the M3 intermediate representation for C and C++ programs and complies with the SABO 1.0 schema. It processes the M3

model, generated using Rascal, rather than raw source code. The source code for the extractor is hosted at <https://github.com/Software-Analytics-Visualisation-Team/M3GraphBuilder>.

A.3.1 Nodes

Table A.5 presents the mapping from graph node labels to M3 concepts extracted from C/C++ source code. These nodes correspond to structural program elements such as operations, variables, and types.

Table A.5: Mapping of SABO 1.0 node labels in the M3GraphBuilder to M3 model concepts.

Graph node label	M3 model concept	Mapping notes
Container	Namespace or translation unit container	Nodes labelled <i>Container</i> represent top-level containers in the M3 model. In the C converter, each source file is converted into a Container node. Thus Container nodes correspond to M3 namespace fragments or, in C, to translation units (files).
Structure	Class, template, specialization, translation unit or macro	Structure nodes correspond to M3 structure fragments. The C converter labels class nodes as Structure. These nodes capture M3 structural entities such as classes, templates, specializations and translation units.
Operation	Function, method, deferred function or function template	Operation nodes model M3 operation fragments. The C converter labels function and method nodes as Operation. These nodes represent executable units in the M3 model.
Variable	Parameter or variable fragment	Variable nodes denote M3 variable fragments. In the C converter, parameter and variable nodes are labelled Variable. These correspond to M3 parameters and variables declared in functions, methods or classes.
Primitive	Primitive type	Primitive nodes model M3 primitive types (e.g., <code>int</code> , <code>float</code>). The C converter labels primitive types as Primitive. These nodes represent built-in scalar types in the M3 model.

A.3.2 Edges

Table A.6 defines the mapping of edge labels to M3-level relationships, including containment, parameterisation, and invocation.

Table A.6: Mapping of SABO 1.0 edge labels in the M3GraphBuilder to M3 model concepts.

Graph edge label	M3 model concept	Mapping notes
hasScript	Parent-script association	Links a parent structure (class, namespace, or translation unit) to a script (function, method, etc.), modelling M3 containment of operations. The converter also adds a <i>contains</i> edge from the file.
hasParameter	Function-parameter association	Connects a function to its parameters, matching the M3 parameter list. Each parameter gets an edge from the function node.
returnType	Function return type	Links a function to its return type, reflecting the M3 return type relation.
specializes	Inheritance/specialization	Models inheritance or specialization between structures. Created from a class's <i>extends</i> property.
hasVariable	Structure-member association	Connects structures (e.g., classes) to their member variables, reflecting M3 member declarations.
invokes	Call graph relation	Links calling operations to callees, modelling dynamic calls in the M3 call graph.
contains	Containment relation	Connects files or containers to contained elements (e.g., functions, variables), matching M3 containment.
type	Variable type annotation	Links a variable or parameter to its declared type, modelling type annotations in M3.

A.4 JavaScript

This SABO 2.0-compliant extractor is built using the Acorn JavaScript parser. It supports analysis of standard JavaScript programs and builds graph-based representations for semantic analysis. The source code is available at <https://github.com/rsatrioadi/jspers>.

A.4.1 Nodes

Table A.7 shows the mapping from node labels to JavaScript language constructs, such as functions, variables, and objects.

A.4.2 Edges

Table A.8 defines the edge semantics, mapping graph edges to relationships like scope containment, function invocation, and object member access.

Table A.7: Mapping of SABO 2.0 node labels in the JavaScript extractor to JavaScript concepts.

Graph node label	JavaScript concept	Mapping notes
Project	JavaScript project	The <i>Project</i> node represents the root of a JavaScript project; its properties capture the project name and path, and <i>includes</i> edges connect it to top-level folders.
Folder	Directory	<i>Folder</i> nodes mirror directories in the project; <i>contains</i> edges relate a folder to its subfolders or files.
File	JavaScript source file (.js)	<i>File</i> nodes model individual JavaScript source files; they belong to folders and may declare modules, classes, functions and variables.
Scope	Module	<i>Scope</i> nodes represent ES module scopes created for files that contain exports; the file declares the module and the module encloses its members.
Type	Class	A <i>Type</i> node corresponds to an ES6 class; each class declaration or class expression becomes a Type node with a <i>kind</i> property set to <code>class</code> .
Operation	Function or method	<i>Operation</i> nodes represent functions and methods (function declarations, function expressions, arrow functions and methods defined in classes); each node records the function signature.
Variable	Parameter or variable	<i>Variable</i> nodes correspond to parameters of functions and variables declared at file or function scope; parameter nodes are connected to their function via <i>parameterizes</i> edges.

Table A.8: Mapping of SABO 2.0 edge labels in the JavaScript extractor to JavaScript concepts.

Graph edge label	JavaScript concept	Mapping notes
contains	Directory containment	The <i>contains</i> edge links a parent folder to a subfolder or file, modelling the hierarchical file system structure.
includes	Project membership	<i>includes</i> edges connect the project node to root folders, indicating that these resources belong to the project.
declares	Declaration of modules, functions, classes and variables	A <i>declares</i> edge is added when a file declares a module, function, class or variable; the edge attaches the declared entity to the file.
encloses	Lexical containment	<i>encloses</i> edges connect a module or enclosing function to the functions, classes or variables declared inside it, modelling lexical nesting.
parameterizes	Parameter association	<i>parameterizes</i> edges link parameter variables to the function or method they belong to, capturing the function signature.
requires	Import dependency	<i>requires</i> edges connect a file to another file that it imports via ES module import statements.
invokes	Function invocation	An <i>invokes</i> edge links a function or method to another function or method it calls, representing dynamic function calls.
uses	Variable use	<i>uses</i> edges connect a function to a variable when the function reads the variable; lexical resolution is used to resolve the variable's qualified name.

A.5 Go

The Go extractor is fully compliant with SABO 2.0 and leverages the standard Go toolchain (`go/ast`, `go/types`, `go/packages`, and `gopls`) to analyze Go source code. The implementation is available at <https://github.com/rayhannp1402/gophers>.

A.5.1 Nodes

Table A.9 provides the mapping from graph node labels to Go-specific constructs such as functions, types, and variables.

Table A.9: Mapping of SABO 2.0 node labels in the Go extractor to Go concepts.

Graph node label	Go concept	Mapping notes
Project	Go module or project root	The <i>Project</i> node models the root of a Go module or project; it records the project path and is connected to folders and files via <i>includes</i> edges.
Folder	Directory	<i>Folder</i> nodes correspond to directories in the file system; <i>contains</i> edges link a parent folder to its subfolders and Go files.
File	Go source file (<code>.go</code>)	A <i>File</i> node represents a Go source file; it belongs to a folder and declares packages, types, variables and functions via <i>declares</i> edges.
Scope	Package	<i>Scope</i> nodes model Go packages; each package node encloses the types declared in that package via <i>encloses</i> edges.
Type	Struct, interface or named type	<i>Type</i> nodes correspond to Go type declarations, including structs, interfaces and named types; primitive types are omitted and the <i>kind</i> property distinguishes the specific category.
Variable	Field, variable or parameter	<i>Variable</i> nodes are created for fields, variables and parameters; the <i>kind</i> property indicates whether the node is a field, variable or parameter.
Operation	Function or method	An <i>Operation</i> node represents a function or method; functions and methods are labelled as both <i>Operation</i> and <i>Type</i> in the extractor so they can participate in type relations.

A.5.2 Edges

The relationships between Go program elements, including calls, declarations, and type usage, are shown in Table A.10.

Table A.10: Mapping of SABO 2.0 edge labels in the Go extractor to Go concepts.

Graph edge label	Go concept	Mapping notes
contains	Directory containment	The <i>contains</i> edge links a parent folder to a subfolder or Go file, modelling the hierarchical file system structure.
includes	Project membership	<i>includes</i> edges connect the project node to folders and files, indicating that these resources belong to the Go project.
declares	Declaration of packages, types, variables and functions	A <i>declares</i> edge indicates that a file declares a package, type, variable or function; separate declarations are created for package scope and file scope.
encloses	Package encloses type	<i>encloses</i> edges link a package (scope) to the types declared in it, modelling that a type belongs to a specific package.
encapsulates	Member ownership	<i>encapsulates</i> edges connect a struct or interface to its fields, and a type to its methods; these edges capture the ownership of fields and methods by their enclosing type.
typed	Type annotation	A <i>typed</i> edge connects a variable (field, variable or parameter) to the type it is declared with, representing static type information.
returns	Return type	The <i>returns</i> edge links a function or method to the type of its return values; multiple return types generate separate edges.
parameterizes	Parameter association	<i>parameterizes</i> edges connect parameter variables to the function or method they belong to, recording the function signature.
invokes	Function or method call	The <i>invokes</i> edge links a function or method to another function or method that it calls, modelling call relationships.
uses	Variable use	<i>uses</i> edges connect a function or method to a variable when the function reads or writes that variable.
requires	Import dependency	<i>requires</i> edges connect a file to another file that it depends on via import statements; the imported package name is stored as an edge property.

A.6 Simulink

The Simulink parser generates a SABO 1.0 graph. Its source code is available at <https://github.com/Software-Analytics-Visualisation-Team/SimulinkParser>.

A.6.1 Nodes

Table A.11 maps the node labels defined in the SABO 1.0 ontology to the elements found in Simulink models.

Table A.11: Mapping of SABO 1.0 node labels to Simulink concepts.

Graph node label	Simulink concept	Mapping notes
Container	Folder or subsystem	Represents a package in SABO. Each Simulink model or <i>SubSystem</i> block becomes a container. Folders on disk also become containers, allowing nested packages; parent packages are connected to child packages or blocks via <i>contains</i> edges.
Structure	Block (S-Function, Reference, primitive blocks, Constant blocks)	Each Simulink block is treated as a class-like structure. For S-Function and Reference blocks, the qualified name is derived from the function name or source block. Primitive and constant blocks are also mapped to structures with the block type stored as a property.
Operation	Port (Inport/Outport)	A block's input or output port becomes an operation. Ports are assigned an identifier and kind (<i>method</i> or <i>function</i>) and are connected back to the parent block or subsystem via a <i>hasScript</i> edge.
Variable	Block parameter or constant value	Parameters declared on blocks (e.g., S-Function parameters) and constant values are represented as variables. For S-Functions, each '<P>' element other than <i>FunctionName</i> produces a variable node connected to the block via a <i>hasVariable</i> edge. Constant blocks create an additional variable representing the constant value and link it to the structure via <i>hasVariable</i> .
Primitive	Generic MATLAB object type	A single primitive node ('matlab_object') represents the return type of all operations because Simulink does not provide type information for ports. Every operation connects to this primitive via a <i>returnType</i> edge.

A.6.2 Edges

Table A.12 maps relationships between Simulink elements to the SABO 1.0 edge labels.

Table A.12: Mapping of SABO 1.0 edge labels to Simulink concepts.

Graph edge label	Simulink concept	Mapping notes
contains	Hierarchical containment	Used for multiple relationships: folders containing subfolders or models, a subsystem containing its child blocks, and blocks containing nested blocks. Created when parsing subsystem and primitive blocks.
hasScript	Port ownership	Connects a structure or container node to each of its operation nodes (ports). When a port is created, a <i>hasScript</i> edge is added from the parent block or subsystem to the operation.
hasVariable	Parameter ownership	Links a structure node to variable nodes representing block parameters or constant values. S-Function parameters and constant block values produce <i>hasVariable</i> edges.
returnType	Operation return type	Connects each operation node (port) to the generic <i>matlab object</i> primitive type to record the port's return type.
invokes	Signal connection between ports	For each line connecting an output port to one or more input ports, the parser creates <i>invokes</i> edges between the corresponding operation nodes, modelling the data-flow between blocks.

B

Interview Questions for DSAR Evaluation at ASML

B.1 Background Questions

Ensure that we have selected a concrete software repository at ASML and that the participant has a clear understanding of a component within that repository.

1. How many years of experience do you have as a software professional?
2. For which company are you working for?
3. What is your role in the company?
4. For how long have you been working in the company (years est.)?
5. Which software repository are you working on?
6. For how long have you been working on the software repository (years est.)?
7. Which component(s) within the software repository are you specifically working on?
8. Could you describe (broadly, no need to be specific) the type of software repository you are working on? (e.g., physics computations, hardware interfacing)
9. How frequently do you work on the architecture of the software repository, e.g., on design documents?
☐ Daily ☐ Weekly ☐ Monthly ☐ Yearly ☐ Incidental ☐ Never

10. How frequently do you work on the source code of the software repository?
☐ Daily ☐ Weekly ☐ Monthly ☐ Yearly ☐ Incidental ☐ Never
11. How familiar are you with the software project?
Likert-scale question, 0: very unfamiliar to 6: very familiar.
12. What other tools have you been using to help manage the repository's architecture?

B.2 Correctness Questions

Determine if participants perceive the architecture and summaries as accurately reflecting the system's actual structure and purpose.

1. Likert statements, -3: strongly disagree to +3: strongly agree
 - The architectural layer classification of packages within layers is accurate based on my knowledge.
 - The architectural layer classification of classes within [Package X] is accurate based on my knowledge.
 - The architectural layer classification of methods within [Class Y] is accurate based on my knowledge.
 - The summaries for packages accurately describe their purpose and functionality.
 - The summaries for classes accurately describe their purpose and functionality.
 - The summaries for methods accurately describe their purpose and functionality.
2. Are there any parts in the diagram that do not align with your understanding of the underlying system? Which ones? Why?
3. Are there any parts in the summaries that do not align with your understanding of the underlying system? Which ones? Why?

B.3 Completeness Questions

Evaluate whether the participants feel that the interactive diagram represents all necessary elements to understand the architecture and key component details.

1. Likert statements, -3: strongly disagree to +3: strongly agree

- The architecture diagram provides sufficient information of the system at a high level and is not missing relevant information.
 - The summaries provides sufficient information of the system at a high level and are not missing relevant information.
2. Are there any parts in the architecture diagram where more detailed information would better support your understanding of the underlying system? Which ones? Why?
 3. Are there any parts in the summaries where more detailed information would better support your understanding of the underlying system? Which ones? Why?

B.4 Discriminative Consistency Questions

Assess if the diagram and summaries clearly differentiate components and relationships, and if the level of abstraction in summaries is appropriate.

1. Likert statements, -3: strongly disagree to +3: strongly agree
 - The architecture diagram provides information at an appropriate level of abstraction (e.g., package, class, method).
 - The architecture diagram clearly shows the difference between architectural layers.
 - The summaries provide information at an appropriate level of abstraction (e.g., package, class, method).
 - The summaries clearly show the difference between architectural layers.
 - The summaries help in differentiating two software elements (e.g., packages, classes, methods) that have similar responsibilities (e.g., different concrete implementations of an abstract strategy).
2. How does the architecture diagram support or hinder the differentiation between levels of abstraction of elements in the underlying system?
3. How do summaries support or hinder the differentiation between levels of abstraction of elements in the underlying system?
4. How does the architecture diagram support or hinder the differentiation between architectural layers and their corresponding elements in the underlying system?

5. How do summaries support or hinder the differentiation between architectural layers and their corresponding elements in the underlying system?
6. How do summaries support or hinder the differentiation between the responsibilities of elements of the same kind in the underlying system?

B.5 Compactness Questions

Assess whether participants find the interactive diagram visually manageable and whether the textual summaries are free from unnecessary detail.

1. Likert statements, -3: strongly disagree to +3: strongly agree
 - The architecture diagram is compact, i.e., does not include unnecessary details.
 - The summaries are compact, i.e., do not include unnecessary details.
2. Are there any elements in the architecture diagram that could be simplified or removed to make it more compact? Which ones? Why?
3. Are there any elements in the summaries that could be simplified or removed to make it more compact? Which ones? Why?

B.6 Composition Questions

Assess whether participants understand the recovered architecture and can accurately interpret layers, dependencies, and interactively displayed summaries, as well as the clarity and arrangement of the diagram.

1. Likert statements, -3: strongly disagree to +3: strongly agree
 - The architecture diagram helps me comprehend the underlying system.
 - The generated summaries help me comprehend the underlying system.
 - The layout and arrangement of elements (e.g., packages, classes, methods) and summaries in the architecture diagram help me understand the structure of the architecture.
 - The architecture diagram correctly emphasises the most important part(s) of the system.
 - The user interface options help with the clarity and navigation of the diagram.
2. How could the architecture diagram support or hinder your understanding of the underlying system?

3. How could the summaries support or hinder your understanding of the underlying system?
4. What aspects of the architecture diagram's layout (e.g., class positioning, package arrangement) make it easy or difficult to understand?
5. Are there any elements in the architecture diagram that could be restructured to improve comprehension? Which ones? Why?

B.7 Task Relevance Questions

Evaluate whether the recovered architecture diagram, including summaries, provides useful contextual information that supports real-world tasks (such as feature additions or bug fixes).

1. Likert statements, -3: strongly disagree to +3: strongly agree
 - The architecture diagram could be useful in assisting my day-to-day tasks.
 - The architectural summaries could be useful in assisting my day-to-day tasks.
2. How could the architecture diagram help or hinder your day-to-day tasks?
3. How could the summaries help or hinder your day-to-day tasks?

B.8 Coherence Questions

Assess whether the recovered architecture, including its terminology and representation, aligns with the team's internal terminology and conceptual understanding (or "ontology") of the system.

1. Likert statements, -3: strongly disagree to +3: strongly agree
 - The terminology and names used in the architecture diagram match the terminology commonly used within our team.
 - The terminology and descriptions used in the summaries match the terminology commonly used within our team.
2. Are there any terms, descriptions, or structural representations in the recovered architecture that do not align with how your team describes or conceptualises these elements? If so, what are they, and what adjustments would make them more coherent?

B.9 Closing

- Any other feedback for the architecture diagram, summaries, or the study itself?
- If you have used other tools with similar goals, how do they compare to this?
- What valuable features could be added on top of our tool?

C

FLASE User Study Documents

C.1 Background Form

1. What is the highest degree or level of school you have completed? If currently enrolled, highest degree received.
☐ Bachelor's degree ☐ Master's degree ☐ Doctorate degree
2. How many years of experience with software development do you have?
3. What is the primary programming language you use?
4. What is your proficiency level in Java?
☐ Beginner ☐ Intermediate ☐ Advanced
5. What is your experience with IDEs? (List any Integrated Development Environments (IDEs) you have used before, e.g., Eclipse, IntelliJ IDEA, Visual Studio Code)
6. Do you have previous experience with software comprehension tools?
☐ Yes ☐ No
Briefly explain your answer.
7. Have you participated in similar user studies before?
☐ Yes ☐ No
Briefly explain your answer.
8. Is there anything else you would like to share that might be relevant to your participation in this study?

C.2 Main Survey

For each of the six questions below, the four answers originating from setup 1, setup 2, setup 3, and the commercial tool Cody are given, and named Arthur, Bibi, Charlotte, and Dirk respectively.

1. What is the functionality of the *CollisionInteractionMap* class?
2. How do the *PointCalculator* and *Game* classes interact?
3. What other classes does the *Launcher* class interact with?
4. What is the functionality of the *level* component?
5. How do the *game* and *level* components interact?
6. What other components does the *game* component interact with?

The following questions are asked to the participant:

1. Rate each answer on a scale of 1–5 based on correctness.
2. Rate each answer on a scale of 1–5 based on completeness.
3. Rate each answer on a scale of 1–10 based on comprehensibility.
4. What answer do you like the most?
Explain your answer.
5. What answer do you dislike the most?
6. Is something missing in all answers that you would like to see included?
7. Is there something included in all answers that you particularly like?
8. Is there something included in all answers that you particularly dislike?

C.3 Post-study Questionnaire

1. Compared to your expectations, did the answers meet, exceed, or fall short in terms of providing insights into the codebase?
☐ Meet ☐ Exceed ☐ Fall short
Briefly explain your answer.
2. Did you find the answers helpful in understanding the codebase?
☐ Yes ☐ No
Briefly explain your answer.

3. Would you consider using a tool capable of generating the provided answers in future software development projects?

☐ Yes ☐ No

Briefly explain your answer.

4. How do you think the answers could be improved to better assist developers in understanding the codebase?
5. Do you have any additional comments or feedback about the provided answers?

Bibliography

- [1] Ala Abuthawabeh and Dirk Zeckzer. IMMV: An Interactive Multi-Matrix Visualization for Program Comprehension. In *2013 First IEEE Working Conference on Software Visualization (VISOFT)*, pages 1–4. IEEE, 2013. doi:10.1109/vissoft.2013.6650549.
- [2] Emad Aghajani, Csaba Nagy, Olga L. Vega-Márquez, Mario Linares-Vásquez, Laura Moreno, Gabriele Bavota, and Michele Lanza. Software Documentation Issues Unveiled. In *41st International Conference on Software Engineering*, pages 1199–1210, 2019. doi:10.1109/ICSE.2019.00122.
- [3] Ali Al-Kaswan, Sebastian Deatc, Begüm Koç, Arie van Deursen, and Maliheh Izadi. Code Red! On the Harmfulness of Applying Off-the-shelf Large Language Models to Programming Tasks. *Proceedings of the ACM on Software Engineering*, 2(FSE):2477–2499, 2025. doi:10.1145/3729380.
- [4] Nedhal A. Al-Saiyd. Source Code Comprehension Analysis in Software Maintenance. In *2nd International Conference on Computer and Communication Systems*, pages 1–5, 2017. doi:10.1109/C-COMS.2017.8075175.
- [5] Domenico Amalfitano, Marco De Luca, Tiziano Santilli, Patrizio Pelliccione, and Anna R. Fasolino. Automated Software Architecture Design Recovery from Source Code Using LLMs. In Vasilios Andrikopoulos, Cesare Pautasso, Nour Ali, Jacopo Soldani, and Xiwei Xu, editors, *Software Architecture*, pages 73–89, Cham, 2026. Springer Nature Switzerland. ISBN 978-3-032-02138-0. doi:10.1007/978-3-032-02138-0_5.

- [6] Megha Anand. About GitHub Copilot Chat in Visual Studio – Visual Studio (Windows). <https://learn.microsoft.com/en-us/visualstudio/ide/visual-studio-github-copilot-chat?view=vs-2022>. [Accessed 08-01-2025].
- [7] Dmitry Anikin, Oleg Borisenko, and Yaroslav Nedumov. Labeled Property Graphs: SQL or NoSQL? In *2019 Ivannikov Memorial Workshop (IVMEM)*, pages 7–13. IEEE, 2019. doi:10.1109/ivmem.2019.00007.
- [8] Maryam Arab, Thomas D. LaToza, Jenny Liang, and Amy J. Ko. An exploratory study of sharing strategic programming knowledge. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, pages 1–15, 2022. doi:10.1145/3491102.3502070.
- [9] ASML Holding NV. ASML 2024 Annual Report (based on US GAAP). Technical report, ASML Holding NV, 2024. URL <https://ourbrand.asml.com/m/79d325b168e0fd7e/original/2024-Annual-Report-based-on-US-GAAP.pdf>. Accessed September 5, 2025.
- [10] Matteo Asuni. Effective Graphical Visualization of Vulnerabilities in C and C++ Programs. Master’s thesis, University of Cagliari, Faculty of Engineering and Architecture, Cagliari, Italy, 2024.
- [11] Althaaf K. Atisomya. Pengembangan Alat Visualisasi Arsitektur Perangkat Lunak untuk Analisis Multidimensi. Bachelor’s thesis, Institut Teknologi Bandung, Sekolah Teknik Elektro dan Informatika, Bandung, Indonesia, 2025.
- [12] Mattia Atzeni and Maurizio Atzori. CodeOntology: RDF-ization of source code. In Claudia d’Amato, Miriam Fernandez, Valentina Tamma, Freddy Lecue, Philippe Cudré-Mauroux, Juan Sequeda, Christoph Lange, and Jeff Heflin, editors, *International Semantic Web Conference*, pages 20–28, Cham, 2017. Springer-Verlag, Springer International Publishing. ISBN 978-3-319-68204-4.

- [13] M. Ali Babar, Patricia Lago, and Arie Van Deursen. Empirical Research in Software Architecture: Opportunities, Challenges, and Approaches. *Empirical Software Engineering*, 16(5):539–543, 2011. doi:10.1007/s10664-011-9168-6.
- [14] Leonardo Banh and Gero Strobel. Generative Artificial Intelligence. *Electronic Markets*, 33(1):63, 2023. doi:10.1007/s12525-023-00680-1.
- [15] Victor R. Basili, Mikael Lindvall, Myrna Regardie, Carolyn Seaman, Jens Heidrich, Jürgen Münch, Dieter Rombach, and Adam Trendowicz. Linking Software Development and Business Strategy Through Measurement. *IEEE Computer*, 43(4):57–65, 2010. doi:10.1109/MC.2010.108.
- [16] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 2021.
- [17] Alvine B. Belle, Ghizlane El Boussaidi, and Sègla Kpodjedo. Combining Lexical and Structural Information to Reconstruct Software Layers. *Information and Software Technology*, 74:1–16, 2016. ISSN 0950-5849. doi:10.1016/j.infsof.2016.01.008.
- [18] Gustav Bergström, Fadhl Hujainah, Truong Ho-Quang, Rodi Jolak, Satrio Adi Rukmono, Arif Nurwidyanoro, and Michel R.V. Chaudron. Evaluating the Layout Quality of UML Class Diagrams Using Machine Learning. *Journal of Systems and Software*, 192:111413, 2022. doi:10.1016/j.jss.2022.111413.
- [19] Tingting Bi, Peng Liang, Antony Tang, and Chen Yang. A Systematic Mapping Study on Text Analysis Techniques in Software Architecture. *Journal of Systems and Software*, 144:533–558, 2018. ISSN 0164-1212. doi:10.1016/j.jss.2018.07.055.
- [20] Christian A. Bird. Interviews. In Tim Menzies, Laurie Williams, and Thomas Zimmermann, editors, *Perspectives on Data Science for Software*

- Engineering*, pages 125–131. Morgan Kaufmann, Boston, 2016. ISBN 978-0-12-804206-9. doi:10.1016/B978-0-12-804206-9.00025-8.
- [21] Linda Birt, Suzanne Scott, Debbie Cavers, Christine Campbell, and Fiona Walter. Member Checking: A Tool to Enhance Trustworthiness or Merely a Nod to Validation? *Qualitative health research*, 26(13): 1802–1811, 2016. doi:10.1177/1049732316654870.
- [22] Gordon Blair. Complex Distributed Systems: The Need for Fresh Perspectives. In *2018 IEEE 38th International Conference on Distributed Computing Systems (ICDCS)*, pages 1410–1421. IEEE, 2018. doi:10.1109/icdcs.2018.00142.
- [23] Remco C. de Boer, Rik Farenhorst, Patricia Lago, Hans van Vliet, Viktor Clerc, and Anton Jansen. Architectural Knowledge: Getting to the Core. In *International Conference on the Quality of Software Architectures*, pages 197–214. Springer-Verlag, 2007. doi:10.1007/978-3-540-77619-2_12.
- [24] Fabiano Borges Ruy, Ricardo de Almeida Falbo, Monalessa Perini Barcellos, Simone Dornelas Costa, and Giancarlo Guizzardi. SEON: A Software Engineering Ontology Network. In *Knowledge Engineering and Knowledge Management: 20th International Conference, EKAW 2016, Bologna, Italy, November 19-23, 2016, Proceedings 20*, pages 527–542. Springer-Verlag, 2016. doi:10.1007/978-3-319-49004-5_34.
- [25] Jan Bosch, Helena H. Olsson, and Ivica Crnkovic. Engineering AI Systems: A Research Agenda. *Artificial Intelligence Paradigms for Smart Cyber-Physical Systems*, pages 1–19, 2021. doi:10.4018/978-1-7998-5101-1.ch001.
- [26] Eric Bouwers, Arie van Deursen, and Joost Visser. Dependency Profiles for Software Architecture Evaluations. In *2011 27th IEEE International Conference on Software Maintenance (ICSM)*, pages 540–543. IEEE, 2011. doi:10.1109/icsm.2011.6080827.

- [27] Ruven Brooks. Towards a Theory of the Comprehension of Computer Programs. *International Journal of Man-Machine Studies*, 18(6):543–554, 1983. ISSN 0020-7373. doi:10.1016/S0020-7373(83)80031-5.
- [28] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language Models Are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [29] Aditya Budi, Lucia, David Lo, Lingxiao Jiang, and Shaowei Wang. Automated Detection of Likely Design Flaws in Layered Architectures. *SEKE 2011: 23rd International Conference on Software Engineering and Knowledge Engineering, Miami Beach, 7-9 July 2011: Proceedings*, July 2011.
- [30] José J. Camacho, Jenny M. Sánchez-Torres, and Ernesto Galvis-Lista. Understanding the Process of Knowledge Transfer in Software Engineering: A Systematic Literature Review. *International Journal of Soft Computing and Software Engineering [JSCSE]*, pages 219–229, 2013. ISSN 2251-7545.
- [31] Donald T. Campbell and Julian C. Stanley. *Experimental and Quasi-Experimental Designs for Research*. Ravenio Books, 2015. ISBN 978-0395307878.
- [32] Gerardo Canfora, Massimiliano Di Penta, Rocco Oliveto, and Sebastiano Panichella. Who Is Going to Mentor Newcomers in Open Source Projects? In *20th International Symposium on the Foundations of Software Engineering*, New York, 2012. Association for Computing Machinery. ISBN 9781450316149. doi:10.1145/2393596.2393647.
- [33] Eason Chen, Ray Huang, Han-Shin Chen, Yuen-Hsien Tseng, and Liang-Yi Li. GPTutor: A ChatGPT-Powered Programming Tool for Code

- Explanation. In *International Conference on Artificial Intelligence in Education*, pages 321–327. Springer-Verlag, 2023.
- [34] Xiaojun Chen, Shengbin Jia, and Yang Xiang. A Review: Knowledge Reasoning Over Knowledge Graph. *Expert Systems with Applications*, 141:112948, 2020. doi:10.1016/j.eswa.2019.112948.
- [35] Noptanit Chotisarn, Leonel Merino, Xu Zheng, Supaporn Lonapala-wong, Tianye Zhang, Mingliang Xu, and Wei Chen. A Systematic Literature Review of Modern Software Visualization. *Journal of Visualization*, 23(4):539–558, 2020. doi:10.1007/s12650-020-00647-w.
- [36] Michael L. Collard, Michael J. Decker, and Jonathan I. Maletic. sr-cML: An Infrastructure for the Exploration, Analysis, and Manipulation of Source Code: A Tool Demonstration. In *2013 IEEE International conference on software maintenance*, pages 516–519. IEEE, 2013. doi:10.1109/ICSM.2013.85.
- [37] Allan Collins, John S. Brown, Ann Holum, et al. Cognitive Apprenticeship: Making Thinking Visible. *American educator*, 15(3):6–11, 1991.
- [38] Simona Colucci, Francesco M. Donini, and Eugenio Di Sciascio. A Review of Reasoning Characteristics of RDF-Based Semantic Web Systems. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 14(4):e1537, 2024. doi:10.1002/widm.1537.
- [39] Thomas D. Cook, Donald T. Campbell, and Arles Day. *Quasi-Experimentation: Design & Analysis Issues for Field Settings*, volume 351. Houghton Mifflin Boston, 1979.
- [40] Juliet Corbin and Anselm Strauss. *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*. Sage publications, 2014.
- [41] Bavo Coremans, Arjen L. Klomp, Satrio Adi Rukmono, Jacob Krüger, Dirk Fahland, and Michel R.V. Chaudron. Process Mining from Jira

- Issues at a Large Company. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 425–435. IEEE, 2023. doi:10.1109/ICSME58846.2023.00055.
- [42] Alvaro Costa Neto, Maria J. Varanda Pereira, and Pedro R. Henriques. An Ontology to Understand Programming Cocktails. In *2024 19th Conference on Computer Science and Intelligence Systems (FedCSIS)*, pages 453–464, 2024. doi:10.15439/2024F7885.
- [43] Jielun Cui, Yutong Zhao, Chong Yu, Jiaqi Huang, Yuanyuan Wu, and Yu Zhao. Code Comprehension: Review and Large Language Models Exploration. In *2024 IEEE 4th International Conference on Software Engineering and Artificial Intelligence (SEAI)*, pages 183–187. IEEE, 2024. doi:10.1109/seai62072.2024.10674263.
- [44] Barthélemy Dagenais, Harold Ossher, Rachel K.E. Bellamy, Martin P. Robillard, and Jacqueline P. de Vries. Moving into a New Software Project Landscape. In *32nd ACM/IEEE International Conference on Software Engineering*, pages 275–284, New York, NY, USA, 2010. Association for Computing Machinery. ISBN 9781605587196. doi:10.1145/1806799.1806842.
- [45] Sayed M.H. Dehaghani and Nafiseh Hajrahimi. Which Factors Affect Software Projects Maintenance Cost More? *Acta Informatica Medica*, 21(1):63, 2013. doi:10.5455/aim.2012.21.63-66.
- [46] Themistoklis Diamantopoulos, Antonis Noutsos, and Andreas Symeonidis. DP-CORE: A Design Pattern Detection Tool for Code Reuse. In *Proceedings of the Sixth International Symposium on Business Modeling and Software Design (BMSD 2016)*, page 160, 2016. doi:10.5220/0006223301600167.
- [47] Stephan Diehl. *Software Visualization: Visualizing the Structure, Behaviour, and Evolution of Software*. Springer-Verlag, 2007.

- [48] Lei Ding and Nenad Medvidovic. Focus: A Light-weight, Incremental Approach to Software Architecture Recovery and Evolution. In *Working IEEE/IFIP Conference on Software Architecture*, pages 191–200, 2001. doi:10.1109/WICSA.2001.948429.
- [49] Carlos E. Duarte. Automated Microservice Pattern Instance Detection Using Infrastructure-as-Code Artifacts and Large Language Models. In *22nd International Conference on Software Architecture Companion*, pages 161–166. IEEE, 2025. doi:10.1109/ICSA-C65153.2025.00030.
- [50] Stéphane Ducasse, Nicolas Anquetil, Muhammad U. Bhatti, Andre C. Hora, Jannik Laval, and Tudor Girba. MSE and FAMIX 3.0: an interexchange format and source code model family. 2011.
- [51] Steve Easterbrook, Janice Singer, Margaret-Anne Storey, and Daniela Damian. Selecting Empirical Methods for Software Engineering Research. In Forrest Shull, Janice Singer, and Dag I.K. Sjøberg, editors, *Guide to Advanced Empirical Software Engineering*, pages 285–311. Springer London, London, 2008. ISBN 978-1-84800-044-5. doi:10.1007/978-1-84800-044-5_11.
- [52] Felipe Ebert, Alexander Serebrenik, Christoph Treude, Nicole Novielli, and Fernando Castor. On Recruiting Experienced GitHub Contributors for Interviews and Surveys on Prolific. In *International Workshop on Recruiting Participants for Empirical Software Engineering*, 2022.
- [53] Thomas W. Edgar and David O. Manz. Exploratory Study. In *Research Methods for Cyber Security*, chapter 4, pages 95–130. 2017. ISBN 978-0-12-805349-2. doi:10.1016/B978-0-12-805349-2.00004-2.
- [54] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert O. Ness, and Jonathan Larson. From Local to Global: A Graph RAG Approach to Query-Focused Summarization, 2025.

- [55] Lisa Ehrlinger and Wolfram Wöß. Towards a Definition of Knowledge Graphs. *SEMANTiCS (Posters, Demos, SuCCESS)*, 48(1-4):2, 2016.
- [56] Ghizlane El Boussaidi, Alvine B. Belle, Stéphane Vaucher, and Hamed Mili. Reconstructing Architectural Views from Legacy Systems. In *19th Working Conference on Reverse Engineering*, pages 345–354, 2012. doi:10.1109/WCRE.2012.44.
- [57] Satu Elo and Helvi Kyngäs. The Qualitative Content Analysis Process. *Journal of Advanced Nursing*, 62(1):107–115, 2008. doi:10.1111/j.1365-2648.2007.04569.x.
- [58] Nils Erben and K Lohr. SAB—The Software Architecture Browser. In *3rd IEEE International Workshop on Visualizing Software for Understanding and Analysis*, pages 1–6. IEEE, 2005.
- [59] Neil A. Ernst and Martin P. Robillard. A Study of Documentation for Software Architecture. *Empirical Software Engineering*, 28:122, 2023. ISSN 1573-7616. doi:10.1007/s10664-023-10347-2.
- [60] Hidde van Esch. Visualizing Software Behavior & Structure in Virtual Reality. Master’s thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, 2024.
- [61] Matteo Esposito, Xiaozhou Li, Sergio Moreschini, Noman Ahmad, Tomas Cerny, Karthik Vaidhyanathan, Valentina Lenarduzzi, and Davide Taibi. Generative AI for Software Architecture. Applications, Challenges, and Future Directions. *Journal of Systems and Software*, 231: 112607, 2026. ISSN 0164-1212. doi:10.1016/j.jss.2025.112607.
- [62] Ilker Etikan, Sulaiman A. Musa, and Rukayya S. Alkassim. Comparison of Convenience Sampling and Purposive Sampling. *American Journal of Theoretical and Applied Statistics*, 5(1):1–4, 2016. doi:10.11648/j.ajtas.20160501.11.
- [63] European Commission. Commission Recommendation of 6 May 2003 Concerning the Definition of Micro, Small and Medium-Sized Enter-

- prises (Text with EEA Relevance) (Notified Under Document Number C(2003) 1422), May 2003.
- [64] Aleksandr Fadeev. Vygotskys Theory of Mediation in Digital Learning Environment: Actuality and Practice. 5(1):24–44, 2019.
- [65] Stefan Feuerriegel, Jochen Hartmann, Christian Janiesch, and Patrick Zschech. Generative AI. *Business & Information Systems Engineering*, 66(1):111–126, 2024. doi:10.1007/s12599-023-00834-7.
- [66] Rosa Filgueira and Daniel Garijo. Inspect4py: A Knowledge Extraction Framework for Python Code Repositories. In *IEEE/ACM 19th International Conference on Mining Software Repositories, MSR 2022, Pittsburgh, PA, USA, May 23-24, 2022*, pages 232–236. IEEE, 2022. doi:10.1145/3524842.3528497.
- [67] Beat Fluri, Michael Wursch, and Harald C. Gall. Do Code and Comments Co-evolve? On the Relation between Source Code and Comment Changes. In *14th Working Conference on Reverse Engineering*, pages 70–79. IEEE, 2007. doi:10.1109/WCRE.2007.21.
- [68] Matthieu Foucault, Marc Palyart, Xavier Blanc, Gail C. Murphy, and Jean-Rémy Falleri. Impact of Developer Turnover on Quality in Open-Source Software. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pages 829–841, New York, USA, 2015. Association for Computing Machinery. ISBN 9781450336758. doi:10.1145/2786805.2786870.
- [69] Martin Fowler. *Patterns of Enterprise Application Architecture*. 2012.
- [70] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. Cypher: An Evolving Query Language for Property Graphs. In *Proceedings of the 2018 International Conference on Management of Data*, pages 1433–1445, 2018. doi:10.1145/3183713.3190657.

- [71] Stan Franklin and Art Graesser. Is it an Agent, or just a Program?: A Taxonomy for Autonomous Agents. In *International Workshop on Agent Theories, Architectures, and Languages*, pages 21–35. Springer-Verlag, 1996. doi:10.1007/bfb0013570.
- [72] Dominik Fuchß, Haoyu Liu, Tobias Hey, Jan Keim, and Anne Koziolk. Enabling Architecture Traceability by LLM-based Architecture Component Name Extraction. In *22nd International Conference on Software Architecture*, pages 1–12. IEEE, 2025. doi:10.1109/ICSA65012.2025.00011.
- [73] Ka Yip Fung. Classifying Java Classes Into Role Stereotypes Based on Their Behavior. Master’s thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, 2023.
- [74] Joshua Garcia, Daniel Popescu, Chris Mattmann, Nenad Medvidovic, and Yuanfang Cai. Enhancing Architectural Recovery Using Concerns. In *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*, pages 552–555. IEEE, 2011. doi:10.1109/ase.2011.6100123.
- [75] Joshua Garcia, Igor Ivkovic, and Nenad Medvidovic. A Comparative Analysis of Software Architecture Recovery Techniques. In *28th IEEE/ACM International Conference on Automated Software Engineering*, pages 486–496. IEEE, 2013. doi:10.1109/ASE.2013.6693106.
- [76] Joshua Garcia, Mehdi Mirakhorli, Lu Xiao, Yutong Zhao, Ibrahim Mujhid, Khoi Pham, Ahmet Okutan, Sam Malek, Rick Kazman, Yuanfang Cai, et al. Constructing a shared infrastructure for software architecture analysis and maintenance. In *2021 IEEE 18th International Conference on Software Architecture (ICSA)*, pages 150–161. IEEE, 2021. doi:10.1109/icsa51549.2021.00022.
- [77] Dennis A. Gioia, Kevin Corley, and Aimee L. Hamilton. Seeking Qualitative Rigor in Inductive Research: Notes on the Gioia Methodology.

- Organizational Research Methods*, 16(1):15–31, jan 2013. ISSN 1094-4281. doi:10.1177/1094428112452151.
- [78] GitHub. What is GitHub Copilot? – GitHub Docs – docs.github.com. <https://docs.github.com/en/copilot/about-github-copilot/what-is-github-copilot>. [Accessed 08-01-2025].
- [79] Jochen Görtler, Christoph Schulz, Daniel Weiskopf, and Oliver Deussen. Bubble Treemaps for Uncertainty Visualization. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):719–728, 2017. doi:10.1109/TVCG.2017.2743959.
- [80] Tom Groot, Lina Ochoa, Bogdan Lazăr, and Jacob Krüger. A Catalog of Unintended Software Dependencies in Multi-Lingual Systems at ASML. In *International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*, pages 240–251. ACM, 2024. doi:10.1145/3639477.3639725.
- [81] Yann-Gaël Guéhéneuc. P-MART: Pattern-Like Micro Architecture Repository. *Proceedings of the 1st EuroPLoP Focus Group on pattern repositories*, pages 1–3, 2007. URL <https://www.ptidej.net/tools/designpatterns/>.
- [82] Greg Guest, Arwen Bunce, and Laura Johnson. How Many Interviews Are Enough? An Experiment with Data Saturation and Variability. *Field methods*, 18(1):59–82, 2006. doi:10.1177/1525822X05279903.
- [83] Greg Guest, Emily Namey, and Mario Chen. A Simple Method to Assess and Report Thematic Saturation in Qualitative Research. *PloS one*, 15(5):e0232076, 2020. doi:10.1371/journal.pone.0232076.
- [84] Frank van Ham. Using Multilevel Call Matrices in Large Software Projects. In *IEEE Symposium on Information Visualization 2003 (IEEE Cat. No. 03TH8714)*, pages 227–232. IEEE, 2003. doi:10.1109/IN-FVIS.2003.1249030.

- [85] Wang Haoyu and Zhou Haili. Basic Design Principles in Software Engineering. In *4th International Conference on Computational and Information Sciences*, pages 1251–1254. IEEE, 2012. doi:10.1109/IC-CIS.2012.91.
- [86] Yeyi He. Enhancing Program Comprehension through Visualization and LLM-based Summarization of Behaviour. Master’s thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, 2025.
- [87] David Heidrich, Andreas Schreiber, and Sebastian Oberdörfer. Towards Generating Labeled Property Graphs for Comprehending C#-Based Software Projects. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, pages 1–4, 2022. doi:10.1145/3551349.3560513.
- [88] Truong Ho-Quang, Michel R.V. Chaudron, Gregorio Robles, and Guntur B. Herwanto. Towards an Infrastructure for Empirical Research into Software Architecture: Challenges and Directions. In *2019 IEEE/ACM 2nd International Workshop on Establishing the Community-Wide Infrastructure for Architecture-Based Software Engineering (ECASE)*, pages 34–41. IEEE, 2019. doi:10.1109/ecase.2019.00014.
- [89] Truong Ho-Quang, Alexandre Bergel, Arif Nurwidyanoro, Rodi Jolak, and Michel R.V. Chaudron. Interactive Role Stereotype-Based Visualization To Comprehend Software Architecture. In *Working Conference on Software Visualization*, pages 122–132, 2020. doi:10.1109/VIS-SOFT51673.2020.00018.
- [90] Truong Ho-Quang, Arif Nurwidyanoro, Satrio Adi Rukmono, Michel R.V. Chaudron, Fabian Fröding, and Duy Nguyen Ngoc. Role Stereotypes in Software Designs and Their Evolution. *Journal of Systems and Software*, 189:111296, 2022. doi:10.1016/j.jss.2022.111296.
- [91] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard De Melo, Claudio Gutierrez, Sabrina Kirrane, José E.L. Gayo,

- Roberto Navigli, Sebastian Neumaier, et al. Knowledge Graphs. *ACM Computing Surveys (Csur)*, 54(4):1–37, 2021. doi:10.1145/3447772.
- [92] Kate Holderer. Single Responsibility Principle: Software Design. *POGIL Activity Clearinghouse*, 3(4), 2022.
- [93] Siw E. Hove and Bente Anda. Experiences from Conducting Semi-structured Interviews in Empirical Software Engineering Research. In *11th IEEE International Software Metrics Symposium*, pages 10 pp.–23. IEEE, 2005. doi:10.1109/METRICS.2005.24.
- [94] ISO/IEC/IEEE. ISO/IEC/IEEE 42010:2022 International Standard for Software, Systems and Enterprise—Architecture Description. Standard, 2022.
- [95] James Ivers and Ipek Ozkaya. Will Generative AI Fill the Automation Gap in Software Architecting? In *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*, pages 41–45. IEEE, 2025. doi:10.1109/icsa-c65153.2025.00014.
- [96] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. Graph Convolutional Networks for Mapping Source Code Entities to Architectural Modules. In *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*, pages 472–481. IEEE, 2025. doi:10.1109/icsa-c65153.2025.00071.
- [97] Jasmin Jahić and Ashkan Sami. State of Practice: LLMs in Software Engineering and Software Architecture. In *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*, pages 311–318. IEEE, 2024. doi:10.1109/ICSA-C63560.2024.00059.
- [98] Christopher Jeffrey, Aditya P. Nugroho, Satrio Adi Rukmono, and Yani Widayani. CodeView: A Tool for Software Visualization in Development View. In *2024 IEEE International Conference on Data and Software Engineering (ICoDSE)*, pages 67–72. IEEE, 2024. doi:10.1109/icodse63307.2024.10829915.

- [99] Nils Johansson, Mauro Caporuscio, and Tobias Olsson. Mapping Source Code to Software Architecture by Leveraging Large Language Models. In Apostolos Ampatzoglou, Jennifer Pérez, Barbora Buhnova, Valentina Lenarduzzi, Colin C. Venters, Uwe Zdun, Khalil Drira, Luciana Rebelo, Daniele Di Pompeo, Michele Tucci, Elisa Yumi Nakagawa, and Elena Navarro, editors, *European Conference on Software Architecture*, pages 133–149, Cham, 2024. Springer Nature Switzerland. ISBN 978-3-031-71246-3. doi:10.1007/978-3-031-71246-3_13.
- [100] Rodi Jolak, Maxime Savary-Leblanc, Manuela Dalibor, Andreas Wortmann, Regina Hebig, Juraj Vincur, Ivan Polasek, Xavier Le Pallec, Sébastien Gérard, and Michel R.V. Chaudron. Software Engineering Whispers: The Effect of Textual vs. Graphical Software Design Descriptions on Software Design Communication. *Empirical software engineering*, 25(6):4427–4471, 2020. doi:10.1007/s10664-020-09835-6.
- [101] Roy Kakkenberg, Satrio Adi Rukmono, Michel R.V. Chaudron, Wim Gerholt, Miguel Pinto, and Claudio R. de Oliveira. Arvisan: An Interactive Tool for Visualisation and Analysis of Low-Code Architecture Landscapes. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, pages 848–855, 2024. doi:10.1145/3652620.3688331.
- [102] Reyhaneh Kalantari, Julian Oertel, Joeri Exelmans, Satrio Adi Rukmono, Vasco Amaral, Matthias Tichy, Katharina Juhnke, Jan-Philipp Steghöfer, and Silvia Abrahão. Systematizing Modeler Experience (MX) in Model-Driven Engineering Success Stories. *Software and Systems Modeling*, 23(4):821–832, 2024. doi:10.1007/s10270-024-01194-w.
- [103] Raj K. Kaur, Babita Pandey, Devendra K. Pandey, and Aditya Khamparia. An Explorative Study on Code Comprehension: Issues and Challenges. In *International Conference On Artificial Intelligence Of Things For Smart Societies*, pages 201–210. Springer-Verlag, 2024. doi:10.1007/978-3-031-63103-0_21.

- [104] Rick Kazman, Phil Bianco, James Ivers, and John Klein. Maintainability. Technical Report CMU/SEI-2020-TR-006, Carnegie Mellon University, 2020.
- [105] Barbara A. Kitchenham, David Budgen, and Pearl Brereton. *Evidence-Based Software Engineering and Systematic Reviews*, volume 4. Chapman & Hall/CRC, 2015. ISBN 1482228653. doi:10.1201/b19467.
- [106] Paul Klint, Tijs van der Storm, and Jurgen Vinju. RASCAL: A Domain Specific Language for Source Code Analysis and Manipulation. In *Ninth IEEE International Working Conference on Source Code Analysis and Manipulation*, pages 168–177, 2009. doi:10.1109/SCAM.2009.28.
- [107] Perry Kloet. Multi-level Layout Algorithms for Visualizing Hierarchical Software Systems. Master’s thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, 2024.
- [108] Kenichi Kobayashi, Manabu Kamimura, Keisuke Yano, Koki Kato, and Akihiko Matsuo. SARF map: Visualizing Software Architecture from Feature and Layer Viewpoints. In *2013 21st International Conference on Program Comprehension (ICPC)*, pages 43–52. IEEE, 2013. doi:10.1109/icpc.2013.6613832.
- [109] Takeshi Kojima, Shixiang S. Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large Language Models Are Zero-Shot Reasoners. *Advances in Neural Information Processing Systems*, 35:22199–22213, 2022.
- [110] Ilianna Kolli, Birte Glimm, and Ian Horrocks. SPARQL Query Answering over OWL Ontologies. In *The Semantic Web: Research and Applications*, pages 382–396. Springer-Verlag, 2011. doi:10.1007/978-3-642-21034-1_26.
- [111] Oliver Krancher and Jens Dibbern. Knowledge Transfer in Software Maintenance Outsourcing: The Key Roles of Software Knowledge and Guided Learning Tasks. In Rudy Hirschheim, Armin Heinzl, and Jens

- Dibbern, editors, *Information Systems Outsourcing: The Era of Digital Transformation*, pages 147–181. Springer International Publishing, Cham, 2020. ISBN 978-3-030-45819-5. doi:10.1007/978-3-030-45819-5_7.
- [112] Philippe Kruchten. The 4+1 View Model of Architecture. *IEEE Software*, 12(6):42–50, 1995. doi:10.1109/52.469759.
- [113] Philippe Kruchten. What do software architects really do? *Journal of Systems and Software*, 81(12):2413–2416, 2008. ISSN 0164-1212. doi:https://doi.org/10.1016/j.jss.2008.08.025.
- [114] Jacob Krüger and Regina Hebig. To Memorize or to Document: A Survey of Developers’ Views on Knowledge Availability. In *International Conference on Product Focused Software Process Improvement (PROFES)*. Springer-Verlag, 2023. doi:10.1007/978-3-031-49266-2_3.
- [115] Juraj Kubelka, Romain Robbes, and Alexandre Bergel. Live Programming and Software Evolution: Questions During a Programming Change Task. In *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*, pages 30–41. IEEE, IEEE, 2019. doi:10.1109/icpc.2019.00017.
- [116] Pi  re van de Laar and Arjan Mooij. Renaissance-Ada: Tools for Analysis and Transformation of Ada Code. *Ada User J*, 43(3):165–170, 2022.
- [117] Jannik Laval, Nicolas Anquetil, Usman Bhatti, and St  phane Ducasse. oZone: Layer Identification in the Presence of Cyclic Dependencies. *Science of Computer Programming*, 78(8):1055–1072, 2013. doi:10.1016/j.scico.2012.08.001.
- [118] Juho Leinonen, Paul Denny, Stephen MacNeil, Sami Sarsa, Seth Bernstein, Joanne Kim, Andrew Tran, and Arto Hellas. Comparing Code Explanations Created by Students and Large Language Models. In *Proceedings of the 2023 Conference on Innovation and Tech-*

- nology in Computer Science Education V. 1, pages 124–130, 2023. doi:10.1145/3587102.3588785.
- [119] Timothy Lethbridge, Janice Singer, and Andrew Forward. How Software Engineers Use Documentation: The State of the Practice. *IEEE Software*, 20(6):35–39, 2003. doi:10.1109/MS.2003.1241364.
- [120] Stanley Letovsky. Cognitive Processes in Program Comprehension. *Journal of Systems and Software*, 7(4):325–339, 1987. ISSN 0164-1212. doi:10.1016/0164-1212(87)90032-X.
- [121] Omer Levy and Dror Feitelson. Understanding Large-Scale Software Systems—Structure and Flows. *Empirical Software Engineering*, 26, 2021. doi:10.1007/s10664-021-09938-8.
- [122] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, pages 9459–9474, 2020.
- [123] Ruiyin Li, Peng Liang, Mohamed Soliman, and Paris Avgeriou. Understanding Software Architecture Erosion: A Systematic Mapping Study. *Journal of Software: Evolution and Process*, 34(3):e2423, 2022. doi:10.1002/smr.2423.
- [124] Grisca Liebel, Jil Klünder, Regina Hebig, Christopher Lazik, Inês Nunes, Isabella Graßl, Jan-Philipp Steghöfer, Joeri Exelmans, Julian Oertel, Kai Marquardt, Katharina Juhnke, Kurt Schneider, Lucas Gren, Lucia Happe, Marc Herrmann, Marvin Wyrich, Matthias Tichy, Miguel Goulão, Rebekka Wohlrab, Reyhaneh Kalantari, Robert Heinrich, Sandra Greiner, Satrio Adi Rukmono, Shalini Chakraborty, Silvia Abrahão, and Vasco Amaral. Human Factors in Model-driven Engineering: Future Research Goals and Initiatives for MDE. *Software and Systems Modeling*, 23(4):801–819, 2024. doi:10.1007/s10270-024-01188-8.

- [125] Johan Linaker, Sardar M. Sulaman, Martin Höst, and Rafael M. de Mello. Guidelines for Conducting Surveys in Software Engineering v.1.1. *Lund University*, 50:1–64, 2015.
- [126] Yvonna S. Lincoln and Egon G. Guba. Naturalistic Inquiry. In *The Blackwell Encyclopedia of Sociology*, chapter 11. John Wiley & Sons, Ltd, 2007. ISBN 9781405165518. doi:10.1002/9781405165518.wbeosn006.
- [127] Daniel Link, Pooyan Behnamghader, Ramin Moazeni, and Barry Boehm. The Value of Software Architecture Recovery for Maintenance. In *12th Innovations on Software Engineering Conference*, pages 1–10, New York, 2019. Association for Computing Machinery. ISBN 9781450362153. doi:10.1145/3299771.3299787.
- [128] David C. Littman, Jeannine Pinto, Stanley Letovsky, and Elliot Soloway. Mental Models and Software Maintenance. *Journal of Systems and Software*, 7(4):341–355, 1987. ISSN 0164-1212. doi:10.1016/0164-1212(87)90033-1.
- [129] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi:10.1162/tacl_a_00638.
- [130] Thibaud Lutellier, Devin Chollak, Joshua Garcia, Lin Tan, Derek Ray-side, Nenad Medvidovic, and Robert Kroeger. Comparing Software Architecture Recovery Techniques Using Accurate Dependencies. In *37th IEEE International Conference on Software Engineering*, volume 2, pages 69–78, 2015. doi:10.1109/ICSE.2015.136.
- [131] Ioanna Lytra, Carlos Carrillo, Rafael Capilla, and Uwe Zdun. Quality Attributes Use in Architecture Design Decision Methods: Research and Practice. *Computing*, 102(2):551–572, 2020. doi:10.1007/s00607-019-00758-9.

- [132] Stephen MacNeil, Andrew Tran, Arto Hellas, Joanne Kim, Sami Sarsa, Paul Denny, Seth Bernstein, and Juho Leinonen. Experiences From Using Code Explanations Generated by Large Language Models in a Web Software Development E-Book. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, pages 931–937, 2023. doi:10.1145/3545945.3569785.
- [133] Jonathan I. Maletic, Andrian Marcus, and Michael L. Collard. A Task Oriented View of Software Visualization. In *Proceedings first international workshop on visualizing software for understanding and analysis*, pages 32–40. IEEE, 2002. doi:10.1109/VISSOF.2002.1019792.
- [134] Jimmie Manning. In Vivo Coding. In *The International Encyclopedia of Communication Research Methods*, pages 1–2. John Wiley & Sons, Ltd, 2017. ISBN 9781118901731. doi:10.1002/9781118901731.iecrm0270.
- [135] Samia Marwan, Nicholas Lytle, Joseph J. Williams, and Thomas Price. The Impact of Adding Textual Explanations to Next-Step Hints in a Novice Programming Environment. In *Proceedings of the 2019 ACM conference on Innovation and Technology in Computer Science Education*, pages 520–526, 2019. doi:10.1145/3304221.3319759.
- [136] Antonio Mastropaolo, Camilo Escobar-Velásquez, and Mario Linares-Vásquez. From Triumph to Uncertainty: The Journey of Software Engineering in the AI Era. *ACM Transactions on Software Engineering and Methodology*, 34(5):1–34, 2025. ISSN 1049-331X. doi:10.1145/3709360.
- [137] Gerardo Maturro, Karina Barrella, and Patricia Benitez. Difficulties of Newcomers Joining Software Projects Already in Execution. In *2017 International Conference on Computational Science and Computational Intelligence (CSCI)*, pages 993–998, 2017. doi:10.1109/csci.2017.171.
- [138] Luis F. Mendivelso, Kelly Garcés, and Rubby Casallas. Metric-Centered and Technology-Independent Architectural Views for Software Com-

- prehension. *Journal of Software Engineering Research and Development*, 6(1):16, 2018. doi:10.1186/s40411-018-0060-6.
- [139] Tom Mens and Michele Lanza. A Graph-Based Metamodel for Object-Oriented Software Metrics. *Electronic Notes in Theoretical Computer Science*, 72(2):57–68, 2002. doi:10.1016/s1571-0661(05)80529-8.
- [140] Martin Misiak, Andreas Schreiber, Arnulph Fuhrmann, Sascha Zur, Doreen Seider, and Lisa Nafeie. IslandViz: A Tool for Visualizing Modular Software Systems in Virtual Reality. In *2018 IEEE Working Conference on Software Visualization (VISOFT)*, pages 112–116. IEEE, 2018. doi:10.1109/vissoft.2018.00020.
- [141] Ahmad H. Mohammadkhani, Chakkrit Tantithamthavorn, and Hadi Hemmatif. Explaining Transformer-Based Code Models: What Do They Learn? When They Do Not Work? In *2023 IEEE 23rd International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pages 96–106. IEEE, 2023. doi:10.1109/scam59687.2023.00020.
- [142] David Moreno-Lumbreras, Jesús M. González-Barahona, Gregorio Robles, and Valerio Cosentino. The Influence of the City Metaphor and Its Derivates in Software Visualization. *Journal of Systems and Software*, 210:111985, 2024. doi:10.1016/j.jss.2024.111985.
- [143] Daye Nam, Youn K. Lee, and Nenad Medvidovic. Eva: A Tool for Visualizing Software Architectural Evolution. In *Proceedings of the 40th international conference on software engineering: companion proceedings*, pages 53–56, 2018. doi:10.1145/3183440.3183490.
- [144] Daye Nam, Andrew Macvean, Vincent J. Hellendoorn, Bogdan Vasilescu, and Brad A. Myers. Using an LLM to Help With Code Understanding. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 2024. doi:10.1145/3597503.3639187. URL <https://doi.org/10.1145/3597503.3639187>.

- [145] Meike Nauta and Christin Seifert. The Co-12 Recipe for Evaluating Interpretable Part-Prototype Image Classifiers. In Luca Longo, editor, *Explainable Artificial Intelligence*, pages 397–420, Cham, 2023. Springer-Verlag, Springer Nature Switzerland. ISBN 978-3-031-44064-9. doi:10.1007/978-3-031-44064-9_21.
- [146] Meike Nauta, Jan Trienes, Shreyasi Pathak, Elisa Nguyen, Michelle Peters, Yasmin Schmitt, Jörg Schlötterer, Maurice van Keulen, and Christin Seifert. From Anecdotal Evidence to Quantitative Evaluation Methods: A Systematic Review on Evaluating Explainable AI. *ACM Comput. Surv.*, 55(13s):1–42, 2023. ISSN 0360-0300. doi:10.1145/3583558.
- [147] Srinivas Nidhra, Muralidhar Yanamadala, Wasif Afzal, and Richard Torkar. Knowledge Transfer Challenges and Mitigation Strategies in Global Software Development—A Systematic Literature Review and Industrial Validation. *International Journal of Information Management*, 33(2):333–355, 2013. ISSN 0268-4012. doi:10.1016/j.ijinfomgt.2012.11.004.
- [148] Sebastian Nielebock, Dariusz Krolikowski, Jacob Krüger, Thomas Leich, and Frank Ortmeier. Commenting Source Code: Is It Worth It for Small Programming Tasks? *Empirical Software Engineering*, 24(3):1418–1457, 2019. doi:10.1007/s10664-018-9664-z.
- [149] Robert Norton and Barbara Warnick. Assertiveness as a Communication Construct. *Human Communication Research*, 3(1):62–66, 03 2006. ISSN 0360-3989. doi:10.1111/j.1468-2958.1976.tb00504.x.
- [150] Bashar Nuseibeh. Weaving Together Requirements and Architectures. *Computer*, 34(3):115–119, 2001. doi:10.1109/2.910904.
- [151] Klaus Oberauer. Working Memory and Attention—A Conceptual Analysis and Review. *Journal of Cognition*, 2(1), 2019. doi:10.5334/joc.58.

- [152] Edward E. Ogheneovo. On the Relationship Between Software Complexity and Maintenance Costs. *Journal of Computer and Communications*, 2(14):1, 2014. doi:10.4236/jcc.2014.214001.
- [153] OpenAI. Introducing ChatGPT. <https://openai.com/blog/chatgpt>, November 2022. Accessed: 2025-10-01.
- [154] Floris M.J. Pex. Exploring Software Explanation using Retrieval Augmented Generation. Master's thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, August 2024.
- [155] Gustavo Pinto, Cleidson De Souza, Thayssa Rocha, Igor Steinmacher, Alberto Souza, and Edward Monteiro. Developer Experiences with a Contextualized AI Coding Assistant: Usability, Expectations, and Outcomes. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*, pages 81–91, 2024. doi:10.1145/3644815.3644949.
- [156] Cristiano Politowski, Foutse Khomh, Simone Romano, Giuseppe Scanniello, Fabio Petrillo, Yann-Gaël Guéhéneuc, and Abdou Maiga. A Large Scale Empirical Study of the Impact of Spaghetti Code and Blob Anti-patterns on Program Comprehension. *Information and Software Technology*, 122:106278, 2020. ISSN 0950-5849. doi:10.1016/j.inf-sof.2020.106278.
- [157] Abdul Qayum, Mengqi Zhang, Simon Colreavy, Muslim Chochlov, Jim Buckley, Dayi Lin, and Ashish R. Sai. A Framework and Taxonomy for Characterizing the Applicability of Software Architecture Recovery Approaches: A Tertiary-Mapping Study. *Software: Practice and Experience*, 55(1):100–132, 2025. doi:10.1002/spe.3364.
- [158] Ernesto Quevedo, Amr S. Abdelfattah, Alejandro Rodriguez, Jorge Yero, and Tomas Cerny. Evaluating ChatGPT's Proficiency in Understanding and Answering Microservice Architecture Queries Using Source Code Insights. *SN Computer Science*, 5(4):422, 2024. doi:10.1007/s42979-024-02664-0.

- [159] Lukas Radosky and Ivan Polasek. Executable Multi-Layered Software Models. In *Proceedings of the 1st International Workshop on Designing Software*, pages 46–51, 2024. doi:10.1145/3643660.3643938.
- [160] Ghulam Rasool. *Customizable Feature Based Design Pattern Recognition Integrating Multiple Techniques*. PhD thesis, Technische Universität Ilmenau, Germany, 2011.
- [161] Sukanya Ratanotayanon and S Sim. When Programmers Don’t Ask. In *Proceedings of the 21st International Conference on Automated Software Engineering*, pages 9–16, 2006.
- [162] Maryam Razavian, Barbara Paech, and Antony Tang. The Vision of On-Demand Architectural Knowledge Systems as a Decision-Making Companion. *Journal of Systems and Software*, 198:111560, 2023. doi:10.1016/j.jss.2022.111560.
- [163] Brittany Reid, Markus Wagner, Marcelo d’Amorim, and Christoph Treude. Software Engineering User Study Recruitment on Prolific: An Experience Report. In *International Workshop on Recruiting Participants for Empirical Software Engineering (RoPES)*, 2022.
- [164] George G. Robertson, Jock D. Mackinlay, and Stuart K. Card. Cone Trees: Animated 3D Visualizations of Hierarchical Information. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, pages 189–194, 1991. doi:10.1145/108844.108883.
- [165] Martin P. Robillard, Andrian Marcus, Christoph Treude, Gabriele Bavota, Oscar Chaparro, Neil Ernst, Marco A. Gerosa, Michael Godfrey, Michele Lanza, Mario Linares-Vásquez, et al. On-Demand Developer Documentation. In *2017 IEEE International conference on software maintenance and evolution (ICSME)*, pages 479–483. IEEE, IEEE, 2017. doi:10.1109/ICSME.2017.17.

- [166] Margaret R. Roller and Paul J. Lavrakas. *Applied Qualitative Research Design: A Total Quality Framework Approach*. Guilford Publications, 2015.
- [167] Steven I. Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D. Weisz. The Programmer’s Assistant: Conversational Interaction With a Large Language Model for Software Development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*, pages 491–514, 2023. doi:10.1145/3581641.3584037.
- [168] Satrio Adi Rukmono. ClassViz: From Inspection Tool to Research Vessel. In *The 24th Belgium-Netherlands Software Evolution Workshop (BENEVOL 2025)*, 2025.
- [169] Satrio Adi Rukmono and Michel R.V. Chaudron. Enabling Analysis and Reasoning on Software Systems through Knowledge Graph Representation. In *20th International Conference on Mining Software Repositories*, pages 120–124. IEEE, 2023. doi:10.1109/MSR59073.2023.00029.
- [170] Satrio Adi Rukmono, Lina Ochoa, and Michel R.V. Chaudron. Achieving High-Level Software Component Summarization via Hierarchical Chain-of-Thought Prompting and Static Code Analysis. In *International Conference on Data and Software Engineering*, pages 7–12. IEEE, 2023. doi:10.1109/ICoDSE59534.2023.10292037.
- [171] Satrio Adi Rukmono, Michel R.V. Chaudron, and Christopher Jeffrey. Layered BubbleTea Software Architecture Visualisation. In *Working Conference on Software Visualization*, pages 122–126. IEEE, 2024. doi:10.1109/VISsOFT64034.2024.00024.
- [172] Satrio Adi Rukmono, Lina Ochoa, and Michel R.V. Chaudron. Deductive Software Architecture Recovery via Chain-of-Thought Prompting. In *44th International Conference on Software Engineering: New Ideas and Emerging Results*, pages 92–96, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400705007. doi:10.1145/3639476.3639776.

- [173] Satrio Adi Rukmono, Lina Ochoa, Thijs M.J. Bressers, Jacob Krüger, and Michel R.V. Chaudron. Replication Package: Evaluating Explanatory Artefacts of DSAR-Recovered Architectures, September 2025.
- [174] Satrio Adi Rukmono, Lina Ochoa, Thijs M.J. Bressers, Jacob Krüger, and Michel R.V. Chaudron. Evaluating Explanatory Artefacts of DSAR-Recovered Architectures from Industrial Codebases, 2026. Accepted at FSE-Industry 2026, to appear.
- [175] Satrio Adi Rukmono, Filip Zamfirov, Lina Ochoa, Floris Pex, and Michel R.V. Chaudron. An Explanation of Software Architecture Explanations. *Empirical Software Engineering*, 31(3):59, 2026. doi:10.1007/s10664-025-10782-3.
- [176] Amir M. Saeidi, Jurriaan Hage, Ravi Khadka, and Slinger Jansen. A Search-based Approach to Multi-view Clustering of Software Systems. In *IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering*, pages 429–438, 2015. doi:10.1109/SANER.2015.7081853.
- [177] Vaskaran Sarcar. Understanding SOLID Principles. In *Java Design Patterns: A Hands-On Experience with Real-World Examples*, pages 3–64. Apress, Berkeley, CA, 2022. ISBN 978-1-4842-7971-7.
- [178] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research-Volume 1*, pages 27–43, 2022. doi:10.1145/3501385.3543957.
- [179] Hugo H. Schoonewille, Werner Heijstek, Michel R.V. Chaudron, and Thomas Kühne. A Cognitive Perspective on Developer Comprehension of Software Design Documentation. In *29th ACM International Conference on Design of Communication*, pages 211–218, New York, 2011. Association for Computing Machinery. ISBN 9781450309363. doi:10.1145/2038476.2038517.

- [180] Ivonne Schröter, Jacob Krüger, Janet Siegmund, and Thomas Leich. Comprehending Studies on Program Comprehension. In *25th International Conference on Program Comprehension*, pages 308–311. IEEE, 2017. doi:10.1109/ICPC.2017.9.
- [181] Christoph D. Schulze, Miro Spönemann, and Reinhard Von Hanxleden. Drawing Layered Graphs with Port Constraints. *Journal of Visual Languages & Computing*, 25(2):89–106, 2014. doi:10.1016/j.jvlc.2013.11.005.
- [182] Carolyn B. Seaman. Qualitative Methods in Empirical Studies of Software Engineering. *IEEE Transactions on software engineering*, 25(4):557–572, 1999. doi:10.1109/32.799955.
- [183] Philip Sedgwick. Convenience Sampling. *BMJ*, 347:f6304, 2013. doi:10.1136/bmj.f6304.
- [184] William R. Shadish, Thomas D. Cook, and Donald T. Campbell. Experimental and Quasiexperimental Designs for Generalized Causal Inference. *Social Service Review*, 76(3):510–514, 2002. ISSN 00377961, 15375404. doi:10.1086/345281.
- [185] Zeinab Shahbazi, Abbas Rasoolzadegan, Zahra Purfallah, and Somayeh Jafari Horestani. A New Method for Detecting Various Variants of GoF Design Patterns Using Conceptual Signatures. *Software Quality Journal*, 30(3):651–686, 2022. doi:10.1007/s11219-021-09576-9.
- [186] Mojtaba Shahin, Peng Liang, and M. Ali Babar. A Systematic Review of Software Architecture Visualization Techniques. *Journal of Systems and Software*, 94:161–185, 2014. doi:10.1016/j.jss.2014.03.071.
- [187] Mojtaba Shahin, Peng Liang, and Zengyang Li. Do Architectural Design Decisions Improve the Understanding of Software Architecture? Two Controlled Experiments. In *22nd International Conference on Program Comprehension*, pages 3–13, New York,

2014. Association for Computing Machinery. ISBN 9781450328791. doi:10.1145/2597008.2597139.
- [188] Sina Sheikholeslami, Hamid Ghasemirahni, Amir H. Payberah, Tianze Wang, Jim Dowling, and Vladimir Vlassov. Utilizing Large Language Models for Ablation Studies in Machine Learning and Deep Learning. In *Proceedings of the 5th Workshop on Machine Learning and Systems*, pages 230–237, 2025. doi:10.1145/3721146.3721957.
- [189] Jonathan Sillito, Gail C. Murphy, and Kris De Volder. Questions Programmers Ask During Software Evolution Tasks. In *Proceedings of the 14th ACM SIGSOFT international symposium on Foundations of software engineering*, pages 23–34. ACM, 2006. doi:10.1145/1181775.1181779.
- [190] Stacy Smulowitz. Interview Guide. In *The International Encyclopedia of Communication Research Methods*, pages 1–2. John Wiley & Sons, Ltd, 2017. ISBN 9781118901731. doi:10.1002/9781118901731.iecrm0124.
- [191] Dominik Sobania, Alina Geiger, James Callan, Alexander Brownlee, Carol Hanna, Rebecca Moussa, Mar Zamorano López, Justyna Petke, and Federica Sarro. Evaluating Explanations for Software Patches Generated by Large Language Models. In *International Symposium on Search Based Software Engineering*, pages 147–152. Springer-Verlag, 2023. doi:10.1007/978-3-031-48796-5_12.
- [192] David Sobrín-Hidalgo, Miguel A. González-Santamaría, Ángel M. Guerrero-Higueras, Francisco J. Rodríguez-Seara, and Vicente Matellán-Olivera. Explaining Autonomy: Enhancing Human-Robot Interaction Through Explanation Generation With Large Language Models. *arXiv preprint arXiv:2402.04206*, 2024.
- [193] Mohamed Soliman. Exploring architectural design decisions in mailing lists and their traceability to issue trackers. In *European Confer-*

- ence on Software Architecture*, pages 307–323. Springer-Verlag, 2024. doi:10.1007/978-3-031-70797-1_20.
- [194] Mohamed Soliman and Jan Keim. Do Large Language Models Contain Software Architectural Knowledge?: An Exploratory Case Study with GPT. In *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*, pages 13–24. IEEE, 2025. doi:10.1109/ICSA65012.2025.00012.
- [195] SourceGraph. Cody—AI Coding Assistant. <https://sourcegraph.com/cody>. [Accessed 08-01-2025].
- [196] Miro Spönemann. *Graph Layout Support for Model-driven Engineering*. PhD thesis, Kiel, 2015.
- [197] Sean Stapleton, Yashmeet Gambhir, Alexander LeClair, Zachary Eberhart, Westley Weimer, Kevin Leach, and Yu Huang. A Human Study of Comprehension and Code Summarization. In *28th International Conference on Program Comprehension*, pages 2–13, New York, 2020. Association for Computing Machinery. ISBN 9781450379588. doi:10.1145/3387904.3389258.
- [198] Igor Steinmacher, Marco A. Graciotto Silva, Marco A. Gerosa, and David F. Redmiles. A Systematic Literature Review on the Barriers Faced by Newcomers to Open Source Software Projects. *Information and Software Technology*, 59:67–85, 2015. doi:10.1016/j.infsof.2014.11.001.
- [199] Igor Steinmacher, Tayana U. Conte, Christoph Treude, and Marco A. Gerosa. Overcoming Open Source Project Entry Barriers with a Portal for Newcomers. In *38th ACM/IEEE International Conference on Software Engineering*, pages 273–284, New York, 2016. Association for Computing Machinery. ISBN 9781450339001. doi:10.1145/2884781.2884806.

- [200] Christoph J. Stettina and Werner Heijstek. Necessary and Neglected? An Empirical Study of Internal Documentation in Agile Software Development Teams. In *Proceedings of the 29th ACM international conference on Design of communication*, pages 159–166, New York, 2011. Association for Computing Machinery. ISBN 9781450309363. doi:10.1145/2038476.2038509.
- [201] Klaas-Jan Stol and Brian Fitzgerald. The ABC of Software Engineering Research. *ACM Trans. Softw. Eng. Methodol.*, 27(3):11:1–51, sep 2018. ISSN 1049-331X. doi:10.1145/3241743.
- [202] Klaas-Jan Stol and Brian Fitzgerald. Guidelines for Conducting Software Engineering Research. In *Contemporary Empirical Methods in Software Engineering*, pages 27–62. Springer-Verlag, 2020. doi:10.1007/978-3-030-32489-6_2.
- [203] Klaas-Jan Stol, Paul Ralph, and Brian Fitzgerald. Grounded Theory in Software Engineering Research: A Critical Review and Guidelines. In *38th International Conference on Software Engineering*, pages 120–131, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450339001. doi:10.1145/2884781.2884833.
- [204] Margaret-Anne Storey. Theories, Methods and Tools in Program Comprehension: Past, Present and Future. In *13th International Workshop on Program Comprehension*, pages 181–191. IEEE, 2005. doi:10.1109/WPC.2005.38.
- [205] Per E. Strandberg. Ethical Interviews in Software Engineering. In *International Symposium on Empirical Software Engineering and Measurement*, pages 1–11. IEEE, IEEE, 2019. doi:10.1109/ESEM.2019.8870192.
- [206] John Sweller. Cognitive Load During Problem Solving: Effects on Learning. *Cognitive science*, 12(2):257–285, 1988. doi:10.1207/s15516709cog1202_4.

- [207] Damian A. Tamburri and Rick Kazman. General Methods for Software Architecture Recovery: A Potential Approach and Its Evaluation. *Empirical Software Engineering*, 23:1457–1489, 2018. doi:10.1007/s10664-017-9543-z.
- [208] Harm Tanis. LLM-Enhanced Code Comprehension by Combining Static and Dynamic Analysis in Large-Scale C++ Systems. Master’s thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, 2025.
- [209] Olle Ten Cate, Linda Snell, Karen Mann, and Jan Vermunt. Orienting Teaching Toward the Learning Process. *Academic Medicine*, 79(3): 219–228, 2004. doi:10.1097/00001888-200403000-00005.
- [210] Maria D.C. Tongco. Purposive Sampling as a Tool for Informant Selection. *Ethnobotany Research & Applications*, 5:147–158, 2007. doi:10.17348/era.5.0.147-158.
- [211] Vincent A. Traag, Ludo Waltman, and Nees J. van Eck. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific reports*, 9(1):1–12, 2019. doi:10.1038/s41598-019-41695-z.
- [212] Christoph Treude, Justin Middleton, and Thushari Atapattu. Beyond Accuracy: Assessing Software Documentation Quality. In *28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1509–1512, New York, 2020. Association for Computing Machinery. ISBN 9781450370431. doi:10.1145/3368089.3417045.
- [213] Vassilios Tzerpos and Richard C. Holt. ACDC: An Algorithm for Comprehension-Driven Clustering. In *Proceedings Seventh Working Conference on Reverse Engineering*, pages 258–267. IEEE, 2000. doi:10.1109/WCRE.2000.891477.
- [214] Gias Uddin and Martin P. Robillard. How API Documentation Fails. *IEEE Software*, 32(4):68–75, 2015. doi:10.1109/MS.2014.80.

- [215] Hataichanok Unphon and Yvonne Dittrich. Software Architecture Awareness in Long-term Software Product Evolution. *Journal of Systems and Software*, 83(11):2211–2226, 2010. ISSN 0164-1212. doi:10.1016/j.jss.2010.06.043.
- [216] Zhiyuan Wan, Yun Zhang, Xin Xia, Yi Jiang, and David Lo. Software Architecture in Practice: Challenges and Opportunities. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1457–1469, 2023. doi:10.1145/3611643.3616367.
- [217] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A Survey on Large Language Model-Based Autonomous Agents. *Frontiers of Computer Science*, 18(6):186345, 2024. doi:10.1007/s11704-024-40231-1.
- [218] Paul T. Ward and Stephen J. Mellor. *Structured Development for Real-Time Systems—Volume 1: Introduction and Tools*. Pearson Education, 1985. ISBN 9780768685176.
- [219] Rogier Wester and John Koster. The Software Behind Moore’s Law. *IEEE Software*, 32(2):37–40, 2015. doi:10.1109/MS.2015.53.
- [220] Richard Wettel and Michele Lanza. Visualizing Software Systems as Cities. In *2007 4th IEEE International workshop on visualizing software for understanding and analysis*, pages 92–99. IEEE, 2007. doi:10.1109/vissof.2007.4290706.
- [221] Jacqueline Whalley, Raymond Lister, Errol Thompson, Tony Clear, Phil Robbins, P.K. Ajith Kumar, and Christine Prasad. An Australasian Study of Reading and Comprehension Skills in Novice Programmers, Using the Bloom and SOLO Taxonomies. Technical report or working paper, 2006. 2006.

- [222] Rebecca Wirfs-Brock, Alan McKean, Ivar Jacobson, and John Vlissides. *Object Design: Roles, Responsibilities, and Collaborations*. Pearson Education, 2002. ISBN 0201379430.
- [223] Rebecca J. Wirfs-Brock. Characterizing Classes. *IEEE Software*, 23(2): 9–11, 2006. doi:10.1109/MS.2006.43.
- [224] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. Planning. In *Experimentation in Software Engineering*, chapter 8, pages 89–116. Springer Berlin Heidelberg, Berlin, Heidelberg, 2012. ISBN 978-3-642-29044-2. doi:10.1007/978-3-642-29044-2_8.
- [225] Tianfu Yang, Zhiyong Jiang, Yanhong Shang, and Monire Norouzi. Systematic Review on Next-generation Web-based Software Architecture Clustering Models. *Computer Communications*, 167:63–74, 2021. ISSN 0140-3664. doi:10.1016/j.comcom.2020.12.022.
- [226] Juyeon Yoon, Robert Feldt, and Shin Yoo. Intent-Driven Mobile GUI Testing With Autonomous Large Language Model Agents. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pages 129–139. IEEE, 2024. doi:10.1109/icst60714.2024.00020.
- [227] Filip Zamfirov, Andrei Radulescu, Jacob Krüger, and Michel R.V. Chaudron. Lessons from Visualizing Software Architecture Structure Conformance at Thermo Fisher Scientific. In *Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. Springer-Verlag, 2025. doi:10.1007/978-3-032-04207-1_25.
- [228] Haisen Zhao and Lin Lu. Variational Circular Treemaps for Interactive Visualization of Hierarchical Data. In *2015 IEEE Pacific Visualization Symposium (PacificVis)*, pages 81–85. IEEE, 2015. doi:10.1109/PACIFICVIS.2015.7156360.
- [229] Junji Zhi, Vahid Garousi-Yusifolu, Bo Sun, Golar Garousi, Shawn Shahnewaz, and Guenther Ruhe. Cost, Benefits and Quality of Soft-

- ware Development Documentation: A Systematic Mapping. *Journal of Systems and Software*, 99:175–198, 2015. ISSN 0164-1212. doi:10.1016/j.jss.2014.09.042.
- [230] Yuxiang Zhu and Minxue Pan. Automatic Code Summarization: A Systematic Literature Review. *ArXiv*, abs/1909.04352, 2019. doi:10.48550/arXiv.1909.04352.

Curriculum Vitae

Satrio Adi Rukmono was born in Jakarta, Indonesia, on 27 September 1988. He obtained his Bachelor's degree in Informatics from Institut Teknologi Bandung (ITB), Indonesia, in 2010. Even before graduation, he joined PT. LAPI Divusi, where he spent more than eight years working on mission-critical software systems, including air traffic control and automated border passage systems. This period provided him with extensive industrial experience in safety-critical and large-scale system development.

He earned his Master's degree in Informatics from ITB in 2018 under the supervision of Achmad Imam Kistijantoro, Ph.D., with a thesis on GPU-based radar signal processing. This work marked a deliberate shift toward an academic career that bridges rigorous engineering practice and research. In 2019, he was appointed as a lecturer at ITB, where he teaches core courses in informatics and software engineering.

In 2021, he moved to Eindhoven, the Netherlands, to pursue doctoral studies at Technische Universiteit Eindhoven (TU/e) under the supervision of Prof. Dr. Michel Chaudron and Dr. Lina Ochoa Venegas. His doctoral research focuses on software architecture explanations, combining empirical studies, knowledge representation, and tool support to improve architectural understanding of complex software systems. After completing his doctorate, he will return to ITB to continue his academic career.

Titles in the IPA Dissertation Series since 2023

S. Kochanthara. *A Changing Landscape: On Safety & Open Source in Automated and Connected Driving.* Faculty of Mathematics and Computer Science, TU/e. 2023-01

L.M. Ochoa Venegas. *Break the Code? Breaking Changes and Their Impact on Software Evolution.* Faculty of Mathematics and Computer Science, TU/e. 2023-02

N. Yang. *Logs and models in engineering complex embedded production software systems.* Faculty of Mathematics and Computer Science, TU/e. 2023-03

J. Cao. *An Independent Timing Analysis for Credit-Based Shaping in Ethernet TSN.* Faculty of Mathematics and Computer Science, TU/e. 2023-04

K. Dokter. *Scheduled Protocol Programming.* Faculty of Mathematics and Natural Sciences, UL. 2023-05

J. Smits. *Strategic Language Workbench Improvements.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2023-06

A. Arslanagić. *Minimal Structures*

for Program Analysis and Verification. Faculty of Science and Engineering, RUG. 2023-07

M.S. Bouwman. *Supporting Railway Standardisation with Formal Verification.* Faculty of Mathematics and Computer Science, TU/e. 2023-08

S.A.M. Lathouwers. *Exploring Annotations for Deductive Verification.* Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2023-09

J.H. Stoel. *Solving the Bank, Lightweight Specification and Verification Techniques for Enterprise Software.* Faculty of Mathematics and Computer Science, TU/e. 2023-10

D.M. Groenewegen. *WebDSL: Linguistic Abstractions for Web Programming.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2023-11

D.R. do Vale. *On Semantical Methods for Higher-Order Complexity Analysis.* Faculty of Science, Mathematics and Computer Science, RU. 2024-01

M.J.G. Olsthoorn. *More Effective Test Case Generation with Multiple*

Tribes of AI. Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-02

B. van den Heuvel. *Correctly Communicating Software: Distributed, Asynchronous, and Beyond*. Faculty of Science and Engineering, RUG. 2024-03

H.A. Hiep. *New Foundations for Separation Logic*. Faculty of Mathematics and Natural Sciences, UL. 2024-04

C.E. Brandt. *Test Amplification For and With Developers*. Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-05

J.I. Hejderup. *Fine-Grained Analysis of Software Supply Chains*. Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-06

J. Jacobs. *Guarantees by construction*. Faculty of Science, Mathematics and Computer Science, RU. 2024-07

O. Bunte. *Cracking OIL: A Formal Perspective on an Industrial DSL for Modelling Control Software*. Faculty of Mathematics and Computer Science, TU/e. 2024-08

R.J.A. Erkens. *Automaton-based Techniques for Optimized Term Rewriting*. Faculty of Mathematics and Computer Science, TU/e. 2024-09

J.J.M. Martens. *The Complexity of Bisimilarity by Partition Refinement*. Faculty of Mathematics and Computer Science, TU/e. 2024-10

L.J. Edixhoven. *Expressive Specification and Verification of Choreographies*. Faculty of Science, OU. 2024-11

J.W.N. Paulus. *On the Expressivity of Typed Concurrent Calculi*. Faculty of Science and Engineering, RUG. 2024-12

J. Denkers. *Domain-Specific Languages for Digital Printing Systems*. Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-13

L.H. Applis. *Tool-Driven Quality Assurance for Functional Programming and Machine Learning*. Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-14

P. Karkhanis. *Driving the Future: Facilitating C-ITS Service Deployment for Connected and Smart Roadways*. Faculty of Mathematics and Computer Science, TU/e. 2024-15

N.W. Cassee. *Sentiment in Software Engineering*. Faculty of Mathematics and Computer Science, TU/e. 2024-16

H. van Antwerpen. *Declarative Name Binding for Type System Specifications*. Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2025-01

I.N. Mulder. *Proof Automation for Fine-Grained Concurrent Separation Logic*. Faculty of Science, Mathematics and Computer Science, RU. 2025-02

T.S. Badings. *Robust Verification of Stochastic Systems: Guarantees in the Presence of Uncertainty*. Faculty of Science, Mathematics and Computer Science, RU. 2025-03

A.M. Mir. *Machine Learning-assisted Software Analysis*. Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2025-04

L.T. Vinkhuijzen. *Data Structures for Quantum Circuit Verification and How To Compare Them*. Faculty of Mathematics and Natural Sciences, UL. 2025-05

D. van der Wal. *What is the Point? Single-Input-Change Testing*

a EULYNX Controller. Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2025-06

A. Rosset. *Uniform Monad Presentations and Graph Quasitoposes*. Faculty of Sciences, Department of Computer Science, VU. 2025-07

L. Guo. *Higher-Order Termination with Logical Constraints*. Faculty of Science, Mathematics and Computer Science, RU. 2025-08

D.A. Manrique Negrin. *A Model Orchestra in Digital Twins: A Model-Driven Approach to Integration and Orchestration*. Faculty of Mathematics and Computer Science, TU/e. 2025-09

C.A. Esterhuyse. *Specification-Centric Multi-Agent Systems*. Faculty of Science, UvA. 2025-10

H.M. Muctadir. *Consistency Matters: Building Consistent Digital Twin Virtual Entities*. Faculty of Mathematics and Computer Science, TU/e. 2025-11

A. Stramaglia. *Model Checking Machine-Control Applications*. Faculty of Mathematics and Computer Science, TU/e. 2025-12

M. Saeedi Nikoo. *Supporting business process management: clone de-*

tection and recommendation techniques. Faculty of Mathematics and Computer Science, TU/e. 2025-13

R.B. Rubbens. *Bridging the Implementation Gap: Advances in Model-Based Concurrent Program Verification.* Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2025-14

F.I. van der Berg. *DMC Model Checker: Delta-Driven Variable-Length Next-State Generation via Recursive Compression.* Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2025-15

D.G. Sprokholt. *Correct Translation between Weak Memory Model Architectures.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2025-16

B. Kohlen. *Verified & Fast Probabilistic Verification: Stepwise Refinement of Probabilistic Model Checking into an IEEE 754 Floating-Point Implementation.* Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2026-01

D. Frolich. *Exploratory Language Development.* Faculty of Science, UvA. 2026-02

L.B. van den Haak. *Verifying Op-*

timised Parallel Code. Faculty of Mathematics and Computer Science, TU/e. 2026-03

A. Papotti. *Human Acceptance of Machine Generated Code Security Analysis.* Faculty of Sciences, Department of Computer Science, VU. 2026-04

N. Doorn. *O bug, where art thou? Improving software testing education.* Faculty of Science, OU. 2026-05

C.R. van der Rest. *Reusable Programming Language Components.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2026-06

T.T.P. Franken. *A Parallel Programming Perspective – Designing AuDaLa and AuDaLogic.* Faculty of Mathematics and Computer Science, TU/e. 2026-07

T.J. Smeding. *Reverse-mode Automatic Differentiation for Functional Array Languages.* Faculty of Science, UU. 2026-08

S.A. Rukmono. *Towards Automated Software Architecture Explanation: An AI-enabled method and industrial evaluation.* Faculty of Mathematics and Computer Science, TU/e. 2026-09

